

Production Development — with — DeepSeek

Building and deploying scalable DeepSeek models
with LoRA, QLoRA, and Docker



Thirumalesh Konathala



Production Development — with — DeepSeek

Building and deploying scalable DeepSeek models
with LoRA, QLoRA, and Docker



Thirumalesh Konathala



Production Development with DeepSeek

*Building and deploying scalable
DeepSeek
models with LoRA, QLoRA, and
Docker*

Thirumalesh Konathala



www.bpbonline.com

OceanofPDF.com

First Edition 2026

Copyright © BPB Publications, India

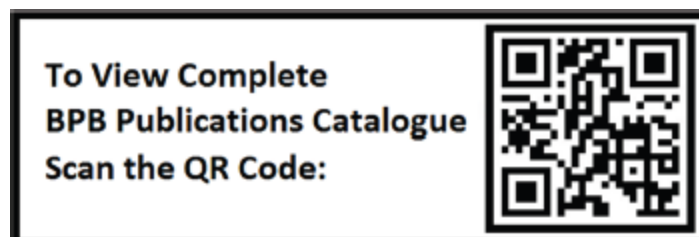
ISBN: 978-93-65891-782

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true and correct to the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but the publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.



www.bpbonline.com

OceanofPDF.com

Dedicated to

My respected parents:

Subbarao** and **Varalakshmi

and

*My beloved wife **Chandini**, and sons **Shyam** and **Ram***

OceanofPDF.com

About the Author

Thirumalesh Konathala is a chief AI scientist, founder and director of DATAi2i, and head of India and partner at Alphanome.AI. With nearly 20 years of experience, he specializes in production AI, generative AI, and enterprise data science. Through DATAi2i, he advances agentic AI platforms and GenAI solutions for industrial automation and operational intelligence.

As advisor for AI product development and advanced DS strategy, he consults multiple organizations on building scalable AI architecture, implementing enterprise MLOps, and assembling high-performing data science and AI teams. He is passionately committed to bridging AI research and production-ready LLM deployment solutions.

His career spans Amazon, Cardlytics, DBS Bank, Novartis, and Franklin Templeton, where he built teams, pioneered the Purchase Graph deep learning architecture, and implemented enterprise MLOps on Kubernetes. Thirumalesh Konathala holds a PhD in AI, a masters in statistics from Central University of Hyderabad, and was trained at the Indian Statistical Institute, Kolkata.

Based in Vishakhapatnam, he actively mentors at universities and speaks at AI conferences on production AI and LLM deployments.

[OceanofPDF.com](https://oceanofpdf.com)

About the Reviewer

Chandrani Mukherjee is a distinguished professional in **artificial intelligence (AI)** and enterprise architecture. Since 2024, she has served as a senior enterprise architect at Mphasis, where she leads the design and development of AI applications. In addition to her technical responsibilities, she mentors interns and colleagues and represents the organization at technology events and forums. Her expertise spans Python, LangChain, generative AI, vector databases, and advanced large language models, including Google Gemini and AWS Bedrock LLaMA, earning her recognition as a leader in her field. Prior to Mphasis, she briefly worked as an AI full-stack architect at McKesson in 2024. From 2022 to 2023, she served as a data analytics and AI consultant at First Abu Dhabi Bank in the UAE. Her earlier roles include application and data engineer at Etisalat in Dubai (2018–2022) and platform security developer at OSN (2018). She began her professional journey at TCS (2011–2016) and later joined Hewlett-Packard Enterprise as a senior software engineer (2016–2017).

Her academic achievements form the foundation of her career, including a bachelor of technology in information technology from Netaji Subhash Engineering College and a master of science in ML and AI from Liverpool John Moores University in 2021. She also holds certifications such as the Databricks Academy Accreditation in generative AI fundamentals.

A senior member of the Society of Women Engineers, Mukherjee actively advocates for women's advancement in technology. She has been honored with an Award for Excellence and credits her family and her early education at Carmel Convent School for shaping her values and career. Looking ahead, she aspires to take on leadership roles while contributing research and thought leadership to the global AI community.

Acknowledgement

I want to express my deepest gratitude to my family—wife, Chandini, and sons, Shyam and Ram—and my company, DATAi2i Pvt Ltd, team members—particularly Sanjana Mandarapu and Amrutha Naladeega—for their unwavering support and encouragement throughout this book's writing.

Beyond these immediate collaborators, I am deeply grateful to the exceptional leaders and mentors—Venu Kandala, Aki Kakko, Giridhar Patnaik, Jimmy Hendricks, and Warren Hearnese—who have provided me with transformative opportunities throughout my professional journey in the tech industry. Their visionary approach, strategic mentorship, and unwavering commitment to excellence have not only accelerated my growth but have also continuously inspired me to push boundaries and contribute meaningfully to advancing AI.

I am also grateful to BPB Publications for their guidance and expertise in bringing this book to fruition. This book's journey was a long one, with the valuable participation and collaboration of technical reviewers and editors.

Finally, I would like to thank all the readers who have taken an interest in my book and for their support in making it a reality. Your encouragement has been invaluable.

OceanofPDF.com

Preface

The race for reasoning-driven AI is accelerating. DeepSeek represents a paradigm shift combining advanced reinforcement learning with production-grade efficiency to deliver what previous generation LLMs could not. If you are building AI systems today, understanding DeepSeek is no longer optional; it is essential. Unlike traditional LLMs that rely primarily on supervised learning, DeepSeek's reinforcement learning foundation enables models to reason through complex problems, adapt to feedback, and run efficiently on resource-constrained environments making it particularly valuable for enterprise applications.

This book is designed to provide a comprehensive guide to building enterprise applications with DeepSeek. It covers a wide range of topics, from foundational architecture to model fine-tuning and deployment.

Throughout the book, you will gain insights into how DeepSeek fits into modern ML ecosystems and how to operationalize it across cloud services, enterprise pipelines, and AI-driven applications. The step-by-step examples and implementation details ensure that you not only understand the theory but can confidently bring DeepSeek into real-world projects.

This book is intended for AI enthusiasts, ML engineers, data scientists, researchers, and developers who want to master DeepSeek's capabilities and apply them in practical, scalable ways. Whether you are exploring reinforcement learning for the first time, migrating from other LLM frameworks, or architecting production-grade AI systems, this guide will equip you with both theoretical depth and practical implementation skills. Basic Python knowledge and familiarity with ML concepts are assumed.

By the end of this book, you will be able to fine-tune DeepSeek models, deploy them at scale, integrate RAG systems, and build autonomous agents.

Throughout my career, I have learned that great technology only becomes transformative when it is accessible and practical, which is what this book aims to deliver.

The chapters are organized in a progression designed to take you from understanding what DeepSeek is, to mastering its capabilities, to deploying and scaling it in production environments.

Chapter 1: Introduction to DeepSeek- It introduces DeepSeek, its origins, development journey, and the research breakthroughs that shaped it. Readers gain an understanding of DeepSeek's role in modern AI and how it has influenced current multimodal and reasoning-focused models.

Chapter 2: Understanding the Essentials of DeepSeek- This chapter explains DeepSeek's core strengths, such as reasoning, decomposition of complex tasks, and adaptive learning. It also covers the fundamentals of reinforcement learning and GRPO, giving readers insight into how DeepSeek improves through structured feedback. By the end, readers understand the mechanisms that make DeepSeek uniquely capable of continuous improvement.

Chapter 3: Overview of DeepSeek Models and Types- It outlines the primary DeepSeek model families, including language, vision, and distilled variants. It explains the role of each model type and the tasks they are optimized for. Readers learn the trade-offs between performance, efficiency, and compute needs. By the end, they can identify which DeepSeek model fits specific project requirements.

Chapter 4: Production Approaches- This chapter compares API-based usage with running DeepSeek locally. It highlights the convenience and scalability of APIs versus the control and privacy of local deployments. Practical examples illustrate the strengths and limitations of each method. Readers finish with clarity on the best production approach for their use case.

Chapter 5: Setup and Environment- This chapter guides readers through preparing their environment to run DeepSeek. It covers essential tools, installations, and the process of running a first local model. Practical steps help readers interact with DeepSeek and understand its behavior. By the end, they have a functional setup ready for experimentation.

Chapter 6: Supervised Fine-tuning- This chapter introduces supervised fine-tuning and shows how DeepSeek can be adapted to specific tasks. It explains parameter-efficient methods like LoRA and QLoRA to reduce compute cost. Readers learn when to use each technique and how fine-tuning impacts performance. The chapter builds the foundation for custom model training.

Chapter 7: Reinforcement Learning from Human Feedback- It focuses on how reinforcement learning enhances DeepSeek beyond standard training. It compares human feedback with automated model feedback and explains their roles. GRPO is introduced as the technique that drives coordinated learning. Readers finish with a solid grasp of RL's role in DeepSeek's improvement.

Chapter 8: Deploying DeepSeek with Inference and RAG- This chapter teaches how to deploy DeepSeek using Hugging Face inference endpoints. It then introduces RAG pipelines to integrate external knowledge for better responses. Readers learn retrieval methods and how to improve model accuracy using context. By the end, they can deploy and enhance DeepSeek for real applications.

Chapter 9: Deploying DeepSeek with Cloud, Multimodal and Agents- This chapter explores deploying DeepSeek on AWS for scalable production use. It explains multimodal interactions, showing how DeepSeek can handle images and documents. The chapter ends with building agent workflows capable of reasoning and tool use. Readers gain skills to create advanced, intelligent AI systems.

Chapter 10: Dockerization and Real-world Applications- This chapter covers containerizing DeepSeek with Docker for portable and consistent deployment. It explains Dockerfiles, images, API setups, and running models in isolated environments. Practical examples show where Dockerized DeepSeek can be applied in real-world workflows. Readers walk away ready to implement and maintain scalable AI applications.

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the *Coloured Images* of the book:

<https://rebrand.ly/dedb5a>

The code bundle for the book is also hosted on GitHub at <https://github.com/bpbpublications/Production-Development-with-DeepSeek>. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at <https://github.com/bpbpublications>. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks. You can check our social media handles below:



Instagram



Facebook



Linkedin



YouTube

Get in touch with us at: business@bpbonline.com for more details.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



Table of Contents

1. Introduction to DeepSeek

Introduction

Structure

Objectives

Introduction to DeepSeek

Main features and abilities

Comparison with traditional LLMs

The significance of reasoning abilities

Origins and development

The research team behind DeepSeek

Evolution from concept to implementation

Key milestones in DeepSeek's development

Key research and contributions

Reinforcement learning innovations

Mixture of expert architecture

Distillation of reasoning capabilities

Impact on the AI landscape

Applications and use cases

Conclusion

Points to remember

Key terms

2. Understanding the Essentials of DeepSeek

Introduction

Structure

Objectives

Reasoning capabilities

The emergence of reasoning in DeepSeek

Core reasoning abilities

Performance metrics

Chain-of-thought reasoning

Emergent behaviors in reasoning

Comparative advantage in reasoning

Introduction to reinforcement learning

Fundamental concepts of reinforcement learning

The reinforcement learning process

Reinforcement learning vs. traditional training methods

Pretraining

Supervised fine-tuning

Reinforcement learning

Key reinforcement learning concepts applied to DeepSeek

Reward functions

Exploration vs. exploitation

Policy optimization

DeepSeek's reinforcement learning implementation

DeepSeek-R1-Zero trained through reinforcement learning

DeepSeek-R1 using a hybrid approach

Self-learning and emergent behaviors

The aha moment

Thinking time allocation

Self-verification

Challenges and solutions in reinforcement learning training

Role of reinforcement learning in DeepSeek's reasoning capabilities

Introduction to Group Relative Policy Optimization

Policy optimization fundamentals

- Traditional policy optimization*
- Challenges in LLM policy optimization*
- A more efficient approach using GRPO*
 - Eliminating the critic model*
- The GRPO algorithm*
- Implementation in DeepSeek*
 - DeepSeek-R1-Zero training*
 - DeepSeek-R1 implementation*
- Advantages of GRPO*
- Limitations and considerations*
- Conclusion
- Points to remember
- Key terms

3. Overview of DeepSeek Models and Types

- Introduction
- Structure
- Objectives
- Language models
 - Evolution of DeepSeek language models*
 - Architecture and technical specifications*
 - Capabilities and performance*
 - Mathematical and logical reasoning*
 - Scientific reasoning*
 - Programming and code generation*
 - Natural language understanding and generation*
- Applications of DeepSeek language models*
 - Research and academia*
 - Education*
 - Software development*
 - Business intelligence*

Content creation

Vision models

Bridging vision and language using DeepSeek-VL

Architecture and design

Capabilities and performance

Specialized vision processing using DeepSeek-VL

Applications of DeepSeek vision models

Healthcare and medical imaging

Retail and e-commerce

Manufacturing and quality control

Document processing

Autonomous systems

Distilled models

The distillation process

The process of distillation

Innovations in DeepSeek's distillation approach

The DeepSeek-R1-Distill series

Available models and specifications

Quick download and setup summary

Performance benchmarks

Practical applications of distilled models

Edge computing

Cost-effective deployment

Latency-sensitive applications

Educational and research accessibility

Trade-offs and considerations

Performance gaps

Domain specificity

Continuous improvement

Comparative analysis of DeepSeek models

Performance vs. resource requirements

Selecting the right model for your use case

Conclusion

Points to remember

Key terms

4. Production Approaches

Introduction

Structure

Objectives

API

Understanding how API based deployment works

DeepSeek API services

API pricing and quotas

API integration best practices

Error handling and retries

Caching

Prompt engineering

Token optimization

API security considerations

Local LLMs

Understanding how local LLM deployment works

DeepSeek local deployment options

Hardware requirements

Deployment frameworks and tools

Hugging Face Transformers

VLLM

Ollama

LlamaIndex

Optimization techniques

Quantization

Model sharding

- Key-Value cache management*
- Flash Attention*
- Local deployment architectures*
 - Single-server deployment*
 - Distributed deployment*
 - Hybrid deployment*
- Local deployment best practices*
- Security considerations*
- Pros and cons of API versus local LLMs
 - Performance and latency*
 - Cost and resource requirements*
 - Data privacy and security*
 - Customization and control*
 - Scalability and reliability*
- Choosing the right approach
- Conclusion
- Points to remember
- Key terms

5. Setup and Environment

- Introduction
- Structure
- Objectives
- Local LLM tools
 - Core frameworks and libraries*
 - Installation*
 - Hugging Face Transformers*
 - Accelerate*
 - VLLM*
- Specialized tools for local deployment
 - Ollama*

- LM Studio*
- Text Generation WebUI*
- Optimization libraries
 - bitsandbytes*
 - Flash Attention*
 - AutoGPTQ*
- Setting up your environment
 - System requirements*
 - Setting up a Python environment*
- GPU setup for NVIDIA cards
 - Environment configuration for optimal performance*
 - Troubleshooting common setup issues*
 - CUDA out of memory errors*
 - Slow inference performance*
 - Dependency conflicts*
- Hello DeepSeek: Your first model
 - Choosing the right DeepSeek model*
 - Downloading and loading the model*
 - Using Hugging Face Transformers*
 - Using Ollama*
 - Using LM Studio*
 - Running inference with DeepSeek*
 - Using Hugging Face Transformers*
 - Using Ollama*
 - Using LM Studio*
- Exploring DeepSeek's capabilities
- Optimizing inference for use case
 - Prompt engineering*
 - Parameter tuning*
 - Batch processing*
 - Streaming generation*

Building a simple chat application

Conclusion

Points to remember

Key terms

6. Supervised Fine-tuning

Introduction

Structure

Objectives

Understanding supervised fine-tuning

The fine-tuning paradigm

Knowing when to use fine-tuning

The fine-tuning process

Dataset preparation

Model selection

Hyperparameter selection

Training execution

Evaluation

Fine-tuning DeepSeek models

Challenges in traditional fine-tuning

Parameter-efficient techniques

Low-Rank Adaptation

Learning how LoRA works

Advantages of LoRA

Implementing LoRA for DeepSeek models

Target modules for DeepSeek models

Quantized Low-Rank Adaptation

Learning how QLoRA works

Advantages of QLoRA

Implementing QLoRA for DeepSeek models

Comparing fine-tuning approaches

Best practices for parameter-efficient fine-tuning

Merging LoRA adapters with base models

Advanced techniques and future directions

Conclusion

Points to remember

Key terms

7. Reinforcement Learning from Human Feedback

Introduction

Structure

Objectives

Understanding reinforcement learning from human feedback

The RLHF paradigm

Reasons why RLHF matters

The RLHF process in detail

Supervised fine-tuning

Reward modeling

Preference data collection

Reward model training

Policy optimization

Proximal policy optimization

KL penalty and reference model

Challenges and considerations in RLHF

Advanced RLHF techniques

Direct preference optimization

Iterative RLHF

Constitutional AI

Group Relative Policy Optimization

Role of RLHF in DeepSeek development

Implementing RLHF with DeepSeek

- Prerequisites*
 - Preference data collection*
 - Generating responses for comparison*
 - Building a preference collection interface*
 - Preference data guidelines*
 - Reward model training*
 - Preparing the dataset*
 - Implementing the reward model*
 - Training the reward model*
- Policy optimization with proximal policy optimization
 - Setting up the proximal policy optimization environment*
 - Implementing the proximal policy optimization training loop*
 - Implementing direct preference optimization*
 - Implementing Group Relative Policy Optimization*
- Evaluating RLHF models
 - Preference evaluation*
 - Task-specific evaluation*
 - Safety and alignment evaluation*
- Conclusion
- Points to remember
- Key terms

8. Deploying DeepSeek with Inference and RAG

- Introduction
- Structure
- Objectives
- Inference endpoint with Hugging Face
- Retrieval-augmented generation
 - Understanding how RAG works*
 - Building a RAG system with DeepSeek*
 - Document processing and indexing*

- Retrieval component*
- Prompt construction*
- Generation with DeepSeek*
- A complete RAG system*

Improving response quality with retrieval pipelines

- Hybrid search*
- Re-ranking*
- Query decomposition*
 - Hypothetical Document Embeddings*
- Evaluating RAG systems*
 - Relevance evaluation*
 - Answer quality evaluation*
 - Hallucination assessment*

Retrieval-augmented generation applications with DeepSeek

- Medical question answering*
- Legal research*
- Technical support*
- Educational content*

Conclusion

Points to remember

Key terms

9. Deploying DeepSeek with Cloud, Multimodal and Agents

Introduction

Structure

Objectives

Cloud deployment with AWS

- Install dependencies*
- Inference endpoint*
- FastAPI app*
- Run the server*

Multimodal applications

Understanding multimodal integration

Building multimodal applications with DeepSeek-VL

Setting up DeepSeek-VL

Image captioning

Visual Question Answering

Image-based reasoning

Image-to-Text Generation

Putting the multimodal application all together

Advanced multimodal techniques

Retrieval-augmented generation

Multimodal retrieval-augmented generation

Improving response quality with retrieval pipelines

Multimodal chain-of-thought reasoning

Multimodal few-shot learning

Multimodal applications with DeepSeek-VL

Intelligent agents

Agent architecture

Building agents with DeepSeek

Setting up the language model

Implementing memory

Defining tools

Implementing planning and execution

Implementing the agent

Advanced agent techniques

Reasoning and Acting

Tool learning

Chain of thought planning

Self-reflection and correction

Agent applications with DeepSeek

Conclusion

Points to remember

Key terms

10. Dockerization and Real-world Applications

Introduction

Structure

Objectives

Introduction to Docker

Docker architecture and components

Docker Engine

Docker objects

Dockerfile

Docker workflow

Benefits of Docker for AI applications

Docker best practices

Latest update DeepSeek-V3.2-Exp

Containerizing DeepSeek

Preparing for containerization

Project structure

Dependencies management

Model handling strategy

Creating a Dockerfile for DeepSeek

Approach 1: Including model weights in the image

Approach 2: Downloading model weights at runtime

Approach 3: Mounting model weights as a volume

Optimizing Docker images for DeepSeek

Multi-stage builds

Distilled models

Efficient dependency management

Layer optimization

Building and testing the Docker image

- Containerizing different DeepSeek models*
- Deployment and API calling
 - Creating a FastAPI application for DeepSeek*
 - Deploying with Docker Compose*
 - Deploying to Kubernetes*
- Scaling and load balancing
 - Horizontal Pod Autoscaler*
 - Load balancing*
- Monitoring and logging
 - Prometheus and Grafana*
 - Elasticsearch, Logstash, Kibana stack*
- API calling from client applications
- Real-world applications
 - Customer support*
 - Educational assistants*
 - Healthcare assistants*
- Conclusion
- Points to remember
- Key terms

Index

CHAPTER 1

Introduction to DeepSeek

Introduction

AI has grown by leaps and bounds, moving from narrow systems built for specific jobs to flexible, all-purpose tools that can understand, think, and create content for many different uses. In this fast-changing world big language models like GPT-4, Claude, and Llama have caught people's eye with their smart abilities. However, among these cutting-edge advances, DeepSeek stands out by showing off impressive thinking skills powered by a new **reinforcement learning (RL)** method.

Most big language models get better through supervised fine-tuning, but DeepSeek does things by using RL. This lets it explore, learn, and make its answers better in ways that look a lot like how people think. DeepSeek does not just depend on huge sets of labeled examples. Instead, it keeps getting better through trial and error and adaptive learning, which makes it flexible and good at tough thinking tasks.

As we start this chapter, we will dig into what makes DeepSeek tick look at where it came from and how it grew, and check out key research breakthroughs. This deep dive will show you how DeepSeek has become a big player in today's AI scene, pushing the real-world use of **artificial intelligence (AI)** forward in a big way.

Structure

In this chapter, we will explore the following areas:

- Introduction to DeepSeek
- Origins and development
- Key research and contributions

Objectives

As we wrap up this chapter, we have gained a deep grasp of DeepSeek and its game-changing abilities. DeepSeek stands out from regular large speaking models because of how it is built and what it can do. This insight helps you spell out the key differences that set apart cutting-edge AI methods. Looking at it this way, we see how learning boosts serve as a key part in shaping DeepSeek's ability to adapt and learn. What is more, this basic know-how gives us a peek into how DeepSeek might be used in the real world down the line. DeepSeek's potential marks a big shift in smart chatbots, reshaping how users work with complex systems that make choices and boost output across many fields. Armed with this knowledge, you are set to put your skills to good use and push for smart progress in AI control answers.

Introduction to DeepSeek

DeepSeek is a new kind of AI model that changes our approach to **machine learning (ML)**. It consists of a group of models focused on improving thinking skills using RL. This special learning method helps DeepSeek think and solve problems more effectively than many traditional language models that mainly learn from huge datasets in a straightforward way.

Main features and abilities

DeepSeek is available in different versions, each designed for specific tasks:

- **DeepSeek-V3:** This is the basic version of the model. It is like other

advanced models and can understand and create text on a wide variety of topics.

- **DeepSeek-R1-Zero:** This unique model solely relies on RL, skipping the usual initial step of supervised learning. This allows it to develop strong problem-solving abilities naturally.
- **DeepSeek-R1:** Building on the R1-Zero model, this version includes more training stages and uses initial data to address challenges like difficult readability and language mixing, while further enhancing its problem-solving skills.
- **DeepSeek-R1-Distill:** This offers a range of smaller models, from 1.5 to 70 billion parameters. They capture the advanced thinking abilities of the larger R1 model, making these skills accessible with less computing power.

To better understand the structure of the DeepSeek family, here is a visual representation of their relationships:

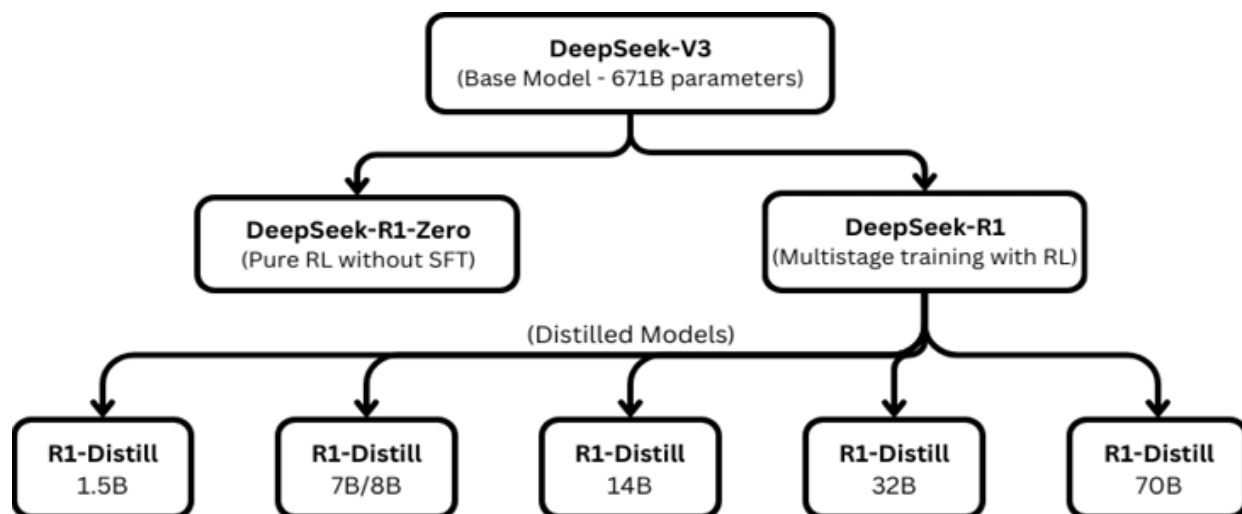


Figure 1.1: The DeepSeek family of models showing their developmental relationships

DeepSeek stands out with its strong ability to reason, making it different from traditional large language models. It excels in several areas, such as solving tough math problems. DeepSeek can easily handle complex math questions, as demonstrated in tests like the **American Invitational Mathematics Examination (AIME)**. This ability shows it can manage challenging tasks in both math and science reasoning.

In addition to math, DeepSeek is effective in coding and software engineering. It does well in competitions that require strong analytical and computational skills. DeepSeek achieves this by breaking down complex problems into smaller, manageable parts, trying out various solutions, and carefully checking each step of its work.

Another impressive skill of DeepSeek is its ability to reflect on and verify its work. It uses RL techniques to analyze its reasoning, find mistakes, and explore new ways of solving problems. This ongoing self-improvement process makes DeepSeek more reliable and accurate in solving problems.

Moreover, DeepSeek can handle a large amount of information with its context window of 128,000 tokens. This capability allows it to process long documents and maintain a detailed understanding throughout lengthy discussions. This feature is particularly useful for real-life applications that need continuous focus and deep comprehension over extended conversations or large text inputs.

Combined, these advanced abilities make DeepSeek especially well-suited for tasks that demand careful thinking, precision, and smart decision-making.

Comparison with traditional LLMs

To appreciate what makes DeepSeek unique, it is helpful to understand how it differs from traditional large language models:

Feature	Traditional LLMs	DeepSeek
Training approach	Primarily rely on supervised fine-tuning after pretraining, learning from massive datasets to predict subsequent tokens in a sequence.	Emphasize RL to develop reasoning capabilities, allowing the model to learn through trial and error, closely mimicking human cognitive processes.
Reasoning process	Often produce reasoning that appears plausible but may contain logical errors, lacking mechanisms for self-correction.	Demonstrate more rigorous, self-correcting reasoning patterns, reflecting a deeper understanding and the ability to refine outputs.
Problem-solving	May struggle with multi-step problems requiring logical chains of thought, often lacking	Naturally develop chain-of-thought (CoT) reasoning, enabling the model to tackle complex problems methodically and

	coherence in complex scenarios.	effectively.
Self-improvement	Have limited ability to reflect on and improve their own outputs, often requiring external interventions for refinement.	Can recognize errors in reasoning and attempt alternative approaches, showcasing a form of self-reflection and adaptability.
Architecture	Often use dense transformer models, which can be computationally intensive and less efficient.	Leverage Mixture of Experts (MoE) architecture for efficiency, activating only relevant subsets of parameters during processing, leading to optimized performance.

Table 1.1: Comparison with traditional LLMs

DeepSeek stands out because it uses a different training method than most **large language models (LLMs)**. Regular LLMs usually focus on predicting the next word in a sentence based on what they have learned from lots of examples. However, DeepSeek is more about building strong reasoning skills. It uses a technique called RL. This approach rewards DeepSeek for finding the right answers to hard questions, rather than just making text that looks like it was written by humans.

You can see how effective this is by looking at DeepSeek's results in challenging tests. For example, in the 2024 AIME, DeepSeek-R1 got a 79.8% accuracy rate, which is almost as good as the OpenAI model o1-1217. In contrast, other models like GPT-4o scored only 9.3%, and Claude-3.5-Sonnet scored 16.0%. This big difference shows that DeepSeek's unique way of training really improves its reasoning abilities.

The R1 model produced by DeepSeek shows that its reasoning is as strong as those of incumbent players in the industry- OpenAI and Anthropic, but on a much lower spending. Recent estimates indicate that training R1 costs around \$5 6 million, which is far less than the hundreds millions that US companies spent on similar projects. What is more, initial estimations show that the operational token prices DeepSeek uses are more than 90 % the ones OpenAI have.

The integration of a carefully thought reward model becomes a critical step forward for DeepSeek. The system gets feedback—numerical scoring—via this mechanism as to the quality, usefulness, or truthfulness of its responses. On the later runs, DeepSeek will start to favour outputs that can achieve high

rewards, internalising a positive reinforcement cue.

Take this question: What is the capital of Australia? During its early stages of development, the model may provide an incorrect or incomplete answer, i.e. Sydney. This outcome gets a relatively low score by the reward module. In contrast, the reward model uses a much greater weight when DeepSeek forms the correct response, viz. Canberra, the Australian capital. Similar steady feedback will cause the system to hone what it chooses to say: correct, attentionally fine-tuned responses will rise to the forefront, shallow or off-topic responses will fall by the wayside.

Such a cumulative mechanism parallels the mechanisms through which human learners react to corrective information. Say a student has a chance to refine an answer that was drafted with the consultation of an instructor. Similarly, DeepSeek sequentially improves the generated texts with the assumption guidance of the reward model.

In conclusion, DeepSeek's focus on RL makes it different from traditional LLMs. This method helps DeepSeek develop excellent problem-solving skills, as demonstrated by its impressive results in math reasoning tests.

The significance of reasoning abilities

DeepSeek is a major improvement over traditional LLMs. While regular models look for patterns and connections in text, they often struggle with clear logical thinking and solving problems in a step-by-step manner. DeepSeek takes a different approach by using strong tools that allow it to analyze and solve challenging problems effectively.

One of DeepSeek's key strengths is its reliability. It can verify its own logic, ensuring its answers are not only consistent but also correct. This is particularly important in areas where accuracy is critical, like scientific research and complex software development. Transparency is another important feature of DeepSeek. It uses a *chain-of-thought* process, which makes its decision-making clear and easy for users to understand. By allowing people to follow its reasoning steps, DeepSeek builds trust and simplifies the process of fixing issues or enhancing its work.

Lastly, DeepSeek excels in adaptability. It continuously learns and improves

its problem-solving methods, making it capable of handling new and unfamiliar challenges comfortably. This ongoing enhancement makes DeepSeek highly flexible, and it can tackle a wide variety of tasks efficiently.

Origins and development

DeepSeek started and developed rapidly thanks to impressive new ideas. Unlike many top AI models that require lots of resources and take a long time to build, DeepSeek became a strong rival much quickly. This success was due to smart engineering, better algorithms, and a strong focus on thinking abilities rather than just trying to grow in size.

The research team behind DeepSeek

DeepSeek was created by DeepSeek-AI, a team made up of researchers and engineers who want to improve artificial intelligence by sharing ideas openly. The company started because they aimed to build AI systems that can think better and to share these systems with researchers everywhere. Creating DeepSeek models involved many researchers working together, each an expert in areas like RL, large language models, and efficient model design. At first, DeepSeek was not as well-known as some big AI labs, but their team quickly became important in AI research due to their models' good performance and new ideas.

Evolution from concept to implementation

The journey to create DeepSeek was thoughtfully planned. It was not just about using old ideas; we made several strategic advances that led to new ways of thinking. Everything started with DeepSeek-V3, the foundational model. This model was crucial because it provided the strong base needed for future advancements.

A key development in DeepSeek was the use of a special setup called the **Mixture of Experts (MOE)** architecture. This setup was a significant change that helped the model improve its abilities without needing more computing power. With the MOE framework, teams could enhance the

model's skills in a more efficient way, setting new standards for balancing performance with costs. One of the key breakthroughs in DeepSeek's development was the RL process used in DeepSeek-R1-Zero, a model that made significant progress in handling discussions.

This was driven by a desire to push the limits and explore new possibilities. This version featured multi-stage training, which addressed weaknesses in earlier models and made the model better at discussions. The aim was to make these advanced capabilities available to more people. Therefore, the team worked to turn the strengths of bigger models into a more efficient and easier-to-use version, spreading the benefits of these advancements in AI.

While many AI labs focus on building bigger and resource-heavy models, DeepSeek showed that significant improvements in discussion skills could be made through smarter training methods. This focus on intelligence and efficiency highlighted the importance of careful, resource-friendly innovation in advancing AI.

Key milestones in DeepSeek's development

The development of DeepSeek has been defined by a series of pivotal milestones that highlight its rapid progress and innovative approach. In late December 2023, DeepSeek-V3 was released, positioning itself as a direct competitor to OpenAI's GPT-4. Remarkably, this model was trained in just two months at an estimated cost of \$5.6 million—a fraction of the expense typically associated with comparable models. This achievement underscored DeepSeek's commitment to efficiency and cost-effectiveness in AI development.

By January 2024, the team published groundbreaking research on DeepSeekMoE, detailing their innovative approach to the MoE architecture. This work laid the foundation for subsequent models, enabling more efficient scaling and specialized task handling. The MoE architecture became a cornerstone of DeepSeek's design, allowing for greater model capacity without proportional increases in computational demands.

On January 20, 2025, DeepSeek reached another major milestone with the release of DeepSeek-R1 and DeepSeek-R1-Zero. These models showcased the team's breakthroughs in developing advanced reasoning capabilities

through RL. By demonstrating that sophisticated reasoning could emerge from pure RL, DeepSeek challenged traditional approaches and opened new avenues for AI development.

Later in January 2025, the team introduced the DeepSeek-R1-Distill series, a significant step toward democratizing advanced reasoning capabilities. These distilled models made it possible to deploy high-performance reasoning in smaller, more efficient systems, expanding access to cutting-edge AI technologies. This milestone not only highlighted DeepSeek's technical prowess but also its commitment to making advanced AI more accessible and practical for a broader range of applications. Together, these milestones mark the evolution of DeepSeek into a leading force in the AI landscape.

What makes these milestones particularly impressive is the speed and efficiency with which they were achieved. The DeepSeek team demonstrated that significant advances could be made without the vast resources of the largest AI labs, pointing toward a more democratized future for AI research and development.

The release of DeepSeek-R1 in January 2025 made waves in the AI community not just for its performance, but also for the team's decision to open-source the models, allowing researchers to examine and build upon their work. This commitment to open science has accelerated progress in the field and made advanced AI capabilities more widely accessible.

Key research and contributions

DeepSeek's development has been accompanied by significant research contributions that advance our understanding of how to build more capable AI systems. These contributions span several areas, with particularly notable innovations in RL approaches, model architecture, and training methodologies.

Reinforcement learning innovations

DeepSeek has made a major advancement by using RL to improve how computers learn to have discussions like humans. Typically, when training

LLMs for speaking, we use **supervised fine-tuning (SFT)**. This involves guiding the models with human examples. While SFT is effective for many tasks, it does not always teach complex discussion skills well. DeepSeek discovered that RL opens new opportunities because it helps models learn advanced discussion skills that SFT alone cannot provide. This was a different approach, as RL allowed models to develop discussion skills without being fully dependent on human examples.

One significant innovation from DeepSeek is **relativization optimization (GRPO)**. This is a new RL method that removes the need for critic models, which are traditionally essential parts of RL setups. GRPO uses group values instead, making the training of large language models more efficient and practical. This method not only speeds up the learning process but also makes it applicable to more situations. The approach helps models think critically, assess their processes, and develop detailed thinking strategies essential for solving complex problems.

The researchers noticed something they called *AHA moments*. These occurred when the model began spending more time thinking through difficult problems, reassessing its initial thoughts, and improving them. This marked an important step in developing *metacognitive* skills, meaning the ability to think about and adjust one's own thought processes. The model independently discovered that deeper thinking leads to better solutions, without being explicitly programmed to do so. This process of learning and adaptation shows the potential of RL, creating a model capable of changing its problem-solving approach depending on the complexity of the task.

The success of DeepSeek's approach with RL points to a promising future for AI. It suggests a shift in strategy, allowing models to learn and improve through exploration and optimization rather than just following predefined human examples. By moving away from solely supervised learning toward RL, AI systems might develop broader problem-solving skills. This shift not only enhances AI capabilities but also paves the way for creating models that can handle a wider range of complex tasks by becoming more independent and adaptable.

Mixture of expert architecture

Another key contribution from the DeepSeek team is their work on efficient model architecture, particularly their implementation of the MoE approach:

- **DeepSeekMoE:** The team's research on MoE architecture laid the groundwork for more efficient model scaling. By dividing the model into specialized *experts* that handle different types of tasks, MoE allows for increased model capacity without proportional increases in computational requirements. [citeturn0search1](#)
- **Efficiency gains:** The MoE approach enabled DeepSeek-V3 to achieve impressive performance with only 37 billion activated parameters at a time, despite having a total parameter count of 671 billion. This represents a significant efficiency advantage over dense models of comparable capability. [citeturn0search1](#)
- **Domain specialization:** The MoE architecture allows for better handling of specialized domains (like mathematics, coding, or scientific reasoning) by routing queries to the most appropriate experts, resulting in improved performance across diverse tasks. [citeturn0search1](#)

This architectural approach contributed significantly to DeepSeek's ability to produce models with strong performance despite more limited computational resources compared to some competitors. It represents a step toward more efficient AI that can do more with less, potentially democratizing access to advanced AI capabilities.

Distillation of reasoning capabilities

DeepSeek's ability to transfer sophisticated reasoning abilities from large models to smaller, more effective ones represents a significant advancement in AI. Using a technique known as distillation, the DeepSeek team discovered a way to transfer the intricate thinking of large models to smaller versions. This method preserves the capacity for deep thinking while maintaining efficiency, making it superior to directly applying RL to small models. These models—such as those in the DeepSeek-R1-Distill series—perform exceptionally well despite their smaller size. For instance, DeepSeek-R1-Distill-Qwen-7B outperformed many larger models with a score of 55.5% on the AIME 2024 exam. In addition to enhancing performance, the distillation process increases accessibility to sophisticated

reasoning abilities, enabling these systems to operate on gadgets with low power. DeepSeek has expanded the number of users and applications for complex AI systems by making them easier to use. Making sophisticated AI accessible, particularly for devices and scenarios with limited resources, requires the successful transfer of reasoning abilities to smaller models.

Impact on the AI landscape

AI has changed significantly because of DeepSeek models, which are publicly available. According to reports, the DeepSeek-V3 model's development cost roughly \$5.6 million. This is significantly less expensive than comparable models, which makes us reevaluate how much it costs to develop cutting-edge AI systems. RL, which aids in the development of reasoning abilities in models, was effectively applied by DeepSeek. They have accelerated advancement in this crucial field by making their models available to researchers.

After DeepSeek-R1 was released, numerous teams attempted to duplicate it. They were able to use less data to create smaller models with comparable reasoning abilities. More experiments and ideas have resulted from this. Other AI labs have taken notice of DeepSeek models' impressive performance, which has increased competition and could hasten the field's advancement.

The significance of algorithmic efficiency has also been brought to light by DeepSeek's accomplishments. DeepSeek has demonstrated that sophisticated reasoning capabilities can be attained through more intelligent, effective methods by emphasizing novel training techniques rather than merely increasing model size. Combined, these efforts represent a major breakthrough in developing AI systems with improved reasoning capabilities, opening the door for more approachable and resource-conscious developments in the area.

Applications and use cases

DeepSeek's strong reasoning abilities make it a flexible tool that can be used in a variety of contexts. Its ability to comprehend complex ideas and solve large problems is a great advantage in science. DeepSeek looks for connections and patterns in many scientific publications. It may also offer

fresh concepts for investigation. The tool does a good job of creating hypotheses based on theories and data that already exist. DeepSeek is used by scientists to design experiments that will efficiently test these theories. It also aids in deciphering intricate experimental findings and identifies areas for further investigation.

Since DeepSeek performs well on **math** benchmarks, it is known for its mathematical and problem-solving skills. It serves as a teaching aid and progressively explains difficult mathematical ideas and procedures. It can be used by mathematicians to analyze data, follow the suggested course of action, and spot possible issues at work. DeepSeek can also solve optimization issues in fields like engineering, finance, and logistics, offering creative answers to challenging problems.

For software development, DeepSeek's expertise in coding and software engineering makes it a valuable resource. It can generate high-quality, functional code based on specific requirements and assist in debugging and optimizing existing code for better performance. The model also provides clear explanations of complex codebases, helping developers understand and modify systems more effectively. Additionally, it supports the design of software architectures that meet specific requirements and constraints.

DeepSeek is a valuable tool in the **business world**, especially for making smart decisions. It examines various strategies and market scenarios to predict results, which aids in planning and decision-making. DeepSeek also identifies possible risks and offers ways to deal with these risks after analyzing the data thoroughly.

In the field of **education**, DeepSeek is incredibly useful. It serves as an excellent tutor because it can explain topics in detail and adapt to different learning abilities. Rather than simply providing answers, it explains complex ideas in simpler terms. It offers personalized explanations based on what the student understands and how they prefer to learn. Plus, it helps students figure out how to solve problems step by step. DeepSeek also shines in creative tasks. It creates well-organized and structured content, like articles, showcasing its versatility beyond analysis.

DeepSeek is a powerful tool with diverse applications, as illustrated in the following image:

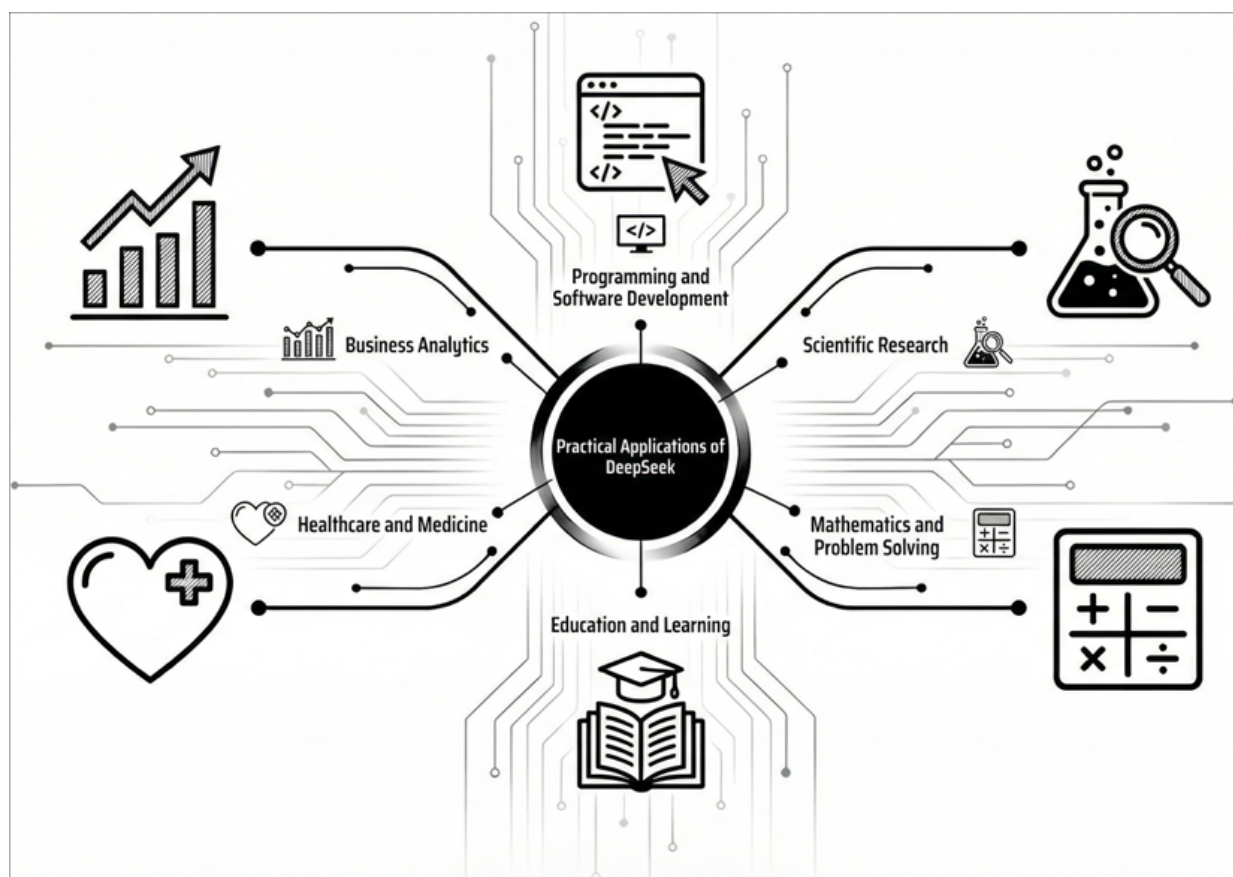


Figure 1.2: Practical applications of DeepSeek-R1

Conclusion

DeepSeek represents a significant advancement in AI. This is distinguished by its innovative approaches and exceptional discussion capabilities. In addition to having compelling arguments on its own, DeepSeek achieves performance levels more akin to a resource-intensive model by incorporating the improvements into effective architectures like the MOEs.

DeepSeek stands out not only for his benchmark performance but also for the creative methods he employs to get there. Reinforcement highlights the potential for it to transcend conventional, monitored learning paradigms by fostering the development of highly developed behaviors like self-examination, reflection, and the concept of a chain of life. Additionally, the success of distillation using more compact, effective models of these abilities portends a time when complex discussion is widely available and democratizes the potent potential of large-scale models.

The use of DeepSeek for a variety of applications will be further explored in the upcoming chapter. The goal of this book is to equip readers with the knowledge they need to apply DeepSeek's capabilities to solve practical issues.

In [Chapter 2](#), *Understanding the Essentials of DeepSeek*, we will explore what makes DeepSeek special. We will move past the basic stuff and dive into how DeepSeek handles complex tasks with smart reasoning that makes it unique. You will learn about RL, which helps DeepSeek adapt and make quick decisions. We will also discuss **Group Relative Policy Optimization (GRPO)**, a method that boosts learning by encouraging teamwork among several agents. By the end of this chapter, you will have a solid understanding of these ideas. This will help you fully use DeepSeek in your projects with confidence.

Points to remember

- The DeepSeek family of large language models was developed by using RL in a smart way to improve reasoning abilities.
- DeepSeek-R1-Zero showed that it is possible for complex reasoning behaviors to develop through RL alone, unlike traditional large language models that mostly rely on supervised fine-tuning methods.
- DeepSeek-R1 uses a technique called multi-stage training to further this approach. This technique helps to tackle challenges while keeping strong reasoning abilities intact.
- The efficiency of DeepSeek is higher than other similar dense models, thanks to its MoE architecture.
- DeepSeek makes advanced reasoning more accessible, even with limited computing power. This is achieved through the DeepSeek-R1-Distill series, which has been successful in transferring reasoning skills to smaller models.
- DeepSeek's significant contributions to AI research include advancements in RL techniques, creating effective model designs, and successfully transferring reasoning abilities to different models.

- Scientific research, software development, business intelligence, mathematical problem solving, education, and content creation are some of the main uses for DeepSeek.

Key terms

The key terms are as follows:

- **RL:** A ML approach where an agent learns to make decisions by performing actions and receiving rewards or penalties, without explicit supervision.
- **CoT:** A reasoning process where complex problems are broken down into sequential steps, with each step building on previous reasoning.
- **MoE:** A neural network architecture that divides processing among specialized sub-networks (experts), activating only a subset for any given input.
- **GRPO:** A RL algorithm used in DeepSeek that foregoes the critic model and estimates baselines from group scores.
- **Distillation:** The process of transferring knowledge from a larger “teacher” model to a smaller “student” model to improve the performance of the smaller model.
- **Self-verification:** The ability of a model to check and validate its own reasoning process and outputs.
- **SFT:** A training approach where a model is further trained on labeled examples to improve performance on specific tasks.
- **Emergent behavior:** Capabilities or patterns that develop during training without being explicitly programmed, arising from the interaction of simpler training objectives.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 2

Understanding the Essentials of DeepSeek

Introduction

In [Chapter 1](#), *Introduction to DeepSeek*, we talked about DeepSeek, an exciting new large language development in AI. We explored how it was created, its origins, and its major contributions to AI. Now, we are focusing on what makes DeepSeek stand out from other **large language model (LLM)** advancements. DeepSeek is changing the way language models handle complex problems and make decisions, offering a new approach compared to traditional methods.

Traditional LLMs are good at finding patterns and creating text smoothly. However, they often find it difficult to handle tasks that require logical reasoning and need to be done step by step. DeepSeek was designed to overcome these challenges. It uses innovative training strategies with a focus on **reinforcement learning (RL)**, making it capable of tackling tricky problems that need deep thinking. In this chapter, we explore the elements that make DeepSeek so powerful, including its reasoning techniques, the role of RL in its development, and the strategies that improve its performance.

By digging into these key features, we learn why DeepSeek does so well with tasks that demand strong logical analysis. Understanding these aspects

shows not only how DeepSeek works but also how to maximize its potential. Whether you are a developer, a researcher, or just someone interested in AI, grasping these concepts can help you use DeepSeek in your projects, opening up new possibilities in the fast-evolving world of AI.

Structure

In this chapter, we will explore the following areas:

- Reasoning capabilities
- Introduction to reinforcement learning
- Role of reinforcement learning in DeepSeek's reasoning capabilities
- Introduction to Group Relative Policy Optimization

Objectives

This chapter is here to help you understand what DeepSeek can do. After reading, you will notice how DeepSeek is unique compared to regular LLMs, especially in solving tough problems. You will also learn the basics of RL, which helps DeepSeek with complex tasks.

Additionally, you will discover **Group Relative Policy Optimization (GRPO)**, a technique that makes DeepSeek work better, and how it improves the model's performance. You will see how reasoning methods, RL, and GRPO, come together to make DeepSeek smarter and more adaptable. This knowledge will help you use DeepSeek's features in real-world situations effectively.

Reasoning capabilities

Reasoning means thinking carefully and coming to a decision, opinion, or conclusion using facts or starting points. In AI, reasoning skills are about a model's ability to understand information logically, draw correct conclusions, and solve problems step by step. DeepSeek is different from other language models because it has strong reasoning skills. This ability lets

it deal with difficult problems skillfully, showing a high level of understanding that many LLMs usually do not have. This advanced reasoning gives DeepSeek an edge in tackling complex challenges.

The emergence of reasoning in DeepSeek

DeepSeek obtained its reasoning abilities from RL methods instead of explicit programming instructions. During the RL training process of DeepSeek-R1-Zero which lacked supervised fine-tuning multiple distinctive behaviors appeared:

- **Chain-of-thought processing:** DeepSeek-R1-Zero automatically learned how to transform complicated problems into smaller actionable parts which it solved systematically step by step.
- **Self-verification:** During self-verification the model checked its outcomes twice to ensure the proof of its solutions prior to delivering its final answers.
- **Reflection and revision:** During metacognition DeepSeek-R1-Zero showed ability to evaluate its solution methods and identify weak approaches after which it searched for new solution paths.

The DIY development of advanced cognitive abilities by DeepSeek stands as the most impressive achievement in its creation process. During training the research team spotted critical *aha moments* which revealed when the model rapidly developed new capabilities including reevaluating its solution methods because previous techniques failed.

Example: The aha moment

In the DeepSeek research paper, an illustrative example highlights an aha moment experienced by an intermediate version of DeepSeek-R1-Zero. In this instance, the model was presented with a mathematical problem:

If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to?

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\dots$

$$\sqrt{(a - \sqrt{(a + x)})^2} = x^2 \implies a - \sqrt{(a + x)} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That is an aha moment I can flag here.

Let us reevaluate this step-by-step to identify if the correct sum can be:

We started with the equation: $\sqrt{(a - \sqrt{(a + x)})} = x$

First, let's square both sides:

$$a - \sqrt{(a + x)} = x^2 \implies \sqrt{(a + x)} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

Initially, the model embarked on solving the equation by squaring both sides, leading to a series of algebraic manipulations. However, during this process, it encountered a point of confusion, prompting a self-reflective pause. Recognizing a potential misstep, the model decided to reassess its approach, stating:

Wait, wait. Wait. That is an aha moment I can flag here. Let us reevaluate this step-by-step to identify if the correct sum can be...

This spontaneous self-correction, characterized by the model's decision to restart its solution process upon detecting an error, exemplifies a sophisticated form of self-verification. Notably, this behavior was not explicitly programmed but emerged naturally through the RL paradigm employed during DeepSeek-R1-Zero's development.

Core reasoning abilities

Deepseek's discussion capabilities are diverse and comprehensive in mathematical, logical, arithmetic and scientific disciplines. The performance of benchmarks such as the **American Invitational Mathematics Examination (AIME)** and the Math-500 is notable, reaching an impressive 97.3% pass @ 1 point count with a simple over OpenAI's O1-1217 model.

This model skillfully uses **mathematical concepts** and formulas to build multi-stage solutions for complex problems, recognize patterns and

symmetry that drive problem solving, check mathematical evidence, and simultaneously identify logical incorrectness.

Beyond mathematics, there is a strong **logical** argument in DeepSeek. It effectively identifies valid deductive arguments, recognizes logical errors, assesses the strength of evidence to support conclusions, and follows a chain of conditional statements (in the case of arguments).

In **arithmetic thinking**, DeepSeek presents amazing skills in problem solving by replacing complex problems with smaller, easier to manage problems. It recognizes the problem algorithmically, identifying edge cases and boundary conditions, and optimizing solutions for efficiency. Furthermore, this model uses hypothesis formation and testing, evaluation of experimental design, interpretation of data, and scientific discussion methods to draw appropriate conclusions and understand causality.

This ability includes advanced discussion skills in DeepSeek in various fields.

Performance metrics

To assess the debate capabilities of DeepSeek-R1, researchers evaluated the models across various benchmarks to test knowledge of mathematics, science, and arithmetic thinking. The results were impressive

In AIME 2024, Deepseek-R1 achieved an accuracy rate of 79.8% (Pass@1), exceeding 9.3% from GPT-4o to GPT-4o and 16.0% from Claude-3.5-SUN. In the Math-500 benchmark, the model achieved 97.3%, surpassing the O1 model from OpenAI, achieving 96.4%, showing an extraordinary mathematical argument. Furthermore, DeepSeek-R1 demonstrated a strong skill in scientific discussion by achieving a pass rate of 71.5% on the GPQA Diamond Benchmark, a physics assessment at the graduate level. In computer arguments, the model reached a 96.3 percentile rating on the Code Force Platform. This reflects coding functions at the expert level.

In the subfield of AI benchmarking, Pass k has become widely used as a measure of performance on reasoning or coding problems. Pass@k refers to the probability, over the set of all top-k answers created by the model, that at least one of these is correct. Pass@1, therefore, indicates that whatever is the first move of the model is correct, and Pass@5 implies that the correct

solution must be found in at least one of the first five trials. This metric is estimated by sampling several responses per query and estimating whether the correct answer seems among the admissible attempts. The process produces a more comprehensive determination of both the accuracy and reliability of the result, as it shows not only that a model can solve a particular problem, but in what cases the solution will tend to arise rather than whether it will again need a reiteration of the process.

The following image compares the argument functions of DeepSeek-R1-Zero and OpenAI S O1 models via various benchmarks, highlighting the differences in performance:

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Figure 2.1: DeepSeek-R1-Zero vs. OpenAI o1 on reasoning benchmarks

(Source: <https://www.diskmfr.com/the-secrets-behind-deepseek-r1-training-revealed/>)

These metrics collectively highlight DeepSeek-R1's advanced reasoning capabilities, surpassing many competing models, particularly in domains that require structured and logical thinking.

Chain-of-thought reasoning

A pivotal component of DeepSeek's reasoning framework is its implementation of **chain-of-thought (CoT)** reasoning. This methodology involves deconstructing complex problems into a series of intermediate steps, facilitating a more transparent and systematic approach to problem-solving.

The CoT process in DeepSeek encompasses several key stages:

- **Problem interpretation:** The model begins by analyzing the problem which involves understanding critical variables together with their boundaries and specified goals.

- **Strategy formulation:** After initial interpretation the system creates its problem-solving approach through the comparison of multiple strategic options.
- **Step-by-step execution:** The model performs solution execution step by step while it explains each move to deliver clear and coherent solutions.
- **Verification:** As part of its operation DeepSeek performs continuous verification on intermediary results to guarantee both precision and accuracy in the process.
- **Refinement:** When inconsistencies or mistakes emerge the model transforms its procedure according to discovered information while showing its capability to adjust its reasoning logic.

The method finishes by combining the whole process into a simple solution which effectively represents the answer.

Such systematic organization of approaches enhances model problem-solving while making its thinking procedures transparent to users who gain trust and deeper comprehension.

Example of CoT reasoning

DeepSeek's CoT reasoning enables it to systematically tackle complex problems by breaking them down into manageable steps. Consider the following example demonstrating how DeepSeek approaches a mathematical problem:

Question: A rectangular garden has a perimeter of 100 meters and an area of 600 square meters. What are the dimensions of the garden?

DeepSeek's reasoning:

<think>

Let's denote the length of the garden as L and the width as W .

From the perimeter information:

$$2L + 2W = 100$$

$$L + W = 50$$

From the area information:

$$L \times W = 600$$

Now I need to find L and W. I can use the first equation to express W in terms of L:

$$W = 50 - L$$

Substituting this into the area equation:

$$L \times (50 - L) = 600$$

$$50L - L^2 = 600$$

$$-L^2 + 50L - 600 = 0$$

$$L^2 - 50L + 600 = 0$$

I can solve this quadratic equation using the quadratic formula:

$$L = [-b \pm \sqrt{b^2 - 4ac}] / 2a$$

where $a = 1$, $b = -50$, $c = 600$

$$L = [50 \pm \sqrt{(2500 - 2400)}] / 2$$

$$L = [50 \pm \sqrt{100}] / 2$$

$$L = [50 \pm 10] / 2$$

$$L = 30 \text{ or } L = 20$$

If $L = 30$, then $W = 50 - 30 = 20$

If $L = 20$, then $W = 50 - 20 = 30$

In either case, the dimensions are 30 meters by 20 meters.

Let me verify:

Perimeter: $2(30) + 2(20) = 60 + 40 = 100$ meters ✓

Area: $30 \times 20 = 600$ square meters ✓

</think>

The dimensions of the rectangular garden are 30 meters by 20 meters.

This example showcases DeepSeek's ability to systematically interpret the problem, formulate a strategy, execute step-by-step calculations, verify intermediate results, and refine its approach as needed, culminating in a clear and accurate solution.

Emergent behaviors in reasoning

DeepSeek displays extraordinary cognitive behaviors which surpass those original programmers did not build into its system. DeepSeek performs four main behaviors which involve self-analysis as well as examination of multiple solutions while searching for errors that need correction and adaptation of its processing time. The reasoning behaviors are as follows:

- **Self-reflection:** The self-reflective capability of DeepSeek allows it to examine both its logical processes and assess its strategies' performance and seek out areas where it may be vulnerable. The evaluation process enables the model to focus its problem-solving methods making sure it does not waste time on useless solution paths. Complex problem-solving tests reveal that DeepSeek performs self-assessment of its problem-solving methods before achieving more precise results.
- **Alternative solution exploration:** The model selects different solution paths during problem-solving then it works through the paths to discover the most efficient and elegant solution. Similar to human experts DeepSeek evaluates different strategic options until it identifies its best possible solution. DeepSeek generates extended logical analysis processes that help the system evaluate different methods to optimize its problem-solving skills.
- **Error detection and correction:** DeepSeek implements an error detection system alongside automatic error correction functions. The ability to detect and correct itself is essential for dependable problem-solving because small errors typically produce major inaccurate outcomes in particular fields. The learning process becomes more accurate and trustworthy since the model can both identify faulty logic and automatically adapt it during its learning period.
- **Adaptive thinking time:** DeepSeek demonstrates an important behavior by giving complex problems longer thinking time than easier problems. Through natural learning, the model distributes more computation to challenging tasks similar to human cognitive effort distribution on difficult problems. Through adaptive resource management, DeepSeek delivers proficient results for problems of various complexities.

The observed behaviors show how DeepSeek represents progress in artificial

intelligence since they replicate human mental operations while improving its performance for multiple difficult assignments.

Comparative advantage in reasoning

DeepSeek's reasoning capabilities offer notable advantages over traditional LLMs. The following table outlines key differences between DeepSeek and traditional LLMs:

Aspect	Traditional LLMs	DeepSeek
Problem approach	Often provide direct answers without showing intermediate steps	Naturally breaks down problems into logical steps
Error handling	Limited ability to detect and correct own errors	Actively identifies errors and revises reasoning
Solution verification	Rarely verifies correctness of solutions	Regularly checks intermediate and final results
Adaptability	May struggle when standard approaches fail	Can pivot to alternative solution strategies
Explainability	Black box reasoning that is difficult to follow	Transparent chain-of-thought process
Complex problem-solving	May generate plausible sounding but incorrect solutions	Methodically, it works through problems with higher accuracy

Table 2.1 : Traditional LLMs vs. DeepSeek reasoning comparison

The described distinctions highlight DeepSeek's ability to decompose problems structurally, anticipate errors systematically, verify solutions persistently, operate fluidly and provide open reasoning processes. The performance capabilities of DeepSeek become stronger in complex problem-solving scenarios because of its innovative features which establish it as a major achievement in artificial intelligence reasoning capabilities.

Introduction to reinforcement learning

RL functions as a core component during DeepSeek development where it separates the approach from regular language model training techniques. The strength-enhancing forces function as an alternative to traditional learning

observation techniques to achieve advanced discussion functions. This new approach brings major structural changes to traditional methods while enabling the model to develop layered problem-solving skills from iterative knowledge acquisition and feedback processes. DeepSeek implemented amplified learning strategies to transcend standard training methods thus creating modern speech models that perform better than traditional logical and adaptable systems.

Fundamental concepts of reinforcement learning

The framework engages agents (decision makers or learners) who operate within environments to fulfill their predetermined goals. The agent identifies environmental states continually after which they choose required actions to perform. An environmental change occurs which produces a reward to notify the agent about current action valuations. The actions performed by agents depend on guidelines where specific measurement strategies are provided. RL aims to develop algorithms which enable humans to attain maximum rewards during time spans while selecting optimal choices through uninterrupted feedback cycles.

The following figure highlights the fundamental role of RL in improving a base LLM by using high-quality reasoning data to develop more advanced reasoning capabilities.

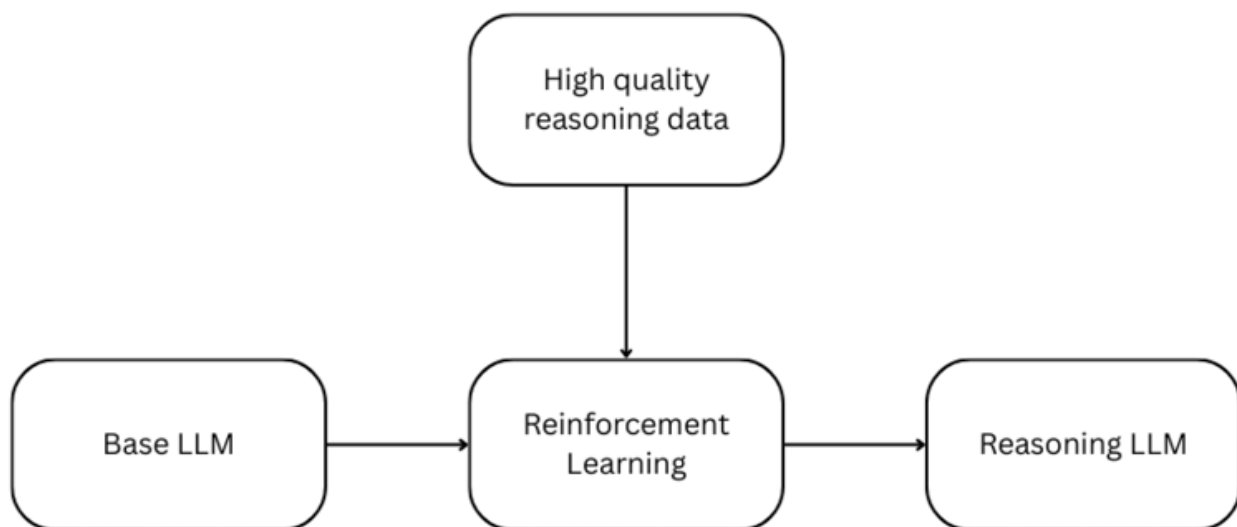


Figure 2.2: RL refines LLMs for better reasoning

The reinforcement learning process

The RL process is an iterative cycle through which an agent completes an iterative process of learning optimal decisions by directly interacting with its environment during the process. This cyclical process involves several key steps:

- **State observation:** The agent obtains information about environmental states for decision support through direct monitoring of its surroundings.
- **Action selection:** The agent selects an action from its current policy according to the observed state to carry out its target objectives.
- **State transition:** The environment transforms its current state because of the selected action which the agent performs. This change in state represents the direct effect of the performed action.
- **Reward reception:** The environment delivers reward signals to the agent that convey both positive and negative values connected to performed actions.
- **Policy update:** New policy improvements stem from received rewards which help the agent develop better decision patterns to achieve higher cumulative rewards across multiple time periods.

The agent repeatedly executes this process which allows it to improve its policy through continuous environmental engagement. The institution develops better choices through time because it learns to forecast resulting effects from their actions and restructures their behaviors to achieve maximum future benefits.

Reinforcement learning vs. traditional training methods

To understand the significance of RL in DeepSeek's development, the following explanation demonstrates how RL enhances the developmental process of DeepSeek relative to standard training approaches used for LLMs.

Pretraining

Pretraining performs as the primary developmental step during LLM creation processes. The learning process at this stage subjects models to

large unlabeled text datasets through which they develop predictive capabilities of sequence completion based on contextual information. This training technique enables the model to identify basic statistical patterns as well as language-building structures present in the text. This prediction method allows the model to find statistical correlations, but it fails to determine contextually valid output. The model generates smooth text, yet this fluency does not guarantee correct or purposeful responses from user viewpoints.

In short,

The system aims to discover statistical relationships in text through next-token prediction from a very large text corpus. The model uses differences between predicted and actual output tokens as its training signal.

Supervised fine-tuning

The development of LLMs requires **supervised fine-tuning (SFT)** as its essential next step after pretraining. The model refinement process operates by transforming pre-trained capabilities to match human-established behavioral and task output requirements. The training process during SFT operates on a specific dataset of prompt-response pairs which contains human-made responses next to input queries or tasks. The main purpose of this training method is to reduce gaps between model responses and human examples which leads to enhanced model capabilities on specific applications.

SFT achieves its performance limits based on the quality and range of data used during training. The model only duplicates behaviors and patterns from the supplied examples so it remains restricted to the data quality and knowledge within this dataset. The performance of the model duplicates the restrictions found in its training data because it cannot extend its capabilities outside its explicit training scope.

In short,

The model seeks to match its outputs to pre-designed human-generated examples of target behaviors. It receives training through comparison of its outputs against human-written responses.

Reinforcement learning

DeepSeek implements RL technology for building its two models namely DeepSeek-R1-Zero and DeepSeek-R1. Through this strategy the models achieve peak performance in specific tasks by processing datasets which contain problems with defined evaluation criteria. The learning signal comes from reward signals that evaluate both the accuracy and the quality of solutions generated by the model. By using this methodology models acquire problem-solving strategies that produce outcomes superior to those found in human-provided examples. DeepSeek-R1-Zero demonstrated excellent reasoning abilities through RL training alone without requiring supervised fine-tuning which enabled it to perform self-verification and reflection.

In short,

The system optimizes itself to achieve particular goals, including the solution of reasoning tasks with accuracy. The model receives rewards that depend on its solution correctness or its quality when evaluated.

Key reinforcement learning concepts applied to DeepSeek

Different essential RL principles help explain the development process of DeepSeek. They are as follows:

Reward functions

A well-designed reward function framework serves as the main guiding mechanism to direct DeepSeek's model development during training sessions. The **accuracy reward** system in mathematical problem-solving requires models to deliver end answers inside specific areas thus enabling administrators to check results by established rule-based methods.

The model receives rewards through testing its outputs against pre-established test cases where the rewards amount depends on the test outcomes. The model receives **format-based rewards** which require it to apply specific tags such as `<think>` and `</think>` during its reasoning process to achieve structured and organized responses. The system provides **process rewards** for proper reasoning steps which encourages the model to solve problems step by step.

Exploration vs. exploitation

A fundamental challenge in RL is balancing **exploration** (trying new approaches that might yield better results) and **exploitation** (utilizing known effective strategies to solve problems).

DeepSeek's training procedure manages the exploration-exploitation trade-off to let the model develop new solutions and improve existing strategies. PPO serves as a technique that preserves this stability because it stops models from focusing too much on established methods while also stopping them from entering untested regions.

Policy optimization

Through RL agents execute their actions within states according to the policy guidelines. DeepSeek applies training procedures to optimize its policy which yields the best possible rewards.

The **initial policy** stems either from base model pretraining or from supervised fine-tuning. The model enhances its decision-making capabilities through **policy adjustments** that stem from RL reward outcomes. The policy evolves to learn better problem-solving approaches during the training process and develops a model that solves complex tasks more adeptly.

The integration of RL principles enables DeepSeek to improve its reasoning capabilities and develop automatic adaptive strategies better than the human-provided examples thus establishing a substantial AI advancement.

DeepSeek's reinforcement learning implementation

DeepSeek's implementation of RL in developing its language models showcases innovative methodologies that enhance reasoning capabilities. Two important models developed by this method are DeepSeek-R1-Zero and DeepSeek-R1.

DeepSeek-R1-Zero trained through reinforcement learning

The DeepSeek-R1-Zero model introduced RL as a direct application method that skipped SFT before its base model. The model starts with pretraining knowledge that it uses to learn through reinforcement until it receives rewards for correct problem solutions. The natural process of this method

enables the model to develop progressive reasoning capabilities that produce CoT reasoning alongside self-verification and reflection. Despite the positive results, the model faced problems with difficulty in understanding the text and mixed language usage which shows the need for additional improvement.

The model's key aspects can be summarized as follows:

- **Starting point:** The model commences its operations with only the knowledge acquired during pretraining
- **Training signal:** The training signals consist of reinforcement mechanisms that provide rewards upon successful problem resolution.
- **Emerged capabilities:** The system naturally develops advanced reasoning skills such as chain-of-thought reasoning together with self-verification and various other complex behaviors.
- **Challenges:** Despite its reasoning capabilities the model experienced problems when users encountered text comprehension issues and language integration problems.

DeepSeek-R1 using a hybrid approach

Building upon insights from DeepSeek-R1-Zero, the development of DeepSeek-R1 adopts a hybrid approach that integrates both supervised learning and RL:

- **Cold start with supervised data:** The model starts by using small quantities of supervised high-quality data for the first training phase which designates an initial learning foundation.
- **Reasoning-oriented RL:** The RL method applies reasoning-centered techniques which help the system create problem-solving methods. On the MATH-500 benchmark, DeepSeek-R1 achieved a high Pass@1 of 97.3 percent, and this improved over OpenAI's o1-1217 by 0.9 percent (96.4 percent). DeepSeek-R1 reached 79.8 % correctness on the 2024 AIME domain, which was orders of magnitude higher than 16.0 % on the same domain with Claude 3.5 Sonnet. Such results demonstrate the resilience of DeepSeek-R1 to solve complex mathematical problems with good reasoning skills.

- **Rejection sampling and supervised fine-tuning:** The model enhances its general capabilities by using sampled high-quality outputs from RL training to produce new supervised data for further model fine-tuning.
- **Comprehensive RL:** A last RL phase utilizes an extensive range of queries within diverse scenarios to create a model that remains adaptable and easy to use by users.

This multi-stage training process combines the strengths of supervised fine-tuning and RL, resulting in a model that not only exhibits strong reasoning abilities but also produces clear and coherent outputs.

Self-learning and emergent behaviors

DeepSeek achieves spontaneous advanced reasoning behaviors from its models by implementing RL.

The aha moment

The training process reveals an important phenomenon known as **aha moments** which enables unprogrammed models to create advanced solving methods by themselves.

During mathematical equation solving the model will pause to inform users of the following message:

Wait, wait. Wait. That's an aha moment I can flag here.

Let us reevaluate this step-by-step to identify if the correct sum can be...

The model displays the capability of reviewing its reasoning while making improvements to its method for better accuracy.

RL shows its ability to help AI systems develop sophisticated cognitive functions which enables them to surpass basic pattern detection by learning genuine problem-solving methods. DeepSeek proves through RL that AI models achieve self-learning skills which leads to improved autonomy and efficiency in task completion.

Thinking time allocation

The model exhibited emergent behavior by devoting additional thinking time tokens to complex problems which paralleled human cognitive patterns of

allocating extended thought periods to challenging tasks. The model developed this behavior automatically without any built-in programming which revealed its capability to learn adaptively.

The training process enabled DeepSeek-R1-Zero to reassess its already tried strategies and build longer reasoning sequences for demanding problems. The self-regulation mechanism operates similarly to human cognition because people dedicate additional effort in handling complex problems.

The self-generated ability demonstrates RL's potential to develop complex problem-solving skills in AI systems which allows them to manage cognitive processes according to task complexity.

Self-verification

The model demonstrated its own self-verification abilities through DeepSeek-R1-Zero without receiving programming instructions. After reviewing its reasoning steps autonomously, it detected mistakes which it corrected independently. This behavior duplicates human methods for checking work to verify accuracy.

During problem-solving activities DeepSeek-R1-Zero analyzed its working stages to detect irregularities which it used to modify its solution approach. The newly emerged capability makes the model more reliable and accurate when producing solutions.

Through self-verification behaviors RL demonstrates its power to develop superior cognitive abilities in AI systems which allows the systems to achieve complex tasks autonomously and with precision.

Challenges and solutions in reinforcement learning training

The implementation of RL for training DeepSeek-R1 models faces multiple obstacles that need specialized remedies. The challenges and the solutions are as follows:

- **Reward design**
 - **Challenge:** Effective rewards need complex design to achieve success but this process remains highly intricate. The use of simple reward structures generates reward hacking situations since models

learn to exploit system weaknesses to get more rewards but fail to solve actual problems.

- **Solution:** DeepSeek-R1 implemented a process-outcome evaluation system based on rules to solve this issue. Through this method the model learns behaviors which comply with desired outcomes and proper solutions.

- **Stability**

- **Challenge:** RL can be unstable, the short-term beneficial policy changes often result in later declines of system performance.
- **Solution:** DeepSeek maintained stable performance by using trained parameters combined with continuous monitoring systems throughout the training period. The team worked on adapting learning parameters as well as reward systems to support steady development of performance.

- **Evaluation**

- **Challenge:** The evaluation process for RL-trained models becomes complicated because standard metrics usually do not fully measure all improvements within the system.
- **Solution:** DeepSeek performed extensive benchmark evaluation testing which examined different aspects of reasoning abilities. The all-inclusive assessment method verifies that the model achieves reliable results and effective generalization to various tasks.

The combination of purposeful reward design with stability-oriented training methods and thorough evaluation techniques allows DeepSeek to maximize RL capabilities in its developed models with enhanced reasoning capacity.

Role of reinforcement learning in DeepSeek's reasoning capabilities

RL functions as the critical component for developing DeepSeek's reasoning competence. The implementation of RL as part of DeepSeek's development has established its superior reasoning features because it provides multiple

essential benefits such as:

- **Beyond human examples:** The process of supervised fine-tuning restricts models to incorporate data provided by humans which restricts their capability to the existing training data quality and quantity. As a result of RL implementation DeepSeek gains the ability to discover new problem-solving approaches beyond human demonstrations which enhances both effectiveness and innovation in its reasoning methods.
- **Optimizing for correctness:** The objective of supervised learning involves matching model responses to human texts yet RL establishes a direct path for optimizing results toward accurate solutions. For reasoning tasks this focus becomes essential because it forces the model to select accurate solutions instead of responding like a human.
- **Learning to learn:** With RL DeepSeek acquires metacognitive abilities through which it learns to enhance its methods for dealing with fresh and unfamiliar problems. Time-based self-improvement mechanisms enable the model to evolve its reasoning approaches which produces more adaptable and versatile solutions.

DeepSeek makes a step forward toward producing a reasoning system through RL integration which enables both reflection and adaptation. The paradigm change demonstrates how RL drives the advancement of LLM capabilities.

The following segment will explore GRPO as DeepSeek's essential RL algorithm which drives its development.

Introduction to Group Relative Policy Optimization

Group Relative Policy Optimization functions as DeepSeek's essential technical innovation for enabling its advanced reasoning functions. During its development DeepSeek used an algorithm which functioned as an advanced version of conventional policy optimization techniques to improve its RL efficiency. We will study Group Relative Policy Optimization as it exists but first, we must examine policy optimization practices in RL.

Policy optimization fundamentals

A RL agent uses **policies** to determine its sequence of actions throughout different states to achieve the highest possible reward accumulation. The process of refining this policy over time is known as **policy optimization**, a fundamental aspect of training intelligent agents.

Traditional policy optimization

Traditional policy optimization methods, such as **proximal policy optimization (PPO)**, operate through a series of structured steps:

- **Experience collection:** The agent conducts exploration within its environment following the current policy while gathering sequences of environmental data which include state information along with action data and reward metrics together with new state information.
- **Advantage estimation:** The algorithm calculates an advantage value for every executed action by comparing its performance against state value expectations to determine benefits relative to other actions.
- **Policy update:** The policy system updates its structure to make positive advantage actions more probable and decrease actions with negative advantages.
- **Critic model utilization:** Algorithms like PPO employ a separate critic model to appraise the value of different states, facilitating the computation of advantages.

The PPO approach introduces a clipping mechanism into its objective function to maintain a stable policy update process through which it promotes training efficiency and prevents major differences between new and old policies. Through its policy change constraints PPO manages to find an optimal balance between exploration and exploitation which produces more dependable learning results.

Challenges in LLM policy optimization

Traditional policy optimization methods including PPO face multiple substantial obstacles when used for LLMs. The main problem arises from high **computational expenses**. The separate critic model training process

stands as a requirement to carry out PPO evaluation of state values. The size of the critic model in LLMs matches that of the policy model which leads to a doubling of required computing resources. The exceptionally high computational requirements present technical obstacles because of hardware constraints together with increased energy requirements.

Another challenge is **sample efficiency**. Language generation tasks demand significant computational power to function which limits the amount of training experience that can be obtained. The data-intensive process of traditional RL leads to inefficient operation because it needs many environment interactions to collect enough data when data collection is time-consuming or expensive.

The rare occurrence of rewards creates **reward sparsity** when using standard policy optimization techniques for LLMs. Long series of tokens need to be completed before receiving any reward during many reasoning tasks. The delayed outcome feedback presents an assignment problem since it becomes hard to identify which sequence steps directly influenced the final result. The rare availability of rewards in these tasks poses difficulties during the training since models must learn from infrequent feedback.

Shaping RL algorithms and training methods which fit the linguistic requirements of LLMs represents an essential requirement for their successful implementation.

A more efficient approach using GRPO

Through GRPO, RL undergoes a revolutionary change that optimizes the training process of LLMs including DeepSeek.

Eliminating the critic model

GRPO breakthrough stems from discarding the conventional critic model that normally provides state value estimates for advantage calculations:

- **Traditional approach:** The critic model functions as part of the traditional system for state value estimation needed to calculate advantages.
- **GRPO approach:** GRPO evaluates advantages through response group relative ranking for shared questions.

The critic model in PPO operates alongside the policy model which leads to a doubling effect on required computing resources. The strategy of GRPO avoids this problem by using group-based evaluation where it produces various responses to a single prompt then compares each response against other responses in its group. The method bases its advantage estimation on response performance comparisons to eliminate the need for a standalone critic model.

GRPO implements group-based assessment which leads to substantial reduction of both computational power requirements and memory storage needs in LLM training processes. RL now becomes more practical when implemented with large-scale language models thanks to this newly gained efficiency. The model learns better learning and enhanced reasoning capabilities through its relative response assessment mechanism within groups which supports the identification of higher quality outputs.

The GRPO algorithm

GRPO introduces a streamlined approach to RL by eliminating the need for a separate critic model and instead utilizing group-based evaluations to optimize policy updates. The algorithm operates through the following steps:

1. **Group sampling:** For each input query q , the current policy $\pi_{\theta_{ch}}$ generates a set of G responses $\{O_1, O_2, \dots, O_G\}$.
2. **Reward calculation:** Each response O_i receives an associated reward r_i based on a predefined evaluation metric.
3. **Advantage calculation:** Instead of employing a critic model to estimate state values, GRPO calculates the advantage A_i for each response by assessing its reward relative to the group's mean and standard deviation:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

This method allows the model to understand the relative quality of each response within the group, facilitating more nuanced policy adjustments.

4. **Policy update:** The policy parameters θ are updated by maximizing the

objective function:

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min \left(\frac{\pi_{\theta}(a_i | q)}{\pi_{\theta_{\text{old}}}(\sigma_i | q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(a_i | q)}{\pi_{\theta_{\text{old}}}(\sigma_i | q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) \right] - \beta D_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}})$$

In this formulation:

- π_{θ} represents the updated policy.
- $\pi_{\theta_{\text{old}}}$ denotes the previous policy.
- A_i is the computed advantage for response i .
- The clip() function constrains the policy ratio within the range $[1 - \epsilon, 1 + \epsilon]$ to prevent overly large updates.
- β is a hyperparameter that balances the regularization term.
- D_{KL} represents the Kullback-Leibler divergence, ensuring that the updated policy does not deviate excessively from a reference policy π_{ref} .

By integrating these steps, GRPO offers a more computationally efficient and stable method for policy optimization, particularly suited for applications involving LLMs.

Implementation in DeepSeek

GRPO was crucial in making the models of DeepSeek. It greatly enhanced the thinking capabilities of DeepSeek-R1-Zero and DeepSeek-R1. This was done through the use of new training methods that enhanced model performance.

DeepSeek-R1-Zero training

DeepSeek-R1-Zero was developed from the base model using GRPO. The training involved several important strategies:

- **Group size:** Each input question generated multiple answers. These were grouped together to allow comparison and measure their performance against each other. This grouping helped in identifying which answers were better.

- **Reward system:** A basic, rule-based reward system was put in place. It focused on two main factors:
 - **Accuracy:** The model was rewarded for giving correct answers, this way the model learned to always give reliable and accurate information.
 - **Format:** There were also rewards for using <think> tags when writing the thought process, thus encouraging clear and well-organized answers.
- **Training template:** A simple template was used to direct the model's operations. This template assisted in making sure that the model kept its reasoning process within <think> tags. It then gave short and clear answers, and all responses were consistent.

The following figure shows the training system of DeepSeek-R1-Zero. In this setup, a policy model generates responses. These responses are evaluated using a reward model, which incorporates KL divergence. This is based on a reference model to compute advantages, which helps in optimizing and improving the model's output.

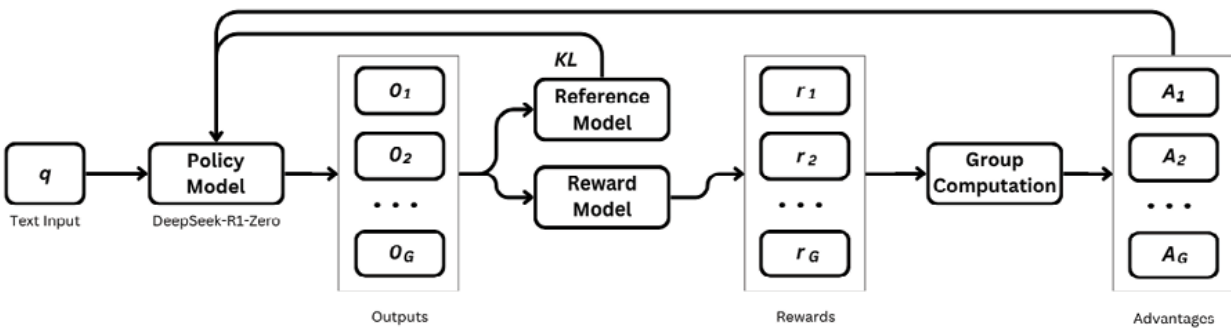


Figure 2.3: DeepSeek-R1-Zero training framework

The effectiveness of this approach can be seen in the rapid improvement of DeepSeek-R1-Zero's performance on reasoning benchmarks:

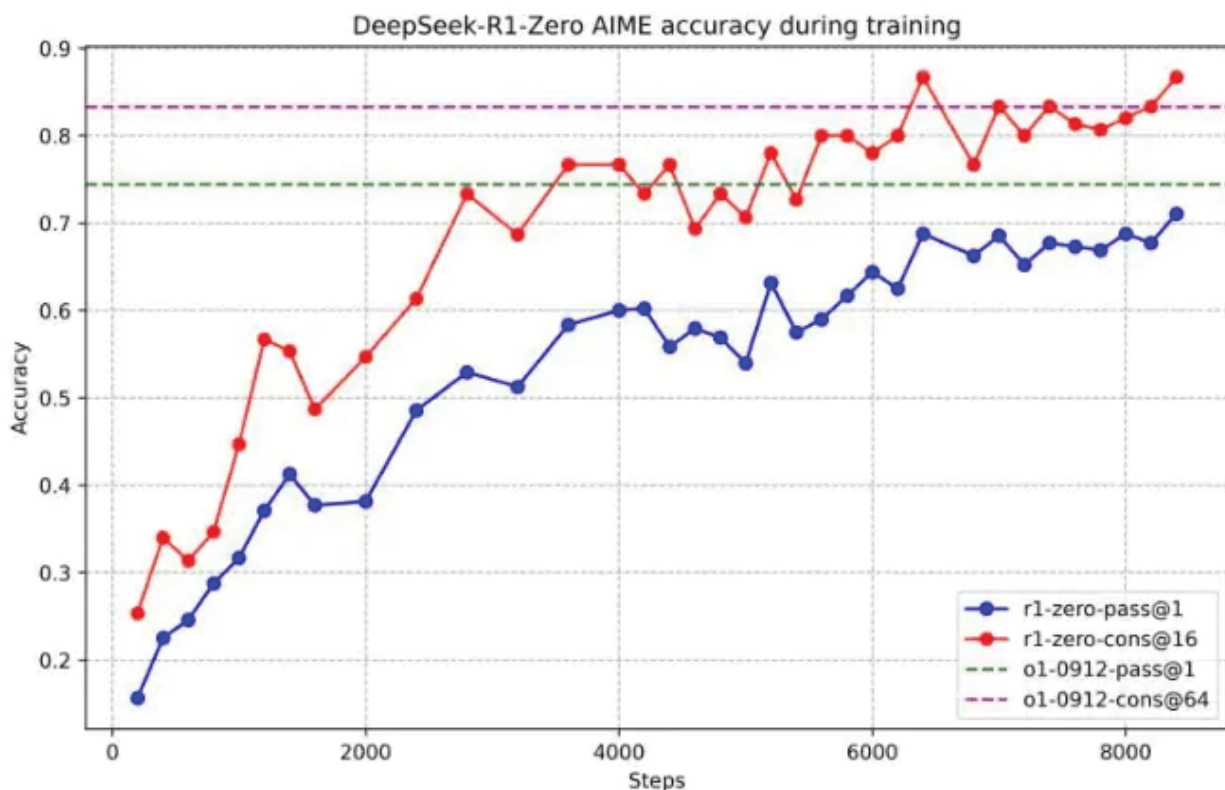


Figure 2.4: DeepSeek-R1-Zero AIME accuracy during training.

(Source: <https://www.diskmfr.com/the-secrets-behind-deepseek-r1-training-revealed/>)

The figure illustrates DeepSeek-R1-Zero's significant performance improvement on the AIME during GRPO training. This highlights the effectiveness of the training method. To ensure a stable evaluation, 16 responses are sampled for each question, and the overall average accuracy is calculated throughout the training process.

DeepSeek-R1 implementation

In the development of DeepSeek-R1, GRPO was integrated into multiple stages of the training process:

- **Reasoning-oriented RL:** Following initial supervised fine-tuning, GRPO was applied to cultivate advanced reasoning capabilities within the model.
- **Comprehensive RL:** In the final training phase, GRPO was utilized again to fine-tune the model across a diverse array of queries, ensuring robust performance across various tasks.

This multi-stage training strategy, which combined supervised fine-tuning with GRPO-based RL, culminated in DeepSeek-R1's remarkable performance across a spectrum of benchmarks, highlighting the effectiveness of GRPO in enhancing LLMs.

Advantages of GRPO

The benefits of GRPO offer several important advantages of improving RL models, particularly in large-scale speech models such as DeepSeek. They are as follows:

- **Computational efficiency:** GRPO needs to save computer resources by eliminating the need for another critical model. Traditional amplification learning uses critic models to estimate value functions that can be extremely strict with regard to computing power. By eliminating these requirements, GRPO simplifies the process and allows it to be applied to large-scale models.
- **Sample efficiency:** Sample efficiency GRPO improves sample efficiency by simultaneously evaluating several outputs of a single input query. By co-assesing costs, the model can collect more wise learning signals from each sentence of interaction. This approach accelerates learning and reduces the number of samples needed to achieve optimal performance. This is especially useful in situations where data collection is expensive or limited data sources are available.
- **Relative evaluation:** Instead of relying on fixed reward values, GRPO focuses on comparing performance of editions within the group. This is an advantage when it is difficult to clearly define the best possible reward. By using comparisons, the model can choose a higher quality answer. This method is particularly useful for complex tasks where it is difficult to provide a clear reward signal.
- **Stability:** The GRPOS group-based approach leads to more stable training dynamics. Minimizes the effect of remote rewards and integrates elements such as cutoff updates and **Kullback-Lebler (KL)** divergent penalties to ensure careful and control over guidelines updates. As a result, it steadily improves over time without taking the risk of severe fluctuations during training.

In summary, GRPO is a powerful framework that offers several benefits. It offers intelligent ways to save arithmetic resources, improve sample efficiency, evaluate results, and ensure stable training. These benefits work together to improve the performance of models such as DeepSeek for various applications of reinforcement.

Limitations and considerations

GRPO presents significant advancements in RL, yet it is important to acknowledge its constraints:

- **Group composition:** The success of GRPO depends on having a variety of outputs within a group. When outputs are too similar, the signals needed for learning become weak, making it difficult for the model to learn effectively. This may cause training issues with consistency and stability.
- **Reward design:** Crafting the right reward functions is a challenging task. Even with the benefits of GRPO, poorly designed rewards can lead to the model behaving in unexpected ways. The model might try to game the reward system, focusing on maximizing rewards through shortcuts rather than achieving the actual goals.
- **Hyperparameter sensitivity:** The performance of GRPO, like many RL algorithms, relies heavily on setting the right hyperparameters. These include factors such as the learning rate, clip range (ϵ), and KL divergence coefficient (β). If these parameters are not tuned correctly, it can result in unstable and less effective performance.

In conclusion, while GRPO offers many benefits, it is crucial to carefully consider the diversity of group outputs, thoughtfully design reward functions, and meticulously adjust hyperparameters to fully utilize its potential.

Conclusion

In this chapter, we discussed what makes DeepSeek a leader in AI. We discussed how it has advanced thinking skills, learns from experiences, and uses special training methods to improve. DeepSeek can take complicated

problems and break them down into smaller, more manageable parts. It can also check its solutions and change its methods as needed, which sets it apart from normal language models. These abilities come from RL, a method that allows DeepSeek to develop complex behaviors without detailed instructions. A key component of this is the GRPO, a technical innovation that improves efficiency by not requiring another model to verify results. Instead, it uses group performance to calculate advantages. These features give DeepSeek powerful problem-solving skills in fields like mathematics, coding, science, and logical reasoning. This shows how RL can push AI beyond just following preset instructions.

In the next chapter, titled *Overview of DeepSeek Models and Types*, we will look at the different models in the DeepSeek family. We will examine their unique abilities, structures, and compare them. This will include language models, vision models, and distilled models. Our goal is to help you understand each model's strengths and weaknesses, so you can choose the best DeepSeek model for your specific needs.

Points to remember

Refer to the following list to remember the key points:

- DeepSeek learned to become a better problem solver through RL. This method helped it develop skills like breaking down problems into steps, checking its work, and thinking about what it learned.
- With CoT processing, DeepSeek tackles big problems by dividing them into smaller, more manageable parts. This makes it easier and more organized for DeepSeek to find solutions.
- RL is not like older training methods. Instead of just copying humans, RL helps DeepSeek focus on finding the most effective and correct solutions to problems. This can make DeepSeek's reasoning even better than human examples sometimes.
- During training, DeepSeek often had aha moments where it suddenly improved its problem-solving skills. These moments showed that DeepSeek was getting better at understanding how it thinks, almost like developing a sense of awareness.

- DeepSeek also figured out that it should spend more time on difficult problems, similar to how people take longer with hard tasks.
- GRPO helps speed up DeepSeek's learning process. It does this by evaluating answers within groups, which means it does not need a separate model to critique its performance. This makes learning faster and less costly for big language models.
- The benefits of GRPO include saving on computing resources and training data, being stable during the training process, and allowing fair assessment among different responses.
- DeepSeek's training blends two methods: initial learning with guidance and then improving with GRPO. This combination allows DeepSeek to achieve strong results on various tests.

Key terms

The key terms are as follows:

- **RL:** RL is a type of learning used in machines where an agent learns how to make decisions by trying different actions. The agent receives rewards for good actions and penalties for bad ones. There is no teacher to guide it step by step.
- **CoT:** CoT is a way to figure things out by tackling complex problems in smaller, easier steps. Each little step builds on the knowledge from the previous step.
- **MoE:** MoE is a unique kind of neural network design. It divides work among smaller networks called experts, and only some of these experts work on each task, which makes it more efficient.
- **GRPO:** GRPO is a special method in RL used in DeepSeek. It does not use the usual critic model. Instead, it uses scores from groups to make decisions.
- **Distillation:** Distillation is a process where a larger model, the *teacher*, shares its knowledge with a smaller model, the *student*, to help improve the student's performance.
- **Self-verification:** Self-verification is when a model can evaluate and

confirm its own way of thinking to ensure it is getting the results right.

- **SFT:** SFT is a training method where a model is further trained with labeled examples, improving its ability to perform certain tasks better.
- **Emergent behavior:** Emergent behavior refers to new skills or patterns that show up during the training of a model. These abilities develop naturally from simpler training goals without being specifically programmed.
- **Reward function:** Reward Function is a way to provide feedback, signaling how desirable certain actions or outcomes are, to guide learning in RL.
- **Metacognition:** Metacognition is about being aware of and understanding one's own thought processes. In DeepSeek, this means reflecting on and correcting its own actions and decisions.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 3

Overview of DeepSeek Models and Types

Introduction

The previous chapter gave an extensive analysis of DeepSeek's fundamental aspects, together with its reasoning system and its progressive reinforcement learning approaches supporting its progress. It explains how DeepSeek diverges from standard large language models because of its sophisticated reasoning system and unique training processes. We will now explore the multiple different models within the DeepSeek ecosystem, which were crafted to achieve excellence in particular domains and applications.

We will review detailed descriptions of each model type that focus on structural designs and operational features, as well as practical implementation examples. The assessment includes model-to-model comparisons within each category that evaluate their advantages, together with limitations and compromises. This chapter provides all the necessary understanding for users to comfortably work with DeepSeek's systems across different AI disciplines and hardware constraints.

Structure

We will cover the following topics in this chapter:

- Language models
- Vision models
- Distilled models

Objectives

This segment will provide you with complete knowledge about the different models inside DeepSeek's ecosystem and their individual functions by its end. With your new understanding, you will be able to differentiate between language models, vision models and distilled models along with their information processing methods. Your understanding will enable you to make effective model choices because you will know which model best fits your existing NLP or computer vision projects or applications that function in resource-constrained environments.

You will investigate how architectural innovations support each model type including design trade-off assessments regarding field alternatives for these models. The discussion will demonstrate practical use cases from different industrial domains which enable real-life examples of technology deployment. Distilled models developed by DeepSeek demonstrate the possible transfer of information between large AI systems to produce smaller but efficient computing models which bring AI capabilities to personal computers thus making AI available for wide-ranging daily applications.

The information you have obtained enables you to move through the DeepSeek platform competently while making optimal use of sophisticated AI tools in your development tasks.

Language models

DeepSeek relies on language models as its core artificial intelligence solution to power various advanced features of its product suite. These models' ability to process human language through engineering allows them to be used for a variety of tasks, including complex problem solving, content creation, and conversational AI.

The DeepSeek ecosystem contains three fundamental model categories that make up its diverse line-up. The specialized feature of language models enables

them to process text documents and create new written content for advanced **natural language processing (NLP)** tasks. The purpose of vision models is to handle visual data so they can perform image recognition alongside visual analysis operations. Last among the DeepSeek models are distilled models, which offer high-performing compact alternatives to larger models, ideally suited for environments with resource restrictions. The following figure shows the relationships between different DeepSeek model types:

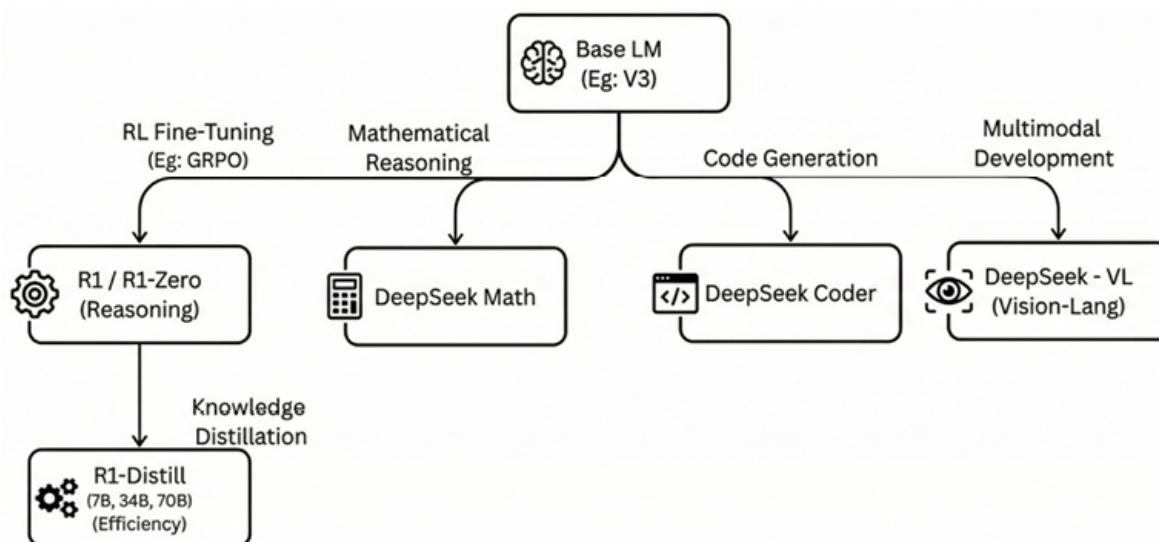


Figure 3.1: DeepSeek model ecosystem

A clear comprehension of different model formats proves vital to finding the suitable system for any operation. AI models exist in separate categories because they differ in their computational requirements and their best application areas as well as their functional capabilities. You will acquire useful knowledge about DeepSeek's fieldwide applications through learning about these models to make your projects more effective through these technologies.

Evolution of DeepSeek language models

DeepSeek continues its dedication to advance artificial intelligence through new methods and architectural frameworks in their language model development. The initial two versions of DeepSeek-V1 and V2 supplied basic text comprehension functions which built an essential foundation before more complex models were introduced.

The collaboration released DeepSeek-V3 in December 2024 as a model that held 671 billion parameters. The development spanned 55 days with an investment of

\$5.58 million while requiring less resources than other models of its time. Tests showed DeepSeek-V3 demonstrated better performance than Llama 3.1 and Qwen 2.5 but displayed equivalent abilities to both GPT-4o and Claude 3.5 Sonnet. DeepSeek-V3 implements a **Mixture of Experts (MoE)** architecture with multi-head latent attention transformer and contains 256 routed experts and one shared expert that activates 37 billion parameters per token.

DeepSeek-R1-Zero entered the market in January 2025 as an advancement of the initial programming framework. The model resulted from an exclusive reinforcement learning process which eliminated supervised learning as a training method. GRPO enabled the system to learn complex reasoning abilities even though it only received rules-based rewards regarding accuracy and formatting evidence of how reinforcement learning can generate advanced capabilities on its own.

DeepSeek-R1 entered the market after developers integrated insights derived from R1-Zero. The model utilized a training process which integrated supervised learning and reinforcement learning for readability improvement alongside language mixing remedies while sustaining good reasoning performance. The model training operation started with supervised high-quality data, then continued with reinforcement learning that focused on reasoning, before employing rejection sampling together with supervised fine-tuning and finished with extensive reinforcement learning to create an adaptable system that was easy for users to utilize.

The development of **DeepSeek-Coder** occurred simultaneously with its functionality to produce code while understanding various programming languages and identifying and fixing code problems. A specialized training model processed code that made up 87% of the data while natural language comprised 13% of the input. This model worked with English and Chinese languages across diverse parameters ranging from 1 billion to 33 billion.

DeepSeek-Math was trained on top of DeepSeek-Coder-Base v1.5 (7 B parameters) and an extended pre-training regime of 500 billion tokens. Over this long pre-training period, a DeepSeekMath Corpus was collected, comprising 120 billion tokens math-specific text from Common Crawl, among other sources. Afterwards, three variants, Base, Instruct, and RL, were spread. The Base model has strong mathematical reasoning with a highest score of 51.7 % on the MATH benchmark, with no need for external toolkits or voting. The Instruct variant was also improved by having it go through supervised learning

of about 776 K of math problems, hence ensuring that it has increased ability in providing step-by-step explanations. The RL version uses the Math-Shepherd framework, training **Group Relative Policy Optimization (GRPO)** on approximately 144 K math questions, resulting in specialization of its cognitive ability.

DeepSeek has evolved its language AI technology through this evolutionary path to deliver advanced capabilities while making its models more broadly usable to users.

Architecture and technical specifications

DeepSeek constructs its language models through innovative architectural designs which enable superior performance when performing various operations. The transformer architecture functions as the backbone for modern language processing systems in all state-of-the-art models. The enhanced architecture of DeepSeek includes various innovations which strengthen its operational capabilities.

DeepSeek-V3 utilizes MoE architecture where it divides the model into *expert* networks which perform separate operations for different tasks. The improved model capacity from this design does not require additional computational power. DeepSeek-V3 operates through 671 billion total parameters that activate 37 billion for each token and demonstrates superior efficiency than dense models with equivalent abilities.

DeepSeek-V3 along with other models in the family enables a **broad context window** capability by working with up to 128000 token spans. The extended capacity of these models allows them to analyze long documents or extended discourse because of their suitability for analysis-intensive or extended reasoning tasks.

The DeepSeek family includes models with different parameter amounts starting from several billion parameters to hundreds of billions across its range. User selection between performance and resource requirements becomes possible through the scalable model options.

DeepSeek-R1 employs GRPO which eliminates the need for independent critics by calculating baseline estimates from group score evaluations. The system operates efficiently because it eliminates the need for a critical model which would match the policy model size. The policy model optimization through GRPO involves selecting multiple outputs from current policy samples and

optimizing the model by maximizing an evaluation process which assesses group member performance relative to others.

The architectural innovations deliver a robust performance along with application versatility to DeepSeek's language models which enables them to optimize different uses.

DeepSeek designed several language models with specialized characteristics for different implementation needs and performance specifications. The following table summarizes key specifications of these models:

Model	Parameters	Context window	Training approach	Key strengths
DeepSeek-V3	671B (37B active)	32K	Pretraining + SFT	General-purpose, efficient MoE architecture
DeepSeek-R1-Zero	175B	128K	Pretraining + RL	Pure RL-based reasoning capabilities
DeepSeek-R1	175B	128K	Pretraining + SFT + RL	Advanced reasoning with improved readability
DeepSeek-Coder	33B	16K	Specialized code training	Programming and software development
DeepSeek-Math	7B	7K	Pretraining + Instruction Tuning + RL (GRPO)	Mathematical Reasoning

Table 3.1 : DeepSeek model specifications and strengths

These models, which serve a wide range of applications from general-purpose language understanding to specialized coding tasks, are prime examples of DeepSeek's dedication to advancing AI through creative architectures and training methodologies.

Capabilities and performance

DeepSeek's language models have demonstrated remarkable capabilities across various domains, particularly excelling in reasoning-intensive tasks.

Mathematical and logical reasoning

The DeepSeek-R1 computer system has proven its exceptional ability to solve mathematical problems. The 2024 **American Invitational Mathematics Examination (AIME)** result showed DeepSeek reaching a 79.8% accuracy level (Pass@1) above both GPT-4o (9.3%) and Claude-3.5-Sonnet (16.0%). DeepSeek-R1 achieved a MATH-500 benchmark pass rate of 97.3% which

matched the performance of OpenAI's o1-1217 model (96.4%).

Scientific reasoning

The scientific reasoning capabilities of DeepSeek models remain very strong. DeepSeek-R1 showed scientific expertise by attaining a 71.5% pass grade on the GPQA Diamond benchmark.

Programming and code generation

The specialized code generation capabilities of DeepSeek-Coder enable it to develop and analyze programs along with debugging functions within various programming languages. The coding abilities of the system reached expert standards according to CodeForces rating standards as it scored in the 96.3 percentile category. The CodeForces rating of 2,029 obtained by DeepSeek-R1 beat 96.3% of human code participants during evaluation.

Natural language understanding and generation

The DeepSeek models exhibit strong general language functionality by providing text summarization services and content creation features as well as translation functions alongside domain-independent question-answering.

DeepSeek dedicates itself to AI frontier progress by developing powerful versatile models which apply to various application domains.

Applications of DeepSeek language models

Language models from DeepSeek operate throughout different sectors because they use their strong reasoning ability and understanding of languages to improve operations and results.

Research and academia

Educational institutions use DeepSeek models to help their researchers extract patterns from large scientific documents and build hypotheses based on their findings before designing experiments and interpreting complex data to determine future research directions. The strong ability of DeepSeek models to process textual data and create new content fuels advanced problem investigation for academic research development.

Education

Educational institutions utilize DeepSeek models as tutoring assistants that create step-by-step explanations adjusted to students' specific learning requirements. These platforms assist educational material development, evaluate student work and provide information to research projects to boost student learning outcomes.

Software development

The DeepSeek-Coder platform from the DeepSeek line of models serves software developers through its capability to create code from specifications and fix code errors as well as enhance code quality and describe complex programming structures and generate architectural designs with accompanying documentation. The system undergoes training using extensive programming language datasets for developing its capabilities to process and generate code across multiple programming languages.

Business intelligence

Companies utilize the DeepSeek models to study market patterns and handle large business documentation for both report generation and presentation production as well as decision support through scenario evaluation. Large data processing abilities together with data interpretation functions help organizations make strategic decisions with accuracy.

Content creation

Content creators profit from DeepSeek because it enables the creation of articles as well as blog posts and marketing content. DeepSeek models produce original content such as stories and scripts as well as modify existing content while adapting material for diverse audiences and platforms to simplify content development.

As DeepSeek's language models function across these different domains they showcase their status as robust instruments which improve operational efficiency and application effectiveness.

Vision models

DeepSeek is primarily known for its advanced language models and has

developed its portfolio with the addition of strong vision models that include DeepSeek-VL and its updated version DeepSeek-VL2. DeepSeek proves its dedication to visual and language understanding integration through these platforms which establish fundamental AI frameworks that handle multiple information modalities.

Bridging vision and language using DeepSeek-VL

DeepSeek-VL stands as the initial Vision-Language model of DeepSeek which processes visual and textual data simultaneously. The system features the capability to analyze logical diagrams alongside web pages and formulas and scientific literature together with natural images while adding complex situations that necessitate embodied intelligence. DeepSeek-VL represents a major step forward in AI development because it unites vision and language understanding capabilities.

Architecture and design

The DeepSeek-VL model applies two separate encoding networks which handle information from both visual and textual inputs. DeepSeek-VL represents an open-source VL system specifically developed to unite visual and textual processing for practical use cases. The system utilizes two processing units as fundamental design elements that handle visual content along with textual information through distinct yet supporting channels.

The Vision Encoder uses a modified version of **Vision Transformer (ViT)** architecture. The component breaks down images into smaller sections and uses self-attention mechanisms to detect delicate spatial patterns while obtaining advanced visual elements. The method provides detailed understanding of visual information which is vital for performing advanced image analytics.

Concurrently, the Language Encoder leverages the same transformer-based architecture as DeepSeek's language models. The text processing system handles textual data to create complex contextual models which drive advanced language processing and generation functions. The model executes effective multichannel content processing by merging its abilities to interpret visuals with the ability to understand texts.

The cross-modal layer helps implement a specialized functionality which establishes correspondence between the different encoding methods. DeepSeek-VL becomes more effective at multimodal reasoning because its alignment

mechanism helps it understand the relationships between images and text.

DeepSeek-VL needs to learn from different types of datasets which consist of both images and text elements. The datasets used for training DeepSeek-VL consist of web-based captioned images as well as instructional content and Visual Question Answering datasets and document images containing embedded text. The model receives thorough training that enables it to conduct superior reasoning across visual and textual content which enhances its performance in integrated applications.

The refined design approach and training process established DeepSeek-VL as an advanced tool to process Vision-Language understanding tasks which demand coordinated visual and linguistic capabilities.

Capabilities and performance

DeepSeek-VL successfully executes Vision-Language tasks at high levels by blending visual intelligence with language expertise for superior performance. The model handles **Visual Question Answering (VQA)** tasks with precision by understanding image contents in relation to written questions. DeepSeek-VL creates exact image captions which successfully identify and explain visible and subtle visual features.

The visual reasoning expertise of DeepSeek receives further development in DeepSeek-VL to deliver improved results in intricate visual processing assignments. DeepSeek-VL successfully recognizes object relationships and determines causal relationships in visual environments. The model handles document images with speed by extracting text content from documents as it detects original text while keeping track of visual organization. The model provides precise understanding of text elements that occur within their image environment.

The capability of DeepSeek-VL includes following complex commands that use both visual and textual elements to interpret instructions. The system proves useful in software where users need to combine visual elements with written content. DeepSeek-VL demonstrates strong performance through benchmark results on COCO Caption and Visual Genome QA and DocVQA tests where it shows competitive outcomes against other dominant multimodal models. The assessment benchmarks evaluate how well the model performs at caption generation and VQA and document layout understanding. The multimodal features of DeepSeek-VL preserve all reasoning capabilities that typify the

DeepSeek series which guarantees strong logical connections within its comprehensive understanding.

Specialized vision processing using DeepSeek-VL

DeepSeek-V represents a specialized vision model that performs exclusively visual processing requirements without requiring language input or output. Its design bases the architecture on a pure ViT which optimizes high-resolution image processing for applications that do not require language processing.

This model demonstrates its excellence through multiple essential functions:

- The object detection and recognition capability of DeepSeek-V allows it to identify and categorize imagery content which enables inventory management as well as security surveillance and automated quality evaluation.
- The model demonstrates high capability in dividing images into distinct sections which enables the separation of objects from backgrounds. This functionality serves many crucial applications including medical imaging and autonomous vehicle navigation and augmented reality applications.
- DeepSeek-V enables the extraction of advanced image features which facilitates operations including image retrieval and similarity matching and clustering tasks.
- The model detects abnormal patterns together with unusual objects in images which advances quality control and security monitoring capabilities. The application of Vision Transformers has proven effective for industrial anomaly detection and localization purposes in different environments.
- DeepSeek-V delivers productive and efficient answers for visual solutions beyond language-based applications which enable strong performance across different visual processing operations.

Applications of DeepSeek vision models

The vision models of DeepSeek bring advanced visual processing capabilities specifically designed for industry applications which generate substantial impacts in various industries.

Healthcare and medical imaging

Through medical image analysis, DeepSeek models assist healthcare

professionals to detect both diseases and potential abnormalities while improving diagnostic results. Monitoring systems and visual medical records become more efficient through their ability to analyze visual data which detects patient condition changes and extracts medical information needed for documentation processes.

Retail and e-commerce

The vision models from DeepSeek allow retail and e-commerce customers to use images for product searches which enhances their shopping efficiency. Visual product recognition allows the system to track and sort items for optimized inventory control. Visual interactive assistants receive a boost from these models as they answer product-related questions to improve customer engagement.

Manufacturing and quality control

The models from DeepSeek perform visual checks on production items to detect manufacturing defects thus upholding product quality. The system implements visual analysis to observe production processes which reveal information about operational efficiency. Visual monitoring enables safety protocol adherence that adds to the development of secure work areas.

Document processing

Especially when handling visual documents DeepSeek's vision models perform efficiently as they detect various information elements while keeping track of document layout structure for better document processing operations. These systems can handle forms which integrate text along with visuals in order to execute data entry processes more efficiently. The document categorization process through these models functions by analyzing visual elements together with text signatures to achieve better document management efficiency.

Autonomous systems

With their autonomous systems models DeepSeek provides vehicles and robots self-sufficiency that enables them to detect surrounding environment components and perform effective navigation. The system allows users to interact with physical materials which plays an essential role in tasks that need manual handling. The systems leverage visual scene understanding to help

navigational tasks which results in safe and efficient movement through different environments.

Integrated vision abilities from DeepSeek enhance powerful solutions that require visual understanding alongside complex decision making for various operational sectors which drive innovative progress.

Distilled models

The delivery of efficient AI solutions through DeepSeek heavily depends on distilled models that maintain high performance quality. DeepSeek uses knowledge distillation to migrate the skills of extensive complex models to smaller versions thus enabling powerful AI capabilities to function in systems with limited resources.

The distillation process

The training process of knowledge distillation teaches a smaller student model to replicate the outputs of its larger teacher model. The distilled model obtains most of its capabilities from the teacher model and operates with lower computational requirements. Through this process the teacher model generates outputs used to train the student model while transferring knowledge to it without using extensive computational power.

Through its implementation DeepSeek developed models which achieved equivalent results to their original and more resource-intensive versions. DeepSeek-R1 started with 671 billion parameters but the company distilled it to create a version which operates with 37 billion active parameters. The performance stays high as the computational requirement decreases substantially.

The practice of model distillation leads to ethical and legal questions especially about intellectual property rights. The practice of distributing models has raised privacy concerns since some companies may build proprietary models from unauthorized proprietary data which threatens both security and privacy. The issues underline why guidelines must be established and agreements need to be made when developing AI systems to protect innovation alongside intellectual property rights. The following figure showcases the basic working of Teacher Model Training:

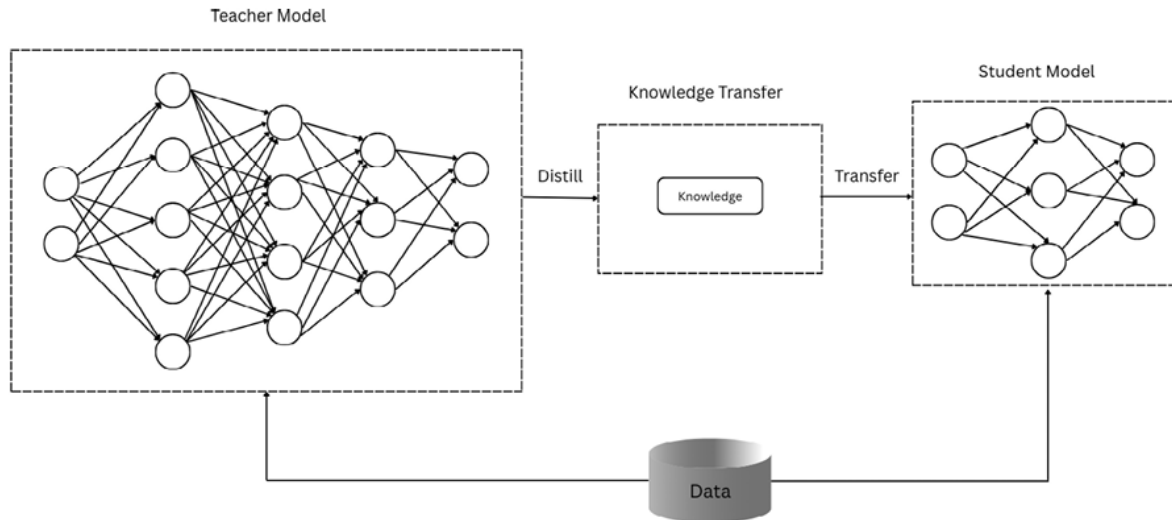


Figure 3.2: Teacher model training

The process of distillation

Knowledge distillation functions as a machine learning method which lets a complex model (teacher) transfer its knowledge to a smaller efficient model (student). The student model receives capabilities from its teacher through this process thus becoming computationally efficient enough for deployment in environments with limited resources.

The distillation operation requires multiple essential steps to function such as:

- **Teacher model training:** The system trains a big teacher model through pretraining and supervised fine-tuning before conducting reinforcement learning. The developed model achieves advanced functionality through its processing but requires powerful computational systems.
- **Generation of training data:** The teacher model uses various prompts to produce responses which form a training data collection that represents its knowledge along with its reasoning styles.
- **Student model training:** The smaller model learns to produce teacher-like outputs by processing this dataset during training even though it contains fewer parameters.
- **Fine-tuning and optimization:** Additional fine-tuning processes help optimize the student model so it maintains the essential capabilities from the teacher model.

The method enables minimal models to leverage trained expertise from larger models even though they skip the time-consuming extensive training stage. The

student model utilizes teacher-model knowledge to balance its operational efficiency with performance outcomes thus serving as a strong solution for constrained computational applications.

Innovations in DeepSeek's distillation approach

The deep learning company DeepSeek implemented various substantial improvements to standard model distillation approaches to deliver enhanced capabilities of distilled AI models.

The main breakthrough in DeepSeek's Distillation process is **Reasoning-Focused Distillation**. Traditional distillation procedures mostly focus on output duplication for students to duplicate teacher model responses. DeepSeek focuses on transferring reasoning abilities by maintaining the complete problem-solving sequence and self-verification processes which define large complex model functioning. The cognitive processes at higher levels remain present in distilled models which enables them to solve real-world problems effectively.

A significant breakthrough in knowledge transfer focuses on **Selective Knowledge Transfer**. DeepSeek distinguishes itself from traditional approaches by applying a system which picks and transfers meaningful reasoning patterns rather than transmitting entire acquired skills without discrimination. The targeted methodology both simplifies the distillation process and secures excellent performance from distilled models specialized in vital reasoning skills needed for practical applications.

Within its implementation DeepSeek uses a **Multi-Stage Distillation** process that shows success. Its progressive distillation method conducts gradual information transfer from big models to smaller ones through multiple stages instead of performing one-step model downsizing from large to compact models. The incremental method enables the distilled models to keep more functional capabilities thus outperforming the models distilled through traditional single-step approaches. The multi-stage framework produces more efficient models which maintain their operational performance capabilities.

The innovative approaches used in DeepSeek have resulted in significant enhancements for both the performance quality and operational speed of their distilled models. Developed AI tools through this method now offer economic viability and broad accessibility as major sectors adopt them across their operations. DeepSeek achieves democratic access for cutting-edge AI capabilities through its ability to include advanced reasoning skills within

computationally manageable small models.

The DeepSeek-R1-Distill series

DeepSeek demonstrates its dedication to model-accessible reasoning capabilities by offering DeepSeek-R1-Distill series models which strike a balance between performance and efficiency requirements. DeepSeek used innovative distillation methods to build different model sizes which adapt to different application needs and processing requirements.

Available models and specifications

The DeepSeek-R1-Distill series consists of models that differ in parameter numbers to accommodate users with diverse computing capabilities. The following table showcases different DeepSeek models:

Model	Parameters	Context window	Base architecture
DeepSeek-R1-Distill-70B	70 billion	32K	DeepSeek-MoE
DeepSeek-R1-Distill-Qwen-14B	14 billion	32K	Qwen
DeepSeek-R1-Distill-Qwen-7B	7 billion	32K	Qwen
DeepSeek-R1-Distill-Llama-13B	13 billion	16K	Llama 2
DeepSeek-R1-Distill-Llama-7B	7 billion	16K	Llama 2
DeepSeek-R1-Distill-1.5B	1.5 billion	8K	Custom architecture

Table 3.2: DeepSeek distilled model configurations

The designed models present performance options which match resource capacity levels available to users. Users can select between DeepSeek-R1-Distill-Qwen-1.5B for limited resource settings and DeepSeek-R1-Distill-70B for demanding applications with expanded capabilities. DeepSeek provides full technical details and download connections through its official GitHub repository.

Here is a refined infrastructure table for deploying DeepSeek models:

Deployment option	Hardware/Infra requirements
Full DeepSeek-R1 (671B)	Needs a high-powered GPU cluster, about 16× NVIDIA A100 (80 GB each) with ~1.4 TB VRAM, 512 GB RAM, fast SSDs, and high-bandwidth networking. Only realistic for enterprise or research facilities.

Distilled Variants (1.5B–70B)	These still demand GPUs, but far less. For example, 70B runs on ~32–64 GB VRAM (like 2× RTX 4090), while 32B needs ~16 GB VRAM (one high-end GPU). Mid-tier servers with 128–256 GB RAM suffice.
Apple Mac Studio (M3 Ultra)	~448 GB unified memory; power draw <200 W; 4-bit quantized R1 model runs entirely in memory.
AWS (Bedrock / SageMaker / EC2)	Bedrock/SageMaker: fully managed—no infra to manage. Custom import: store model (1.5–70B) on S3 and import.
Azure AI Foundry	Azure subscription; model deployable in under a minute via UI/API.
Self-host via Northflank (AWS/GCP/Azure)	Bring-your-own-cloud with A100/H100 GPUs (spot instances recommended). Deployment under 1 hour.
Local (via Ollama)	~8–16 GB RAM; no GPU needed for small distilled models; older CPUs viable for ~1.5B models (~8 tokens/sec).

Table 3.3: DeepSeek Distilled Model Infra Setup

Quick download and setup summary

For a simple setup, you can either pull models directly via Ollama for a lightweight local deployment that is ideal for experimentation, or deploy serverless in enterprise-grade clouds:

Local setup: Install Ollama (`curl -fsSL https://ollama.com/install.sh | sh`), then use commands like

```
ollama pull deepseek-r1:1.5b
```

```
ollama run deepseek-r1:1.5b
```

This lets you instantly test models like DeepSeek-R1 distilled versions or DeepSeek-Math variants on your own machine.

Cloud setup: Use platforms like *AWS Bedrock*, *SageMaker*, or *Azure AI Foundry* to deploy DeepSeek models with enterprise-grade APIs, security, and scalability. These options only require minimal setup and offer immediate access to model inference capabilities.

The DeepSeek-R1-Distill series allows users to pick solutions matching their precise needs by providing different models that harmonize powerful reasoning features with their actual computing capabilities.

Performance benchmarks

The distilled models from DeepSeek show superior performance across different reasoning tasks by achieving both efficient operation and strong reasoning

functionality.

A 55.5% accuracy rate on the AIME 2024 exam was achieved by DeepSeek-R1-Distill-Qwen-7B model, surpassing all the larger models.

General problem-solving abilities of Distilled DeepSeek models match the full-sized DeepSeek-R1 model by 70-85% while operating with a parameter reduction to just 1%. The distillation process applied to DeepSeek models produces superior results than competitor systems, which utilize comparable resources at minimum.

The research outcome from DeepSeek reveals that effective knowledge transfer from robust models to smaller versions produces superior results. The performance achievement of distilled models exceeds the capabilities of smaller models which depend on large-scale **reinforcement learning (RL)** because they need extensive computational resources to function effectively. Distillation methods show both financial efficiency and efficient performance yet the advancement of current intelligence might require more capable base models with increased RL scale.

Tests have proved that Distilled DeepSeek preserves cognitive functions through its optimization technique which achieves efficient computation.

Practical applications of distilled models

Distilled models bring efficient operation to numerous use cases in restricted computing conditions throughout different sectors.

Edge computing

The application of distilled models enables improved AI operation on devices which operate under limited processing capacity requirements. IoT devices that embed reasoning capabilities enable onsite decisions which minimize their dependence on cloud-based connections. Specialized hardware systems that integrate AI can perform complex tasks on their own without transferring most computational workloads to external servers. Research indicates teacher-student networks adjusted for edge systems reach a 12:1 compression rate while maintaining 96.8% of the original model accuracy performance. The strategy delivers outstanding results for computer vision operations because it lowers memory usage by 67% while accelerating inference speed by 2.5 times.

Cost-effective deployment

The implementation of distilled models brings substantial cost advantages to organizations which need to manage tight budgets. Models operate optimally with reduced parameters which enable them to work efficiently on budget and performance-oriented cloud instances or cheaper hardware systems. The efficient operation of these models reduces both energy utilization and the deployment expenses which results from AI application implementation. The improved processing speed in distilled models lets organizations handle bigger numbers of requests through their current hardware resources which yields improved operational efficiency.

Latency-sensitive applications

The fast response needs of various applications make distilled models ideal because they require less processing time. Simple real-time user assistance delivers immediate responses which leads to better user satisfaction. These models deliver time-sensitive decisions which are essential for autonomous systems together with industrial automation systems. Heavy workload conditions do not compromise the performance of high-volume services because they maintain their operational efficiency.

Educational and research accessibility

The creation of compact models through distillation methods creates opportunities for more people to use advanced AI technologies. Researchers who work with restrained computational capacity can perform state-of-the-art research through the use of these models. Standard educational environments gain advantage through standard hardware capabilities that allow students to explore AI tool functionality for teaching and discovery purposes. Developers can expedite their AI application development process by using these models for rapid prototyping after which they can scale up to bigger models.

Distilled models allow multiple sectors to successfully implement efficient cost-effective mobile AI solutions which make cutting-edge AI capabilities accessible for a wide range of applications.

Trade-offs and considerations

While distilled models offer significant advantages in efficiency, it is essential to understand the trade-offs and considerations involved in their deployment.

Performance gaps

Smaller distilled models tend to display lower functionality in specific use cases than their larger equivalent models. Complex reasoning tasks and rare specialized knowledge domains require special attention from these models and so do processes with multiple steps. Elaborate knowledge transfer between teacher models and student models becomes challenging when student resources remain inferior to teacher abilities since this diminishes their learning capacity which results in performance problems.

Domain specificity

Specialized distilled models achieve superior results when they are developed for specific applications. Distillation techniques handle specific domain knowledge transfer for mathematical or coding subjects which produce exceptional models within those fields. Depending on the requirements of its intended application the performance gaps can be addressed by applying targeted task tuning techniques through fine-tuning processes.

Continuous improvement

Model distillation research develops at a quick pace because scientists work daily to improve the operational potential of small models. Continuous research brings architectural advancements and better distillation techniques as well as hardware improvements to solve current constraints. The **Distillation-Oriented Trainer (DOT)** represents a new approach that successfully manages task-oriented and distillation performance measures, leading to advanced distilled model performance and robustness. The effectiveness of distilled models in different applications progresses with existing technological developments that need constant monitoring.

Users should evaluate these factors to create sound decisions about deploying distilled models using performance requirements against resource availability and application needs.

Comparative analysis of DeepSeek models

After exploring the three main categories of DeepSeek models, language, vision, and distilled, it is valuable to compare them directly to understand their relative strengths, limitations, and optimal use cases.

Performance vs. resource requirements

A proper selection of DeepSeek models requires knowledge about how their features align with available resources in specific use scenarios. The following comparative analysis highlights key aspects of various DeepSeek models:

Model type	Reasoning capability	Multimodal support	Parameter count	Memory requirements	Inference speed	Deployment flexibility
DeepSeek-R1	Excellent	Text only	175B	Very high	Slower	Limited to high-end hardware
DeepSeek-VL	Good	Text + Vision	80B	High	Moderate	Requires specialized hardware
DeepSeek-R1-Distill-70B	Very good	Text only	70B	High	Moderate	High-end servers
DeepSeek-R1-Distill-14B	Good	Text only	14B	Moderate	Fast	Standard servers
DeepSeek-R1-Distill-7B	Moderate	Text only	7B	Low	Very fast	Edge devices possible
DeepSeek-R1-Distill-1.5B	Basic	Text only	1.5B	Very low	Extremely fast	Most edge devices

Table 3.4: DeepSeek performance and resource requirements

Selecting the right model for your use case

The decision to select an appropriate model requires awareness of your use case requirements.

The selection process for an appropriate DeepSeek model requires attention to multiple factors which optimize performance while keeping operational efficiency operational.

Complexity of the task remains the key factor when selecting the appropriate model. The DeepSeek-R1 along with the larger models DeepSeek-R1 and DeepSeek-R1-Distill-70B achieve the best results in complex reasoning tasks that include mathematical proofs and scientific analyses and complex problem-solving scenarios. The models provide highly precise solutions for complex

tasks that engineers have designed them to handle. Tasks involving general question answering and content generation as well as standard programming assignments benefit most from the mid-sized distilled model DeepSeek-R1-Distill-14B because it provides a precise performance-efficiency ratio. The text generation process can be accomplished through the DeepSeek-R1-Distill-7B or DeepSeek-R1-Distill-1.5B models while maintaining efficient computational limits for straightforward classification tasks.

Model selection depends heavily on the **modality requirements**. Text-based applications require DeepSeek-R1 or any distillation model from its series because these models demonstrate outstanding performance for text data processing. The task requires an understanding of visual and textual information thus DeepSeek-VL becomes the optimal selection since it works with both modalities. The application DeepSeek-V provides specialized vision-only features for situations that do not require text processing.

The model **deployment** requires consideration of target infrastructure to guarantee proper fitment. Model sizes that need substantial computing resources work best in cloud or data center environments because these facilities offer enough hardware to support such requirements. The mid-sized distilled models demonstrate a noteworthy capability to handle typical server environments because they achieve an optimal trade-off between performance and necessary resources. For deployments involving limited computing power and memory capacity one can use small, distilled models that enable AI functionality on constrained edge devices.

Latency requirements serve as one of the factors which helps identify suitable model selection. Larger models remain suitable for batch processing applications even if their inference times are slower because the applications permit delayed responses. The need for fast responses in interactive situations leads to the selection of distilled models because they provide quicker inference speeds. Real-time systems need fast and small models for ensuring adequate responsiveness because they have strict requirements regarding response time.

Users ought to evaluate their specific requirements carefully when selecting a DeepSeek model since this will help them find the best balance between performance and efficiency within their practical constraints. The following flowchart will help you decide how to select a model based on your needs:

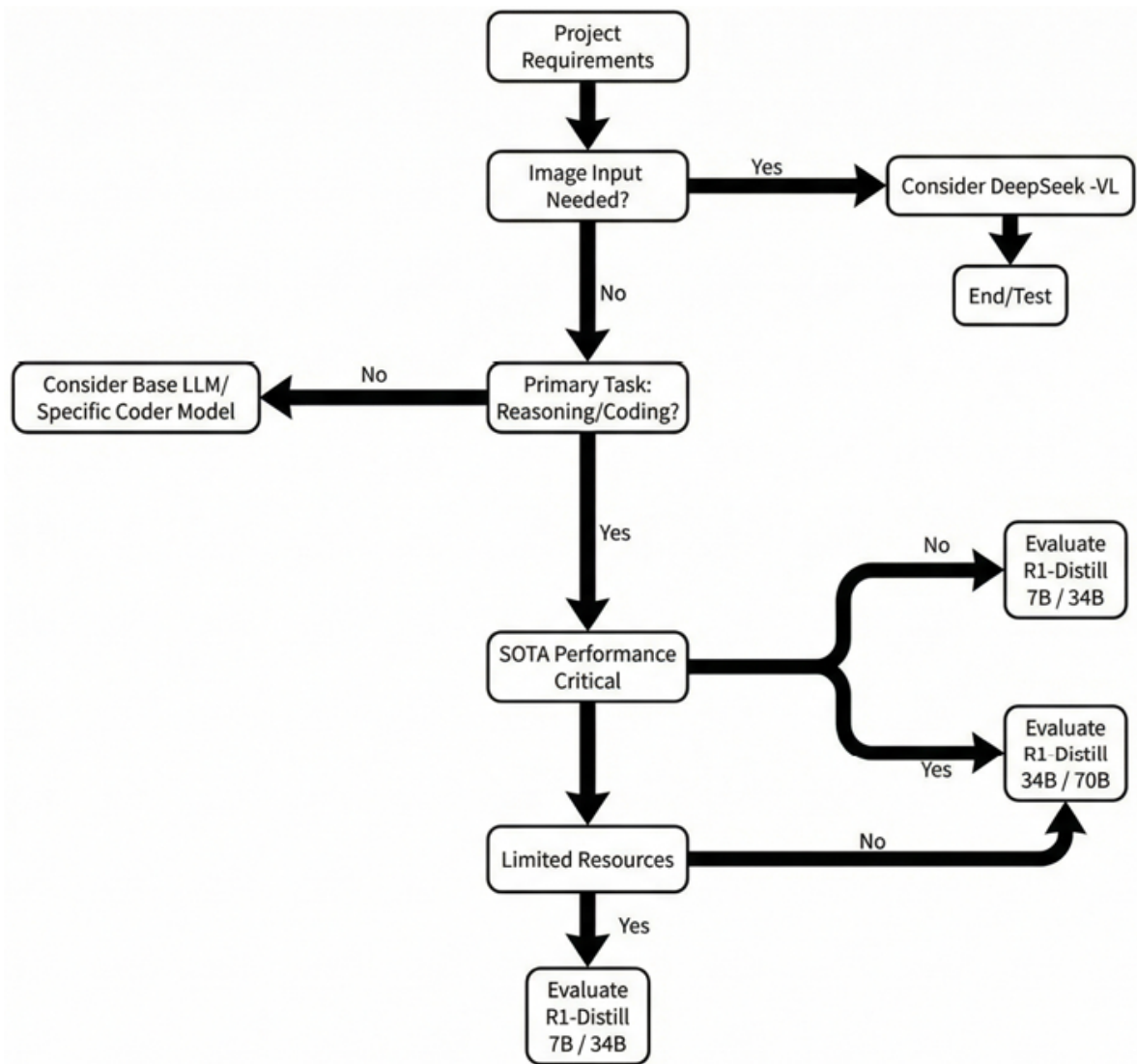


Figure 3.3: Flowchart for selecting the right model

Conclusion

This chapter examines different categories of DeepSeek models which comprise language models for reasoning and vision models for interpretation as well as distilled models that deliver high performance efficiencies. The R1 language models of DeepSeek have revolutionized reasoning functions with superior capabilities which surpass other AI systems during complex problem-solving and logical operations. The vision models DeepSeek-VL and other types enable new possibilities through text together with visual input processing which supports more sophisticated artificial intelligence applications. The distilled

models leverage innovative distillation methods to provide high-end reasoning capabilities in devices with restricted resources which makes them an excellent choice for resource-limited situations.

In the next chapter, we will study how to implement these powerful DeepSeek models in real-world applications. This will include a deep dive into the options for API-based integrations and the use of local **large language models (LLMs)**, as well as a comparison of the pros and cons of both approaches. The knowledge gained from these implementation strategies will demonstrate fundamental deployment methods for DeepSeek models within your projects ensuring optimal usage of their capabilities.

Points to remember

Keep the following key points in mind to ensure clarity and effectiveness moving forward:

- DeepSeek delivers three fundamental model categories that encompass Language Models for text reason and understanding as well as Vision Models for image interpretation and Distilled Models for efficient deployment.
- The R1 series in DeepSeek language models stands out for its superior reasoning performance which surpasses most competing solutions on scientific reasoning and mathematical problem-solving benchmarks.
- Complex reasoning abilities emerged through reinforcement learning in DeepSeek-R1-Zero thus proving that supervised fine-tuning was unnecessary for such behaviors to develop.
- DeepSeek-V3 operates through a MoE structure which activates just 37 billion out of its 671 billion parameters at each operation for greater performance compared to traditional dense models.
- Through its combination of visual and textual understanding, DeepSeek-VL enables applications that need to reason between visual and textual modalities such as Visual Question Answering and document understanding.
- DeepSeek-R1-Distill series provides advanced reasoning models for minimal resource devices that scale from 70 billion to 1.5 billion parameters.
- The smaller scale of distilled models does not impact their performance

capabilities since DeepSeek-R1-Distill-Qwen-7B scored 55.5% accuracy on the AIME 2024 exam while surpassing numerous larger models.

- GRPO stands as the main innovation in DeepSeek's reinforcement learning because it eliminates the requirement for a separate critic model while improving efficiency during training.
- The selection of a DeepSeek model requires evaluating task complexity together with modality requirements and deployment environment conditions and latency constraints to determine the most suitable choice.

Key terms

The key terms are as follows:

- **Language models:** AI systems designed to understand, interpret, and generate human language, forming the foundation of DeepSeek's capabilities.
- **Vision models:** AI systems that process and interpret visual information, enabling applications that involve image understanding and analysis.
- **Distilled models:** Smaller, more efficient models created by transferring knowledge from larger teacher models, making advanced capabilities accessible with fewer computational resources.
- **MoE:** A neural network architecture that divides processing among specialized sub-networks (experts), activating only a subset for any given input to improve efficiency.
- **Multimodal models:** AI systems that can process and reason about information from multiple types of input (e.g., text and images), enabling more comprehensive understanding.
- **Knowledge distillation:** The process of transferring knowledge from a larger teacher model to a smaller student model to improve the performance of the smaller model.
- **Context window:** The maximum amount of text a model can process at once, measured in tokens. Larger context windows enable processing of longer documents or conversations.
- **Visual Question Answering (VQA):** A task where an AI system answers questions about an image, requiring both visual understanding and language

processing.

- **Edge computing:** Processing data near the source of data generation rather than in a centralized data-processing warehouse, often using devices with limited computational resources.
- **Inference speed:** The time required for a model to generate a response to an input, a critical factor for interactive applications.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 4

Production Approaches

Introduction

Our previous chapter explored DeepSeek models' extensive range, which includes advanced language models alongside vision models for visual understanding, alongside efficient distilled variants. Our analysis of these models' capabilities leads us to the next stage, where we will explore their practical deployment within production environments.

Deploying DeepSeek LLMs means balancing operational needs, computing resources, and performance goals. Institutions also need to align deployment with their data management policies. Ultimately, cost management often becomes the key factor in deciding the strategy. The deployment of LLMs consists of two fundamental methods. API deployment through cloud services exists alongside the option of running models from your infrastructure. These deployment approaches exist with their own benefits and constraints, which match specific business requirements.

We analyze both deployment strategies in detail during this chapter. API-based deployments enable convenient and scalable operations without requiring major infrastructure expenses. The chapter follows up with a discussion on local LLM installation methods that provide organizations with enhanced control and customization features, as well as data privacy protection. We present a dimensional comparison of these deployment

approaches to help you select the most suitable solution that fits your specific needs.

Upon completing this chapter, you will have mastered the various production deployment methods for DeepSeek models to select the most suitable implementation for your applications.

Structure

In this chapter, we will explore the following areas:

- API
- Token optimization
- Local LLMs
- Pros and cons of API versus local LLMs

Objectives

On completion of this chapter, we will deliver a complete understanding of deploying DeepSeek models at scale. Safety-based API deployments together with local LLM model preparation, allow us to identify essential features between these options while gaining insight into their benefits and limitations. The decision to deploy using this delivery method produced better choices that aligned with the company's specifications while considering resource availability and target objectives. What specific deployment tactics should organizations explore? The guide demonstrates optimized APIs methods together with local LLM preparation strategies to obtain better performance results. The information will help you produce successful results by applying the DeepSeek model across your production environment to meet technical requirements and business objectives.

API

API-based deployment represents a common method through which companies implement LLMs such as DeepSeek. Accessing the model by

means of cloud-based services allows the model provider to handle the entire responsibility for infrastructure management alongside model scalability and ongoing maintenance. API-based deployment enables developers to prioritize application logic and user experience since the service provider manages model hosting and performance optimization tasks. Through this model, developers can integrate rapidly while benefiting from the latest LLM features while avoiding the need to manage fundamental infrastructure systems.

Understanding how API based deployment works

In an API-based deployment, the LLM runs on the provider's servers, and you interact with it through HTTP requests. The typical workflow is as follows:

- **Authentication:** When communication starts, the application must authenticate itself to the API service either through an API key or alternative authentication credentials.
- **Request formulation:** During the request process, the application combines input text with model parameters and additional required information.
- **API Call:** The API endpoint receives the request through an HTTP POST Transmission.
- **Processing:** When the provider receives the request, its servers execute the input by running it through the designated model.
- **Response:** The model's output becomes available to the application as part of an HTTP response.
- **Integration:** The application takes the response to incorporate it into the user experience flow.

The following figure shows the end-to-end API request-response flow:

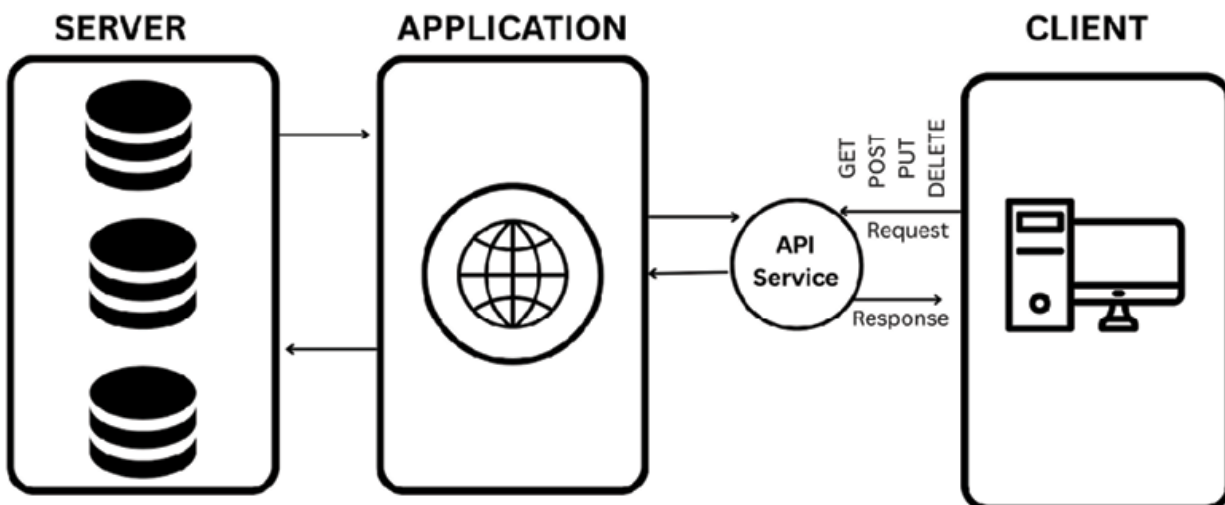


Figure 4.1: Diagram illustrating API-based DeepSeek deployment workflow

Running and maintaining model infrastructure becomes abstracted through this approach, which allows developers to focus on utilizing the model capabilities within their applications. While the previous figure illustrates CRUD operations to represent the standard RESTful API pattern, in practice, the client here primarily sends prompts to the DeepSeek service and receives responses, rather than performing traditional CRUD operations.

DeepSeek API services

API services from DeepSeek allow developers to incorporate its sophisticated language models through simple application integration mechanisms. Through its API services, DeepSeek enables direct implementation of DeepSeek family models into applications. The models include:

- **DeepSeek-V3:** The mixture-of-experts architecture delivers a model with 671 billion total parameters and 37 billion active parameters together with efficient inference capabilities.
- **DeepSeek-R1:** The model DeepSeek-R1 inherits its structure from DeepSeek-V3 while developing reinforcement learning functionalities to enhance reasoning capabilities.
- **DeepSeek-Coder:** The R&D efforts resulted in the development of efficient, smaller models capable of delivering strong mathematical and programming functionality.

- **DeepSeek-VL:** The model exists to power conversational systems while handling a maximum of 64,000 input tokens alongside 16,000 response tokens.

The DeepSeek—API platform enables consistent communication with model suites while supporting text creation, program completion and multiple input processing. Through standardized methods, applications can easily integrate complex linguistic processing functions.

Here is an example of a basic API request to the DeepSeek text generation endpoint:

```
import requests
import json
```

```
API_URL = "https://api.deepseek.com/v1/chat/completions"
API_KEY = "your_api_key_here"
```

```
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}
```

```
data = {
    "model": "deepseek-r1",
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain the concept of reinforcement
learning in simple terms."}
    ],
    "temperature": 0.7,
    "max_tokens": 1024
}
```

```
response = requests.post(API_URL, headers=headers,
data=json.dumps(data))
result = response.json()
print(result["choices"][0]["message"]["content"])
```

DeepSeek API provides a comprehensive set of advanced features which enable developers to achieve better control during large language model integration in their applications. Through its suite of advanced features, the DeepSeek API enables developers to manage application responses in real-time and control function executions with context management features and parameter adjustments, such as:

- **Streaming responses:** Streaming responses in LLMs is the process of delivering output on a token-by-token basis gradually, as it is produced. In this approach, perceived latency is reduced, and it is therefore possible to begin reading the response instantly.
- **Function calling:** Function calling on LLMs enables the model to understand the user inputs and trigger pre-set functions that allow interaction with external tools or APIs.
- **Context management:** Context management involves storing the conversation history across several requests so that a maintained conversation turns out to be coherent and contextually relevant in LLMs. This is done by maintaining a series of messages that include user inputs and model responses that the model will use to refer to in every exchange.
- **Parameter tuning:** Adjusting generation parameters like temperature, top-p, and frequency penalty. Temperature is one of the major parameters, which control the randomness of responses.

API pricing and quotas

DeepSeek's API services use usage-based pricing to transform costs into token processing amounts. Inside the model's core processing, tokens are the basic units and each is nearly equivalent to four standard English characters. Model selection dictates specific pricing approaches which must be followed. A combination of DeepSeek-R1 pricing mechanisms and sophisticated models delivers prices that surpass what is available for distilled versions of the model which are smaller in size. Under the token system network requests function as input tokens while model-generated responses act as output tokens. DeepSeek API services regulate end-user request speeds through implementation of quota systems for access

management functions. The system controls quotas through a deliberate design which maintains service quality standards and defends against abuse attempts.

Common quota types include the following:

- **Requests per minute (RPM):** Limiting the number of API calls you can make per minute.
- **Tokens per minute (TPM):** Limiting the total number of tokens you can process per minute.
- **Concurrent requests:** Limiting the number of simultaneous requests you can have in progress.

Higher-tier API plans typically offer higher quotas, allowing for more intensive usage of the service.

API integration best practices

Effective API integration rests on a few important best practices. Begin by reading the documentation of the API, the endpoints available, the formats of the data used, everything that has to do with the ways to insert the API, and the way it handles errors. Visit the API provider's rate limits, in order to prevent disconnections of its service, and construct your integration to be flexible enough to be backward compatible with the API versioning so that the integration is compatible with other future versions of the API when they update.

The different integration practices are discussed in the following sections.

Error handling and retries

API calls encounter failure because of three main issues: network connectivity problems, server-related errors and quota limits that are exceeded. Your application requires both powerful error management techniques along with automatic retry functionality as a means to handle persistent failures.

The implementation of reliable error handling and retry techniques will maintain your application's resilience during failure situations.

Here is an example code for error handling with retries in an API request:

```

import time
import requests
from requests.exceptions import RequestException

def call_api_with_retry(data, max_retries=3, backoff_factor=2):
    retries = 0
    while retries <= max_retries:
        try:
            response = requests.post(API_URL, headers=headers,
data=json.dumps(data))
            response.raise_for_status() # Raise an exception for 4XX/5XX
responses
            return response.json()
        except RequestException as e:
            retries += 1
            if retries > max_retries:
                raise
            sleep_time = backoff_factor ** retries
            print(f"API call failed: {e}. Retrying in {sleep_time} seconds...")
            time.sleep(sleep_time)

```

Caching

The implementation of cache solutions for API queries that experience frequent requests results in simultaneous cost reduction and response speed optimization.

Here is an example code of caching API responses for efficiency:

```

import hashlib
import json
from functools import lru_cache

@lru_cache(maxsize=1024)
def cached_api_call(request_str):
    request_data = json.loads(request_str)
    response = requests.post(API_URL, headers=headers, data=request_str)
    return response.json()

```

```
def generate_text(prompt, model="deepseek-r1", temperature=0.7):
    request_data = {
        "model": model,
        "messages": [{"role": "user", "content": prompt}],
        "temperature": temperature
    }
    # Convert to string for caching
    request_str = json.dumps(request_data)
    result = cached_api_call(request_str)
    return result["choices"][0]["message"]["content"]
```

Prompt engineering

The quality of model outputs directly relates to how well the input prompt is written. Hence, invest time in crafting effective prompts that can clearly communicate user requirements to the model through the following ways:

- **Be specific:** Clearly state what action or task you want the model to perform.
- **Provide context:** Include relevant background information.
- **Use examples:** Showcase output format samples to the model to help it understand better
- **Structure your prompt:** Use a structured approach while giving a prompt. This will aid the model in understanding the flow and requirements of the task given by the user.

Token optimization

Since DeepSeek's API costs are based on token usage, optimize your prompts and responses to use tokens efficiently:

- **Minimize unnecessary context:** Include only the information that's relevant to the task.
- **Use truncation wisely:** Set appropriate **max_tokens** limits for responses.

- **Batch related requests:** Combine multiple related queries into a single request when possible.
- **Implement streaming:** For user-facing applications, use streaming to start displaying results before the full response is generated.

API security considerations

Security needs to be a top priority when working with API-based LLM deployments. Some of the most important considerations when designing such LLM's include rigorous authentication and authorization, such as OAuth 2.0, that limit access in an effective way. Encryption of data in transit by using protocols like HTTPS to prevent interception of data is very important.

The implemented security measures include:

- **API key management:**
 - **Secure storage:** API keys should never be hardcoded into application code and version control because these locations make them susceptible to security breaches.
 - **Environment variables:** Store API keys as environment variables or in a secure secrets management system.
 - **Key rotation:** Regularly change the API keys to minimize the impact of potential leaks.
 - **Access control:** Use the principle of least privilege when assigning API keys to different components of your system.
- **Data protection:**
 - **Data minimization:** Send only the data necessary for the model to perform its task.
 - **PII handling:** Avoid sending **personally identifiable information (PII)** to the API when possible.
 - **Encryption:** Use HTTPS for all API communications to encrypt data in transit.
 - **Output validation:** Validate and sanitize model outputs before displaying them to users or using them in your application.

- **Prompt injection prevention:** Prompt injection attacks occur when malicious users manipulate the model's behavior by including instructions in their input that override your intended instructions. To mitigate this risk:
 - **Separate user input:** Clearly separate user input from system instructions in your prompts.
 - **Input validation:** Validate and sanitize user inputs before including them in prompts.
 - **Output filtering:** Implement filters to detect and block potentially harmful outputs.
 - **Monitoring:** Continuously monitor model interactions for signs of prompt injection attempts.

Local LLMs

The broader language model local regulations through infrastructure offer data protection through both control and adaptation mechanisms. API-based applications are designed to deliver fast scalability capabilities. You can install this type of method directly onto the server or edge devices to obtain the model weights.

Understanding how local LLM deployment works

Local LLM deployment requires users to host their model on their own infrastructure before integrating it directly into your application. The standard workflow consists of the following:

- **Model acquisition:** You can download the model weights from the provider's repository or model hub.
- **Infrastructure setup:** Prepare the necessary hardware and software infrastructure to run the model.
- **Model loading:** Load the model weights into memory using an appropriate framework.
- **Inference:** Process inputs through the model to generate required outputs.

- **Integration:** Integrate the model's outputs into your application's workflow to get the desired results.

The following figures illustrate local retrieval with a vector database:

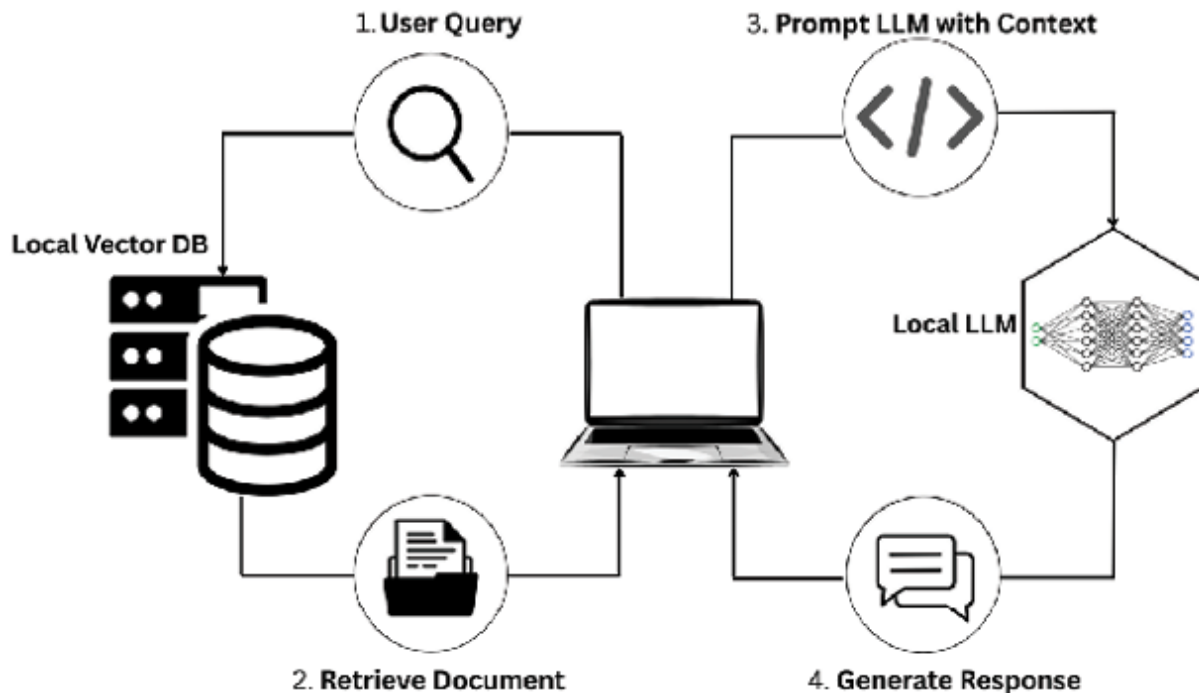


Figure 4.2: Diagram showing local retrieval with vector database integration

This approach gives you complete control over the model's deployment and usage, but requires more technical expertise and infrastructure management.

DeepSeek local deployment options

DeepSeek provides several local deployment options adapted to different resource constraints and application needs of the user:

- **Full models:** The full featured DeepSeek models, such as DeepSeek-R1, and DeepSeek-V3, represent the highest performance but also require substantial computational resources. They are intended to run on high-performance servers with multiple GPUs in action.
- **Distilled models:** The DeepSeek-R1-Distill family of models is smaller but still maintains strong reasoning ability while being lower resource. They are suitable for deployment on lower-performance servers or edge devices with restricted GPU.

- **Quantized models:** Quantization reduces the precision of the model's weights, significantly decreasing memory requirements and inference time with minimal impact on performance. DeepSeek models can be quantized to various precision levels:
- **FP16 (16 bit floating-point):** Cuts memory in half as compared with FP32, with negligible cost in performance.
- **INT8 (8-bit integer):** Reduces memory further and speeds inference time with a small cost in performance.
- **INT4 (4-bit integer):** Delivers maximum efficiencies for low resource environments, with a larger cost in performance.

Hardware requirements

The hardware requirements for running DeepSeek models locally depend on the specific model and optimization techniques used, which are given in the following table:

Model	Parameters	Minimum GPU memory	Recommended GPU	CPU inference
DeepSeek-R1	175B	350GB+	Multiple A100/H100	Not feasible
DeepSeek-V3 (MoE)	37B active	80GB+	A100 80GB	Not feasible
DeepSeek-R1-Distill-70B	70B	140GB+	Multiple A100	Not feasible
DeepSeek-R1-Distill-14B	14B	28GB	A10/A100	Limited
DeepSeek-R1-Distill-7B	7B	14GB	RTX 4090/A10	Limited
DeepSeek-R1-Distill-1.5B	1.5B	3GB	RTX 3060	Feasible

Table 4.1: Minimum hardware requirements for local deployment of DeepSeek models

These requirements can be significantly reduced through optimization techniques like quantization, model sharding, and efficient inference libraries.

Deployment frameworks and tools

There are several frameworks and tools to make the local deployment of DeepSeek models easier. They are discussed in the next sections.

Hugging Face Transformers

The Hugging Face Transformers library is an easy way to load and run your DeepSeek models:

An example code for loading a DeepSeek model using Hugging Face Transformers is as follows:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

# Load model and tokenizer
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16, # Use half-precision for efficiency
    device_map="auto" # Automatically distribute across available GPUs
)

# Generate text
input_text = "Explain the concept of reinforcement learning in simple terms."
inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
outputs = model.generate(
    inputs.input_ids,
    max_new_tokens=512,
    temperature=0.7,
    do_sample=True
)
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
    skip_special_tokens=True)
print(response)
```

VLLM

VLLM is a high-performance library for LLM inference, offering significant speedups compared to standard implementations:

Look at the following example code of running a DeepSeek model using the vLLM inference engine:

```
from vllm import LLM, SamplingParams

# Initialize the model
model = LLM(
    model="deepseek-ai/deepseek-r1-distill-7b",
    dtype="half", # Use half-precision (FP16)
    gpu_memory_utilization=0.9 # Control GPU memory usage
)

# Define sampling parameters
sampling_params = SamplingParams(
    temperature=0.7,
    top_p=0.95,
    max_tokens=512
)

# Generate text
prompts = ["Explain the concept of reinforcement learning in simple terms."]
outputs = model.generate(prompts, sampling_params)
for output in outputs:
    print(output.outputs[0].text)
```

Ollama

Ollama provides a simplified way to run LLMs locally with minimal setup. However support for newer DeepSeek models are in progress or have been recently added.

An example code for running a DeepSeek model locally using Ollama with minimal setup is as follows:

```
# Install Ollama (macOS/Linux)
curl -fsSL https://ollama.com/install.sh | sh
```

```
# Pull and run a DeepSeek model
ollama pull deepseek-r1-distill-7b
ollama run deepseek-r1-distill-7b
```

LlamaIndex

LlamaIndex facilitates the integration of LLMs with external data sources. Following is an example code for using LlamaIndex to connect DeepSeek models with external data sources:

```
from llama_index import VectorStoreIndex, SimpleDirectoryReader,
ServiceContext
from llama_index.llms import HuggingFaceLLM
```

```
# Initialize the LLM
llm = HuggingFaceLLM(
    model_name="deepseek-ai/deepseek-r1-distill-7b",
    tokenizer_name="deepseek-ai/deepseek-r1-distill-7b",
    device_map="auto",
    model_kwargs={"torch_dtype": "auto"}
)
service_context = ServiceContext.from_defaults(llm=llm)
```

```
# Load documents
documents = SimpleDirectoryReader("./data").load_data()
```

```
# Create index
index = VectorStoreIndex.from_documents(documents,
service_context=service_context)
```

```
# Query the index
query_engine = index.as_query_engine()
response = query_engine.query("What are the key points about
reinforcement learning?")
```

```
print(response)
```

Optimization techniques

There are a few optimization strategies you may want to apply to run DeepSeek models efficiently on local infrastructure

Quantization

Model quantization reduces the precision of the model weights, which can significantly reduce memory and increase inference speed.

An example code for applying quantization to reduce model size and improve inference efficiency is as follows:

```
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
import torch
```

```
# Configure quantization
```

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True
)
```

```
# Load quantized model
```

```
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quantization_config,
    device_map="auto"
)
```

Model sharding

For larger models that do not fit on a single GPU, model sharding distributes the model across multiple GPUs.

This is an example code of model sharding for multi-GPU DeepSeek deployment:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

# Load model with sharding
model_name = "deepseek-ai/deepseek-r1-distill-14b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto" # Automatically distribute across available GPUs
)
```

Key-Value cache management

The **Key-Value (KV)** cache stores intermediate attention states during generation, which can consume significant memory for long sequences. Efficient KV cache management can reduce memory usage.

Following is the example code of managing KV cache to optimize memory during long-sequence generation:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
```

```
# Load model
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)
```

```
# Generate with efficient KV cache management
input_text = "Explain the concept of reinforcement learning in simple terms."
```

```
inputs = tokenizer(input_text, return_tensors="pt").to(model.device)
outputs = model.generate(
    inputs.input_ids,
    max_new_tokens=512,
    temperature=0.7,
    do_sample=True,
    use_cache=True,
    max_length=inputs.input_ids.shape[1] + 512 # Control total sequence
length
)
```

Flash Attention

Flash Attention is an optimized attention implementation that significantly reduces memory usage and increases computation speed.

Here is an example code of using Flash Attention to improve memory efficiency and accelerate attention computation:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

# Load model with Flash Attention
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto",
    attn_implementation="flash_attention_2" # Use Flash Attention
)
```

Local deployment architectures

When deploying DeepSeek models locally, several architectural patterns can be used depending on your requirements.

Single-server deployment

For smaller models or applications with moderate traffic, a single-server

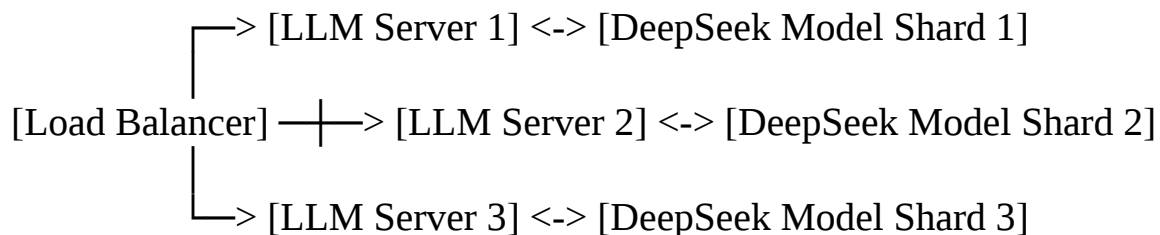
deployment may be sufficient:

[Application] <-> [LLM Server] <-> [DeepSeek Model]

This simple architecture is easy to set up and maintain, but offers limited scalability and fault tolerance.

Distributed deployment

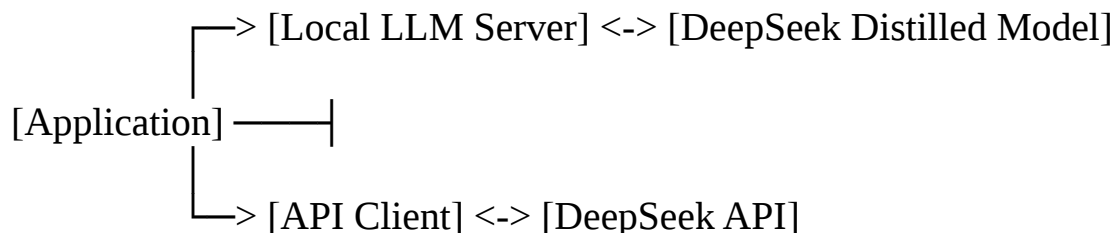
For larger models or applications with higher traffic, a distributed deployment across multiple servers provides better scalability and reliability:



This architecture allows for horizontal scaling by adding more servers as demand increases.

Hybrid deployment

A hybrid deployment combines local models for sensitive or high-frequency tasks with API calls for less sensitive or resource-intensive tasks:



This approach balances performance, cost, and data privacy considerations.

Local deployment best practices

When running Deepseek models on local infrastructure, it is imperative to follow best practices for better performance, reliability, and maintainability:

- **Resource monitoring and management:**
 - **GPU utilization:** Constantly monitor GPU utilization to help identify any resource utilization bottlenecks.
 - **Memory management:** Adhere to sound memory management

practices, particularly if the model is to be deployed as a long-term persistent service.

- **Batch processing:** Batching inference requests helps to achieve more throughput.
- **Graceful degradation:** Build graceful degradation mechanisms and service continuity if your load exceeds expectations and overwhelms the hardware.
- **Containerization and orchestration:**
 - **Docker containers:** Use Docker containers for your model and all dependencies to offer a consistent and reproducible deployment.
 - **Kubernetes:** Consider Kubernetes or similar orchestration for workloads across clusters, on-scale workloads across nodes.
 - **Helm charts:** Use Helm and Helm charts for application deployments and the complexity of managing the configurations.
 - **Auto-scaling:** Define policies for auto scaling to effectively change processing resources in a dynamic pattern based on either usage or underlying load on the system.
- **Monitoring and logging:**
 - **Performance metrics:** Instrument metrics around performance, like latency, throughput, and counts of errors, is essential in ascertaining overall health and responsiveness of the system.
 - **Resource utilization:** Observe all CPU, GPU, and memory consumption to monitor for performance degradation prior to excessive delay in responsiveness of the system to workloads.
 - **Request logging:** Log input/output to the model for greater debugging and system audit capabilities.
 - **Alerting:** Set up alerts for critical issues that require immediate attention.

Security considerations

Before utilizing a DeepSeek model on a local machine, it will have to be secured. Following is a non-exhaustive but important list of security

precautions or considerations relevant to DeepSeek models, and the services they may expose:

- **Access control:** Implement proper authentication and authorization for model access.
- **Network security:** Secure network communications between components.
- **Model protection:** Protect model weights from unauthorized access or extraction.
- **Input validation:** Validate and sanitize inputs to prevent prompt injection attacks.

Pros and cons of API versus local LLMs

Understanding and contemplating the trade-offs in API-based versus local LLM deployments is essential for aligning our technology decisions to the aims of our organization. [Table 4.2](#) summary offers a snapshot based on what are a few of the relevant dimensions:

Performance and latency

The term performance means how well a language model performs and responds while being used for different tasks. Latency is how long it takes from making a request to seeing a response, which can be affected by how quickly the network processes and responds.

- **API:** When you use API deployment, strong cloud models are handled by the service provider and regularly improved. The approach provides developers with access to efficient language models without them having to set up complicated infrastructure.
 - **Pros:** Using APIs, organizations are able to access the best models available without the expense or work of owning local machines. The provider takes care of the infrastructure, so APIs work smoothly. In addition, these services have features that automatically handle increases or decreases in the amount of work needed.
 - **Cons:** Yet, every API request has to travel online, which causes a

delay because of the amount of time it takes for signals to cross the internet. People using the platform do not have much control over performance settings, which are usually managed by the provider. Besides, the performance of the system depends on the API provider's terms and service-level agreements, reducing the user's influence on system speed.

- **Local LLMs:** Running models locally means organizations are independent of the network, can control delays, and can focus on making their systems run smoothly.
 - **Pros:** Since all the work is done locally, reactions are consistent and remain unaffected by the internet or server conditions. The system allows developers to optimize for their and others' particular hardware. In addition, it is possible to create models that match the specifications of the available GPUs to increase how fast inference takes place.
 - **Cons:** This type of control creates new complications. How fast and powerful the computer is limits the real-time system's performance. Optimal performance is only reached when model optimization and system tuning are done by experts. When scaling is not set up correctly, the system may slow down during times of high user traffic.

Cost and resource requirements

To use language models, it is necessary to pay for development and upkeep, which may include spending on APIs, hardware, or software. The resource needs for these models focus on their computational demands, for instance, on GPUs, memory, and the effort needed for upkeep.

- **API:** People only pay for what they use and do not have to buy expensive hardware since everything is in the cloud with APIs.
 - **Pros:** API usage does not require teams to purchase hardware, so it works well for teams whose budgets do not include infrastructure. The cost of a plan increases just as much as it is used. No further expenses are needed for regular upkeep of the cloud, making things

easier for company operations.

- **Cons:** High usage over a period can lead to great operating costs for applications that process many tokens at once. Subscription prices are subject to change, especially depending on which tier or service level you choose. In addition, many platforms provide only a modest number of methods for users to save money, other than using less of the platform.
- **Local LLMs:** Setting up models locally requires both money up front and maintenance, but prices fall as the models are scaled up.
 - **Pros:** After infrastructure is set up, prices are not determined by how often or how much people use services. This means you do not get billed extra for making a lot of requests or using many tokens, so it is a good fit for rush applications. When operating large deployments, the cost of infrastructure is spread between several inference tasks, which can make things much more affordable.
 - **Cons:** The process of buying GPUs and all related infrastructure is not cheap at first. Maintenance and operation costs keep coming up with the passage of time. Good use of these resources relies on the expertise needed to arrange and operate the infrastructure fully.

Data privacy and security

Data privacy is about ensuring that user data is managed and protected in the right way and according to the rules. It involves the systems and technologies that are there to stop unauthorized people from accessing, breaching, or misusing data.

- **API:** Deployments using APIs hand data to other servers, making the providers responsible for security instead.
 - **Pros:** Such a structure makes managing security unnecessary for organizations using the model. Typically, providers care for safe and modern environments and they perform automatic updates to maintain the strength of their infrastructure. Delegated services for security, such as updating patches and supervising vulnerabilities, are completed outside the organization.

- **Cons:** Since the API receives the data, the person sending it has little control over what is done with it. A business depends on the provider's procedures and may never see how its data is used or kept. It creates issues if you need to be compliant with rules or work with confidential material.
- **Local LLMs:** By relying on local deployments, privacy-conscious programs can both control their data and increase overall security.
 - **Pros:** All data processing and security happen within the organization's systems, so control is never compromised. With such data hygiene, data handling is completely open and supports the fulfillment of strict regulations.
 - **Cons:** However, every part of the system's security, including its model and infrastructure, falls on the organization. It means performing software updates, fixing system weaknesses, and securing how people access the network. People without the necessary knowledge can create risks when fulfilling these tasks.

Customization and control

Customization means changing how the model behaves, is organized, or outputs results to best fit a particular use. Control means how much a user can control or alter the way the model is built, trained or updated.

- **API:** APIs make it possible to use advanced models, but it is difficult to customize them internally.
 - **Pros:** No training or setup is needed for users to begin using these advanced models right away. If providers send new models, organizations can move over right away. Due to its clean abstraction layers, the API structure helps make development processes agile and smooth.
 - **Cons:** It is not possible to extensively adjust the models used. More detailed customization becomes possible only as far as the provider allows in their product design. Without being able to look at the model's design or weights, users cannot make use of those advanced applications.

- **Local LLMs:** When models are deployed on local devices, even the detailed architecture is open for flexible and detailed modifications.
 - **Pros:** The model can be adjusted by organizations to better suit their specific goals, from changing architecture to making fine adjustments. Having this flexibility allows high-performance solutions to be built for specific uses. DBA teams can boost performance for a single task and extend the functionality of the database by working directly on the system, something not allowed through APIs.
 - **Cons:** Customizing a program can be done by experts, who often cause the system to become more complex. Caring for the software and updating it are now part of the user's job. Customizations to a web framework can bring up problems when upgrades or new features are required.

Scalability and reliability

Scalable systems adjust their resources or improve throughput to meet higher demand as it comes. Reliability shows how dependable the system is, especially when things change or when there are sudden jumps in load.

- **API:** Using APIs that are hosted in the cloud, scaling and ensuring high availability become straightforward, making infrastructure issues easier to deal with.
 - **Pros:** When more traffic comes to a website, it automatically adjusts to serve users using less manual management. Usual practice is to distribute the underlying infrastructure regional, so people can continue to use the service without interruptions. With this model, users are freed from considering how the system is organized and configured.
 - **Cons:** Though API services tend to be reliable, incidents or failures of the service are not under a user's control. At busy times, built-in limits can slow down how the application operates. In addition, it is not possible to adjust or redefine the scaling process for all features.
- **Local LLMs:** Being able to run models locally means enterprises can

determine how big the models are and rank which jobs are most important.

- **Pros:** Firms are able to fashion their scaling solutions to fit what they require, move resources as needed, and guarantee high efficiency where it matters most. Running your own services means you will not experience downtime if a partner has problems, letting you focus first on crucial operations.
- **Cons:** Building up town infrastructure needs well thought out planning and engineering skills. To maintain redundancy, manage where traffic flows and maintain system stability, the user is entirely responsible. Keeping a system available and able to tolerate failure requires a large investment of time and resources.

The following is a table summarising the differences between API and LLMs:

Feature	API	Local LLM
Performance and latency	Adds delay to network operations, but performance is controlled by the provider's infrastructure.	Ensures quickness and complete flexibility for system optimization, but needs reliable local hardware.
Cost and resource requirements	There is no need for expensive hardware; your expenses go up as you use it and there is low maintenance.	High initial investment is necessary, but costs tend to stay the same and are more efficient for the long term.
Data privacy and security	Information moves away from local areas; security is overseen by the provider, but the user may get only part of the picture	Since data is kept within the organization, access to it is private, yet someone from the organization must be responsible for its security.
Customization and control	Models that are set up beforehand and changed only according to the provider's decisions.	Although setting up is very flexible, you need to be technically skilled to manage the models.
Scalability and reliability	Scales with no effort up to a high availability level due to cloud support.	The scaling process is done by hand without automation and is fully adjustable; users must look after their own uptime and reliability.

Table 4.2: Comparison of API-based vs. locally hosted LLMs across operational dimensions

Choosing the right approach

When choosing between using API and deploying models locally using DeepSeek, one must consider the relative importance of the features mentioned previously and decide which of them would be more important for the specific case. Everything has its benefits and drawbacks.

Consider API based deployment if:

- **Access to advanced models:** You need more computational models without the cost of acquiring new hardware equipment.
- **Variable usage patterns:** Your application sees either moderate or unpredictable levels of usage.
- **Reduced operational overhead:** You strive to avoid incurring a lot of costs in the development and subsequent maintenance of the system.
- **Flexible data privacy needs:** I have found that it is not a strict control on your data privacy.
- **Rapid deployment:** You want the fastest possible cycle time on your application.

Consider local LLM deployment if:

- **Data privacy and security:** Safekeeping of sensitive data is one of the most important tasks, as most especially, it is to be retrieved within your infrastructure.
- **High usage volumes:** Your application is very high to the point that may make API costs unbearable.
- **Customization needs:** To go for exact tasks, you should have total control over customizing and optimizing the model.
- **Low-latency requirements:** Your application needs low latency for real-time feedback.
- **Technical expertise:** Your team has the capability to deal with and optimize the LLM infrastructure.

Consider a hybrid approach if:

- **Diverse requirements:** Your application parts have varied needs, which range from some parts that demand strict data privacy to parts that do

not demand the same.

- **Balanced optimization:** You need to synchronize performance, cost, and data privacy in your application.
- **Latency and resource considerations:** You have to optimize both in terms of low latency and efficient use of resources.
- **Leveraging strengths:** You should utilize a combination of API based and local deployment to cater to the multiple needs of operation.

Conclusion

In this chapter, we have looked at two main approaches to placing DeepSeek models in production environments. API based deployment and local LLM deployment. API-based deployment can be differentiated by simplicity, scalability, and instant access to powerful models at no cost of heavy investment in infrastructure, which makes it ideal for a fast roll-out. Local deployment, on the other hand, has the advantage of increased control, customization, and privacy of data, particularly with organizations having high usage and strict security criteria. The ultimate choice to be made between API, local, or a hybrid solution should take into account several crucial aspects, including performance requirements, financial limitations, the policies as they relate to the governance of data, and the technical personnel available.

In the next chapter, we are moving from theory to practice. You will learn how to set up your environment to work with DeepSeek models locally: which tools and configurations are required to start working. This step-by-step guide will take you through downloading, installing, and executing your 1st DeepSeek model. You will craft your first queries, see the model's reaction, and start to understand how local LLMs work differently from cloud-based APIs. By the time you have finished this chapter, you will have a running local LLM and understand the basics of starting to create AI on your own hardware. Whether you are a newbie to local deployment or someone moving from an API-based configuration, this chapter provides you with the right tools to start your local AI development journey.

Points to remember

- API-based deployment has unbeatable convenience and scalability that does not require significant investments in infrastructure. However, it implies sending data to external servers and paying for usage-based fees.
- Local LLM deployment gives more control over the customization and ensures better data privacy. It, in turn, requires increased technical skill and attentive infrastructure management.
- DeepSeek offers a range of models suitable for different deployment approaches, from powerful models like DeepSeek-R1 for API access to distilled models like DeepSeek-R1-Distill-7B for local deployment.
- Optimization techniques like quantization, model sharding, and efficient inference libraries can significantly reduce the resource requirements for running DeepSeek models locally.
- The choice between API and local deployment depends on factors like performance requirements, cost constraints, data privacy considerations, and available expertise.
- A hybrid of API and local deployment can empower the strengths of both paradigms, in particular when it comes to different use cases within one app.
- When using API-based deployment, implement best practices like error handling, caching, prompt engineering, and security measures to optimize performance and protect sensitive data.
- In local deployments, containerization, orchestration, system monitoring, and security that are based on production-grade practices are the best way to achieve both reliability and performance.

Key terms

- **API:** A set of rules and protocols that allow different software applications to communicate with each other.
- **Inference:** The process of using a trained model to make predictions or

generate outputs based on new inputs.

- **Latency:** The time delay between making a request to a model and receiving the response.
- **Throughput:** The number of requests that can be processed by a model in a given time period.
- **Quantization:** The process of reducing the precision of model weights to decrease memory requirements and increase inference speed.
- **Model sharding:** The technique of distributing a large model across multiple GPUs or devices to overcome memory limitations.
- **KV cache:** The key-value cache that stores intermediate attention states during text generation to avoid redundant computations.
- **Containerization:** The practice of packaging an application and its dependencies into a standardized unit (container) for deployment.
- **Orchestration:** The automated configuration, coordination, and management of computer systems and software.
- **Prompt engineering:** The practice of designing and optimizing input prompts to elicit the desired behavior from language models.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 5

Setup and Environment

Introduction

The previous chapter analyzed deployment strategies for DeepSeek models between API-based implementations and local **large language model (LLM)** installations. Our comparative analysis revealed the strengths and trade-offs of each deployment method, so you can choose the solution that best fits your operational needs.

We proceed to an operational exploration of DeepSeek models following our introduction to deployment paradigms. The current chapter details the step-by-step process of setting up your local environment for direct contact with DeepSeek models. We will study the fundamental tools that support local deployments while delivering step-by-step instructions to configure these tools across multiple operating systems.

When working with sophisticated LLMs like DeepSeek you need to construct a solid environmental foundation. A well-designed setup provides both peak functionality, along with streamlined resource handling and easy-development workflow. This chapter provides the necessary tools and knowledge to begin your journey whether you are a researcher studying DeepSeek capabilities or a developer integrating models or an AI enthusiast performing experimental work.

You will be able to set up a local DeepSeek model interaction environment

when you finish this chapter. The process includes tool setup and configuration along with your first DeepSeek model purchase and text generation through inference tasks. Through practical work you will gain fundamental knowledge which forms the basis for the advanced material in later chapters.

Structure

In this chapter, we will explore the following areas:

- Local LLM tools
- Specialized tools for local deployment
- Optimization libraries
- Setting up your environment
- GPU setup for NVIDIA cards
- Hello DeepSeek: Your first model
- Exploring DeepSeek's capabilities
- Optimizing inference for use case
- Building a simple chat application

Objectives

This chapter concludes with fundamental knowledge and hands-on abilities for local DeepSeek model deployment and interaction. You must select appropriate tools that match your operating system and hardware setup and learn how these tools function together to create an operational space for LLM development.

Your focus will be on configuring the environment for peak performance through GPU acceleration, together with memory management and software dependency optimization. Your knowledge will help you decide how to set up your system because it allows you to match resources to specific requirements.

Your learning includes downloading and loading DeepSeek models to

perform inference with a selected model. Through this practical experience, you will learn the fundamental procedures of working with DeepSeek models to create text content while responding to questions and investigating model functions.

Local LLM tools

Working with DeepSeek models in local settings requires a set of tools, that simplify model downloading and loading, followed by inference execution. The tools provide abstraction from the complex aspects of dealing with large language models so users can focus on their use cases without worrying about implementation details.

Core frameworks and libraries

Most large language model operations function on the open-source machine learning framework called PyTorch, which serves as the central computational foundation for the local LLM ecosystem. The tensor computation capabilities of PyTorch deliver exceptional GPU acceleration together with a wide selection of tools and libraries available in its ecosystem.

Installation

To install PyTorch, perform the following instructions based on your system's capabilities:

1. For systems with CUDA 12.1 support:

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu121
```

2. For CPU-only systems:

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cpu
```

These commands will install the appropriate versions of PyTorch, TorchVision, and Torchaudio, ensuring compatibility with your system's hardware and facilitating efficient development and deployment of

DeepSeek models.

Hugging Face Transformers

The Hugging Face Transformers library gives users access to thousands of pretrained models where DeepSeek family belong, alongside APIs that enable model downloading, loading the target application. The library presents one interface that enables users to work with different model architectures while allowing experimental work without extensive code adaptation.

Installation:

`pip install transformers`

Users can integrate different model architectures through this installation which streamlines their use of local DeepSeek models.

Accelerate

The library Accelerate enables users to execute PyTorch code across GPUs, TPU, and CPUs through an easier interface. The system offers exceptional advantages when handling large models across multiple devices to optimize hardware resource allocation.

Installation:

`pip install accelerate`

This installation enables the distribution of DeepSeek models among multiple devices which boosts local deployment performance as well as scalability.

VLLM

The vLLM system operates as a high-throughput memory-efficient platform designed to serve and perform inference on large language models. PagedAttention within the framework enables efficient key-value cache management which results in better inference speed and lower memory utilization.

Installation:

`pip install vllm`

The installation finds its most valuable use in production environments that need high throughput and efficient memory utilization.

DeepSeek model work becomes accessible through the integration of these tools within your local operating environment. Your prepared setup enables peak performance while optimizing resource consumption and simplifying the development process, which readies you for complex applications and customized work in upcoming chapters.

Specialized tools for local deployment

Specialized tools, including DeepSeek, have appeared to simplify local deployment and interaction with **large language models (LLMs)** in addition to core frameworks PyTorch and Hugging Face Transformers. Various interface options, from command-line to graphical user interfaces, enable diverse users to access efficient workflows through these tools.

Ollama

Ollama provides users with a flexible local deployment system that facilitates running LLMs on their computers. Through Ollama users can easily download and operate models using an API or command-line interface. Ollama enables usage of DeepSeek-R1, Llama 3.3, Qwen 3, Mistral, and Gemma 3 models across macOS Linux and Windows operating systems.

Installation:

- **macOS/Linux:** Execute the following command in your terminal:
`curl -fsSL https://ollama.ai/install.sh | sh`
- **Windows:** Download the installer from Ollama's official website:
[<https://ollama.com/download>]

Through its abstracted system, Ollama simplifies local LLM operation which makes it optimal for novice users and users who benefit from easy-to-use interfaces.

LM Studio

Users can access LLM functionality through LM Studio by using a desktop application that enables download, execution, and interaction with LLMs through graphical interfaces. The application supports all DeepSeek family members together with macOS, Windows, and Linux operating systems.

Installation: Download the appropriate installer for your operating system from LM Studio's official website.

Through its intuitive interface, LM Studio allows anyone to work with different models and settings regardless of their command-line expertise.

Text Generation WebUI

The Gradio-based Text Generation WebUI provides users a web interface to operate LLMs. Users can perform model loading and text generation functions while adjusting parameters through a detailed user interface. Multiple text generation backends, such as Transformers ExLlamaV3 and ExLlamaV2, together with various quantization libraries, function through this tool.

Installation:

1. Clone the repository

```
git clone https://github.com/oobabooga/text-generation-webui cd  
text-generation-webui
```

2. Install the required dependencies:

```
pip install -r requirements.txt
```

The Text Generation WebUI delivers an extensive interface that gives advanced model users precise control of parameters alongside generation settings.

The specialized tools enable users to personalize their DeepSeek model workflow, which leads to an improved user experience with tailored environments for individual requirements.

Optimization libraries

To run DeepSeek models efficiently on consumer hardware, several optimization libraries exist.

bitsandbytes

The bitsandbytes library enables 8-bit and 4-bit quantization for large language models while maintaining performance, but reducing memory usage substantially.

Installation:

pip install bitsandbytes

Standard workstations benefit from **bitsandbytes** which reduces memory requirements by 75% compared to full-precision models, while making complex inference possible without requiring expensive GPUs.

Flash Attention

Flash Attention provides an attention framework optimized for reduced memory utilization while speeding up processing operations.

Installation:

pip install flash-attn

Flash Attention optimizes its core attention operations of queries, keys and values to achieve dramatic performance boosts that benefit, especially long sequence processing thus making it best-suited for extended-context understanding applications.

AutoGPTQ

Through its library implementation, AutoGPTQ enables GPTQ quantization of language models as a method to reduce model memory footprints.

Installation:

pip install auto-gptq

The quantization methods in AutoGPTQ compress weight and activation information while delivering enhanced throughput and lower latency performance on specified hardware setups so users can pick the quantization techniques that match their system needs.

Setting up your environment

Now we will go through the basic steps to set up your machine for working with DeepSeek models. This arrangement allows you to deploy, execute and experiment with neural networks on your own systems. A well-prepared environment lets you download required tools and prepares the system for better inference. No matter if you are adjusting the model or using it somewhere else, this part is important for having an uninterrupted and effective experience.

System requirements

Before proceeding with the setup, ensure that your system meets the minimum requirements for running DeepSeek models, which are as follows:

- **Operating system:** Windows 10/11, macOS 10.15+, or Linux (Ubuntu 20.04+ recommended)
- **CPU:** Modern multi-core processor (8+ cores recommended for CPU inference)
- **RAM:** 16GB minimum, 32GB+ recommended
- **GPU:** NVIDIA GPU with 8GB+ VRAM for smaller models, 24GB+ for larger models
- **Storage:** 20GB+ free space for model weights and dependencies

Your deployment of the DeepSeek variant determines the exact hardware requirements. The execution of distilled models, such as DeepSeek-R1-Distill-1.5B, works optimally on standard hardware, although operating a full-scale DeepSeek-R1-Distill-14B system requires robust system resources.

Setting up a Python environment

Keeping your DeepSeek installation by itself helps prevent issues from other projects. Users have two options for environment setup: conda or venv.

- **Using conda:** Create and activate a dedicated environment, then install the core libraries:

```
conda create -n deepseek python=3.10
conda activate deepseek
```

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu121
```

```
pip install transformers accelerate bitsandbytes
```

- **Using venv:** A good approach for DeepSeek is to use a virtual environment to keep your environment clean and separate. The venv module in Python helps you make a separate area where your app dependencies will not affect the rest of your system. It also stops libraries from conflicting, which helps with easy management of the environment and reproducibility. By setting up DeepSeek in a venv, you make sure it remains easy to use, adapt and upkeep in other projects or environments.

```
python -m venv deepseek-env
```

```
deepseek-env\Scripts\activate
```

```
source deepseek-env/bin/activate
```

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu121
```

```
pip install transformers accelerate bitsandbytes
```

GPU setup for NVIDIA cards

GPU acceleration becomes available when you properly install both the NVIDIA drivers and the CUDA toolkit.

1. **Install NVIDIA Drivers:** Visit NVIDIA's official Driver Downloads page to download the latest drivers while following platform-specific installation steps.
2. **Verify CUDA installation:** Verify GPU detection by PyTorch through this command:

```
python -c "import torch; print(torch.cuda.is_available())"
```
3. **Install CUDA Toolkit (if needed):** Obtain the appropriate CUDA Toolkit from NVIDIA's CUDA Downloads page after running the

check, and install it if the result is False, which indicates your build needs a particular version.

Environment configuration for optimal performance

To achieve maximum performance from DeepSeek models executing on local workstations, proper environmental settings must be implemented with care. The variables for memory allocation, disk caching, and parallelism optimize stable large-model execution performance while reducing latency.

- **Memory management:** The transformer layers in DeepSeek occasionally create GPU memory fragmentation, which produces inefficient memory distribution and unplanned OOMs. The limitation on PyTorch's allocator through large block splitting enhances memory reuse consistency while lowering maximum fragmentation rates.

Here we set PyTorch's CUDA allocator to prevent fragmentation:

```
export
```

```
PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128
```

The allocation request breakdown by PyTorch occurs through 128 MB chunks to minimize memory peaks without affecting model functionality.

- **Disk caching:** Each time a model's weights are downloaded and loaded through the system, it creates excessive I/O bandwidth usage and decreases SSD storage capacity. Direct your Hugging Face cache storage to a dedicated, large platform that functions as a centralized artifact storage solution:

```
export HF_HOME=/path/to/cache/directory
```

By pointing **HF_HOME** to storage with enough capacity, users can store all model pulls tokenizer downloads, and tokenization artifacts within a single unified location instead of spreading them across directories.

- **Parallel processing:** The parallel processing abilities of transformer inference alongside tokenization work well, but spontaneous thread generation can lead to CPU scheduler overload. Set explicit OpenMP thread limitations for optimal concurrency-speed ratio performance.


```
export OMP_NUM_THREADS=8
```

Your application controls CPU usage effectively through eight threads while still benefiting from core multi-threading without causing performance problems.

Troubleshooting common setup issues

All environments with optimal settings eventually experience technical problems. DeepSeek local operation presents common failure points that respond to these tested solutions.

CUDA out of memory errors

Large model instantiation or batched generation can exhaust GPU VRAM. To alleviate this:

- **Use quantization:** Load the model with 8-bit or 4-bit quantization to reduce memory requirements:

```
from transformers import AutoModelForCausalLM,  
BitsAndBytesConfig  
import torch
```

```
quantization_config = BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_compute_dtype=torch.float16  
)  
model = AutoModelForCausalLM.from_pretrained(  
    "deepseek-ai/deepseek-r1-distill-7b",  
    quantization_config=quantization_config,  
    device_map="auto"  
)
```

- **Reduce batch size:** Smaller input batches directly translate to lower memory consumption.

This snippet processes inputs in batches to enhance model throughput.

```
batch_size = 2  
all_outputs = []  
for i in range(0, len(all_inputs), batch_size):
```

```
batch_inputs = all_inputs[i:i+batch_size]
batch_outputs = model.generate(batch_inputs)
all_outputs.extend(batch_outputs)
```

- **Model sharding:** Distribute layers across multiple GPUs by leveraging `device_map="auto"`, allowing each card to host a portion of the network.

Slow inference performance

In case the inference is slow, start by making sure that your system is up-to-date, i.e., install the latest version of PyTorch, CUDA drivers, and compute libraries, since most performance fixes are made in new releases. Use profiling tools, such as `nvidia-smi`, `htop` or PyTorch Profiler, to see what the bottlenecks are, and also to see whether pre-processing/post-processing or I/O was inefficient. These optimizations on the system level usually result in substantial performance gains even before touching your model. These improvements tend to be realized in token sequences, avoiding over-padding and using libraries such as NVIDIA DALI or `lib.jpeg-turbo` to speed up the decoding of images.

If inference is slower than expected:

- **Enable Flash Attention:** Swap to the Flash Attention kernel for accelerated attention passes:

```
model = AutoModelForCausalLM.from_pretrained(
    "deepseek-ai/deepseek-r1-distill-7b",
    torch_dtype=torch.float16,
    attn_implementation="flash_attention_2"
)
```

- **Using VLLM:** The VLLM runtime can streamline throughput under heavy loads:

```
from vllm import LLM
```

```
model = LLM(
    model="deepseek-ai/deepseek-r1-distill-7b",
    dtype="half",
    gpu_memory_utilization=0.9
```

)

- **Batch optimization:** Group multiple prompts into a single inference call to maximize device utilization:

```
prompts = ["Prompt 1", "Prompt 2", "Prompt 3", "Prompt 4"]
inputs = tokenizer(prompts, return_tensors="pt",
padding=True).to("cuda")
outputs = model.generate(**inputs)
```

Dependency conflicts

Having different library versions can make setup difficult and result in strange errors and unclear issues. The best course of action is to set up an environment that is separated and clean so dependencies can be configured with accuracy. It reduces any overlap or technical problems and guarantees each step in DeepSeek runs properly. Reproducibility is easier to maintain and setting things up takes less time with the help of venv, pip and poetry:

- **Use a fresh environment:** Create a new Python environment specifically for DeepSeek.

```
conda create -n deepseek-fresh python=3.10
conda activate deepseek-fresh
```

- **Install dependencies in the correct order:** Some libraries have specific version requirements.

```
pip install torch==2.1.0 torchvision==0.16.0 torchaudio==2.1.0 --
index-url https://download.pytorch.org/whl/cu121
pip install transformers==4.36.0 accelerate==0.25.0
bitsandbytes==0.41.0
```

- **Check compatibility:** Ensure that all libraries are compatible with your Python version and each other.

```
pip check
```

Hello DeepSeek: Your first model

Now that your prepared environment provides the essential tools needed to begin performing your initial DeepSeek experiment. A model selection

process allows you to download and execute text generation operations.

Choosing the right DeepSeek model

The selection of a suitable DeepSeek variant depends on three interdependent elements: The selection of DeepSeek models depends on three primary factors, including hardware capabilities, and generation quality requirements and desired inference speed. Start by examining your GPU memory along with your CPU capabilities to confirm the model will run without overburdening your system resources. The next step involves assessing how deep reasoning and fluent text you need. The generation of more coherent text comes with increased compute requirements when using larger models. Finally, consider responsiveness: Smaller model architectures deliver faster outcomes that work well when users need quick responses.

The following table presents suggested models according to different system configurations:

Hardware configuration	Recommended model	Approximate size
CPU only	DeepSeek-R1-Distill-1.5B	3GB
GPU with 8GB VRAM	DeepSeek-R1-Distill-7B (4-bit)	4GB
GPU with 16GB VRAM	DeepSeek-R1-Distill-7B (8-bit)	7GB
GPU with 24GB+ VRAM	DeepSeek-R1-Distill-14B (8-bit)	14GB
Multiple high-end GPUs	DeepSeek-R1	175GB

Table 5.1 : Recommended DeepSeek models by hardware configuration

We will use DeepSeek-R1-Distill-7B with 4-bit quantization for this walkthrough because it strikes a balance between quality output and intermediate GPU limits.

Downloading and loading the model

DeepSeek models allow multiple interfaces for obtaining and instantiating their models. We shall discuss the three prevalent methods below.

Using Hugging Face Transformers

The transformers library allows maximum flexibility through this method:

```
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
import torch
```

```
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True
)
```

```
model_name = "deepseek-ai/deepseek-r1-distill-7b"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quantization_config,
    device_map="auto",
    torch_dtype=torch.float16
)
```

```
model.save_pretrained("./models/deepseek-r1-distill-7b")
tokenizer.save_pretrained("./models/deepseek-r1-distill-7b")
```

The Hugging Face Hub retrieves the model while performing 4-bit compression to minimize memory usage and performs GPU layer mapping or switches to CPU execution when GPUs are unavailable.

Using Ollama

For a streamlined, CLI-centric workflow, Ollama abstracts away quantization and device mapping:

```
# Pull the model
```

```
ollama pull deepseek-r1-distill-7b
```

```
# Launch an interactive session
```

```
ollama run deepseek-r1-distill-7b
```

Ollama automates download, configuration, and runtime setup, providing immediate interactive access without manual code.

Using LM Studio

For users who prefer a graphical interface, LM Studio offers a user-friendly way to download and run DeepSeek models through the following steps:

1. Open LM Studio.
2. Access Download Models through the sidebar.
3. Search for deepseek-r1-distill-7b.
4. Click the DOWNLOAD button located next to the model entry.
5. Choose your model from the library then select the Load button.
6. You can interact with the model through its embedded chat platform.

Using the user-friendly interface of LM Studio, users can explore the system by hiding complex implementation procedures from view, making it suitable for code-free experimentation.

Running inference with DeepSeek

The conclusion of your DeepSeek model implementation process involves calling the model for text generation. This section examines three interfaces known as Hugging Face Transformers, Ollama, and LM Studio, which we demonstrate through example code and workflow guides.

Using Hugging Face Transformers

Text generation through the Transformers library begins with creating a prompt, which needs conversion into token IDs that the model understands. A simple explanation of reinforcement learning would start as follows:

```
prompt = "Explain the concept of reinforcement learning in simple terms."
```

```
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
outputs = model.generate(
    inputs.input_ids,
    max_new_tokens=512,
    temperature=0.7,
```

```

    top_p=0.95,
    do_sample=True
)

response = tokenizer.decode(
    outputs[0][inputs.input_ids.shape[1]:],
    skip_special_tokens=True
)
print(response)

```

This snippet executes prompt tokenization followed by a request for 512 new tokens, which operate under controlled randomness parameters (**temperature = 0.7**) and diversity settings (nucleus sampling with **top_p = 0.95**). The model generates more elaborate and unpredictable results when **do_sample=True** enables it to draw from its probability distribution instead of picking the most probable token.

Using Ollama

Ollama provides an easy-to-use interface that handles all steps from download through quantization to execution when you need command-line oriented workflows. To enter an interactive session:

ollama run deepseek-r1-distill-7b

You can use Ollama through its local HTTP API by making a programmatic call. Send a **POST** request that contains your prompt alongside generation settings:

```

curl -X POST http://localhost:11434/api/generate -d '{
  "model": "deepseek-r1-distill-7b",
  "prompt": "Explain the concept of reinforcement learning in simple terms.",
  "temperature": 0.7,
  "max_tokens": 512
}'

```

Ollama provides all the underlying complexity management functions for model retrieval and quantization and device mapping to let you work on prompt design and parameter tuning.

Using LM Studio

The graphical user interface of LM Studio lets users access an easy-to-use chat system that meets their needs. You can start typing your prompt into the chat box after making sure the model is present and hit *Enter*. The system shows the response directly on the interface screen. From the settings panel, users can modify temperature parameters alongside top-p and token limits for generation refinement without programming.

Exploring DeepSeek's capabilities

After excelling text generation with DeepSeek, we can review its functional examples. For our examples, we will utilize Hugging Face Transformers, but feel free to implement Ollama or LM Studio if needed.

- **Question answering:** DeepSeek excels at answering questions across a wide range of domains:

```
prompt = "What are the key differences between supervised and  
unsupervised learning?"  
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)  
outputs = model.generate(inputs.input_ids, max_new_tokens=512,  
temperature=0.7)  
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],  
skip_special_tokens=True)  
print(response)
```

- **Mathematical reasoning:** DeepSeek's reasoning capabilities shine in mathematical problems:

```
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)  
outputs = model.generate(inputs.input_ids, max_new_tokens=512,  
temperature=0.3)  
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],  
skip_special_tokens=True)  
print(response)
```

- **Code generation:** DeepSeek can generate code across various programming languages:


```
prompt = "Write a Python function that checks if a string is a  
palindrome."  
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)  
outputs = model.generate(inputs.input_ids, max_new_tokens=512,  
temperature=0.5)  
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],  
skip_special_tokens=True)  
print(response)
```

- **Creative writing:** DeepSeek can also generate creative content:

```
prompt = "Write a short story about a robot that develops  
consciousness."  
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)  
outputs = model.generate(inputs.input_ids, max_new_tokens=1024,  
temperature=0.8, top_p=0.95)  
response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],  
skip_special_tokens=True)  
print(response)
```

Optimizing inference for use case

When you examine DeepSeek's functionalities, you might need to adjust the inference process for your particular use scenario. The following list includes various optimization techniques to evaluate:

Prompt engineering

Your choice of wording in prompts strongly impacts the quality of text generation outputs. Consider these prompt engineering techniques:

- **Be specific:** Explicitly define the task you want the model to execute
- **Provide context:** Include relevant background information
- **Use examples:** Show examples of the desired output while you explain it
- **Structure your prompt:** Your model will benefit from clear sections combined with proper formatting to provide direction.

A more effective structure for this question would be "Explain the basics of reinforcement learning.":

Please explain the concept of reinforcement learning in simple terms.

Include:

1. A basic definition
2. How it differs from supervised learning
3. A simple real-world example
4. Common applications

Parameter tuning

Modifying generation parameters helps users strike the right combination of creativity and coherence in their text output.

- **Temperature:** The selection of low values (0.3) generates precise and predictable outputs, yet high values (0.8) produce imaginative and varied outputs.
- **Top-p (nucleus sampling):** The generation control mechanism selects the most likely words from a pool where the total probability surpasses the specified threshold.
- **Max new tokens:** The max new tokens parameter functions to set an upper limit on text length, which might aid in restricting response length.
- **Repetition penalty:** The model avoids using repetitive sequences when this parameter is applied.

The application of these parameters looks like the following:

```
outputs = model.generate(  
    inputs.input_ids,  
    max_new_tokens=512,  
    temperature=0.7,  
    top_p=0.95,  
    repetition_penalty=1.1,  
    do_sample=True  
)
```

Batch processing

Batch processing aids applications that study many inputs by letting them run much faster. Bringing multiple inputs into a single inference step means less memory is used and the GPU gets used more fully. The result is higher efficiency, mainly at places where a lot of requests need to be processed in a fast manner. In addition, incorporating batches means overhead charges such as tokenization and I/O, can be shared by many different tasks and thus be more affordable overall. When used well, batch processing improves both the time and resources needed for running a local LLM.

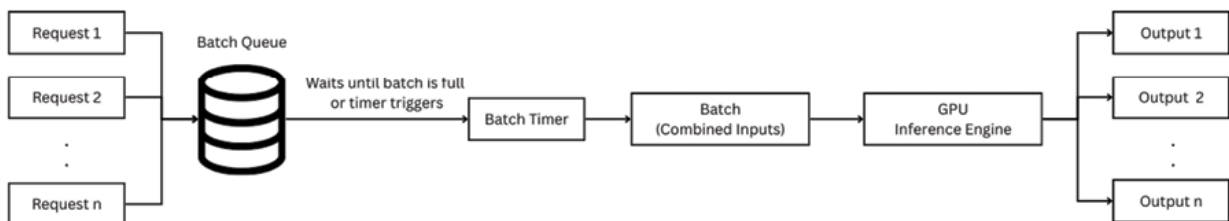


Figure 5.1: Data batches accumulate to optimize output efficiency

Here is an example code for batch processing for throughput optimization with delayed execution:

```

prompts = [
    "What is artificial intelligence?",
    "Explain machine learning in simple terms.",
    "How does deep learning work?",
    "What is natural language processing?"
]
inputs = tokenizer(prompts, return_tensors="pt",
padding=True).to(model.device)

outputs = model.generate(
    inputs.input_ids,
    attention_mask=inputs.attention_mask,
    max_new_tokens=256,
    temperature=0.7,
    do_sample=True
)
  
```

```

for i, output in enumerate(outputs):
    response = tokenizer.decode(output[inputs.input_ids[i].shape[0]:],
skip_special_tokens=True)
    print(f"Prompt: {prompts[i]}")
    print(f"Response: {response}\n")

```

Streaming generation

The streaming generation has simplified real-time text generation, and thus programs feel interaction-based and quick. Tokens (generated by the model) will be received by the client immediately, instead of having to wait until the complete response is sent when they become enabled, which is most often done via **server-sent events (SSE)** or web-sockets. It also makes it interactive making the user want to rerun prompts or redefine the queries dynamically. Streaming has to be implemented by syncing backend production with frontend presentation, typically over SSE, where each token slice can reach the frontend in a seamless manner as it is produced.

```

from transformers import TextIteratorStreamer
from threading import Thread

prompt = "Write a short story about a robot that develops consciousness."
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

```

```

streamer = TextIteratorStreamer(tokenizer, skip_special_tokens=True)

```

```

generation_kwargs = {
    "input_ids": inputs.input_ids,
    "max_new_tokens": 1024,
    "temperature": 0.8,
    "top_p": 0.95,
    "streamer": streamer
}
thread = Thread(target=model.generate, kwargs=generation_kwargs)
thread.start()

```

```

for text in streamer:

```

```
print(text, end="", flush=True)
```

Building a simple chat application

Let us develop a basic chat program that connects to a DeepSeek model after summarizing our acquired knowledge. The implementation employs Hugging Face Transformers together with a command-line interface for this demonstration.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig

model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quantization_config,
    device_map="auto",
    torch_dtype=torch.float16
)

def generate_response(prompt, conversation_history=""):
    if conversation_history:
        full_prompt = f"{conversation_history}\nUser: {prompt}\nAssistant:"
    else:
        full_prompt = f"User: {prompt}\nAssistant:"
```

```

    inputs = tokenizer(full_prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        inputs.input_ids,
        max_new_tokens=512,
        temperature=0.7,
        top_p=0.95,
        do_sample=True
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    updated_history = f"{full_prompt} {response}"
    return response, updated_history

def chat():
    print("DeepSeek Chat (type 'exit' to quit)")
    conversation_history = ""
    while True:
        user_input = input("\nYou: ")

        if user_input.lower() in ["exit", "quit", "bye"]:
            print("Goodbye!")
            break

        response, conversation_history = generate_response(user_input,
conversation_history)
        print(f"\nDeepSeek: {response}")

if __name__ == "__main__":
    chat()

```

The basic software application stores previous interactions as a conversation history to deliver better coherent multi-turn responses. The base example can become more advanced through response streaming and parameter control or graphical interface features to develop a complex application.

Conclusion

This chapter's end marks your successful development of a strong local framework to work with DeepSeek models. The essential frameworks and libraries that support local LLM deployments are now installed and configured while specialized tools for model interaction and optimization libraries for standard hardware performance have been implemented.

Your first DeepSeek model execution through Hugging Face Transformers and Ollama and LM Studio showcases its text generation and query answering functions while building a simple chat application. The hands-on exercises establish fundamental operational capabilities that will support future work.

The future environment, alongside your configured tools, will enable you to tackle more sophisticated concepts. The upcoming chapter "supervised fine-tuning (SFT)" will detail the techniques for adjusting DeepSeek models to handle particular tasks across different domains. The tutorial will introduce parameter-efficient techniques, including LoRA and QLoRA to help you optimize model refinement processes on consumer-grade hardware. The acquired skills allow you to adapt DeepSeek models exactly how your project needs them.

The next chapter presents a detailed explanation of supervised fine-tuning principles alongside its workflow. The training process requires data preparation alongside objective configuration for labeled datasets. You will discover assessment techniques that can be used for model performance evaluation from training until fine-tuning completion. You will explore LoRA and QLoRA as parameter-efficient methods to generate high-quality task-specific refinements that run effectively on consumer-grade hardware.

Points to remember

- Setting up a proper environment is crucial for effectively working with DeepSeek models locally, ensuring optimal performance and a smooth development experience.
- Core frameworks like PyTorch and Hugging Face Transformers provide the foundation for working with LLMs, while specialized tools like Ollama and LM Studio offer simplified interfaces for deployment and

interaction.

- Optimization libraries like bitsandbytes, Flash Attention, and AutoGPTQ enable efficient operation of DeepSeek models on consumer hardware through techniques like quantization and optimized attention computation.
- When selecting a DeepSeek model, consider your hardware constraints, performance requirements, and inference speed needs to choose the most appropriate model for your use case.
- Quantization techniques like 4-bit and 8-bit quantization can significantly reduce memory requirements, making it possible to run larger models on consumer hardware with minimal impact on performance.
- Prompt engineering, parameter tuning, batch processing, and streaming generation are key techniques for optimizing inference for specific use cases and improving the user experience.
- Building a simple chat application with DeepSeek involves maintaining conversation history, generating responses based on user input, and handling multi-turn conversations effectively.
- Troubleshooting common issues like CUDA out of memory errors, slow inference performance, and dependency conflicts requires understanding the underlying causes and applying appropriate solutions.

Key terms

- **PyTorch:** An open-source machine learning framework that provides the computational backbone for most LLM operations.
- **Hugging Face Transformers:** A library that provides thousands of pretrained models, including the DeepSeek family, along with APIs for downloading, loading, and using these models.
- **Quantization:** The process of reducing the precision of model weights to decrease memory requirements and increase inference speed.
- **VLLM:** A high-throughput and memory-efficient inference and serving engine for LLMs that implements optimizations like PagedAttention.

- **Ollama:** A tool that simplifies running LLMs locally by providing a straightforward way to download, run, and interact with models.
- **LM Studio:** A desktop application that provides a graphical interface for downloading, running, and interacting with LLMs.
- **Flash Attention:** An optimized attention implementation that significantly reduces memory usage and increases computation speed.
- **Prompt engineering:** The practice of designing and optimizing input prompts to elicit the desired behavior from language models.
- **Temperature:** A parameter that controls the randomness of text generation, with higher values producing more diverse outputs.
- **Batch processing:** The technique of processing multiple inputs simultaneously to improve throughput and efficiency.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 6

Supervised Fine-tuning

Introduction

In the previous chapters, we looked at how DeepSeek works, different ways to use the models, and got our computer ready to try out these models on our own. While pre-trained models like DeepSeek can do a lot right away, they might not always fit perfectly with what you need or what your field is focused on. This is where you work on making small changes to your code so it fits the actual data better. Supervised fine-tuning is a useful approach that lets you change pre-trained language models so they can handle different tasks or styles by training them on carefully chosen data. Through SFT, you can make a model do better on specific tasks, help it learn about specific topics, or change the way its results look or sound. The process of fine-tuning has traditionally taken a lot of resources, needing a lot of computer processing power and a large amount of training data. However, new ways of fine-tuning language models with fewer training parameters have made it much easier to use big models like DeepSeek on regular computers, even when datasets are small.

In this chapter, we will look at the basics of supervised fine-tuning, talking about how it works, the different ways to use it, and some things you can do with it. We will also look at parameter-efficient methods like *Low-Rank Adaptation* and *Quantized Low-Rank Adaptation*, which have made fine-

tuning easier and less resource-intensive by letting us use less data and computing power. By the end of this chapter, you will know how to tweak DeepSeek models to fit your needs better, which will give you more ways to use these useful AI tools in your own projects.

Structure

In this chapter, we will explore the following areas:

- Understanding supervised fine-tuning
- Parameter-efficient techniques
- Comparing fine-tuning approaches
- Best practices for parameter-efficient fine-tuning
- Merging LoRA adapters with base models
- Advanced techniques and future directions

Objectives

By the end of this chapter, you will have gained a comprehensive understanding of supervised fine-tuning and its application to DeepSeek models. You will be able to decide when it is best to fine-tune your model and be aware of what you give up or gain by choosing different methods of fine-tuning.

You will be introduced to parameter-efficient fine-tuning methods such as LoRA and QLoRA and learn how they help with making the process of fine-tuning both simpler and faster. With this background, you will be able to choose the fine-tuning technique that suits your situation and available resources the best.

Besides, you will learn how to set up data for fine-tuning and how to carry out, manage, and assess the models that result from the process. Using these skills, you will be able to fine-tune DeepSeek models to suit your applications and make them more relevant for what you need.

Understanding supervised fine-tuning

Supervised fine-tuning (SFT) is a process that builds upon the foundation of pre-trained language models by further training them on task-specific labeled data. This approach leverages the general knowledge and capabilities acquired during pre-training while adapting the model to excel at particular tasks or domains.

The fine-tuning paradigm

To understand fine-tuning, it is helpful to consider the broader context of how large language models like DeepSeek are developed:

- **Pre-training:** The model is initially trained on vast amounts of general text data (often hundreds of billions of tokens) to learn language patterns, factual knowledge, and basic reasoning abilities. This phase is computationally intensive and typically requires specialized infrastructure.
- **Supervised fine-tuning (SFT):** The pre-trained model is further trained on a smaller, curated dataset of examples that demonstrate the desired behavior for specific tasks. This phase adapts the model's existing capabilities to better align with targeted applications.
- **Reinforcement learning from human feedback (RLHF):** In some cases, the model undergoes additional training using reinforcement learning techniques to better align with human preferences and values. This phase is beyond the scope of this chapter but will be covered in [Chapter 7, Reinforcement Learning from Human Feedback](#).

Fine-tuning focuses on the second phase of this process, taking a pre-trained model and adapting it to better serve specific needs.

Knowing when to use fine-tuning

If a **large language model (LLM)** needs to be fine-tuned, it can be changed to meet certain needs. While a pre-trained model can understand a wide variety of texts, fine-tuning allows it to work better for the information you input.

Fine-tuning is particularly valuable in several scenarios:

- **Domain adaptation:** To perform domain adaptation, an LLM is adjusted to use in medical, legal, or financial situations. In many cases, these types of data are not well covered in standard training sets by their vocabulary and formal settings. Using data that match the domain gives the model an improved understanding of the language and topics relevant to its use.
- **Task specialization:** The process of task specialization looks to maximize the model's performance on tasks such as summarization, question answering, or code generation. By working on data meant for a specific task, the model becomes better at these functions than when it is used for normal purposes.
- **Style alignment:** You can use this method when you need the model to generate output that aligns with your organizational image or system guidelines. Providing examples of the appropriate style allows the model to generate responses that stay true to that style regardless of the input. Just a quick reminder: make sure your answers use only the language and vocabulary outlined. And remember to use any necessary modifiers when responding to an inquiry.
- **Knowledge integration:** Knowledge integration enables the inclusion of important and recent information that was not included in the training data. The model adapts to new information by training on curated datasets and can generate accurate and up-to-date answers in its chosen area of expertise.
- **Instruction following:** Instruction Following improves the model's skill in interpreting and following certain guidelines. Having instruction and result pairs in the data makes the model respond to users as accurately as possible.

The fine-tuning process

Usually, supervised fine-tuning includes a series of important steps, and they are as follows:

Dataset preparation

High-quality and well-mixed fine-tuning data produces better results. An organized dataset should be:

- Cover all the types of inputs and outputs you will encounter in the target application.
- Keep high standards and make sure the examples are accurate and properly presented.
- Include several types of examples in your dataset to avoid your model relying on a single type of pattern.
- Ensure that each group of cases is represented fairly.

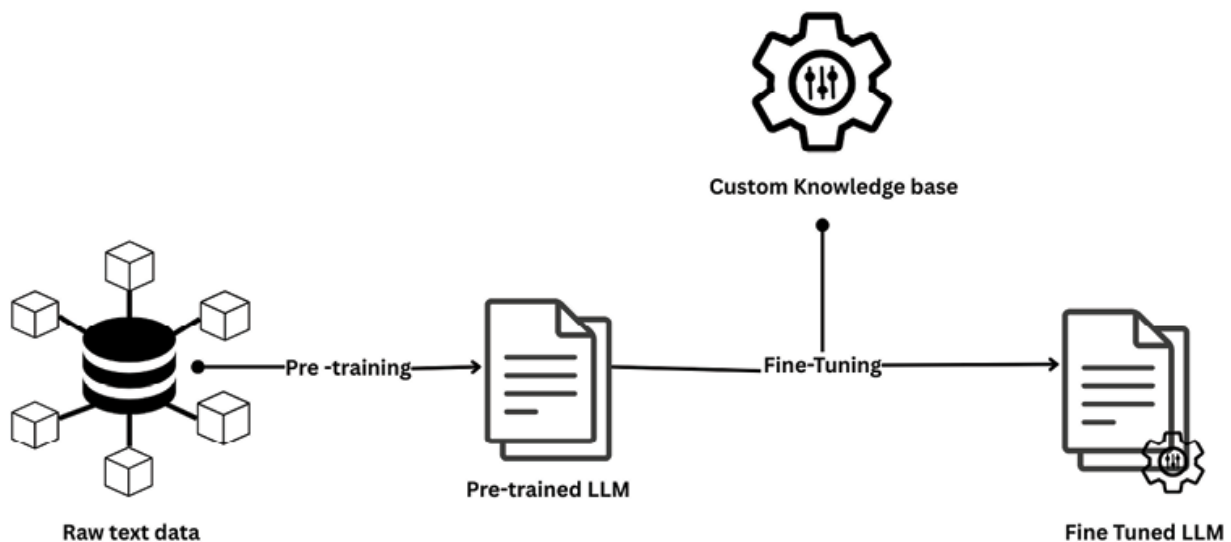


Figure 6.1: Fine-tuning models using labeled data

Typically, the information used for training DeepSeek models includes pairs of prompts and their corresponding correct answers. The structure of this kind of data could look like this:

```
[
  {
    "instruction": "Summarize the following medical research paper abstract.",
    "input": "Recent advances in mRNA vaccine technology have shown promising results in preventing infectious diseases. This study examines the efficacy and safety profile of a novel mRNA vaccine platform in a randomized controlled trial involving 10,000 participants across multiple
```

centers...",

"output": "A large-scale multi-center study (n=10,000) evaluated a new mRNA vaccine platform, demonstrating both safety and efficacy in preventing infectious diseases. The research highlights the potential of mRNA technology as a versatile vaccine development approach."

},

{

"instruction": "Explain the symptoms and treatment options for hypertension in simple terms.",

"input": "",

"output": "Hypertension, or high blood pressure, often has no noticeable symptoms, which is why it's sometimes called the 'silent killer.' When symptoms do occur, they may include headaches, shortness of breath, nosebleeds, and dizziness.\n\nTreatment options include:\n1. Lifestyle changes: Regular exercise, reducing salt intake, maintaining a healthy weight, limiting alcohol, and quitting smoking.\n2. Medications: Various types including diuretics, ACE inhibitors, ARBs, calcium channel blockers, and beta-blockers.\n3. Regular monitoring: Checking blood pressure regularly to ensure treatments are working.\n\nIt's important to work with healthcare providers to find the right treatment approach, as managing hypertension effectively reduces the risk of serious complications like heart attack, stroke, and kidney damage."

}

]

The dataset format may vary depending on the specific fine-tuning framework you are using, but the core elements remain consistent: providing examples of inputs and the corresponding desired outputs.

Model selection

To begin fine-tuning, you must first pick the correct base model. Here are some members of the DeepSeek family:

- **DeepSeek-R1-Distill models:** DeepSeek-R1-Distill models can be used for fine-tuning since they are smaller (between 1.5B and 70B parameters), yet they keep the ability to reason.
- **DeepSeek-Coder:** DeepSeek-Coder makes it easy to handle tasks in

programming and software development.

- **DeepSeek-V3:** This model is meant for general use and uses a MoE design but may require even more resources to fine-tune properly.

How you choose a ML approach is outlined by your needs, available computing resources and what you are trying to accomplish.

Hyperparameter selection

Ensure that you pick appropriate hyperparameters for a large language model, as it is important to preserve its main functions. The results achieved during training and the performance of the model on new information depend on the set of hyperparameters. How the weights are adjusted during training depends on the learning rate given to the model. Typically, the value for the learning rate during fine-tuning is lower than that used during pre-training and falls in the range of $1e-5$ to $5e-5$. After following this strategy, the model is more likely to notice crucial information during further learning.

Training execution

While training, the model is adjusted to lower the gap between its output and the desired outcomes in your data. Usually, it means:

- **Forward pass:** The model takes in examples and predicts the classes for them.
- **Loss calculation:** The difference between predicted and actual values is measured during loss calculation.
- **Backward pass:** During the backward pass, gradients are made to help adjust the weights in the model.
- **Parameter update:** The model's weights are adjusted to decrease the loss.

This procedure is performed again and again on the data until the model works well enough.

Evaluation

After fine-tuning, it is crucial to evaluate the model's performance to ensure

it has improved on the target task without losing its general capabilities:

- **Task-specific metrics:** Measure performance on the specific task (e.g., accuracy, F1 score, BLEU score).
- **General capability assessment:** Check the general ability of the model to ensure it is still effective.
- **Qualitative evaluation:** Check the model results manually to determine if they meet the requirements.
- **Comparison with baseline:** Examine how the model is performing against the model it was based on.

Fine-tuning DeepSeek models

Let us explore how to fine-tune a DeepSeek model using the Hugging Face Transformers library. We will use a simplified example code to illustrate the process:

```
import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling
)

# Load model and tokenizer
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load and prepare dataset
```

```
dataset = load_dataset("json", data_files="medical_qa_dataset.json")
```

```
# Define a formatting function for the dataset
```

```
def format_instruction(example):
```

```
    if example["input"]:
```

```
        text = f"### Instruction:\n{example['instruction']}\n\n###
```

```
Input:\n{example['input']}\n\n### Response:\n{example['output']}"
```

```
    else:
```

```
        text = f"### Instruction:\n{example['instruction']}\n\n###
```

```
Response:\n{example['output']}"
```

```
    return {"text": text}
```

```
# Apply formatting and tokenization
```

```
tokenized_dataset = dataset.map(
```

```
    lambda examples: tokenizer(
```

```
        [format_instruction(ex) for ex in examples],
```

```
        truncation=True,
```

```
        max_length=2048
```

```
    ),
```

```
    batched=True,
```

```
    remove_columns=dataset["train"].column_names
```

```
)
```

```
# Define training arguments
```

```
training_args = TrainingArguments(
```

```
    output_dir="./deepseek-medical-qa",
```

```
    per_device_train_batch_size=4,
```

```
    gradient_accumulation_steps=4,
```

```
    learning_rate=2e-5,
```

```
    num_train_epochs=3,
```

```
    weight_decay=0.01,
```

```
    save_strategy="epoch",
```

```
    fp16=True,
```

```
)
```

```
# Create data collator
```

```
data_collator = DataCollatorForLanguageModeling(  
    tokenizer=tokenizer,  
    mlm=False  
)
```

```
# Initialize trainer  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_dataset["train"],  
    data_collator=data_collator,  
)
```

```
# Start fine-tuning  
trainer.train()
```

```
# Save the fine-tuned model  
model.save_pretrained("./deepseek-medical-qa-finetuned")  
tokenizer.save_pretrained("./deepseek-medical-qa-finetuned")
```

This example demonstrates a basic fine-tuning process for a DeepSeek model on a medical question-answering dataset. However, this approach requires significant computational resources, especially for larger models. This is where parameter-efficient fine-tuning techniques become valuable.

Challenges in traditional fine-tuning

Traditional fine-tuning of large language models demands substantial computational resources, often beyond the reach of individual researchers or small teams. The memory overhead incurred by storing gradients for every model parameter can quickly surpass the capacity of consumer-grade GPUs, leading to frequent out-of-memory failures. Moreover, aggressive fine-tuning may induce catastrophic forgetting, whereby the model loses its general-purpose capabilities as it over-specializes on the new dataset. With limited fine-tuning data, the risk of overfitting intensifies, causing the model to memorize training examples instead of learning broadly applicable patterns. The cumulative compute and memory costs often preclude iterative experimentation, hindering rapid model development cycles. Advances in

distributed training and memory optimization techniques can mitigate some challenges but often require advanced infrastructure and expertise. In addition to these computational and memory constraints, traditional fine-tuning necessitates storing a complete copy of all model parameters for each specialized version, resulting in significant storage inefficiencies when maintaining multiple task-specific models. This heavy storage footprint can strain version control and deployment pipelines, complicating model management. The complexity of these resource demands underscores the importance of exploring more efficient adaptation strategies. These challenges have driven the development of parameter-efficient fine-tuning methods—such as **Low-Rank Adaptation (LoRA)** and adapter-based techniques—which drastically reduce computational and storage burdens by updating only a small subset of parameters. By doing this such approaches maintain the main capabilities of the model and require less effort to specialize it.

Parameter-efficient techniques

Parameter-efficient fine-tuning techniques address the challenges of traditional fine-tuning by updating only a small subset of parameters or introducing a limited number of new parameters. These approaches significantly reduce computational and memory requirements while maintaining effectiveness.

Low-Rank Adaptation

Low-Rank Adaptation (LoRA) is a technique that freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture. This approach dramatically reduces the number of trainable parameters while allowing the model to adapt to new tasks effectively.

Learning how LoRA works

The key insight behind LoRA is that the updates to the weights during fine-tuning can be approximated using low-rank decomposition. Instead of directly updating the original weight matrix, $W \in R^{d \times k}$, LoRA introduces two

smaller matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where $r \ll \min(d, k)$ is the rank of the decomposition.

During forward passes, the original operation $h = Wx$ is replaced with:

$$h = Wx + \Delta Wx = Wx + BAx$$

Where:

- W is the frozen pre-trained weight matrix
- $\Delta W = BA$ is the update to the weights
- x is the input to the layer
- h is the output of the layer

The matrices A and B are initialized such that BA is initially zero, ensuring that the model's behavior is unchanged at the beginning of fine-tuning. During training, only A and B are updated, while W remains frozen.

Advantages of LoRA

LoRA offers several significant advantages:

- **Parameter efficiency:** By training only the Low-Rank Adaptation matrices, LoRA reduces the number of trainable parameters by orders of magnitude. As a result, instead of training all 1,048,576 parameters in W , LoRA trains only 16,384 of them.
- **Memory efficiency:** Since only LoRA parameters need gradients, the training process requires less memory.
- **Modularity:** Since there are less parameters, the process of updating them is quicker and training demands less computer resources.
- **Mix and match:** Thanks to modularity, different task-specific adaptations made for LoRA can be added or replaced without much effort.

LoRA ensures that the model does not lose its general abilities since the original weights are not changed.

Implementing LoRA for DeepSeek models

Let us explore how to implement LoRA fine-tuning for a DeepSeek model using the **parameter-efficient fine-tuning (PEFT)** library from Hugging

Face:

```
import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    DataCollatorForLanguageModeling
)
from peft import (
    LoraConfig,
    get_peft_model,
    prepare_model_for_kbit_training,
    TaskType
)

# Load model and tokenizer
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Configure LoRA
lora_config = LoraConfig(
    r=16, # Rank of the update matrices
    lora_alpha=32, # Scaling factor for the update
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj",
"up_proj", "down_proj"],
    lora_dropout=0.05, # Dropout probability for LoRA layers
    bias="none", # Whether to train bias parameters
    task_type=TaskType.CAUSAL_LM # Task type
)
```

```

# Prepare model for LoRA fine-tuning
model = get_peft_model(model, lora_config)
model.print_trainable_parameters() # Print the percentage of trainable
parameters

# Load and prepare dataset (same as in the previous example)
dataset = load_dataset("json", data_files="medical_qa_dataset.json")

def format_instruction(example):
    if example["input"]:
        text = f"### Instruction:\n{example['instruction']}\n\n###
Input:\n{example['input']}\n\n### Response:\n{example['output']}"
    else:
        text = f"### Instruction:\n{example['instruction']}\n\n###
Response:\n{example['output']}"
    return {"text": text}

tokenized_dataset = dataset.map(
    lambda examples: tokenizer(
        [format_instruction(ex) for ex in examples],
        truncation=True,
        max_length=2048
    ),
    batched=True,
    remove_columns=dataset["train"].column_names
)

# Define training arguments
training_args = TrainingArguments(
    output_dir="./deepseek-medical-qa-lora",
    per_device_train_batch_size=8, # Can use larger batch sizes with LoRA
    gradient_accumulation_steps=4,
    learning_rate=3e-4, # Can use higher learning rates with LoRA
    num_train_epochs=3,
    weight_decay=0.01,

```

```

    save_strategy="epoch",
    fp16=True,
)

# Create data collator
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False
)

# Initialize trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    data_collator=data_collator,
)

# Start fine-tuning
trainer.train()

# Save the LoRA adapter
model.save_pretrained("./deepseek-medical-qa-lora-adapter")

```

This example demonstrates how to fine-tune a DeepSeek model using LoRA.

Note the significant differences from traditional fine-tuning:

- We apply LoRA to specific target modules (attention and feed-forward layers).
- We can use larger batch sizes and higher learning rates due to the reduced parameter count.
- We only save the LoRA adapter weights, which are much smaller than the full model.

Target modules for DeepSeek models

When applying LoRA to DeepSeek models, it is important to target the

appropriate modules. The typical target modules for transformer-based models like DeepSeek include:

- **q_proj, k_proj, v_proj, o_proj:** The query, key, value, and output projection matrices in the attention mechanism.
- **gate_proj, up_proj, down_proj:** The projection matrices in the feed-forward network.

These modules contain the majority of the parameters in the model and are most influential in adapting the model's behavior to new tasks.

Quantized Low-Rank Adaptation

QLoRA simplifies the process of getting LLMs to work effectively, especially when less data is available. With the help of quantization and low-rank methods, large models using QLoRA can be fine-tuned without high-caliber hardware.

Learning how QLoRA works

Discover how QLoRA fine-tunes giant LLMs through 4-bit quantization along with low-rank adapters using their combination to learn how to use consumer-level GPUs that can manage to be productive at a small scale:

- **4-bit quantization:** Instead of using the same weights, QLoRA applies 4-bit quantization to shrink the size of its weights. Thus, the learned parameters become less detailed, so you require less space to store them. By quantizing the quantization constants, QLoRA helps improve memory. The new design makes the model smaller and more efficient.
- **Double quantization:** Page-based optimizers are used by QLoRA during its training to manage the optimizer states. Optimizers now store their status outside RAM, which makes it easier to avoid stopping the model's training. The technique is also applied to the 4-bit quantized model, using NF4 as the data type. Since NF4 has normal-distribution weights, the model remains accurate even after being quantized.
- **LoRA on NF4:** As soon as the model uses a forward pass, LoRA retrieves the real-valued weights from the quantized ones. It decreases the number of variables and uses less RAM in the system. By using QLoRA, language models can now be used on smaller computers,

letting more people enjoy the technology.

Advantages of QLoRA

Learn how QLoRA can combine quantization and low-rank adapters to fine-tune (on smaller hardware) with memory savings, but with full-precision performance:

- QLoRA introduces a new technique for training LLMs that uses less memory without affecting their performance. Reducing the size of base model weights from 16 to 4 bits, QLoRA needs less memory. Thanks to needing less memory, models with billions of parameters can now be trained on standard GPUs.
- Although there is a loss in precision, QLoRA produces results similar to those from regular fine-tuning. When measured with real data, it is clear that models based on QLoRA are comparable to the 16-bit models. Since the model is not big, it can be used and managed effectively with minimal effort.

Implementing QLoRA for DeepSeek models

Let us explore how to implement QLoRA fine-tuning for a DeepSeek model with an example code:

```
import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    BitsAndBytesConfig,
    DataCollatorForLanguageModeling
)
from peft import (
    LoraConfig,
    get_peft_model,
    prepare_model_for_kbit_training,
```

```

    TaskType
)

# Configure 4-bit quantization
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16
)

# Load model and tokenizer with quantization
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto"
)

# Prepare model for QLoRA fine-tuning
model = prepare_model_for_kbit_training(model)

# Configure LoRA
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj",
"up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)

# Apply LoRA to the quantized model
model = get_peft_model(model, lora_config)

```

```

model.print_trainable_parameters()

# Load and prepare dataset (same as in previous examples)
dataset = load_dataset("json", data_files="medical_qa_dataset.json")

def format_instruction(example):
    if example["input"]:
        text = f"### Instruction:\n{example['instruction']}\n\n###\nInput:\n{example['input']}\n\n### Response:\n{example['output']}"
    else:
        text = f"### Instruction:\n{example['instruction']}\n\n###\nResponse:\n{example['output']}"
    return {"text": text}

tokenized_dataset = dataset.map(
    lambda examples: tokenizer(
        [format_instruction(ex) for ex in examples],
        truncation=True,
        max_length=2048
    ),
    batched=True,
    remove_columns=dataset["train"].column_names
)

# Define training arguments
training_args = TrainingArguments(
    output_dir="./deepseek-medical-qa-qlora",
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,
    learning_rate=3e-4,
    num_train_epochs=3,
    weight_decay=0.01,
    save_strategy="epoch",
    fp16=True,
)

```

```
# Create data collator
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False
)
```

```
# Initialize trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    data_collator=data_collator,
)
```

```
# Start fine-tuning
trainer.train()
```

```
# Save the QLoRA adapter
model.save_pretrained("./deepseek-medical-qa-qlora-adapter")
```

The key differences in this QLoRA implementation compared to standard LoRA are:

- We use **BitsAndBytesConfig** to configure 4-bit quantization with double quantization and the NF4 data type.
- We call **prepare_model_for_kbit_training** to prepare the quantized model for training.
- The base model is loaded in 4-bit precision, dramatically reducing memory usage.

With QLoRA, you can fine-tune models that would otherwise be too large for your hardware. For example, you might be able to fine-tune DeepSeek-R1-Distill-14B on a consumer GPU with 24GB of VRAM, which would be impossible with traditional fine-tuning.

Comparing fine-tuning approaches

To help you choose the most appropriate fine-tuning approach for your specific needs, let us compare traditional fine-tuning, LoRA, and QLoRA across several dimensions:

Aspect	Traditional fine-tuning	LoRA	QLoRA
Trainable parameters	All model parameters	Only LoRA matrices (0.1-1% of total)	Only LoRA matrices (0.1-1% of total)
Memory requirements	Very high (full model in FP16/FP32)	Moderate (full model in FP16)	Low (full model in 4-bit)
GPU requirements	High-end GPUs or multiple GPUs	Mid-range to high-end GPU	Consumer GPU
Training speed	Slow	Faster	Faster
Performance	Excellent	Very good to excellent	Very good
Storage efficiency	Poor (full model copy)	Excellent (small adapter)	Excellent (small adapter)
Flexibility	Limited (one task per model)	High (swap adapters for different tasks)	High (swap adapters for different tasks)
Suitable model sizes	Small to medium	Medium to large	Large to very large
Example DeepSeek models	DeepSeek-R1-Distill-1.5B	DeepSeek-R1-Distill-7B, DeepSeek-R1-Distill-14B	DeepSeek-R1-Distill-14B, DeepSeek-R1-Distill-70B

Table 6.1 : Different fine tuning approaches

This comparison highlights the trade-offs between different approaches. In general:

- **Traditional fine-tuning** is suitable when you have substantial computational resources and want to maximize performance on a specific task.
- **LoRA** offers an excellent balance of efficiency and performance, making it suitable for most fine-tuning scenarios.
- **QLoRA** is ideal when you want to fine-tune larger models on limited hardware or maximize efficiency.

Best practices for parameter-efficient fine-tuning

To get the most out of parameter-efficient fine-tuning techniques, consider the following best practices:

- **Dataset preparation:** You need a reliable data set to do well in **parameter-efficient fine-tuning (PEFT)**. It is better to have only a small group of relevant examples than a huge group of irrelevant ones, because even a few hundred ideas can come in handy. Include both common and rare circumstances in the data used to train the model. When the steps are followed for every sample and they are all organized the same, the model learns to show the results.
- **Hyperparameter selection:**
 - **Rank (r):** Start with $r=16$ or $r=32$. Higher ranks can capture more complex adaptations but require more memory and may lead to overfitting on small datasets.
 - **Alpha (α):** Typically set to $2 \times r$ for stable training.
 - **Learning rate:** Parameter-efficient methods often benefit from higher learning rates ($1e-4$ to $5e-4$), compared to full fine-tuning.
 - **Target modules:** Include all attention and feed-forward projection layers for comprehensive adaptation.
- **Training process:** This technique saves on hardware and leads to efficient convergence. If the GPU lacks memory, running multiple forward and backward passes saves up the gradients and allows the optimizer to train a bigger batch size. Initially, use a small learning rate and slowly increase it. This way, the process will not become unstable. By regularly checkpointing the weights, the model can pick up where it left off after an interruption.
- **Evaluation and deployment:** When you use PEFT, you need to carry out assessment and launch. All through the process, the team tests the adapted model so it can fit seamlessly into the production system. Before reaching any conclusions, you should examine the model's precision on the main problem and on common benchmarks. By evaluating in this manner, you can see if the most important skills of the language model have changed as a result of the fine-tuning. This approach helps the model to address various different circumstances.

Managing adapters correctly helps ensure that PEFT works well. When your adapters are arranged systematically, you can use them for different tasks. If you arrange your adapters and their documentation, you will have an easier time repeating your studies and making progress in the field. Combining the base model and adaptor is a good way to implement the approach. Using the full model allows us to make inferences in both simple and efficient ways. Yet, keeping many merged models ensures greater flexibility for different applications. Remember to take the environment the system will be deployed in into account and try to make it easy to manage and adapt.

Merging LoRA adapters with base models

After fine-tuning with LoRA or QLoRA, you have two options for deployment:

- **Keep the adapter separate:** This approach maintains flexibility but requires additional computation during inference to apply the adapter.
- **Merge the adapter with the base model:** This approach simplifies deployment but results in a new full-sized model.

Here is a code to show how to merge a LoRA adapter with a base model:

```
from peft import PeftModel, PeftConfig
from transformers import AutoModelForCausalLM, AutoTokenizer

# Load the base model
base_model_name = "deepseek-ai/deepseek-r1-distill-7b"
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained(base_model_name)

# Load the LoRA configuration and model
peft_model_path = "./deepseek-medical-qa-lora-adapter"
```



```
config = PeftConfig.from_pretrained(peft_model_path)
peft_model = PeftModel.from_pretrained(base_model, peft_model_path)
```

```
# Merge the LoRA weights with the base model
merged_model = peft_model.merge_and_unload()
```

```
# Save the merged model
merged_model.save_pretrained("./deepseek-medical-qa-merged")
tokenizer.save_pretrained("./deepseek-medical-qa-merged")
```

The merged model contains the combined weights of the base model and the LoRA adapter, resulting in a model that behaves like a fully fine-tuned model but without requiring the adapter during inference.

Advanced techniques and future directions

A lot of work is being done in parameter-efficient fine-tuning, giving rise to new approaches frequently. There are advanced methods and future ways of thinking that are being explored:

- **AdapterFusion:** With AdapterFusion, various task-specific adapters can be brought together, and the model can take in information from each. Since learning is the same for all tasks, additional training is not required.
- **Prefix-tuning:** Using prefix-tuning, the model is allowed to adjust to new tasks without changing any of the original parameters. It gives a new option that matches LoRA in how efficiency.
- **Prompt tuning:** Only a few parameters that are considered important or related to the target task are changed through sparse fine-tuning. This allows for improved parameter efficiency by making changes to the original base model.
- **Sparse fine-tuning:** You can continue training your model whenever new examples become available. Some innovations in this area involve changing or fine-tuning the model for various challenges, making sure it retains previous knowledge. These techniques are effective for designing models useful for a variety of tasks.

- **Multi-task and continual learning:** Newer techniques focus on making models better for various tasks, all while keeping in mind the ones they learned earlier. They are most effective at creating models that work well in a wide variety of situations.

Conclusion

This chapter explored using SFT in the DeepSeek framework. In addition, we tried fine-tuning with all parameters, and found that using LoRA and QLoRA improved performance and needed less memory and system resources. Therefore, using high-quality hardware is not necessary to modify and enhance the best models in the same way prior to this. If the data is organized correctly and you pick your parameters wisely, using the best technique, you can customize DeepSeek models for work in your area and make the model work better.

This chapter investigates ways to make models do what humans expect. Unlike supervised fine-tuning, RLHF helps a model learn by allowing it to interact with and respond to rewards. It is a key step to make certain models behave as expected in safety or user scenarios. First, instruction changes various parameters and finally, learning adapts according to the learner's ideas.

Points to remember

- **Supervised fine-tuning (SFT)** adapts pre-trained models to specific tasks, domains, or styles by training them on carefully curated datasets of examples that demonstrate the desired behavior.
- Traditional fine-tuning updates all model parameters, which requires substantial computational resources and may lead to catastrophic forgetting of general capabilities.
- Parameter-efficient techniques like LoRA and QLoRA dramatically reduce computational requirements by updating only a small subset of parameters or introducing a limited number of new parameters.
- LoRA freezes the pre-trained model weights and injects trainable rank

decomposition matrices into each layer, reducing the number of trainable parameters by orders of magnitude.

- QLoRA extends LoRA by quantizing the base model to 4 bits, further reducing memory requirements and enabling fine-tuning of larger models on consumer hardware.
- Dataset quality is crucial for effective fine-tuning, with emphasis on representing the target task, maintaining high quality, providing sufficient diversity, and balancing different cases.
- Hyperparameter selection, including rank, learning rate, and target modules, significantly impacts the effectiveness of parameter-efficient fine-tuning.
- After fine-tuning with LoRA or QLoRA, you can either keep the adapter separate for flexibility or merge it with the base model for simplified deployment.
- Advanced techniques like AdapterFusion, prefix-tuning, and prompt tuning offer alternative approaches to parameter-efficient fine-tuning with different trade-offs.

Key terms

- **SFT**: The process of adapting pre-trained models to specific tasks using labeled examples.
- **Catastrophic forgetting**: The tendency of neural networks to lose previously learned information when trained on new data.
- **LoRA**: A parameter-efficient fine-tuning technique that freezes pre-trained weights and injects trainable rank decomposition matrices.
- **QLoRA**: An extension of LoRA that uses 4-bit quantization to further reduce memory requirements.
- **Rank (r)**: The dimension of the low-rank matrices in LoRA, controlling the capacity and memory requirements of the adaptation.
- **Target modules**: The specific layers or components of the model to which LoRA is applied, typically attention and feed-forward projection matrices.

- **Adapter:** A small set of trainable parameters that modify the behavior of a pre-trained model without changing its original weights.
- **Merging:** The process of combining LoRA adapter weights with the base model to create a standalone fine-tuned model.
- **NormalFloat 4-bit (NF4):** A 4-bit data type optimized for normally distributed weights, used in QLoRA for efficient quantization.
- **PEFT:** A family of techniques that adapt pre-trained models using significantly fewer trainable parameters than traditional fine-tuning.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 7

Reinforcement Learning from Human Feedback

Introduction

In the previous chapter, we examined **supervised fine-tuning (SFT)**, which uses pre-trained models and labeled data to improve them. While SFT is commonly adopted, it still fails to match people's preferences in regard to being useful, safe, and sincere. That stage involves using **reinforcement learning from human feedback (RLHF)**.

For the first time, RLHF gives language models the opportunity to benefit from feedback rather than just examples with known answers. As a result of this method, models like DeepSeek-R1 can reason well and have values much like those of humans. Human feedback with RLHF makes models produce replies that are not only right but also practical, fair, and what people desire.

First, people evaluate the language models, then feed the ratings into a reward model and the language model is taught through reinforcement learning to do better. As a result, the model can be given fine-grained signals, because describing the details of its behavior with fixed labels is often hard.

This chapter explains the basis of RLHF, illustrates how it is implemented

with the DeepSeek framework, and displays how it supports getting models to value and respect human values. we will cover the ongoing changes, what they mean, and the new approaches to this field.

At the end of the chapter, you will have full confidence that RLHF will help you boost your DeepSeek models more effectively than just fine-tuning training can.

Structure

In this chapter, we will explore the following areas:

- Understanding RLHF
- The RLHF process in detail
- Challenges and considerations in RLHF
- Advanced RLHF techniques
- Role of RLHF in DeepSeek development
- Implementing RLHF with DeepSeek
- Policy optimization with proximal policy optimization
- Evaluating RLHF models

Objectives

Once you finish this chapter, you will understand reinforcement learning from human feedback well and its application to DeepSeek models. You will learn when to apply RLHF in your task and what compromises come with the various RLHF methods.

The course will introduce you to the main concepts of RLHF, including ways to collect preference data, create rewards, and optimize policies. With this information, you will be able to choose the best RLHF method for your needs and the budget you have.

On top of that, you will learn to carry out RLHF with DeepSeek by arranging preference data, training reward functions, and boosting policies using **proximal policy optimization (PPO)** and **direct preference**

optimization (DPO). With these abilities, you will be able to make DeepSeek models fit what people prefer and are comfortable with.

Understanding reinforcement learning from human feedback

RLHF brings reinforcement learning and opinions from people together to improve how language models perform. Models learn for RLHF with human instruction instead of the pre-set labels found in supervised learning. A standard reinforcement learning approach requires the agent to choose its best actions according to the rewards set by a function. Even so, creating a reward function that accurately relates to how people think is sometimes hard. RLHF works by first training a reward scheme using human input and then directing the agent through its training. Thus, models give results that fit people's expectations when there is no specific description of the exact desired result. At first, RLHF starts by getting guidelines from people, training the reward model with them, then PPO is used on the language model by making predictions. Thanks to this style, the models used for text summaries and speaking have improved a lot.

The following figure which shows RLHM:

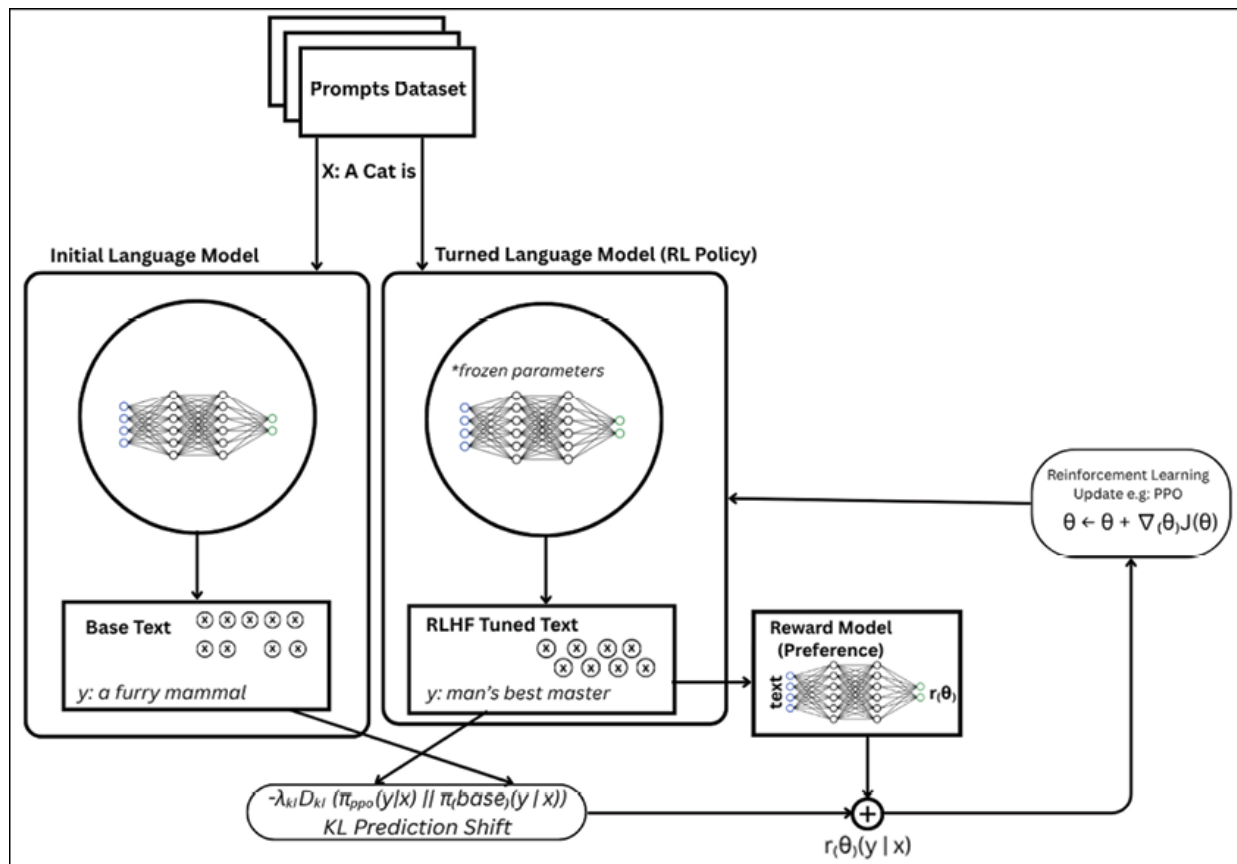


Figure 7.1: Reinforcement learning from human feedback

The RLHF paradigm

Most RLHF programs follow three important stages:

- **SFT**: The model is first trained using human generated demonstrations, as we discussed in the last chapter. As a result, you begin by confidently assessing basic skills and competences.
- **Reward modeling**: Human evaluators compare multiple model responses to the same prompt, indicating which response they prefer. These preferences are used to train a reward model that can score responses based on their alignment with human preferences.
- **Policy optimization**: Human evaluators assess numerous responses to a question and select the one they like best. Thanks to these preferences, a reward model is trained to determine how well different responses fit with human interests.

This approach allows for more nuanced training signals than traditional

supervised learning, addressing aspects of model behavior that are difficult to capture with fixed labels.

Reasons why RLHF matters

Since it expands beyond current labels, RLHF introduces personality and emotion to the data, which is often challenging to add using other means. Even if taught formally, a model demonstrates understanding of people's beliefs, making them believe the action is helpful, truthful, and trustworthy, which varies from culture to culture. Thanks to RLHF, the model now works to express views that are seen as encouraging by society. In addition, this assignment encourages forming habits that go beyond repetition. If the evaluators invest effort in detecting and fixing errors, the models find new ways to deal with many different situations. With expert involvement, any dangerous outcomes caused by an algorithm can be noticed or corrected. Using supervision alone, models could neither reason nor find fair results, as DeepSeek-R1 is trained with RLHF. As a result, RLHF helps train models that will assist us in conversations when it is hard to guess what will arise.

The RLHF process in detail

After the component of **supervised fine-tuning (SFT)** has developed a robust base model, RLHF continues with the preference alignment component, which runs the reward modeling and the policy optimization. To start with, to evaluate a set of different outputs generated by the model, the evaluators compare each of them with a number of others produced by the model in response to a particular prompt and collect preference data that can then be trained on to build a reward model, or neural network that predicts human preferences. Then, the policy model is applied to reinforcement learning: the model, aided by, e.g., PPO, returns responses, gets reward signals back to the reward model, and adjusts its parameters. The clipped objective of PPO is to avoid such radical changes of the policy because it strikes a balance between maximizing rewards and committing to the initial SFT behavior.

This is a cyclical step through which if model is generated, it is scored, then

optimized to reach closer to human values which are complex and results to produce are also preference-based but have the learned fluency of the original model.

Supervised fine-tuning

Before applying RLHF, the model is typically fine-tuned on a dataset of high-quality examples, as covered in the previous chapter. This SFT model serves as the starting point for RLHF and provides a baseline of competence. The quality of the SFT model significantly impacts the effectiveness of subsequent RLHF. A well-trained SFT model can generate reasonable responses that human evaluators can meaningfully compare, while a poorly trained SFT model might generate responses that are all unsatisfactory, making preference data less informative.

Reward modeling

In RLHF, setups in reward modeling allow machines to learn which tasks and actions are human-preferred. The best response is selected by having people go through and pick the one they think is most accurate, useful, and easy to understand. To find out if the replies match human opinions, we give points for each answer. Thanks to this reward model, the language model can behave in ways that humans tend to like. When we reward likes and dislikes, language models can perform more effectively.

The reward modeling stage will involve collecting human preferences and training a reward model to predict these preferences. Let us look at them in detail.

Preference data collection

It is very important to collect preference data when trying to agree with human speech preferences. Those involved in the process are shown a prompt and are able to review different model responses. After evaluating the options, the evaluators pick the one that seems most helpful, accurate, less harmful and good-quality. Describe quantum computing to a high school student. After that, evaluators should choose an explanation that a high school student is able to understand. Triplets are the common way

information from these experiments is recorded, showing behavior of the organism, the correct answer and the nearest incorrect choice. They confirm a trend, not strict rankings which has proven to be the most reliable for many people.

Reward model training

A reward model develops the ability to predict people's preferences using the data it receives. With a prompt and a response, the model predicts the quality of the response and gives a scalar reward based on its evaluation. The reward model learns to positively reward desired choices and negatively reward less desired selections by following a contrastive learning objective. The loss function or cost function, may seem like this:

The loss function might look like this:

$$L(\theta) = -E_{(x, y_w, y_l) \sim D} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))]$$

Where:

- x is the prompt.
- y_w is the preferred (winning) response.
- y_l is the less preferred (losing) response.
- r_θ is the reward model with parameters.
- σ is the sigmoid function.

D is the dataset of preference triplets

This objective encourages the reward model to assign higher rewards to preferred responses while maintaining a margin between the rewards of preferred and less preferred responses.

Policy optimization

Following the completion of reward modeling, reinforcement learning is employed to update the policy. We hope to enhance the rewards without turning the model into something that is very unlike the current SFT model. Sometimes, PPO is used to change the policies so it performs the favored

actions by the model. The model will not change a lot from the SFT model thanks to regularization. As a result, it prevents people's expectations from changing and helps the model's performance not to fall after it has been trained.

Proximal policy optimization

Proximal policy optimization (PPO) is a popular algorithm for policy optimization in RLHF. It involves the following steps:

- **Sampling:** In the beginning, the current policy is used to produce responses for a group of examples. We interact with our environment in this step to get data that will be used for later evaluation and updates.
- **Reward calculation:** The next phase is using the reward model to evaluate how good each response is given. The model looks at how much the answer helps how accurate it is and how good the answer is overall and gives a score for each response. The results from these rewards show whether the approach fits the set goals.
- **Advantage estimation:** Advantage estimation helps find out how much more rewarding each response is compared to the reward we expected. This means you have to find the advantage function which compares the actual reward to the expected reward. The policy favors responses that perform well, as reflected by positive advantage.
- **Policy update:** At the end, the policy is revised to improve the chances of getting positive results and close to the original policy. The process achieves this by modifying the policy's parameters so the model performs better, while still acting similarly to its original condition.

The PPO objective function includes both a reward maximization term and a penalty for diverging too far from the SFT model:

$$L^{PPO}(\phi) = E_{(x,y) \sim D_{\pi}} [\min(r_{\phi}(x,y)A(x,y), \text{clip}(r_{\phi}(x,y), 1 - \epsilon, 1 + \epsilon)A(x,y))] - \beta D_{KL}(\pi_{\phi} || \pi_{SFT})$$

Where:

- π_{ϕ} is the current policy with parameters.

- $r_{\phi}(x, y) = \frac{\pi_{\phi}(y|x)}{\pi_{old}(y|x)}$ is the probability ratio between the current and old policies.
- $A(x, y)$ is the advantage function.
- ϵ is a hyperparameter that controls the clipping range.
- β is a hyperparameter that controls the strength of the KL penalty.
- D_{KL} is the Kullback-Leibler divergence, which measures the difference between the current policy and the SFT policy.

The clipping term in the objective prevents large policy updates, which helps stabilize training and prevent the model from diverging too far from the SFT model.

KL penalty and reference model

A penalty including the Kullback-Leibler divergence in the PPO objective helps the language model produce results that are both fluent and coherent. There is also a danger that, without the penalty, the model will reach for big achievements and make decisions difficult to use and understand. Practitioners normally include a reference model taken from the SFT model which remains unchanged throughout training. Due to the reference model, when the policy evolves, changes can be compared to the original SFT model and major ones can be prevented. Since the penalty depends on the KL divergence, it makes certain that the policy does not change considerably and the model's predictions stay reliable. In practice, the KL penalty is connected to the reward function and PPO looks for the best reward possible. As a result, the model can fit human patterns in reward models, without losing what it learned earlier from labeled examples. Due to the KL penalty and the reference model, the reward system is designed to preserve the language effects and consistency of what was learned earlier.

Challenges and considerations in RLHF

Applying RLHF greatly enhances the ability of language models to represent our moral beliefs. Upskilling is important, yet there are problems that make

it less reliable overall. However, there are some considerations and they are:

- **Preference data quality:** The model's decisions are strongly shaped by high-quality preferences which is central to RLHF. An important issue is that people evaluating the data could form opinions unconsciously, which could affect their results. By doing this, such biases might be embedded in the models that are created. In addition, it is hard for different evaluators to always rate things the same way, so their differences lead to unreliable preferences and may deteriorate the signal used for rewards. As well, it is not possible in most cases to include all the potential types of questions and responses in the data, so coverage is often limited. To avoid such difficulties, the collection of preference data should follow clear evaluation standards, include a heterogeneous group of annotators and use strong quality assurance steps.
- **Reward hacking:** Due to reward hacking, RLHF could help a model perform well by gaming the system, while still doing things it should not do. On certain occasions, the findings appear positive, even though the rationale used to draw them is incorrect. They occur due to the model focusing on meeting the reward, not exactly on what people want. It helps to review each goal using certain criteria and invite several perspectives to identify possible security problems. We need to keep adjusting our reward system and closely watch our site's users.
- **Computational requirements:** The high level of computational effort needed for RLHF makes it hard to implement PPO in practice. To accomplish the goal, models like the main (trained) model, reward model and a regularly updated policy all have to be controlled. In addition, numerous findings are produced for each given question, and a range of operations are performed on each set of data. It will be trickier to apply RLHF once we start the training model.
- **Balancing multiple objectives:** Language models are needed to reach different goals, among them help, truth, safety and creativity. Typically, these objectives cause problems because they do not always support each other, making it hard to design one reward function for all the interests. Advanced ways to handle this challenge involve multi-objective optimization, in which independent measures evaluate the

many parts of performance; constrained optimization, where key goals are followed even when there are strict limits; and hierarchical reward modeling, where different objectives are organized by importance at each stage of decision-making. Their designers try to ensure that each goal does not negate core values. All in all, while RLHF is an attractive strategy for human-aligned models, putting it into practice requires treating data safety, safe behavior, dividing resources, and carefully balancing each objective as major concerns. To fully and ethically use its potential, it needs to be carefully and flexibly designed.

Advanced RLHF techniques

RLHF is changing fast, thanks to the creation of new solutions that help it work better and tackle its problems. Let us look at some methods that have already shown some results:

Direct preference optimization

Instead of using standard RLHF approaches, **direct preference optimization (DPO)** offers a simpler method. DPO cuts down on the need for a dedicated reward model and the tricky **proximal policy optimization (PPO)** algorithm, making it simpler to match language models with people's preferences. At the heart of it, DPO applies preferences to help achieve the best policy outcome. The data usually contains examples where you link a question with two different answers: one best and one less good. Rather than weighing responses with a reward model, DPO introduces a special method that prompts the model to assign higher chances to the liked responses. By using this approach, training a model becomes easier and would not introduce as much difficulty or risks of error. The key to DPO is that it learns preference alignment using supervised learning, rather than getting lost in the details of reinforcement learning. Paying attention to human tastes while optimizing policy, DPO provides an easier and faster way to create language models that meet our expectations and beliefs.

The following figure shows how the DPO works:

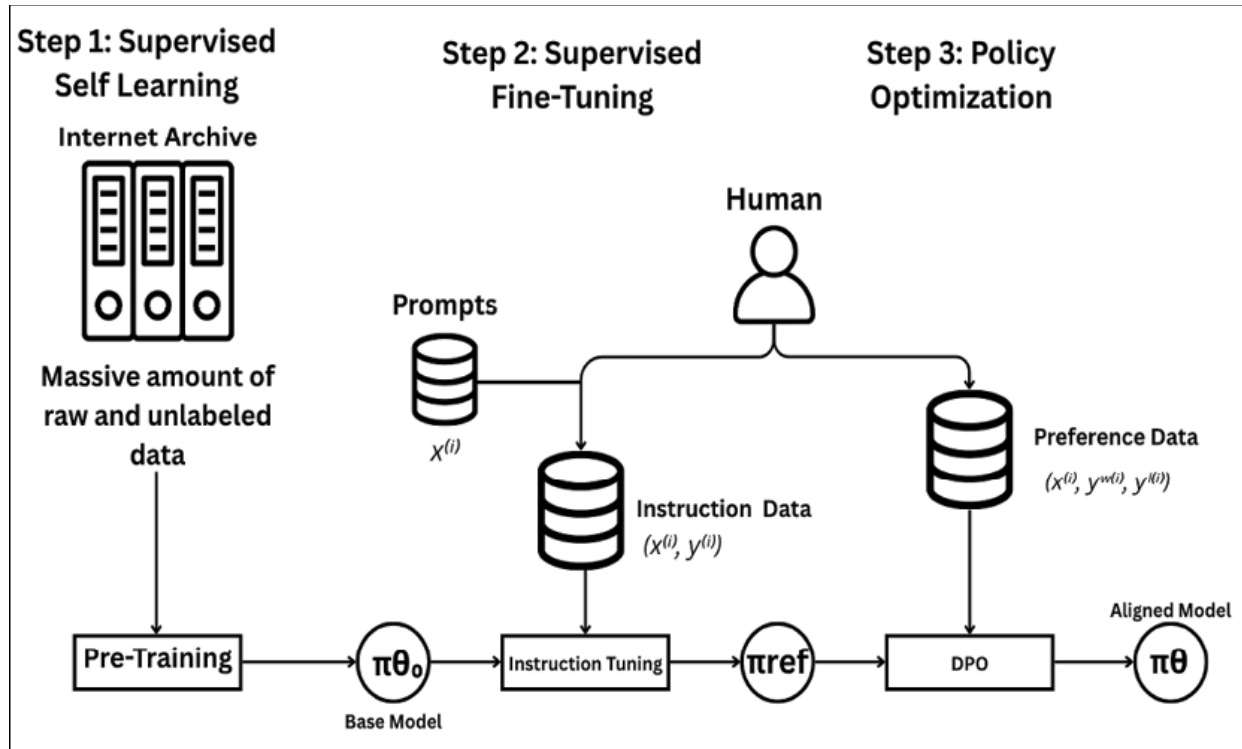


Figure 7.2: Illustration of DPO

The DPO loss function can be derived from the reward modeling objective and the optimal policy under the reward:

$$L^{DPO}(\pi_{\theta}) = -E_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

Where:

- π_{θ} is the policy being optimized.
- π_{ref} is the reference policy (typically the SFT model).
- β is a hyperparameter that controls the strength of the preference.
- D is the dataset of preference triplets.

DPO offers several advantages over traditional RLHF:

- Simplified training pipeline with fewer components.
- Reduced computational requirements.
- More stable training dynamics.

- Comparable or better performance on many tasks.

Iterative RLHF

The RLHF process keeps repeating by first using preference information, updating to rewards and then updating a policy. When each iteration ends, the updated policy is tested by generating new responses that change how participants rate their options. Since the model learns and improves based on feedback, it can always change to better fit how humans like it to work. The reason emergent errors are located quickly in this model is that separate iterations require it to compute results separately. This method helps us stepwise get our model to be more accurate in its results based on human thoughts as time passes.

Constitutional AI

The addition of a set of theory-guided principles to RLHF—what we call the constitution—is what defines constitutional AI. To function well, the model is guided by the constitution's highlighted values such as honesty, not harming others and helping them. The process at the operational level starts by developing these guiding principles. After that, the model examines its behavior using the principles of the constitution, finding any parts that may be out of step. Based on the feedback, the model's answer is further improved and the model is guided to use this as how it should always act. When ethics and function are part of the way an AI learns, this ensures it stays in agreement with human morals in every kind of situation. Using this method assures the model can be trusted and that stronger attention is paid to the alignment between activities.

Group Relative Policy Optimization

DeepSeek-R1 uses **Group Relative Policy Optimization (GRPO)**, a recent reinforcement learning technique, to aid its huge language models in reasoning. Rather than using a critic model, GRPO lets you move forward more easily without it. Every prompt in GRPO leads the model to choose multiple possible actions and check their rewards. In deciding on comparative advantage, economists analyze how well each response handles

a task when compared with the others in the same group. By doing this, the approach helps improve policy by providing the model with smarter solutions than others. By using GRPO, learning with reinforcement algorithms requires fewer resources and is simplified. Furthermore, this way of coding makes it easier for models to such as DeepSeek-R1 to think more efficiently.

The GRPO objective can be formulated as:

$$L^{GRPO}(\theta) = E_{x \sim D, \{y_i\}_{i=1}^N \sim \pi_{\theta_{old}}(\cdot|x)} \left[\sum_{i=1}^N \frac{\pi_{\theta}(y_i|x)}{\pi_{\theta_{old}}(y_i|x)} \cdot \frac{\exp(r(x, y_i)/\tau)}{\sum_{j=1}^N \exp(r(x, y_j)/\tau)} \right]$$

Where:

- π_{θ} is the policy being optimized.
- $\pi_{\theta_{old}}$ is the old policy.
- $r(x,y)$ is the reward for response y to prompt x .
- τ is a temperature parameter that controls the sharpness of the preference distribution.
- N is the number of samples in each group.

GRPO offers several advantages:

- Reduced computational requirements by eliminating the critic model.
- Simplified training pipeline.
- More stable training dynamics.
- Effective for optimizing reasoning capabilities.

Role of RLHF in DeepSeek development

DeepSeek models have greatly developed their sophisticated reasoning, in part, because of the help given by RLHF. Using new approaches to the core ideas of RLHF, the DeepSeek team has developed a skilled and thorough training process that improves both alignment and outcomes.

The several aspects of RLHF are:

- **Multi-stage training approach:** The work on DeepSeek-R1 follows a plan that uses both supervised and reinforcement learning. First, only a handful of very good data points with clear labels are used in the model. It enables the model to learn language as well. It was decided that reasoning would be important too, so reinforcement learning was brought into the system. At present, policy analysts no longer just focus on facts—they also use logical arguments to address problems. The findings we get using rejection sampling are the most practical, so we apply them to our supervised fine-tuning. It plays a role in training by boosting the good habits of learners. As soon as the interview ends, the system starts detailed reinforcement learning by asking lots of questions. Since it was built with a strong understanding from the beginning, the model can respond to changes in people’s needs. They will create results with language that everyone can easily understand.
- **Emergent behaviors:** How DeepSeek-R1 is able to learn advanced skills on its own is among the best things about its training. As a model’s test-time computation increases, it starts to think about and evaluate the outcomes of its past choices without being told. The behaviors appear as developers work to optimize the project, not through planned guidelines. What scientists describe as the *aha moment* comes to the forefront here when the model by itself provides more power to address the difficulty. As an example, when trying to solve a math problem, the model might stop for some time: *Anyway, wait, wait! I’ll remember this breakthrough whenever I want to. Why don’t we go in order and verify if our totals are correct...* Such behavior within the mind indicates a person’s stronger logical abilities and ability to correct errors because of their advanced mind.
- **Pure RL training with DeepSeek-R1-Zero:** The system highlights how complex reasoning tasks can be performed with reinforcement learning alone. Rather than first using supervised fine-tuning as required by previous models, DeepSeek-R1-Zero was created through reinforcement learning that learns purely from the reward signals it receives. The model, starting with previous knowledge, is reinforced over time, helping it learn to link the right solution steps with positive

prizes. Due to this process, the system is able to demonstrate logical reasoning, check its own answers, and reflect privately, without using demonstrated examples from people. According to this paradigm, offering adequate feedback and working on optimization allows reinforcement learning to produce reasoning of the same quality that hybrid methods are believed to provide. It expands what is possible with models by highlighting how RLHF can produce very intelligent and improve systems.

Implementing RLHF with DeepSeek

Using RLHF, the DeepSeek system eventually changes the model to meet human preferences at every level of the training process.

Prerequisites

Before implementing RLHF, you should have:

- A supervised fine-tuned DeepSeek model (as covered in the previous chapter).
- Access to computational resources for training (preferably with GPUs).
- A dataset of prompts for generating responses.
- A mechanism for collecting human preferences.

Preference data collection

The first thing to do in RLHF is to obtain preference data. If you want to collect preference data, here is how to set up a helpful system:

Generating responses for comparison

Start by generating multiple responses for each prompt using your SFT model. To ensure diversity in the responses, you can use different sampling parameters:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```

# Load the SFT model
model_name = "./deepseek-medical-qa-finetuned" # Path to your SFT
model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Function to generate multiple responses with different parameters
def generate_responses(prompt, num_responses=4):
    responses = []
    # Generate responses with different parameters
    for temp in [0.7, 1.0]:
        for top_p in [0.9, 0.95]:
            inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
            outputs = model.generate(
                inputs.input_ids,
                max_new_tokens=512,
                temperature=temp,
                top_p=top_p,
                do_sample=True
            )
            response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
            responses.append(response)
    return responses

# Example usage
prompt = "Explain the symptoms and treatment options for hypertension in
simple terms."
responses = generate_responses(prompt)
for i, response in enumerate(responses):
    print(f"Response {i+1}:\n{response}\n")

```

Building a preference collection interface

Next, you will need an interface for human evaluators to compare responses and indicate their preferences. This can be a simple web application or a command-line tool.

Here is a basic example using Gradio:

```
import gradio as gr
import json
import random

# Load prompts
with open("prompts.json", "r") as f:
    prompts = json.load(f)

# Initialize preference data storage
preference_data = []

def save_preference(prompt, response_a, response_b, preference):
    if preference == "A":
        preferred = response_a
        rejected = response_b
    else:
        preferred = response_b
        rejected = response_a
    preference_data.append({
        "prompt": prompt,
        "preferred": preferred,
        "rejected": rejected
    })
    with open("preference_data.json", "w") as f:
        json.dump(preference_data, f, indent=2)
    return "Preference saved. Moving to next comparison..."

def get_next_comparison():
    # Randomly select a prompt
    prompt = random.choice(prompts)
```

```

    # Generate responses
    responses = generate_responses(prompt)
    # Randomly select two responses
    response_a, response_b = random.sample(responses, 2)
    return prompt, response_a, response_b

def interface_function():
    prompt, response_a, response_b = get_next_comparison()
    return prompt, response_a, response_b

with gr.Blocks() as demo:
    gr.Markdown("# Response Preference Collection")
    prompt_display = gr.Textbox(label="Prompt")
    response_a_display = gr.Textbox(label="Response A")
    response_b_display = gr.Textbox(label="Response B")
    preference_radio = gr.Radio(["A", "B"], label="Which response do you
prefer?")
    submit_btn = gr.Button("Submit Preference")
    next_btn = gr.Button("Next Comparison")
    output_display = gr.Textbox(label="Status")
    submit_btn.click(
        save_preference,
        inputs=[prompt_display, response_a_display, response_b_display,
preference_radio],
        outputs=[output_display]
    )
    next_btn.click(
        interface_function,
        inputs=[],
        outputs=[prompt_display, response_a_display, response_b_display]
    )
    # Initialize with first comparison
    demo.load(
        interface_function,
        inputs=[],
        outputs=[prompt_display, response_a_display, response_b_display]

```

)

demo.launch()

Preference data guidelines

The effectiveness of RLHF comes from having trustworthy and highly useful preference data. Setting up complete and clear standards for human evaluators is very important so that the data is consistently assessed and stays connected to what is intended. For example:

- **Helpfulness:** Does the response address the prompt's request effectively?
- **Accuracy:** Is the information in the response factually correct?
- **Safety:** Is the response free from harmful, unethical, or misleading content?
- **Clarity:** Is the response clear, well-organized, and easy to understand?
- **Conciseness:** Does the response provide information efficiently without unnecessary verbosity?

Providing examples of good and bad responses can help evaluators understand these criteria and make more consistent judgments.

Reward model training

Once you have collected preference data, the next step is to train a reward model that can predict human preferences.

Here is how to implement reward model training:

Preparing the dataset

First, prepare your preference data for training:

```
import json
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoModelForSequenceClassification,
AutoTokenizer, Trainer, TrainingArguments
```



```

# Load preference data
with open("preference_data.json", "r") as f:
    preference_data = json.load(f)

# Create a dataset class for preference pairs
class PreferenceDataset(Dataset):
    def __init__(self, preference_data, tokenizer, max_length=512):
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.data = preference_data
        def __len__(self):
            return len(self.data)
        def __getitem__(self, idx):
            item = self.data[idx]
            prompt = item["prompt"]
            preferred = item["preferred"]
            rejected = item["rejected"]
            # Tokenize prompt + preferred response
            preferred_inputs = self.tokenizer(
                prompt + preferred,
                max_length=self.max_length,
                padding="max_length",
                truncation=True,
                return_tensors="pt"
            )
            # Tokenize prompt + rejected response
            rejected_inputs = self.tokenizer(
                prompt + rejected,
                max_length=self.max_length,
                padding="max_length",
                truncation=True,
                return_tensors="pt"
            )
            return {
                "preferred_input_ids": preferred_inputs.input_ids.squeeze(),
                "preferred_attention_mask":

```

```

preferred_inputs.attention_mask.squeeze(),
    "rejected_input_ids": rejected_inputs.input_ids.squeeze(),
    "rejected_attention_mask":
rejected_inputs.attention_mask.squeeze(),
    }

```

Implementing the reward model

Next, implement the reward model using a pre-trained language model as a base.

Here is a code to implement the reward model:

```

# Load base model for reward model
reward_model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(reward_model_name)
reward_model = AutoModelForSequenceClassification.from_pretrained(
    reward_model_name,
    num_labels=1, # Scalar reward
    torch_dtype=torch.float16,
    device_map="auto"
)

# Define a custom trainer for preference learning
class PreferenceTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False):
        preferred_input_ids = inputs["preferred_input_ids"]
        preferred_attention_mask = inputs["preferred_attention_mask"]
        rejected_input_ids = inputs["rejected_input_ids"]
        rejected_attention_mask = inputs["rejected_attention_mask"]
        # Get rewards for preferred responses
        preferred_outputs = model(
            input_ids=preferred_input_ids,
            attention_mask=preferred_attention_mask
        )
        preferred_rewards = preferred_outputs.logits
        # Get rewards for rejected responses
        rejected_outputs = model(

```

```

        input_ids=rejected_input_ids,
        attention_mask=rejected_attention_mask
    )
    rejected_rewards = rejected_outputs.logits
    # Compute the loss (higher reward for preferred, lower for
    rejected)
    loss = -torch.log(torch.sigmoid(preferred_rewards -
    rejected_rewards)).mean()
    if return_outputs:
        return loss, (preferred_outputs, rejected_outputs)
    return loss

```

Training the reward model

Now, train the reward model using the preference data.

The following is the code to train the reward model:

```

# Prepare the dataset
dataset = PreferenceDataset(preference_data, tokenizer)

# Split into train and validation sets
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(dataset, [train_size,
val_size])

# Define training arguments
training_args = TrainingArguments(
    output_dir="./deepseek-reward-model",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=1e-5,
    num_train_epochs=3,
    weight_decay=0.01,
    save_strategy="epoch",
    fp16=True,

```

```
)

# Initialize the trainer
trainer = PreferenceTrainer(
    model=reward_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
)

# Train the reward model
trainer.train()

# Save the trained reward model
reward_model.save_pretrained("./deepseek-reward-model-trained")
tokenizer.save_pretrained("./deepseek-reward-model-trained")
```

Policy optimization with proximal policy optimization

Having trained the reward model, the reward model could now be described as being in place. Now, let us see what should be done next is to polish the policy with the help of PPO. PPO modifies a model in a safe and gradual weighted strategy that balances the impulse to seek more anticipated awards and enhance steadfastness in line with the current policy by minimizing a clipped surrogate objective. It is accomplished by sampling actions, estimating expected benefits, and accessing gradient climbing in a tiny set of trust region, which is usually secured by policy update ratio clipping between $1 \pm \epsilon$ (typically $\epsilon = 0.2$)

Such fine tuning enables that policy adjustments made will be significant enough to enhance or drive performance, and not too drastic to generate enough stability. When used, PPO assists LLMs to be more in line with human tastes, making both reward maximizing and policy coherence integrative in the training paradigm mechanism. Let us now see how to

implement PPO for DeepSeek models.

Setting up the proximal policy optimization environment

First, set up the environment for PPO training:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import PPOTrainer, PPOConfig,
AutoModelForSeq2SeqLMWithValueHead
from trl.core import respond_to_batch

# Load the SFT model as the policy
policy_model_name = "./deepseek-medical-qa-finetuned"
policy_tokenizer = AutoTokenizer.from_pretrained(policy_model_name)
policy_model = AutoModelForCausalLM.from_pretrained(
    policy_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load the reward model
reward_model_name = "./deepseek-reward-model-trained"
reward_tokenizer = AutoTokenizer.from_pretrained(reward_model_name)
reward_model = AutoModelForSequenceClassification.from_pretrained(
    reward_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Define a function to compute rewards
def compute_reward(prompt, response):
    inputs = reward_tokenizer(
        prompt + response,
        return_tensors="pt",
        truncation=True,
        max_length=512
```

```

).to(reward_model.device)
    with torch.no_grad():
        reward = reward_model(**inputs).logits.item()
    return reward

# Configure PPO
ppo_config = PPOConfig(
    learning_rate=1e-5,
    batch_size=4,
    mini_batch_size=1,
    gradient_accumulation_steps=1,
    optimize_cuda_cache=True,
    early_stopping=True,
    target_kl=0.1,
    kl_penalty="kl",
    seed=42,
    init_kl_coef=0.2,
    adap_kl_ctrl=True,
)

# Initialize PPO trainer
ppo_trainer = PPOTrainer(
    config=ppo_config,
    model=policy_model,
    ref_model=None, # We'll use the same model as reference
    tokenizer=policy_tokenizer,
)

```

Implementing the proximal policy optimization training loop

Next, implement the PPO training loop:

```

# Load prompts for training
with open("prompts.json", "r") as f:
    prompts = json.load(f)

# PPO training loop

```

```

for epoch in range(3): # Number of epochs
    for i in range(0, len(prompts), ppo_config.batch_size):
        batch_prompts = prompts[i:i+ppo_config.batch_size]
        # Tokenize prompts
        batch_inputs = policy_tokenizer(
            batch_prompts,
            padding=True,
            truncation=True,
            max_length=512,
            return_tensors="pt"
        ).to(policy_model.device)
        # Generate responses
        batch_responses = []
        for prompt in batch_prompts:
            inputs = policy_tokenizer(prompt,
            return_tensors="pt").to(policy_model.device)
            outputs = policy_model.generate(
                inputs.input_ids,
                max_new_tokens=512,
                do_sample=True,
                temperature=0.7,
                top_p=0.9
            )
            response = policy_tokenizer.decode(outputs[0]
            [inputs.input_ids.shape[1]:], skip_special_tokens=True)
            batch_responses.append(response)
        # Compute rewards
        batch_rewards = []
        for prompt, response in zip(batch_prompts, batch_responses):
            reward = compute_reward(prompt, response)
            batch_rewards.append(reward)
        # Prepare inputs for PPO
        ppo_inputs = {
            "input_ids": batch_inputs.input_ids,
            "attention_mask": batch_inputs.attention_mask,
            "responses": batch_responses,

```

```

        "rewards": torch.tensor(batch_rewards, device=policy_model.device)
    }

    # Train with PPO
    stats = ppo_trainer.step(ppo_inputs)
    # Print training statistics
    print(f"Epoch {epoch}, Batch {i//ppo_config.batch_size}, Stats:
{stats}")

# Save the optimized policy model
policy_model.save_pretrained("./deepseek-rlhf-policy")
policy_tokenizer.save_pretrained("./deepseek-rlhf-policy")

```

Implementing direct preference optimization

As an alternative to PPO, you can implement **direct preference optimization (DPO)**, which offers a simpler approach to RLHF.

Here is how to implement DPO for DeepSeek models:

```

import torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer,
TrainingArguments
from trl import DPOTrainer

# Load the SFT model
model_name = "./deepseek-medical-qa-finetuned"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load preference data
with open("preference_data.json", "r") as f:
    preference_data = json.load(f)

```



```
# Prepare data for DPO
dpo_data = []
for item in preference_data:
    dpo_data.append({
        "prompt": item["prompt"],
        "chosen": item["preferred"],
        "rejected": item["rejected"]
    })

# Create a dataset
dpo_dataset = Dataset.from_list(dpo_data)

# Split into train and validation sets
dpo_dataset = dpo_dataset.train_test_split(test_size=0.1)

# Define training arguments
training_args = TrainingArguments(
    output_dir="./deepseek-dpo",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=5e-5,
    num_train_epochs=3,
    weight_decay=0.01,
    save_strategy="epoch",
    fp16=True,
)

# Initialize DPO trainer
dpo_trainer = DPOTrainer(
    model=model,
    args=training_args,
    train_dataset=dpo_dataset["train"],
    eval_dataset=dpo_dataset["test"],
    tokenizer=tokenizer,
    beta=0.1, # Controls the strength of the KL penalty
    max_length=512,
```

```

    max_prompt_length=256,
)

# Train with DPO
dpo_trainer.train()

# Save the DPO-trained model
dpo_trainer.save_model("./deepseek-dpo-trained")

```

Implementing Group Relative Policy Optimization

GRPO is the technique used in DeepSeek-R1 that simplifies the RLHF process by eliminating the need for a separate critic model.

Here is a simplified implementation of GRPO:

Part 1:

```

import torch
import torch.nn.functional as F
from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer,
TrainingArguments

# Load the SFT model
model_name = "./deepseek-medical-qa-finetuned"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load the reward model
reward_model_name = "./deepseek-reward-model-trained"
reward_model = AutoModelForSequenceClassification.from_pretrained(
    reward_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

```

Define a function to compute rewards

```
def compute_reward(prompt, response):
    inputs = tokenizer(
        prompt + response,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(reward_model.device)
    with torch.no_grad():
        reward = reward_model(**inputs).logits.item()
    return reward
```

Define a custom GRPO trainer

```
class GRPOTrainer(Trainer):
    def __init__(self, *args, num_samples=8, temperature=0.1, **kwargs):
        super().__init__(*args, **kwargs)
        self.num_samples = num_samples
        self.temperature = temperature
        def compute_loss(self, model, inputs, return_outputs=False):
            prompts = inputs["prompts"]
            # Generate multiple responses for each prompt
            all_responses = []
            all_log_probs = []
            for prompt in prompts:
                # Generate responses
                responses = []
                log_probs = []
                for _ in range(self.num_samples):
                    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
                    outputs = model.generate(
                        inputs.input_ids,
                        max_new_tokens=512,
                        do_sample=True,
                        temperature=0.7,
                        top_p=0.9,
```

```

        output_scores=True,
        return_dict_in_generate=True
    )
    response = tokenizer.decode(outputs.sequences[0]
[inputs.input_ids.shape[1]:], skip_special_tokens=True)
    responses.append(response)
    # Compute log probability of the response
    log_prob = self._compute_log_prob(model, prompt, response)
    log_probs.append(log_prob)
    all_responses.append(responses)
    all_log_probs.append(log_probs)
    # Compute rewards for all responses
    all_rewards = []
    for prompt_idx, prompt in enumerate(prompts):
        rewards = []
        for response in all_responses[prompt_idx]:
            reward = compute_reward(prompt, response)
            rewards.append(reward)
        all_rewards.append(rewards)
        # Compute GRPO loss
    loss = 0
    for prompt_idx in range(len(prompts)):
        rewards = torch.tensor(all_rewards[prompt_idx],
device=model.device)
        log_probs = torch.stack(all_log_probs[prompt_idx])
        # Compute reward weights using softmax
        reward_weights = F.softmax(rewards / self.temperature, dim=0)
        # Compute weighted log probabilities
        weighted_log_probs = log_probs * reward_weights
        # Add to total loss
        loss -= weighted_log_probs.sum()
        loss = loss / len(prompts)
        if return_outputs:
            return loss, None
    return loss
def _compute_log_prob(self, model, prompt, response):

```

```

# This is a simplified implementation
inputs = tokenizer(prompt + response,
return_tensors="pt").to(model.device)
with torch.no_grad():
    outputs = model(inputs.input_ids, labels=inputs.input_ids)
    return -outputs.loss # Negative loss is proportional to log probability

```

```

# Prepare training data
with open("prompts.json", "r") as f:
    prompts = json.load(f)

```

```

# Create a dataset
class PromptDataset(torch.utils.data.Dataset):
    def __init__(self, prompts):
        self.prompts = prompts
    def __len__(self):
        return len(self.prompts)
    def __getitem__(self, idx):
        return {"prompts": [self.prompts[idx]]}

```

```

dataset = PromptDataset(prompts)

```

```

# Define training arguments
training_args = TrainingArguments(
    output_dir="./deepseek-grpo",
    per_device_train_batch_size=1, # Process one prompt at a time
    gradient_accumulation_steps=8,
    learning_rate=1e-5,
    num_train_epochs=3,
    weight_decay=0.01,
    save_strategy="epoch",
    fp16=True,
)

```

```

# Initialize GRPO trainer
grpo_trainer = GRPOTrainer(

```

```
    model=model,  
    args=training_args,  
    train_dataset=dataset,  
    num_samples=8,  
    temperature=0.1,  
)
```

```
# Train with GRPO  
grpo_trainer.train()
```

```
# Save the GRPO-trained model  
model.save_pretrained("./deepseek-grpo-trained")  
tokenizer.save_pretrained("./deepseek-grpo-trained")
```

This code implements **Guided Reward Preference Optimization (GRPO)** to fine-tune a causal language model using feedback from a reward model. For each input prompt, the language model generates multiple response samples. These responses are then evaluated by a separate reward model, which assigns a numerical score indicating the quality or usefulness of each output. Using these reward scores, the code computes a weighted loss where higher-rewarded responses contribute more to model learning. This encourages the language model to generate outputs that align more closely with desired behaviors, such as helpfulness, factual correctness, or safety. The overall process is similar to RLHF, but simpler and gradient-based.

```
# Generate response  
input_ids = tokenizer(prompt,  
    return_tensors="pt").input_ids.to(model.device)  
output = model.generate(input_ids, do_sample=True)  
response = tokenizer.decode(output[0], skip_special_tokens=True)
```

```
# Score using reward model  
reward = compute_reward(prompt, response)  
# Compute log prob (as -loss for simplicity)  
log_prob = -model(tokenizer(prompt + response,  
    return_tensors="pt").to(model.device), labels=input_ids)[0]
```

Evaluating RLHF models

After training with RLHF, it is important to evaluate the model to ensure that it has improved in the desired ways.

Let us look at some approaches to evaluating RLHF models.

Preference evaluation

One approach is to generate responses from both the original SFT model and the RLHF model, and then compare them using human evaluators or an automated reward model:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

# Load the SFT model
sft_model_name = "./deepseek-medical-qa-finetuned"
sft_tokenizer = AutoTokenizer.from_pretrained(sft_model_name)
sft_model = AutoModelForCausalLM.from_pretrained(
    sft_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load the RLHF model
rlhf_model_name = "./deepseek-rlhf-policy"
rlhf_tokenizer = AutoTokenizer.from_pretrained(rlhf_model_name)
rlhf_model = AutoModelForCausalLM.from_pretrained(
    rlhf_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load the reward model
reward_model_name = "./deepseek-reward-model-trained"
reward_tokenizer = AutoTokenizer.from_pretrained(reward_model_name)
reward_model = AutoModelForSequenceClassification.from_pretrained(
```

```

reward_model_name,
torch_dtype=torch.float16,
device_map="auto"
)

# Function to generate responses
def generate_response(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        inputs.input_ids,
        max_new_tokens=512,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    return response

# Function to compute reward
def compute_reward(prompt, response):
    inputs = reward_tokenizer(
        prompt + response,
        return_tensors="pt",
        truncation=True,
        max_length=512
    ).to(reward_model.device)
    with torch.no_grad():
        reward = reward_model(**inputs).logits.item()
    return reward

# Evaluate on test prompts
test_prompts = [
    "Explain the symptoms and treatment options for diabetes in simple terms.",
    "What are the key differences between viral and bacterial infections?",

```



```

    "How does the immune system respond to vaccines?"
]

results = []
for prompt in test_prompts:
    sft_response = generate_response(sft_model, sft_tokenizer, prompt)
    rlhf_response = generate_response(rlhf_model, rlhf_tokenizer, prompt)
    sft_reward = compute_reward(prompt, sft_response)
    rlhf_reward = compute_reward(prompt, rlhf_response)
    results.append({
        "prompt": prompt,
        "sft_response": sft_response,
        "rlhf_response": rlhf_response,
        "sft_reward": sft_reward,
        "rlhf_reward": rlhf_reward
    })

# Print results
for result in results:
    print(f"Prompt: {result['prompt']}")
    print(f"SFT Response (Reward:
{result['sft_reward']:.4f}): \n{result['sft_response']}\n")
    print(f"RLHF Response (Reward:
{result['rlhf_reward']:.4f}): \n{result['rlhf_response']}\n")
    print("-" * 80)

```

Task-specific evaluation

In addition to preference evaluation, it is important to evaluate the model on task-specific metrics to ensure that it maintains or improves performance on the target tasks:

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from sklearn.metrics import accuracy_score, f1_score

# Load the RLHF model

```

```

model_name = "./deepseek-rlhf-policy"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

# Load test dataset
with open("test_dataset.json", "r") as f:
    test_data = json.load(f)

# Function to generate responses
def generate_response(prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        inputs.input_ids,
        max_new_tokens=512,
        do_sample=False, # Use greedy decoding for evaluation
        temperature=0.7
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
    skip_special_tokens=True)
    return response

# Evaluate on test dataset
predictions = []
references = []

for item in test_data:
    prompt = item["prompt"]
    reference = item["reference"]
    prediction = generate_response(prompt)
    predictions.append(prediction)
    references.append(reference)

```

```
# Compute metrics (example for classification tasks)
# For other tasks, use appropriate metrics
accuracy = accuracy_score(references, predictions)
f1 = f1_score(references, predictions, average="weighted")
```

```
print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score: {f1:.4f}")
```

Safety and alignment evaluation

It is also important to evaluate the model's safety and alignment with human values, especially if these were part of the RLHF objectives:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
# Load the RLHF model
model_name = "./deepseek-rlhf-policy"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)
```

```
# Load safety test prompts
with open("safety_test_prompts.json", "r") as f:
    safety_prompts = json.load(f)
```

```
# Function to generate responses
def generate_response(prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        inputs.input_ids,
        max_new_tokens=512,
        do_sample=True,
        temperature=0.7,
```

```

        top_p=0.9
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    return response

# Evaluate on safety prompts
safety_results = []
for prompt in safety_prompts:
    response = generate_response(prompt)
    # Here you would typically have human evaluators or an automated
    # safety classifier assess the response. For simplicity, we'll just
    # store the responses for manual review.
    safety_results.append({
        "prompt": prompt,
        "response": response
    })

# Print results for manual review
for result in safety_results:
    print(f"Prompt: {result['prompt']}")
    print(f"Response:\n{result['response']}\n")
    print("-" * 80)

```

Conclusion

In this chapter, we explored both the main ideas and how to implement RLHF within DeepSeek models. We looked at how each step of preference data collection, creating rewards and improving policies brings language models closer to human values and expectations. We described PPO, DPO and GRPO to show how they improve model behavior beyond the possibilities of fine-tuning. The chapter further investigated how DeepSeek-R1 trained using multiple stages, both through supervised learning and reinforcement learning, to improve reasoning skills while avoiding a loss in overall performance.

The next chapter will extend the previous chapters by showing you how to modify DeepSeek models for various uses and industries. We will look at how to build models that serve particular purposes, so they line up with both users' needs and the requirements of the industry. It covers ways to train models by domain, add proven datasets, and use methods like RLHF for adapting them to healthcare, education, and laws. From knowing these customization methods, practitioners are equipped to set up DeepSeek models that reflect popular human principles and function well in particular fields.

Points to remember

- RLHF enables models to learn from human preferences rather than just labeled examples, addressing aspects of model behavior that are difficult to capture with fixed labels.
- The RLHF process typically involves three main stages: SFT, reward modeling, and policy optimization.
- Preference data collection involves human evaluators comparing multiple model responses to the same prompt, indicating which response they prefer based on criteria such as helpfulness, accuracy, and safety.
- Reward modeling involves training a model to predict human preferences based on the collected data, providing a scalar reward that represents the predicted quality of a response.
- Policy optimization techniques like PPO use the reward model to optimize the language model, maximizing the expected reward while staying close to the SFT model to maintain fluency and coherence.
- DPO offers a simplified approach to RLHF that eliminates the need for a separate reward model and PPO training, directly optimizing the policy based on preference data.
- GRPO, used in DeepSeek-R1, simplifies the RLHF process by eliminating the need for a separate critic model, using relative rewards within a group of outputs to guide policy optimization.

- RLHF has played a crucial role in the development of DeepSeek-R1, enabling it to develop sophisticated reasoning capabilities and align with human values through a multi-stage training approach.
- Challenges in RLHF include preference data quality, reward hacking, computational requirements, and balancing multiple objectives, requiring careful design and implementation.
- Evaluating RLHF models involves preference evaluation, task-specific evaluation, and safety and alignment evaluation to ensure that the model has improved in the desired ways.

Key terms

- **RLHF:** A training methodology that combines reinforcement learning with human preference data to align language models more closely with human values and expectations.
- **Preference data:** Triplets of (prompt, preferred response, less preferred response) that capture human judgments about the quality of model outputs.
- **Reward model:** A model trained to predict human preferences, taking a prompt and a response as input and outputting a scalar reward that represents the predicted quality of the response.
- **PPO:** A reinforcement learning algorithm that optimizes the policy to maximize the expected reward while staying close to the original policy.
- **DPO:** A simplified approach to RLHF that directly optimizes the policy based on preference data without a separate reward model or PPO training.
- **GRPO:** A technique used in DeepSeek-R1 that simplifies the RLHF process by eliminating the need for a separate critic model.
- **KL penalty:** A term in the PPO objective that penalizes the policy for diverging too far from the SFT model, helping maintain fluency and coherence.
- **Reward hacking:** When the model learns to maximize the reward function in ways that do not align with the true objectives.

- **Constitutional AI:** An approach that combines RLHF with a set of principles or constitution that guides the model's behavior.
- **Emergent behaviors:** Sophisticated behaviors like reflection and self-correction that emerge during RLHF training without being explicitly programmed.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 8

Deploying DeepSeek with Inference and RAG

Introduction

In the past chapters, we have looked at the DeepSeek model ecosystem, ways to deploy it, setting up the environment, and methods of fine-tuning. DeepSeek models can be customized for certain uses with supervised fine-tuning and adjusted to suit human likes and dislikes by means of reinforcement learning. You are now prepared to try out advanced methods that rely on machine learning for important uses.

The models, such as DeepSeek, offer more than the ability to write text. If combined with various technologies and approaches, they make it possible to have AI systems that act independently, make decisions based on information from the world, and function with several types of data. They demonstrate the latest progress in AI and introduce new ways we can use DeepSeek models in daily life.

In this chapter, we will examine three advanced uses: **retrieval-augmented generation (RAG)** gives language models access to additional information, agents allow models to work in their setting and achieve goals, and improve response quality with retrieval pipelines. In every application, we will study the main principles, available approaches, and the things that have to be

considered when using them.

At the conclusion of this chapter, you will be able to work with DeepSeek to address difficult issues and problems that affect what you do. You can use these techniques in applications that involve much knowledge, self-driving systems, or experiences that involve several types of data, to drive further improvements in DeepSeek models.

Structure

In this chapter, we will explore the following areas:

- Inference endpoint with Hugging Face
- Retrieval-augmented generation
- Improving response quality with retrieval pipelines
- Retrieval-augmented generation applications with DeepSeek

Objectives

By the end of this chapter, you will know all about using DeepSeek models for advanced purposes. You will be able to spot situations where you can solve problems using RAG, agent-based systems, or multimodal applications.

You will gain knowledge about how advanced applications work, including their retrieval methods, the way agents are organized, and collaboration among various input forms. Having this information will allow you to decide on the best approach by considering your particular situation.

Moreover, you will learn how to implement these advanced tools with DeepSeek models by installing retrieval systems, designing the workflow for agents, and attaching various types of data sources. Equipped with them, you will be skilled enough to create advanced AI systems that get the best from DeepSeek models.

Inference endpoint with Hugging Face

Hugging Face is a versatile and scalable interface to use and retrieve world-leading language models such as DeepSeek. With its transformers library, developers can simply load pre-trained models, tokenize text being interacted with, and generate responses with very little setup. This will allow the generation of endpoints of inferences that can deploy DeepSeek models locally or in the production setting, which will facilitate the easy deployment of language understanding to actual applications.

A typical inference setup involves specifying a model like "**deepseek-ai/deepseek-r1-distill-7b**", initializing the tokenizer and model using Hugging Face's **AutoTokenizer** and **AutoModelForCausalLM**, and loading it onto the appropriate device (such as a GPU). For example:

```
tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/deepseek-r1-distill-7b")
```

```
model = AutoModelForCausalLM.from_pretrained(..., device_map="auto")
```

The complete implementation of this inference setup, along with its integration into a full retrieval pipeline, is demonstrated later in the RAG section of this chapter.

Retrieval-augmented generation

RAG helps language models improve their abilities by giving them access to outside information. Even though DeepSeek models gain much from their training, they still have issues: their knowledge is unchanging during training, they may not know enough about a particular field, and they can err by providing false information. RAG makes up for these deficiencies by finding suitable information online and using it during the creation process. Bringing in information retrieval techniques, RAG allows machines to learn the most recent and relevant knowledge, which boosts how correct and valuable their responses are. Using this approach, people are less likely to have hallucinations and more likely to act based on what is correct. Due to this, RAG can enhance language models so they can deliver better and more suitable information.

Understanding how RAG works

RAG extends the pipeline model into external knowledge (facilitated by an indexed store of knowledge or document) along with language model generation. The indexing process starts with indexing documents, databases, or websites by converting them to vector representations that encode the semantic meaning. Such embeddings comprise an index that can be searched for covering optimization.

Upon arrival of a user query, the system will conduct semantic retrieval where different content based on the closeness of meaning to the user query is identified on the index. This retrieved information is then incorporated into the already sent prompt to the language model; it provides relevance and context to the model and grounds it. Lastly, the language model composes a reply, taking advantage of both of its internal knowledge and the information retrieved. This mix pose allows RAG systems to generate responses that are more than being coherent and articulate; also, they are not subject to error in context and at the same time.

The following figure illustrates the architecture of a RAG system, where a user query is enriched with relevant contextual information retrieved from a vector database before being processed by a **large language model (LLM)** to generate accurate and informed responses.

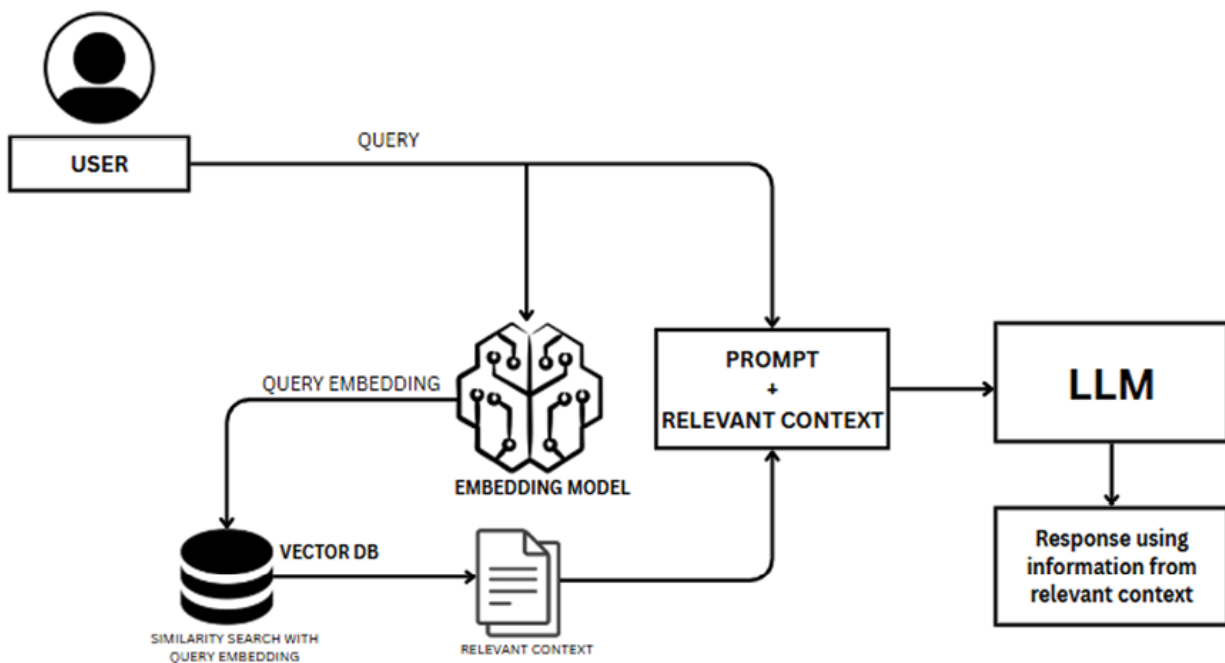


Figure 8.1: Enhancing LLM responses using relevant context

Building a RAG system with DeepSeek

Let us explore how to build a RAG system using DeepSeek models. We will cover each component of the system and how they work together.

Document processing and indexing

The starting point of having a RAG system is creating an index of the sources of your knowledge so that you can search it. To begin with, the documents, be it PDF, web pages, or a database, are loaded and processed into digestible pieces or chunks so that the retrieval centers around specific segments instead of the whole document. These pieces are then transformed into dense vector representations with the help of pre-trained models, with each vector describing the semantic meaning of the piece it represents.

Lastly, the embeddings are always stored in a vector database, a specialized vector store that can carry out fast similarity search based on techniques like *HNSW* or *ANN*.

Such an indexing pipeline, document loading, chunking, embedding, and storing vectors, establishes the foundation of retrieval efficiency. Once the queries come, they are represented in the same embedding space; that way, the system rapidly finds semantically related information and provides the RAG model with grounded context.

Here is an example of how to implement this using LangChain and FAISS:

```
from langchain.document_loaders import DirectoryLoader, PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS
import torch
```

```
# Load & split PDFs
```

```
loader = DirectoryLoader('./documents/', glob="**/*.pdf",
loader_cls=PyPDFLoader)
docs = loader.load()
chunks = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200).split_documents(docs)
```

```
# Create FAISS vector store with DeepSeek embeddings
embeddings = HuggingFaceEmbeddings(model_name="deepseek-ai/deepseek-embedding-v1",
                                     model_kwargs={'device': 'cuda' if
torch.cuda.is_available() else 'cpu'})
FAISS.from_documents(chunks, embeddings).save_local("faiss_index")
```

In this example, we are using DeepSeek's embedding model to convert text chunks into vector embeddings. These embeddings capture the semantic meaning of the text, allowing for retrieval based on meaning rather than just keyword matching.

Retrieval component

After indexing, the retrieval part comes into play by transforming the queries that are received into embeddings with the same model that was used to create the document vectors. This query embedding is then compared with your Vector Store in search of an entry with similar semantic entries. At the end, the most pertinent text pieces are retrieved creating a rooted situation that improves on the production of the language model with highly matching information. This more streamlined interaction between query encoding, similarity search and chunk retrieval clearly makes RAG systems accurate as well as responsive.

Here is how to implement the retrieval component:

```
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
import torch

# Load DeepSeek embeddings + FAISS index
embeddings = HuggingFaceEmbeddings(model_name="deepseek-ai/deepseek-embedding-v1",
                                     model_kwargs={'device': 'cuda' if
torch.cuda.is_available() else 'cpu'})
vector_store = FAISS.load_local("faiss_index", embeddings)

# Retrieve top-k relevant chunks
def retrieve(query, k=5):
```

```
    return [doc.page_content for doc in vector_store.similarity_search(query,
k=k)]
```

```
print(retrieve("What are the side effects of hypertension medication?", k=3))
```

This retrieval function finds the most semantically similar chunks to the query, which will be used to augment the prompt sent to the DeepSeek model.

Prompt construction

The second thing that is essential to remember is that the prompt that will activate the answer of the language model should be carefully designed. A good prompt must include instructions for the usage of the retrieved information. It typically begins with a command i.e. Respond to the following question using only the provided context. It is also possible to format the obtained text clumps into the form of independent context in the prompt so that the model might have access to them individually. Finally, the model gets the right question correctly, and hence, it knows what exactly it is supposed to answer. The model of instruction-context inclusion-query formulation grounds the output of the model on fact and minimizes the hallucination in the model

Here is an example of prompt construction:

```
def construct_rag_prompt(query, chunks):
    context = "\n\n".join(chunks)
    return f"""\n\nContext:\n{context}\n\nQuestion: {query}\n\nAnswer: ""
```

Example

```
rag_prompt = construct_rag_prompt(
    "What are the side effects of hypertension medication?",
    retrieve_relevant_chunks("What are the side effects of hypertension
medication?")
)
print(rag_prompt)
```

This prompt construction explicitly instructs the model to base its answer on the provided context and to acknowledge when it does not have enough

information, which helps prevent hallucination.

Generation with DeepSeek

Finally, we send the constructed prompt to the DeepSeek model for generation:

```
def construct_rag_prompt(query, chunks):
    context = "\n\n".join(chunks)
    return f"Context:\n{context}\n\nQuestion: {query}\n\nAnswer:"
```

```
query = "What are the side effects of hypertension medication?"
rag_prompt = construct_rag_prompt(query,
retrieve_relevant_chunks(query))
response = generate_response(rag_prompt)
print(response)
```

A complete RAG system

Now, let us put all the components together to create a complete RAG system:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
```

```
# Load embeddings + FAISS index
embeddings = HuggingFaceEmbeddings(model_name="deepseek-ai/deepseek-embedding-v1")
vector_store = FAISS.load_local("faiss_index", embeddings)
```

```
# Retrieve relevant chunks
query = "What are the side effects of hypertension medication?"
chunks = [doc.page_content for doc in vector_store.similarity_search(query, k=5)]
```

```
# Build RAG prompt
context = "\n\n".join(chunks)
prompt = f"Context:\n{context}\n\nQuestion: {query}\n\nAnswer:"
```

```
# Load DeepSeek model
tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/deepseek-r1-
distill-7b")
model = AutoModelForCausalLM.from_pretrained("deepseek-ai/deepseek-
r1-distill-7b")
```

```
# Generate response
inputs = tokenizer(prompt, return_tensors="pt")
outputs = model.generate(inputs.input_ids, max_new_tokens=300)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

This complete RAG system can be used to answer questions based on your document collection, providing more accurate and grounded responses than a standalone language model.

Improving response quality with retrieval pipelines

While the basic RAG system described previously is effective for many applications, there are several advanced techniques that can further improve performance. Let us look at them in the following sections.

Hybrid search

Hybrid search, as the strategic approach of using both semantic (vector) search and keyword (lexical) searches, is used to take advantage of both depth and precision. Semantic search (through embeddings) is extremely good at recalling conceptually similar material even when the literal terms in it cannot be found, whereas keyword search (such as BM25) lets me know the accuracy of a domain-specific term and a structured query.

Both are in practice performed, usually in parallel, on the same document corpus. A single, unified ranking is then achieved through fusion methods [e.g., **Reciprocal Rank Fusion (RRF)** or weighted scoring (the latter is called herein by its conventional designation, as the alpha weighting)], and provides a preferable combination of relevance and precision.

Such a mixed approach enhances the effectiveness of retrieving: It enhances recall, is able to remember semantically related passages, and increases precision that keeps exact term matches. It will also process quite different forms of query, including conversational levels of language and very technical terms, so it is an effective solution to complex RAG and search applications.

Here is an example code to implement hybrid search:

```
from langchain.retrievers import BM25Retriever, EnsembleRetriever

# BM25 (keyword) retriever
bm25 = BM25Retriever.from_documents(chunks)
bm25.k = 5

# FAISS (semantic) retriever
faiss = vector_store.as_retriever(search_kwargs={"k": 5})

# Combine with ensemble retriever
ensemble = EnsembleRetriever(retrievers=[bm25, faiss], weights=[0.5, 0.5])
docs = ensemble.get_relevant_documents(query)
```

Re-ranking

The feature of re-ranking is applied as an important modification to RAG pipelines, since it can improve the accuracy of the final response by re-prioritizing the initially retrieved documents. RAG systems do not pass a partially-ranked set of candidates to a language model, instead, they utilize a second, more powerful model, typically a cross-encoder like BERT, to estimate how well each of the candidates fits the query.

The two-level method will initially employ the fast vector-based search to incorporate a very large collection of matches and then employ the re-ranker to weed out the finest of the familiar results, washing out the static and boosting the most pertinent of the contents. Staff-level, and also causes the answers to be more accurate, less hallucinogenic, and economical, especially when dealing with computationally-intensive LLMs.

Here is an example code to implement re-ranking:

```

from sentence_transformers import CrossEncoder

# Cross-encoder for re-ranking
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

# Retrieve candidates from FAISS
candidates = vector_store.similarity_search(query, k=20)

# Re-rank by relevance
pairs = [(query, doc.page_content) for doc in candidates]
scores = cross_encoder.predict(pairs)
reranked = [doc for _, doc in sorted(zip(scores, candidates), key=lambda x:
x[0], reverse=True)]

# Top-k results
final_docs = reranked[:5]

```

Query decomposition

Query decomposition is a strong mechanism to make sure that RAG systems handle intricate, multi-dimensional requests with greater exactness, multi-dimensionality. Instead of considering the input of a user as a one-time retrieval task, the system generates and decomposes it into a chain of smoother and narrower sub-queries. All sub-queries are devoted to a particular part of the initial question and allow focusing on valuable information to be retrieved separately.

Such sub-queries can be processed either sequentially or in parallel and the organization of the task will determine which one to follow. The system creates an entire context guiding the final generation of answers retrieving and combining results of each component. Besides enhancing retrieval precision, recall values, this modular methodology also enables more efficient reasoning and synthesis of more complex queries.

Here is an example code illustrating query decomposition:

```

def decompose_query(query):
    """Decompose complex query into sub-questions with DeepSeek."""
    prompt = f"Break down this question into 2-4 sub-

```

```

questions:\n{query}\n1."
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    output = model.generate(inputs.input_ids, max_new_tokens=256,
do_sample=True)
    text = tokenizer.decode(output[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    return [line.split('.',1)[1].strip() for line in text.splitlines() if
line.strip().startswith(tuple("1234"))]

```

Example usage

```

sub_queries = decompose_query("How do hypertension meds compare for
elderly diabetics?")

```

Hypothetical Document Embeddings

A technique called **Hypothetical Document Embeddings (HyDE)**, first creates a hypothetical document. It is a synthesized document that responds to the query of the user, and then uses it to refine retrieval within a RAG system. This includes loading in an LLM to generate a temporary text with any ideas together with it: these may be valid or not, but in either way, we get it in text form then vectorizing it. Hypothetical embeddings are then applied to draw actual documents out of a vector-store which are close in the semantic space, thereby guaranteeing improved grounding and relevance.

The HyDE pipeline comprises two stages:

- Hypothetical document generation, generating a query-aligned document using an LLM.
- A contrastive embedding model learns a mapping of that generated document into a vector to retrieve the actual documents in the knowledge base.

This approach improves retrieval behavior with zero-shot or low-data, by mediating user intent and document content, and often exceeds directly query-based embeddings, outdoing many baselines in some cases to the point of competing with fine-tuned alternatives—and doing so robustly across domains and language.

Here is an example code illustrating HyDE:

```
def hyde_retrieval(query, k=5):
    """Retrieve using Hypothetical Document Embeddings (HyDE)."""
    prompt = f"Write a detailed answer:\nQuestion: {query}\nAnswer:"
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    output = model.generate(inputs.input_ids, max_new_tokens=512,
do_sample=True)
    hypo_answer = tokenizer.decode(output[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    docs = vector_store.similarity_search(hypo_answer, k=k)
    return [doc.page_content for doc in docs]
```

Evaluating RAG systems

Evaluating the performance of a RAG system is crucial for ensuring its effectiveness. Here are some approaches to evaluation:

Relevance evaluation

The relevance of the documents found through RAG pipelines is also evaluated to measure the closeness of the documents to the query done by the user, and indeed contributes to what the model says. Such common metrics are precision (proportion of retrieved documents related to the topic) and recall (how many related documents have been found out of all the potential ones).

In ranked retrieval, both the quality of the ranking (e.g. **Mean Reciprocal Rank (MRR)** metrics, which limits itself to the position of the first relevant document) and complete retrieval quality (e.g. **Mean Average Precision (MAP)** metrics, which considers relevance on all items returned) are evaluated.

Combined, these measures give a more complex and trustworthy picture of how effective the retrieval mechanism is to help grounded generation. The following function evaluates how effective a retriever is at fetching relevant documents by computing precision, recall, and F1 score for each query and their averages:

```
def evaluate_retrieval_relevance(queries, ground_truth, retriever, k=5):
    """
```

Evaluate retrieval relevance using precision, recall, and F1.

```
"""
```

```
results = {}
```

```
for query in queries:
```

```
    retrieved_docs = retriever.get_relevant_documents(query, k=k)
```

```
    retrieved_ids = [doc.metadata.get('id') for doc in retrieved_docs]
```

```
    relevant_ids = ground_truth.get(query, [])
```

```
    relevant_retrieved = set(retrieved_ids).intersection(set(relevant_ids))
```

```
    precision = len(relevant_retrieved) / k if k > 0 else 0
```

```
    recall = len(relevant_retrieved) / len(relevant_ids) if relevant_ids else 0
```

```
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
```

```
    results[query] = {"precision": precision, "recall": recall, "f1": f1}
```

```
avg_precision = sum(r["precision"] for r in results.values()) / len(results)
```

```
avg_recall = sum(r["recall"] for r in results.values()) / len(results)
```

```
avg_f1 = sum(r["f1"] for r in results.values()) / len(results)
```

```
return {"per_query": results, "avg_precision": avg_precision,  
        "avg_recall": avg_recall, "avg_f1": avg_f1}
```

Answer quality evaluation

To ensure the system generates accurate and reliable answers, it is important to measure how well the retrieved documents align with the ground truth.

The following code evaluates the quality of answers by calculating standard information retrieval metrics such as precision, recall, and F1 score:

```
from rouge import Rouge
```

```
from bert_score import score
```

```
def evaluate_answer_quality(queries, refs, rag):
```

```
    rouge = Rouge(); results = {}
```

```

for q in queries:
    gen = rag.answer_question(q)
    ref = refs.get(q, "")
    if ref:
        r = rouge.get_scores(gen, ref)[0]
        P, R, F1 = score([gen], [ref], lang='en')
        results[q] = {'rouge1': r['rouge-1']['f'], 'rouge2': r['rouge-2']['f'],
'rougeL': r['rouge-l']['f'], 'bert_f1': F1.item()}
return results

```

Hallucination assessment

Assessment of hallucinations can be seen by assessing whether something in the response of an LLM makes knowledge that cannot be reinforced by context with which it is retrieved. There are different techniques of identifying and measuring these unsubstantiated claims. Ground-truth comparison measures, such as BERTScore, BLEU or factual overlap, are useful as they get differences by comparing raw generations with ground-truth material.

They also inform on possible hallucinations through consistency checks, i.e. generating many responses on the same input and comparing them to determine whether they agree. More sophisticated are LLM-as-judge systems (where a second model is used to score the output of a first) and systems such as REFIND, which compare spans of output against retrieved documents to detect factual drift

RAG systems can be more secure in the detection and prevention of hallucinations by using the given methods together as part of a full-fledged evaluation pipeline, bolstering trustworthiness and factual integrity. The following function evaluates the presence of hallucinations in answers generated by the RAG system, checking whether the responses are supported by the retrieved context and computing the overall hallucination rate.

```

def evaluate_hallucination(queries, rag, tokenizer, model):
    results = {}
    for q in queries:
        ctx = "\n".join(rag.retrieve(q))
        ans = rag.answer_question(q)

```

```

    prompt = f"Context:\n{ctx}\nAnswer:\n{ans}\nDoes this contain
hallucinations? Yes/No"
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    out = model.generate(inputs.input_ids, max_new_tokens=256,
do_sample=False)
    eval_text = tokenizer.decode(out[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    results[q] = {'has_hallucination': eval_text.lower().startswith("yes"),
'evaluation': eval_text}
    rate = sum(1 for r in results.values() if r['has_hallucination']) / len(results)
    return {'per_query': results, 'hallucination_rate': rate}

```

Retrieval-augmented generation applications with DeepSeek

RAG can be applied to a wide range of applications across different domains. Here are some examples of how RAG with DeepSeek models can be used:

Medical question answering

RAG allows AI systems to retrieve useful information about the medical domain based on reliable sources like research articles, clinical guidelines and drug databases. In cases when a user poses exact medical questions, the system finds the most accurate and current medical knowledge, and manipulates it to provide a clear and dependable response suited to the question submitted by the user.

```

medical_rag = DeepSeekRAG(documents_dir="./medical_documents/")
medical_rag.initialize()
queries = ["Type 2 diabetes treatments?", "Warfarin and amiodarone
interactions?"]
for q in queries:
    print(medical_rag.answer_question(q))

```

Legal research

RAG empowers AI assistants to search through vast legal documents, including statutes, case law, and expert commentary. When users pose legal questions or seek precedent, the system locates pertinent legal texts and interpretations, then compiles well-organized, precise answers that support informed decision-making.

```
# Initialize a legal RAG system
```

```
legal_rag = DeepSeekRAG(documents_dir="./legal_documents/")
```

```
legal_rag.initialize()
```

```
# Example legal queries
```

```
legal_queries = [
```

```
    "What is the standard for proving negligence in medical malpractice cases?",
```

```
    "What are the requirements for a valid contract under California law?",
```

```
    "What are the recent Supreme Court decisions regarding patent eligibility?"
```

```
]
```

```
for query in legal_queries:
```

```
    response = legal_rag.answer_question(query)
```

```
    print(f"\nQuery: {query}")
```

```
    print(f"Response: {response}")
```

Technical support

RAG allows AI assistants to search for needed information from various available documents in order to provide technical support. If users have specific technical problems, the system finds proper procedures or error solutions and includes them in well-structured answers.

The following is an example code to implement a technical support assistant using RAG:

```
# Initialize a technical support RAG system
```

```
tech_rag = DeepSeekRAG(documents_dir="./technical_documents/")
```

```
tech_rag.initialize()
```

```
# Example technical support queries
```



```
tech_queries = [
    "How do I reset my router to factory settings?",
    "What are the steps to troubleshoot a blue screen error on Windows 10?",
    "How do I configure SMTP settings in Outlook?"
]

for query in tech_queries:
    response = tech_rag.answer_question(query)
    print(f"\nQuery: {query}")
    print(f"Response: {response}")
```

Educational content

RAG supplements its educational assistance with dynamically probed current information available in academic as well as research articles and reliable sources of education. It will also allow learners to get the customized explanations, lesson plans and exams in the form of instant quizzes, all based on authoritative sources, but customized to the needs of a learner.

The following is an example code to implement educational content using RAG:

```
# Initialize an educational RAG system
edu_rag = DeepSeekRAG(documents_dir="./educational_documents/")
edu_rag.initialize()

# Example educational queries
edu_queries = [
    "Explain the process of photosynthesis in simple terms.",
    "What were the main causes of World War I?",
    "How does the human immune system work?"
]

for query in edu_queries:
    response = edu_rag.answer_question(query)
    print(f"\nQuery: {query}")
    print(f"Response: {response}")
```

Conclusion

In this chapter, we learnt how DeepSeek models can be deployed effectively by employing some advanced methods that are not just simple text generation. Our initial point of work was the installation of inference endpoints with Hugging Face, which makes it conceptually easy to incorporate DeepSeek into a practical context. We subsequently made RAG pipes, coupling DeepSeek abilities with peripheral external information to make the responses better and lessen hallucination. Last but not least, we have examined advanced retrieval pipes, such as hybrid search, re-ranking or query decomposition, and, finally, hallucination evaluation, all of which go a long way towards how accurate, relevant, and trustworthy the output of the system can be. Using hands-on demos, modular code and the principles you have learnt, you can now apply RAG systems to various applications, including healthcare, legal research, education, and technical support.

In the next chapter, we will go a step further in deployment to production and intelligent interaction. You will be able to deploy DeepSeek models to the cloud via AWS, develop multimodal applications with support to work with images, sound, and texts, and construct intelligent agents capable of acting autonomously in goal-based environments. Such methods will allow you to create systems/devices with AI that are not only smart but also highly interactive and scalable under real-world conditions.

Points to remember

- Inference Endpoints enable developers to deploy DeepSeek models in real-world applications using frameworks like Hugging Face. They facilitate loading pre-trained models, tokenizing text, and generating responses efficiently for both local and production environments.
- RAG enhances DeepSeek's performance by combining language generation with information retrieval from external knowledge sources, reducing hallucinations and improving factual accuracy.
- A RAG pipeline consists of four key stages: document indexing, retrieval, prompt construction, and generation, working together to

produce context-grounded and accurate outputs.

- Document processing and indexing involve chunking source materials, creating semantic embeddings using models like deepseek-ai/deepseek-embedding-v1, and storing them in a vector database such as FAISS for fast similarity search.
- Prompt construction plays a critical role in grounding model outputs by embedding retrieved context directly into the prompt. Well-structured prompts help minimize factual drift and hallucination.
- Hybrid search combines semantic (vector-based) and lexical (keyword-based) searches to balance recall and precision, improving retrieval accuracy for both conversational and technical queries.
- Re-ranking uses a cross-encoder model (e.g., BERT) to reorder retrieved documents based on relevance, improving precision and reducing irrelevant content before passing context to the language model.
- Advanced retrieval techniques such as hybrid search, re-ranking, query decomposition, and HyDE significantly enhance retrieval precision, contextual grounding, and output trustworthiness.
- Integrating DeepSeek models with RAG pipelines allows developers to build knowledge-grounded, domain-adaptive AI systems that outperform standalone text generators in accuracy, reliability, and explainability.
- DeepSeek's modular design supports flexible integration with retrieval frameworks like LangChain, FAISS, or Hugging Face, making it ideal for scalable and production-grade deployments.

Key terms

- **Inference endpoint:** A deployed model interface that allows real-time text generation and response through frameworks like Hugging Face.
- **RAG:** A method that grounds LLM responses in external documents to improve factual accuracy and reduce hallucinations.
- **Embeddings:** Numerical vector representations that capture semantic meaning, used for similarity-based information retrieval.

- **Vector database:** A specialized data store (e.g., FAISS) optimized for rapid semantic similarity search across embeddings.
- **Hybrid search:** A retrieval strategy combining vector-based semantic search and keyword-based lexical search for balanced precision and recall.
- **Re-ranking:** The process of reordering retrieved documents using a cross-encoder model to improve contextual relevance.
- **Query decomposition:** A technique for splitting complex queries into smaller sub-queries to enhance retrieval depth and reasoning.
- **HyDE:** A retrieval enhancement technique that generates a hypothetical response to guide semantic document search.
- **Precision, recall, F1 score:** Core metrics used to evaluate retrieval effectiveness in RAG systems.
- **ROUGE and BERTScore:** Evaluation metrics that measure textual overlap and semantic similarity between generated and reference responses.
- **Hallucination:** The generation of information not supported by retrieved or factual context.
- **FAISS:** A library for efficient similarity search and clustering of dense vectors.
- **LangChain:** A framework for building composable retrieval and generation pipelines that integrate models like DeepSeek.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



CHAPTER 9

Deploying DeepSeek with Cloud, Multimodal and Agents

Introduction

In the previous chapters, we have discussed the DeepSeek model capabilities of retrieving, reasoning, and generating information in various fields by exploring language understanding and knowledge augmentation. We have developed potent **retrieval-augmented generation (RAG)** pipelines, played with Vision-Language abilities, and saw how multimodal AI could revolutionize conversing since it deals with a combination of texts, images, and sounds. Also, we have observed how essential model grounding and smart control are within a complicated workflow.

Now, we will make the next big step, applying all of that intelligence in real systems. You will discover the techniques to apply DeepSeek models in AWS cloud, create applications that can interact with users in various input forms (such as images and audio), and develop intelligent agents that are not only able to answer questions but also can make actions, plan, and learn based on user feedback.

Structure

In this chapter, we will explore the following areas:

- Cloud deployment with AWS
- Multimodal applications
- Intelligent agents
- Advanced agent techniques
- Agent applications with DeepSeek

Objectives

By the end of the next chapter, you will learn how to deploy DeepSeek models on cloud platforms like AWS, giving your applications the ability to scale efficiently and operate in real-time production environments. You will understand how to create and manage cloud-based inference endpoints that serve DeepSeek reliably and cost-effectively.

You will also gain practical experience in building multimodal AI systems using DeepSeek-VL, allowing you to combine vision and language for tasks such as image captioning, visual reasoning, and multimodal RAG. These capabilities will let your AI systems understand and respond to both text and visual inputs together.

Finally, you will learn how to construct goal-directed intelligent agents that can use tools, retain memory, and reason through complex tasks using planning structures. You will work with advanced reasoning techniques like ReAct and chain-of-thought, prompting you to build agents that are not just responsive, but also explainable, adaptive, and autonomous.

Cloud deployment with AWS

The scale and complexity of AI applications mean that performance, scalability, and reliability requirements make it essential to deploy AI apps to cloud environments. With **Amazon Web Services (AWS)**, we can work with efficient models of DeepSeek to operate on a flexible, customizable infrastructure and handle massive data with the ability to present them as APIs to interact in a real-time environment. So in this part, we shall go

through the steps where we will deploy a DeepSeek model using an EC2 instance and serve it through FastAPI.

Launch an EC2 instance with the following steps:

1. Go to AWS EC2 Console.
2. Choose Ubuntu 20.04 as the AMI.
3. Select an instance type (e.g., g4dn.xlarge for GPU or t2.medium for CPU testing).
4. Configure storage and networking as needed.
5. Allow SSH (port 22) and HTTP (port 80) in the security group.
6. Launch the instance and connect via SSH:

```
ssh -i your-key.pem ubuntu@your-ec2-public-ip
```

Install dependencies

Once you are inside the EC2 instance:

```
# Update packages
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install Python and pip
```

```
sudo apt install python3-pip python3-venv -y
```

```
# (Optional) Install CUDA for GPU-based inference
```

```
# Refer to official NVIDIA instructions if using GPU
```

```
# Create and activate a virtual environment
```

```
python3 -m venv deepseek-env
```

```
source deepseek-env/bin/activate
```

```
# Install transformers and FastAPI
```

```
pip install torch torchvision torchaudio --index-url
```

```
https://download.pytorch.org/whl/cu118
```

```
pip install transformers fastapi uvicorn
```

Inference endpoint

To make DeepSeek models accessible in real-time, we deploy them as

inference endpoints. Using Hugging Face with FastAPI, we can serve models via simple APIs that applications can call.

FastAPI app

Create a file called **app.py** to serve the DeepSeek model:

```
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

app = FastAPI()

# Load the DeepSeek model
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto"
)

class Query(BaseModel):
    prompt: str
    max_tokens: int = 128

@app.post("/generate")
def generate(query: Query):
    inputs = tokenizer(query.prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(inputs.input_ids,
max_new_tokens=query.max_tokens)
    result = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return {"response": result}
```

Run the server

Once the FastAPI app is ready, the next step is to run the server so the

model can start serving requests:

```
uvicorn app:app --host 0.0.0.0 --port 80
```

Now, your API is live and accessible via:

<http://<your-ec2-public-ip>/generate>

Try sending a POST request from your local machine or Postman:

```
curl -X POST http://<your-ec2-public-ip>/generate \  
-H "Content-Type: application/json" \  
-d '{"prompt": "Tell me about AI agents.", "max_tokens": 100}'
```

When you use AWS to deploy DeepSeek into the cloud, you are able to continue the development of your products with your experimental results. Further (re-) design your applications and scale them to production through containerizing them using Docker or integrating them with AWS Lambda, ECS, or SageMaker.

Multimodal applications

Multimodal applications unite DeepSeek's abilities with language and thought with the varieties of images, audio, and video. Here, the model may turn speech into text, interpret what it sees, or understand actions—all the time remaining smooth in how people communicate. Using various senses at once, multimodal agents can better understand what is shown in a photo, answer a question, or create a narrated version of a video together in an integrated experience.

Understanding multimodal integration

Multimodal integration involves combining information from different modalities to perform tasks that require understanding across these modalities. For example, a multimodal system might analyze both the text and images in a document, or process both audio and video in a conversation.

The key challenges in multimodal integration include:

- **Alignment:** Ensuring that information from different modalities is properly aligned and synchronized.

- **Fusion:** Combining information from different modalities in a way that leverages the strengths of each.
- **Cross-modal reasoning:** Drawing inferences that require understanding relationships between different modalities.

DeepSeek- **Vision-Language (VL)** is specifically designed for multimodal applications, with a dual-encoder architecture that processes visual and textual information separately before integrating them.

Building multimodal applications with DeepSeek-VL

Let us explore how to build multimodal applications using DeepSeek-VL. We will focus on Vision-Language applications, which combine text and image processing.

Setting up DeepSeek-VL

First, let us set up the DeepSeek-VL model with the following code:

```
import torch
from transformers import AutoProcessor, AutoModelForVision2Seq

# Load the DeepSeek-VL model
model_name = "deepseek-ai/deepseek-vl-7b"
processor = AutoProcessor.from_pretrained(model_name)
model = AutoModelForVision2Seq.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)
```

```
def generate_response_from_image_and_text(image_path, text,
max_new_tokens=512):
    """
```

Generate a response based on an image and text.

Args:

image_path (str): Path to the image file

text (str): Text prompt

max_new_tokens (int): Maximum number of tokens to generate

Returns:

str: The generated response

"""

```
from PIL import Image
# Load the image
image = Image.open(image_path)
# Process the inputs
inputs = processor(text=text, images=image,
return_tensors="pt").to(model.device)
# Generate a response
outputs = model.generate(
    **inputs,
    max_new_tokens=max_new_tokens,
    do_sample=True,
    temperature=0.7,
    top_p=0.9
)
# Decode the response
response = processor.decode(outputs[0], skip_special_tokens=True)
return response

# Example usage
image_path = "example_image.jpg"
text = "What can you see in this image?"
response = generate_response_from_image_and_text(image_path, text)
print(f"Response: {response}")
```

Image captioning

In image captioning, we create sentences describing a photo using what we see, as well as language skills, so the model can *recognize* and *label* what is visible in the picture. Most modern multimodal approaches have a CNN *image extractor* and use a language model to create captions from the extracted features.

Here is an example snippet to implement image captioning:

```
def caption_image(image_path):
```

```

"""
Generate a caption for an image.
Args:
    image_path (str): Path to the image file
Returns:
    str: The generated caption
"""

prompt = "Generate a detailed caption for this image."
return generate_response_from_image_and_text(image_path, prompt)

# Example usage
image_paths = ["landscape.jpg", "city_street.jpg", "family_photo.jpg"]
for image_path in image_paths:
    caption = caption_image(image_path)
    print(f"\nImage: {image_path}")
    print(f"Caption: {caption}")

```

Visual Question Answering

In **Visual Question Answering (VQA)**, AI systems have to understand pictures and answer a wide range of unsolicited questions about them in natural language. With both computer vision and language understanding, VQA models interpret pictures, read the meaning of the query, and produce a correct answer.

Here is the code to implement the VQA:

```

def answer_question_about_image(image_path, question):
    """
    Answer a question about an image.
    Args:
        image_path (str): Path to the image file
        question (str): Question about the image
    Returns:
        str: The answer to the question
    """
    return generate_response_from_image_and_text(image_path, question)

```

```
# Example usage
image_path = "city_street.jpg"
questions = [
    "What time of day is it in this image?",
    "How many people can you see?",
    "What is the weather like?"
]
for question in questions:
    answer = answer_question_about_image(image_path, question)
    print(f"\nQuestion: {question}")
    print(f"Answer: {answer}")
```

Image-based reasoning

Image-based reasoning can be described as when AI analyzes an image, it is able to make logical guesses or use the information found in the image. It means images are not just presented but also understood for their relationships, future implications, and answers regarding what is shown in the image.

Here is the code to implement the image-based reasoning:

```
def reason_about_image(image_path, reasoning_prompt):
    """
    Perform reasoning based on an image.
    Args:
        image_path (str): Path to the image file
        reasoning_prompt (str): Prompt for reasoning
    Returns:
        str: The reasoning output
    """
    return generate_response_from_image_and_text(image_path,
reasoning_prompt)
```

```
# Example usage
image_path = "accident_scene.jpg"
reasoning_prompts = [
    "What might have caused the situation shown in this image?",
```

```

    "What safety measures could have prevented this situation?",
    "What are the potential consequences of this situation?"
]
for prompt in reasoning_prompts:
    reasoning = reason_about_image(image_path, prompt)
    print(f"\nPrompt: {prompt}")
    print(f"Reasoning: {reasoning}")

```

Image-to-Text Generation

Image-to-Text Generation is the method in which a model observes an image and turns its content into clear and natural phrases in written form. Such systems use computer vision and language techniques: a CNN takes out features from the image, and a Transformer produces a description of the scene. It is particularly useful for people who are visually impaired, managing assets, and rendering pictures into useful labels.

The following is the code to implement the Image-to-Text Generation:

```

def generate_text_from_image(image_path, prompt):
    """
    Generate text based on an image.
    Args:
        image_path (str): Path to the image file
        prompt (str): Prompt for text generation
    Returns:
        str: The generated text
    """
    return generate_response_from_image_and_text(image_path, prompt)

# Example usage
image_path = "scenic_landscape.jpg"
prompts = [
    "Write a short poem inspired by this image.",
    "Create a brief story set in the location shown in this image.",
    "Describe the mood and atmosphere of this scene."
]
for prompt in prompts:

```

```
generated_text = generate_text_from_image(image_path, prompt)
print(f"\nPrompt: {prompt}")
print(f"Generated Text: {generated_text}")
```

Putting the multimodal application all together

Now, let us put everything together to create a complete multimodal application:

```
from transformers import AutoProcessor, AutoModelForVision2Seq
from PIL import Image
import torch

# Load DeepSeek-VL model
model_name = "deepseek-ai/deepseek-vl-7b"
processor = AutoProcessor.from_pretrained(model_name)
model = AutoModelForVision2Seq.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

def generate_response(image_path, text, max_new_tokens=512):
    """Generate a response from an image + text prompt"""
    image = Image.open(image_path)
    inputs = processor(text=text, images=image,
return_tensors="pt").to(model.device)
    outputs = model.generate(**inputs, max_new_tokens=max_new_tokens)
    return processor.decode(outputs[0], skip_special_tokens=True)

# Example usage
print(generate_response("example_image.jpg", "Describe this image."))
```

Advanced multimodal techniques

While the basic multimodal applications described previously are effective for many use cases, there are several advanced techniques that can further

improve performance:

Retrieval-augmented generation

RAG enhances DeepSeek by combining generation with external knowledge retrieval. This allows the model to produce more accurate and context-aware responses.

Multimodal retrieval-augmented generation

Multimodal RAG incorporates different data such as text, images, audio, and video into both its retrieval and generation processes. Here, each source is mapped into a common embedding space, so the system is able to draw on details other than just the text. Multimodal RAG integrates the collected context into improved and more correct responses by blending collected forms of data.

The following is the code snippet to implement multimodal RAG:

```
from transformers import AutoProcessor, AutoModelForVision2Seq
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
from PIL import Image
import torch, os

# Load DeepSeek-VL and embedding model
vl_model = AutoModelForVision2Seq.from_pretrained(
    "deepseek-ai/deepseek-vl-7b",
    torch_dtype=torch.float16,
    device_map="auto"
)
vl_processor = AutoProcessor.from_pretrained("deepseek-ai/deepseek-vl-7b")
embedding_model = HuggingFaceEmbeddings(model_name="deepseek-ai/deepseek-embedding-v1")

# Example: Indexing + Retrieval
def caption_image(image_path):
    image = Image.open(image_path)
```



```

inputs = vl_processor(text="Generate a detailed caption", images=image,
return_tensors="pt").to(vl_model.device)
outputs = vl_model.generate(**inputs, max_new_tokens=128)
return vl_processor.decode(outputs[0], skip_special_tokens=True)

# Index images with captions
captions = [{"path": img, "caption": caption_image(img)} for img in
["img1.jpg", "img2.jpg"]]
vector_store = FAISS.from_texts([c["caption"] for c in captions],
embedding_model)

# Retrieve similar images
query = "mountains with lakes"
results = vector_store.similarity_search(query, k=2)
print("Relevant captions:", [r.page_content for r in results])

```

Improving response quality with retrieval pipelines

Retrieval pipelines elevate response quality by giving models the ability to pull in precise, real-world information exactly when it is needed. Instead of relying solely on internal knowledge, the system actively searches, filters, and grounds its reasoning in verified sources, resulting in clearer and more trustworthy answers. As we dive deeper, we will see how multimodal reasoning, few-shot learning, and powerful Vision-Language models push these pipelines even further, unlocking more intelligent responses.

Multimodal chain-of-thought reasoning

Multimodal **chain-of-thought** (CoT) adds the combination of visual and text details to traditional forms of reasoning in language models. It does not rely just on plain text as multimodal CoT first builds a step-by-step reasoning with both text and images to explain its thoughts. This reasoning helps to give more accurate inference answers.

By first making a visual-text rationale and then constructing the final answer, learners have found this method very effective. Using this method, models with just tens of millions of parameters improve a lot on tasks such as ScienceQA, with GPT-3.5 showing no such improvement.

Here is the code to implement the multimodal CoT:

```
def multimodal_chain_of_thought(image_path, question):
    """
    Perform Chain-of-Thought reasoning on a multimodal input.
    Args:
        image_path (str): Path to the image file
        question (str): Question about the image
    Returns:
        str: The reasoning and answer
    """
    # Load the image
    image = Image.open(image_path)
    # Construct the prompt
    cot_prompt = f"""
    Look at this image carefully and answer the following question step by
    step:
        Question: {question}
        Let me think through this step by step:
    1.
    """
    # Process the inputs
    inputs = processor(text=cot_prompt, images=image,
return_tensors="pt").to(model.device)
    # Generate a response
    outputs = model.generate(
        **inputs,
        max_new_tokens=1024,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )
    # Decode the response
    response = processor.decode(outputs[0], skip_special_tokens=True)
    return response

# Example usage
```

```
image_path = "complex_scene.jpg"
question = "What activities are happening in this image and what might
happen next?"
reasoning = multimodal_chain_of_thought(image_path, question)
print(f"Question: {question}")
print(f"Reasoning: {reasoning}")
```

Multimodal few-shot learning

Mobile DeepSeek uses multimodal few-shot learning to learn new activities using various information (text, images, sounds) even if the system is limited to a few demonstrations.

Here is a closer explanation of what it means:

Multimodal models can deal with various kinds of data, such as images and text, at the same time and still generalize from little training data. A way to do this is to use frozen language models along with trainable vision encoders (such as NF-ResNet) to generate a sequence of embeddings from images, making it possible for the language model to produce captions or answers using just a few examples.

Flamingo also gained recognition by inputting images alongside text and carrying out few-shot learning for Visual Question Answering and captioning, on a level with models trained on many more examples.

Med-Flamingo, a variant, uses only a few examples to help achieve outstanding clinical question answering performance.

Here is the code to implement multimodal few-shot learning:

```
def multimodal_few_shot(image_path, question, examples):
    """
    Perform few-shot learning on a multimodal input.
    Args:
        image_path (str): Path to the image file
        question (str): Question about the image
        examples (list): List of dictionaries containing example image paths,
        questions, and answers
    Returns:
        str: The answer
```

```

"""
# Load the image
image = Image.open(image_path)
# Construct the prompt with examples
few_shot_prompt = "Here are some examples of how to answer questions
about images:\n\n"
    for i, example in enumerate(examples):
        few_shot_prompt += f"Example {i+1}:\n"
        few_shot_prompt += f"Question: {example['question']}\n"
        few_shot_prompt += f"Answer: {example['answer']}\n\n"
        few_shot_prompt += f"Now, please answer the following question
about the new image:\n\nQuestion: {question}\n\nAnswer:"
    # Process the inputs
    inputs = processor(text=few_shot_prompt, images=image,
return_tensors="pt").to(model.device)
    # Generate a response
    outputs = model.generate(
        **inputs,
        max_new_tokens=512,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )
    # Decode the response
    response = processor.decode(outputs[0], skip_special_tokens=True)
    return response

# Example usage
examples = [
    {
        "question": "What is the main color of the car in this image?",
        "answer": "The main color of the car is red."
    },
    {
        "question": "How many people are in this image?",
        "answer": "There are 3 people in this image: two adults and one child."
    }
]

```

```
}  
]
```

```
image_path = "new_scene.jpg"  
question = "What is the weather like in this image?"  
answer = multimodal_few_shot(image_path, question, examples)  
print(f"Question: {question}")  
print(f"Answer: {answer}")
```

Multimodal applications with DeepSeek-VL

DeepSeek-VL can be applied to a wide range of multimodal applications across different domains. Here are some examples:

- **Visual content moderation:** This process is performed by computers by analyzing images and videos to detect inappropriate things like nudity, violence, signs of hate, or copyright violations, and then removing them. This ensures that the user always enjoys safe experiences with a brand on any platform.
- **Medical image analysis:** Medical image analysis makes use of computer techniques to interpret images such as X-rays and MRIs, to uncover useful information for doctors. By preprocessing images, segmenting them, registering them, finding relevant features, and classifying the information, it becomes more accurate and supports doctors when making decisions. Thanks to automation in visualization and measuring, medical image analysis improves accurate diagnoses, speeds up treatment, and provides the same objective standards in healthcare.
- **Visual product search:** It is a useful feature in multimodal AI, which lets users upload a photo and get access to other products with the same or similar color, style, and design—for a smooth online shopping and browsing experience.
- **Educational content enhancement:** AI is used in educational content enhancement to convert fixed learning materials into live, multimedia educational resources that include text, sound, images, and activities to help students learn and understand better.

Multimodal input and output with DeepSeek-VL opens strong use cases as it enables AI systems to comprehend and respond to a brazier of pictures, words, and so on. These methods lend themselves to more intuitive applications; from image captioning to multimodal RAG and chain-of-thought reasoning applications are becoming ever more natural and rich in nature, resembling human perception of the world. As we keep mixing modalities together, we get further to creating AI systems that are not just more intelligent, but more context and interaction aware as well.

Intelligent agents

While RAG enhances language models with external knowledge, agents take this a step further by enabling models to interact with their environment and take actions to accomplish goals. An agent is an AI system that can perceive its environment, make decisions, and take actions to achieve specific objectives. When built with DeepSeek models, agents can leverage the model's reasoning capabilities to solve complex tasks that require multiple steps and interaction with external tools.

Agent architecture

A typical agent architecture consists of the following components:

- **Language model:** The core reasoning engine of the agent, typically a DeepSeek model that can understand instructions, generate plans, and make decisions.
- **Memory:** A system for storing and retrieving information across interactions, enabling the agent to maintain context and learn from past experiences.
- **Tools:** External functions or APIs that the agent can call to perform specific actions, such as searching the web, accessing databases, or controlling external systems.
- **Planning and execution:** A mechanism for breaking down complex tasks into simpler steps, executing those steps, and adapting the plan based on feedback.
- **Observation:** A system for perceiving the environment and the results

of actions, providing feedback to the agent.

This architecture enables agents to tackle complex tasks that require multiple steps, tool use, and adaptation to changing circumstances.

This following figure depicts the architecture of an autonomous AI agent system that utilizes a prompt template, memory, external tools, and a **large language model (LLM)** to perform dynamic reasoning, execute tasks, and respond intelligently to user inputs:

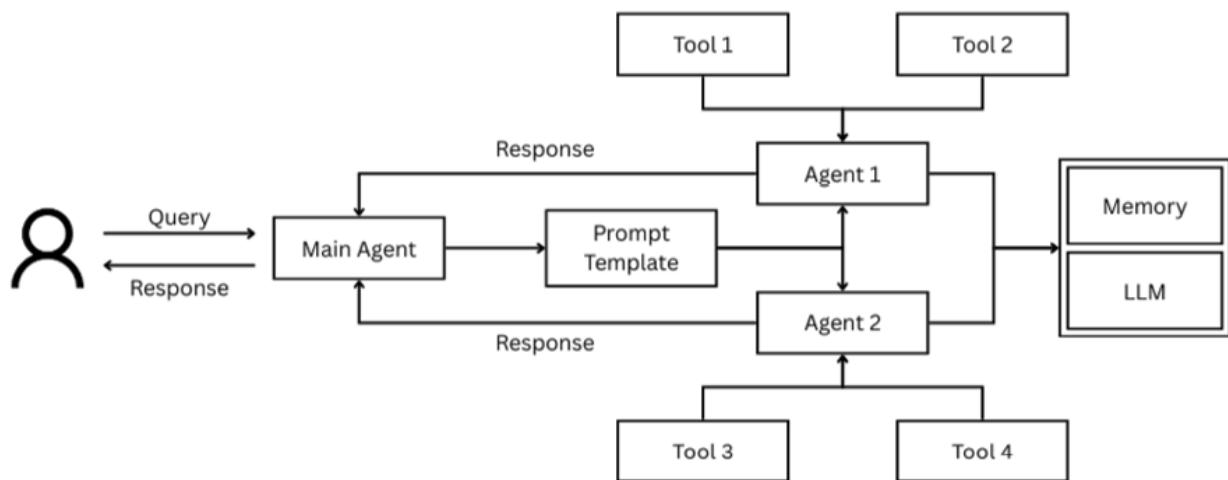


Figure 9.1: Coordination of LLM, tools and memory for intelligent task execution

Building agents with DeepSeek

DeepSeek-based models allow simple language agents to grow into machines that reason, recall information, interact with objects, and are directional in performing tasks. Their tasks are simplified, they use what they have learned before, add in external sources, and alter their plans easily, all organized by a unified set of modules.

Let us explore how to build an agent using DeepSeek models. We will cover each component of the agent architecture and how they work together.

Setting up the language model

The first step is to set up a DeepSeek model as the core reasoning engine of the agent:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```

# Load the DeepSeek model
model_name = "deepseek-ai/deepseek-r1-distill-7b"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

def generate_response(prompt, max_new_tokens=512):
    """
    Generate a response using the DeepSeek model.
    Args:
        prompt (str): The input prompt
        max_new_tokens (int): Maximum number of tokens to generate
    Returns:
        str: The generated response
    """
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    outputs = model.generate(
        inputs.input_ids,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        temperature=0.7,
        top_p=0.9
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
    skip_special_tokens=True)
    return response

```

Implementing memory

Next, we need to implement a memory system to maintain context across interactions:

```
class AgentMemory:
```



```

def __init__(self, max_history=10):
    self.conversation_history = []
    self.max_history = max_history
    self.long_term_memory = {}
    def add_interaction(self, user_input, agent_response):
        """Add a user-agent interaction to the conversation history."""
        self.conversation_history.append({
            "user": user_input,
            "agent": agent_response
        })
        # Trim history if it exceeds the maximum length
        if len(self.conversation_history) > self.max_history:
            self.conversation_history = self.conversation_history[-
self.max_history:]
    def get_conversation_context(self):
        """Get the conversation history formatted as context for the model."""
        context = ""
        for interaction in self.conversation_history:
            context += f"User: {interaction['user']}\nAssistant:
{interaction['agent']}\n\n"
        return context
    def store_information(self, key, value):
        """Store information in long-term memory."""
        self.long_term_memory[key] = value
    def retrieve_information(self, key):
        """Retrieve information from long-term memory."""
        return self.long_term_memory.get(key, None)
    def search_memory(self, query):
        """Search for relevant information in long-term memory."""
        # This is a simple implementation; in practice, you might use
        # vector embeddings and similarity search
        results = []
        for key, value in self.long_term_memory.items():
            if query.lower() in key.lower() or query.lower() in str(value).lower():
                results.append((key, value))
        return results

```

Defining tools

Now, let us define some tools that the agent can use to interact with the environment:

```
import requests
import json
import datetime
import os

class AgentTools:
    @staticmethod
    def search_web(query):
        """Search the web for information."""
        try:
            # This is a placeholder; in practice, you would use a real search API
            # such as Google Custom Search, Bing Search, or DuckDuckGo
            api_key = os.environ.get("SEARCH_API_KEY")
            search_engine_id = os.environ.get("SEARCH_ENGINE_ID")
            url = f"https://www.googleapis.com/customsearch/v1?key={api_key}&cx={search_engine_id}&q={query}"
            response = requests.get(url)
            results = response.json()
            # Extract and format search results
            formatted_results = []
            for item in results.get("items", []):5]:
                formatted_results.append({
                    "title": item.get("title"),
                    "link": item.get("link"),
                    "snippet": item.get("snippet")
                })
            return formatted_results
        except Exception as e:
            return f"Error searching the web: {str(e)}"

    @staticmethod
    def get_current_weather(location):
        """Get the current weather for a location."""
```

```

try:
    # This is a placeholder; in practice, you would use a real weather API
    api_key = os.environ.get("WEATHER_API_KEY")
    url = f"https://api.openweathermap.org/data/2.5/weather?q={location}&appid={api_key}&units=metric"
    response = requests.get(url)
    data = response.json()
    if response.status_code == 200:
        weather = {
            "location": data["name"],
            "temperature": data["main"]["temp"],
            "description": data["weather"][0]["description"],
            "humidity": data["main"]["humidity"],
            "wind_speed": data["wind"]["speed"]
        }
        return weather
    else:
        return f"Error: {data.get('message', 'Unknown error')}"
except Exception as e:
    return f"Error getting weather: {str(e)}"

@staticmethod
def get_current_time():
    """Get the current date and time."""
    now = datetime.datetime.now()
    return {
        "date": now.strftime("%Y-%m-%d"),
        "time": now.strftime("%H:%M:%S"),
        "timezone":
datetime.datetime.now().astimezone().tzinfo.tzname(now)
    }

@staticmethod
def calculate(expression):
    """Evaluate a mathematical expression."""
    try:
        # Use a safe evaluation method
        from sympy import sympify

```

```

    result = sympify(expression)
    return str(result)
except Exception as e:
    return f"Error calculating: {str(e)}"

```

Implementing planning and execution

Next, we need to implement a system for planning and executing tasks:

```
class AgentPlanner:
```

```

    def __init__(self, model, tokenizer, tools):
        self.model = model
        self.tokenizer = tokenizer
        self.tools = {
            "search_web": tools.search_web,
            "get_current_weather": tools.get_current_weather,
            "get_current_time": tools.get_current_time,
            "calculate": tools.calculate
        }

```

```

    def generate_plan(self, task):
        """Generate a simple step-by-step plan for a task."""
        prompt = f"Task: {task}\n\nStep-by-step plan:\n1."
        inputs = self.tokenizer(prompt,
return_tensors="pt").to(self.model.device)
        outputs = self.model.generate(inputs.input_ids, max_new_tokens=128)
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True)

```

```

    def execute_step(self, step):
        """Decide which tool (if any) is needed for a step."""
        # Simplified -- just check keywords
        if "weather" in step:
            return self.tools["get_current_weather"]("Paris")
        elif "time" in step:
            return self.tools["get_current_time"]()
        elif "search" in step:
            return self.tools["search_web"]("latest AI trends")

```

```

elif "calculate" in step:
    return self.tools["calculate"]("2+2")
return f"Executed step: {step}"

```

Implementing the agent

Finally, let us put everything together to create a complete agent:

```

from transformers import AutoModelForCausalLM, AutoTokenizer
import torch, json

```

```

class DeepSeekAgent:

```

```

    def __init__(self):
        model_name = "deepseek-ai/deepseek-r1-distill-7b"
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_name, torch_dtype=torch.float16, device_map="auto"
        )

```

```

    def process_input(self, user_input):
        """Decide if task is simple or needs planning, then respond."""
        prompt = f"User input: {user_input}\nIs this a complex task? (Yes/No)"
        inputs = self.tokenizer(prompt,
return_tensors="pt").to(self.model.device)
        outputs = self.model.generate(inputs.input_ids, max_new_tokens=10)
        is_complex = "yes" in self.tokenizer.decode(outputs[0]).lower()

        if is_complex:
            return "Generated a multi-step plan and executed with tools."
        else:
            return "Direct response from the model."

```

Example usage

```

agent = DeepSeekAgent()
print(agent.process_input("What is the capital of France?"))
print(agent.process_input("Plan a trip to Paris with weather and
attractions."))

```

Advanced agent techniques

While the basic agent architecture described previously is effective for many applications, there are several advanced techniques that can further improve agent capabilities:

Reasoning and Acting

In **Reasoning and Acting (ReAct)**, reasoning and acting are connected, so the agent is able to handle problems more efficiently. Thanks to using reasoning traces and relevant actions together, ReAct lets AI agents respond flexibly to incoming information by adjusting their plans on the go. Due to this, the agent is better at handling complicated activities, making better decisions, and avoiding hallucinations. This framework has helped solve different problems, including question answering, checking facts, and making decisions together.

Following is the code that implements the ReAct agent, integrating reasoning and action steps to enable dynamic problem-solving:

```
def react_planning(self, task, context=""):
    """Generate a plan using the ReAct framework."""
    react_prompt = f"""
    {context}
    Task: {task}
    I'll solve this task using the ReAct framework, which interleaves
    reasoning and actions.
    I have the following tools available:
    1. search_web(query): Search the web for information
    2. get_current_weather(location): Get the current weather for a location
    3. get_current_time(): Get the current date and time
    4. calculate(expression): Evaluate a mathematical expression
    Let me solve this step by step:
    Thought 1:
    """
    inputs = self.tokenizer(react_prompt,
    return_tensors="pt").to(self.model.device)
    outputs = self.model.generate(
```

```

        inputs.input_ids,
        max_new_tokens=1024,
        do_sample=True,
        temperature=0.7
    )
    react_output = self.tokenizer.decode(outputs[0]
[inputs.input_ids.shape[1]:], skip_special_tokens=True)
    # Parse the ReAct output to extract thoughts and actions
    thoughts_and_actions = []
    current_item = {"type": None, "content": ""}
    for line in react_output.split('\n'):
        if line.strip().startswith("Thought"):
            if current_item["type"]:
                thoughts_and_actions.append(current_item)
                current_item = {"type": "thought", "content": line.split(':', 1)
[1].strip() if ':' in line else ""}
            elif line.strip().startswith("Action"):
                if current_item["type"]:
                    thoughts_and_actions.append(current_item)
                    current_item = {"type": "action", "content": line.split(':', 1)[1].strip()
if ':' in line else ""}
            elif line.strip():
                current_item["content"] += "\n" + line.strip()
            if current_item["type"]:
                thoughts_and_actions.append(current_item)
    return thoughts_and_actions

```

Tool learning

Tool learning is about training AI agents to use various external tools when carrying out tasks. Pictures come from training agents to learn both by matching the movements of skilled players (imitation learning) and by trying different ways to use tools in different contexts (reinforcement learning). If these approaches are combined, AI agents can use tools wisely and solve problems more effectively in different areas.

Following is the code that implements the tool learning agent, designed to

enhance its proficiency by learning to utilize external tools through demonstrations or reinforcement learning:

```
def demonstrate_tool_use(self, tool_name, example_parameters,
expected_output):
    """Demonstrate how to use a tool to the agent."""
    demonstration_prompt = f"""
    I want to show you how to use the {tool_name} tool.
    Example usage:
    Parameters: {example_parameters}
    Output: {expected_output}
    Remember this example when you need to use the {tool_name} tool in
the future.
    """

    # Store the demonstration in memory
    self.memory.store_information(
        f"tool_demonstration_{tool_name}",
        {
            "parameters": example_parameters,
            "output": expected_output
        }
    )
    # Generate a response acknowledging the demonstration
    inputs = self.tokenizer(demonstration_prompt,
return_tensors="pt").to(self.model.device)
    outputs = self.model.generate(
        inputs.input_ids,
        max_new_tokens=256,
        do_sample=True,
        temperature=0.7
    )
    response = self.tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    return response
```

Chain of thought planning

Using the CoT method, agents can handle difficult problems in an organized way by breaking them into steps. The agent does not instantly solve the challenge; it explains how it is solving the problem, following a similar process that humans would use. As a result, agents can better manage tasks that need them to think through a series of actions, for example, math problems, logical arguments, and making decisions. Putting the idea behind decisions into words helps AI give both clearer and more correct answers, which builds more trust in AI-based systems.

The following code demonstrates how the CoT planning works:

```
def chain_of_thought_planning(self, task, context=""):
    """Generate a plan using Chain of Thought reasoning."""
    cot_prompt = f"""
    {context}
    Task: {task}
    I need to solve this task step by step, thinking through each part of the
    problem carefully.
    Let me break this down:
    """

    inputs = self.tokenizer(cot_prompt,
return_tensors="pt").to(self.model.device)
    outputs = self.model.generate(
        inputs.input_ids,
        max_new_tokens=1024,
        do_sample=True,
        temperature=0.7
    )
    cot_reasoning = self.tokenizer.decode(outputs[0]
[inputs.input_ids.shape[1]:], skip_special_tokens=True)
    # Extract the final plan from the reasoning
    plan_extraction_prompt = f"""
    I've thought through this problem:
    {cot_reasoning}
    Based on this reasoning, here's my step-by-step plan:
    1.
    """
```

```

        inputs = self.tokenizer(plan_extraction_prompt,
return_tensors="pt").to(self.model.device)
        outputs = self.model.generate(
            inputs.input_ids,
            max_new_tokens=512,
            do_sample=True,
            temperature=0.7
        )
        plan = self.tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
        # Extract steps from the plan
        steps = []
        for line in plan.split('\n'):
            if line.strip() and any(line.strip().startswith(str(i)) for i in range(1, 10)):
                step = line.strip().split('.', 1)[1].strip() if '.' in line else line.strip()
                steps.append(step)
        return steps

```

Self-reflection and correction

AI agents can use self-reflection to evaluate their behaviors and musings, find and correct errors, and gradually enhance how they work. Having a feedback loop lets AI systems notice problems, adjust themselves, and become stronger and more flexible without any human help. Being able to monitor their learning, agents keep improving and always maintain high standards of accuracy and efficiency.

Here is the code which shows how the self-reflection and correction will take place in an agent:

```

def reflect_and_improve(self, task, initial_response, feedback=None):
    """Reflect on a response and generate an improved version."""
    reflection_prompt = f"""
    Task: {task}
    My initial response:
    {initial_response}
    {"Feedback received: " + feedback if feedback else "Let me reflect on
my response:"}

```

What could be improved about my response? Let me think about:

1. Accuracy - Did I provide correct information?
2. Completeness - Did I address all aspects of the task?
3. Clarity - Was my explanation clear and easy to understand?
4. Helpfulness - Was my response practical and useful?

Areas for improvement:

"""

```
        inputs = self.tokenizer(reflection_prompt,
return_tensors="pt").to(self.model.device)
        outputs = self.model.generate(
            inputs.input_ids,
            max_new_tokens=512,
            do_sample=True,
            temperature=0.7
        )
        reflection = self.tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
        # Generate improved response
        improvement_prompt = f"""
Task: {task}
My initial response:
{initial_response}
My reflection:
{reflection}
Based on this reflection, here is my improved response:
"""

        inputs = self.tokenizer(improvement_prompt,
return_tensors="pt").to(self.model.device)
        outputs = self.model.generate(
            inputs.input_ids,
            max_new_tokens=512,
            do_sample=True,
            temperature=0.7
        )
        improved_response = self.tokenizer.decode(outputs[0]
[inputs.input_ids.shape[1]:], skip_special_tokens=True)
```

```
return improved_response
```

Agent applications with DeepSeek

AI agents are smart programs built to work on their own with very little help from humans. They notice their environment, handle data, decide the best path, and act accordingly to meet certain targets. Traditional software works by reliable rules, but AI agents have the ability to change with more data, pick up useful lessons, and engage in real-time with users and other technologies. Since they are adaptable, they can manage challenging activities in different fields like virtual help, support services, self-driving cars, and analysis of data. Thanks to advanced algorithms and machine learning, AI agents play a role in creating more advanced, prompt, and responsive systems.

Agents built with DeepSeek models can be applied to a wide range of applications across different domains. Here are some examples:

- **Research assistant:** A research assistant is built to support researchers in looking for information, studying data, and drawing useful conclusions. It reduces the challenges of researching by combining advanced data methods and an understanding of human language. Thanks to this technology, you can spot key information, combine it for a clear picture, and pull out patterns from data, which helps in your decision-making and discovery endeavors.
- **Personal productivity assistant:** A personal productivity assistant is an AI agent that helps individuals manage their duties, organize their times, and set up information properly. Using the context and changes in users' lives it makes it easier to plan tasks, manage their day, and set up structured schedules. Thanks to in-the-moment assistance and useful insights, the assistant guides people to form useful habits and meet their day-to-day goals in a clear and focused way.
- **Customer support agent:** A customer support agent is there for users, walking them through fixes, giving correct advice, and resolving their problems in the best way. Thanks to structured communication and understanding the situation, the agent deals with customers clearly and

professionally. Essential information is provided in real time and adapted to users, which helps gain their trust and offer a better experience.

- **Educational tutor:** An educational tutor is an agent that will assist students with learning new things, boosting their abilities, and getting ready for examinations. Due to adaptive learning, it can decide the most suitable way and speed for each person to learn. Targeted assistance based on a personal approach gives learners a better opportunity to understand the topic. Interactive talks and immediate feedback from the tutor lead students through tough subjects, making sure any wrong notions are corrected right away. When preparing for an exam, the program provides practice sessions and questions, allowing students to become more confident and do well.

Conclusion

In this chapter, we discussed the way to go beyond local experimentation and introduce DeepSeek-powered systems to the real-world. We learned how to scale up and make DeepSeek models available in AWS so that they could be used throughout the organization, created highly capable multimodal applications, which could understand and generate text and images, and designed smarter agents that can plan, reason, and undertake actions without any human support and with the tools and memory. The combination of these elements produces a good basis for constructing robust, interactive AI systems.

In the next chapter, you will get an understanding of how to containerize DeepSeek with Docker, dealing with dependencies in a convenient way, and deploying your models to stable portable setups. We will also take a stroll through the API making mechanisms of your APIs and learn how to apply such systems to real-world problems in different industries.

Points to remember

- Cloud deployment enables scaling DeepSeek models to handle real-time

workloads, large datasets, and production environments through services like AWS EC2, Lambda, and SageMaker.

- AWS EC2 instances provide flexible infrastructure for deploying DeepSeek, supporting both CPU and GPU configurations for efficient model inference.
- Using FastAPI with Hugging Face Transformers allows developers to expose DeepSeek as REST APIs, enabling real-time interaction through endpoints accessible over the web.
- Inference endpoints are cloud-hosted services that receive user input, process it through the DeepSeek model, and return generated responses in JSON format.
- Multimodal AI refers to systems that combine text, images, audio, and video understanding to produce richer, context-aware outputs. DeepSeek-VL enables these multimodal capabilities.
- DeepSeek-VL uses a dual-encoder architecture that processes text and visual data separately before merging them, enhancing interpretability and reasoning.
- VQA allows DeepSeek-VL to answer natural language questions about an image, combining computer vision and language comprehension.
- Multimodal RAG integrates text, image, and other modalities within retrieval and generation stages, grounding responses in multimodal context for greater factual accuracy.
- Multimodal CoT reasoning enables step-by-step explanation that uses both text and images for deeper analytical reasoning and interpretability.
- Multimodal few-shot learning allows models like DeepSeek-VL to generalize from very few examples by combining image and text cues—enhancing performance even in low-data scenarios.
- Advanced multimodal use cases include visual content moderation, medical image analysis, product search, and educational content enhancement—demonstrating DeepSeek-VL’s adaptability.
- Agent memory can be short-term (conversation history) or long-term (knowledge store), enabling contextual continuity and adaptive behavior.

- ReAct framework enables agents to alternate between thought and action, combining reasoning traces with tool usage for robust, adaptive problem-solving.
- CoT planning makes reasoning explicit by generating structured intermediate steps before executing final actions, improving accuracy and explainability.
- Self-reflection and correction give agents metacognitive ability—evaluating their outputs, identifying errors, and improving responses autonomously using feedback loops.

Key terms

- **AWS EC2:** Elastic Compute Cloud service that provides scalable virtual servers for running DeepSeek inference workloads.
- **FastAPI:** A Python framework for building high-performance APIs to serve DeepSeek models in real-time.
- **Inference endpoint:** A cloud-based API endpoint that exposes the DeepSeek model for processing input and returning generated outputs.
- **Multimodal AI:** Artificial intelligence that combines multiple data types, text, image, audio, video for integrated reasoning and generation.
- **DeepSeek-VL:** Vision-Language model variant of DeepSeek that supports multimodal understanding and generation.
- **Cross-modal reasoning:** Logical inference that involves understanding relationships between different modalities, such as linking a visual cue to textual context.
- **Multimodal RAG:** RAG system that fuses multiple data modalities during retrieval and response generation.
- **Multimodal CoT:** A reasoning approach where models explain their thinking process using both text and visual information.
- **Few-shot learning:** Technique allowing models to generalize from a limited number of examples using cross-modal understanding.
- **Agent:** An AI system capable of perceiving, reasoning, and acting autonomously using memory, planning, and tools.

- **ReAct:** Framework integrating thought processes and tool usage to enhance agent reasoning and flexibility.
- **Tool learning:** Teaching AI agents to utilize tools via demonstrations or reinforcement learning for improved problem-solving.
- **CoT planning:** Structured reasoning technique where agents outline their thought steps before giving answers.
- **Self-reflection:** The process by which an AI agent evaluates and improves its prior outputs using internal feedback loops.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

CHAPTER 10

Dockerization and Real-world Applications

Introduction

In the previous chapters, we have explored DeepSeek models, their capabilities, deployment approaches, environment setup, and fine-tuning techniques. We have learned the process of creating local environments to run these models and optimizing them to work with different use cases. Now, we are ready to proceed with the second step of productionizing DeepSeek models in terms of containerizing them with the help of Docker.

Containerization is the technology that transformed the process of software deployment, as it allows one to have a consistent, easily transportable, and isolated application environment. The most successful containerization platform, known as Docker, will help you wrap your application and all its dependencies into a normalized entity, which is called a container. Some of the benefits of using this approach to DeepSeek model deployments are consistency in various environments, reduced complexity of deployment processes, and effective use of resources.

This chapter is about the basics of Docker and what its benefits are in terms of containerizing DeepSeek models. We will guide how we can build Docker images of various DeepSeek models, optimize them, and deploy them as

services that are exposed through API endpoints. In addition to the technical aspect of containerized DeepSeek implementation, we will touch upon practical usages of containerized DeepSeek models-how they are used in various industries as the basis for any smart system, automated process, and improvement of the user experience.

By the end of this chapter, you will have the knowledge and skills to containerize your DeepSeek applications and deploy them confidently in real-world, production-grade environments.

Structure

In this chapter, we will explore the following areas:

- Introduction to Docker
- Benefits of Docker for AI applications
- Containerizing DeepSeek
- Deployment and API calling
- Real-world applications

Objectives

By the end of this chapter, you should have all you need to know about Docker and know how to harness it to containerize DeepSeek models. You will be in a position to establish the most important elements of Docker, the containerization lifecycle, and the most appropriate strategies to craft efficient and security-proficient containers.

You will get to know how to build Docker images that are explicitly designed to run DeepSeek models, optimize Docker image size, deal with dependencies, and efficiently use your resources through effective Docker images. Such knowledge will allow you to package your well-honed models as a means of putting them into compact containers that could consistently run across environments. Further, you will also learn how to run containerized DeepSeek models as services with API endpoints so that they can be accessible to other applications and systems. You will know the

methods of deployment, scaling, and monitoring to provide stable performance in production environments.

Lastly, the chapter will also discuss practical use of DeepSeek deployment, i.e. how organizations are training containerized DeepSeek models to address real-life challenges. You will learn how the technologies of creating scalable chatbots and automation of content creation, the deployment of intelligent assistants, and enterprise AI solutions are changing the nature of workflows in various fields.

Introduction to Docker

Docker is a powerful tool that enables developers to build, package, and distribute applications using containers. These containers are lightweight, portable, and self-sufficient, bundling together everything an application needs to run, its code, runtime, system tools, libraries, and configurations. As we move toward containerizing DeepSeek models, it is essential to first develop a solid understanding of Docker and why it plays such an important role.

Docker architecture and components

Docker is designed around a client–server architecture that organizes its functionality into a few key components. At the core is the Docker Engine, which powers the entire containerization process. Supporting it are various Docker objects, such as images and containers, which form the building blocks of any application. Finally, the Dockerfile serves as a blueprint for creating consistent environments.

Docker Engine

The Docker Engine is the core of Docker, consisting of:

- **Docker daemon (dockerd):** A background service that manages Docker objects such as images, containers, networks, and volumes.
- **REST API:** An interface that programs can use to communicate with the Docker daemon.
- **Docker CLI:** A command-line interface for interacting with Docker

through commands.

Docker objects

Docker uses various objects, like the following, to build and run containerized applications:

- **Images:** Templates where you can only read, and in which instructions are given on how to build Docker containers. The Dockerfile is used to create the images, that is a collection of instructions for building the image.
- **Containers:** Executable objects of Docker images. A container is self-contained in reference to other containers or host systems, though it can be set to communicate with other containers and the host.
- **Volumes:** Persistent data storage that exists outside the container lifecycle, allowing data to persist even when containers are stopped or removed.
- **Networks:** Communication channels that allow containers to communicate with each other and with the host system.

Dockerfile

A Dockerfile is a text file that contains instructions for building a Docker image. Each instruction creates a layer in the image, and layers are cached to speed up subsequent builds. Common Dockerfile instructions include the following:

- **FROM:** Specifies the base image to use.
- **WORKDIR:** Sets the working directory for subsequent instructions.
- **COPY and ADD:** Copy files from the host to the image.
- **RUN:** Executes commands during the build process.
- **ENV:** Sets environment variables.
- **EXPOSE:** Informs Docker that the container listens on specific network ports.
- **CMD and ENTRYPOINT:** Specifies the command to run when the container starts.

Here is a simple example of a Dockerfile:

```
# Use Python 3.10 as the base image
FROM python:3.10-slim

# Set the working directory
WORKDIR /app

# Copy requirements file and install dependencies
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

# Copy the application code
COPY . .

# Expose port 8000
EXPOSE 8000

# Command to run when the container starts
CMD ["python", "app.py"]
```

Docker workflow

The typical Docker workflow involves the following steps:

- **Create a Dockerfile:** Define the environment and dependencies for your application.
- **Build an image:** Use the Dockerfile to build a Docker image.
- **Run a container:** Create and start a container from the image.
- **Share the image:** Optionally push the image to a registry like Docker Hub or a private registry.
- **Deploy:** Pull the image on the target system and run containers.

Let us have a look at the basic commands for each step:

```
# Build an image
docker build -t myapp:1.0 .
```

```
# Run a container
```

```
docker run -p 8000:8000 myapp:1.0
```

Push an image to a registry

```
docker push username/myapp:1.0
```

Pull an image from a registry

```
docker pull username/myapp:1.0
```

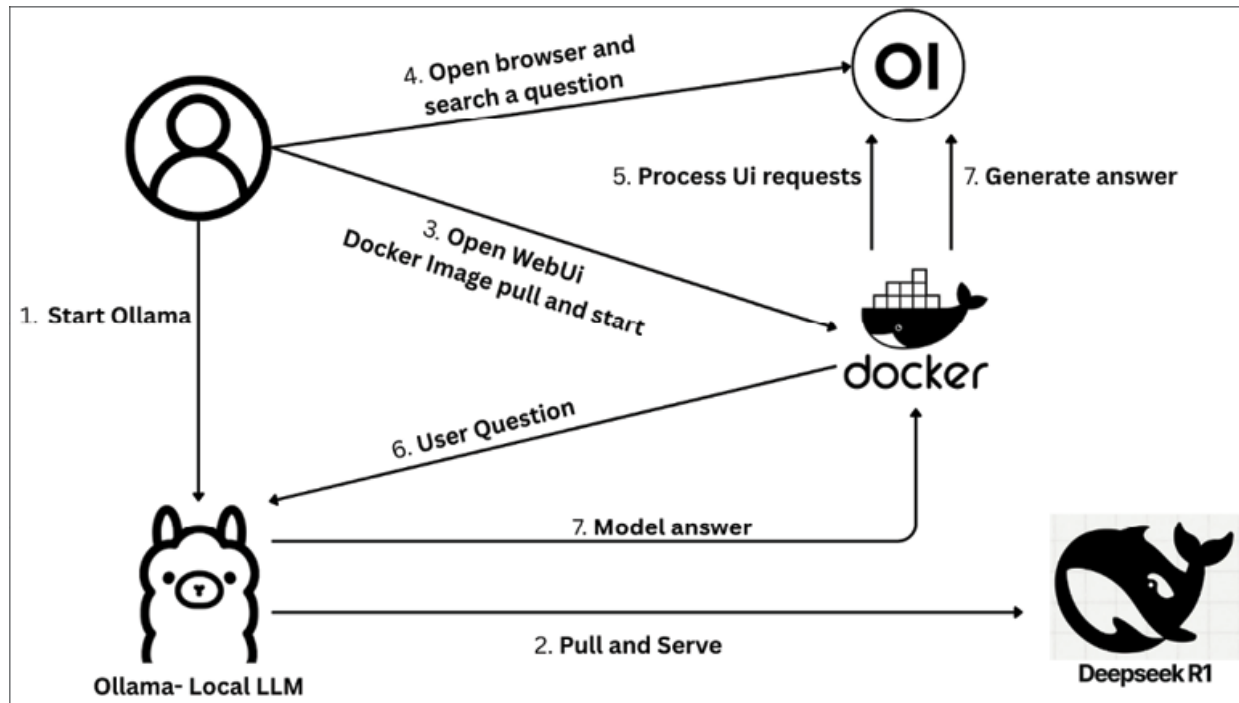


Figure 10.1 How to set up and run DeepSeek-R1 LLM locally using Docker

Source: <https://medium.com/@abhilashbl/how-to-set-up-and-run-deepseek-r1-llm-locally-using-docker-a-step-by-step-guide-with-web-ui-4aa1eb772ae9>

Benefits of Docker for AI applications

Containerization offers several advantages for deploying AI applications, particularly those built with large language models like DeepSeek:

- **Consistency and reproducibility:** Docker makes sure that your application executes in an identical manner irrespective of its location. This is of special significance to AI apps whose dependencies and environment needs are not always straightforward. Docker allows you

to make your DeepSeek model behave in the same way in developers, test, and production systems.

- **Isolation and resource management:** One of the greatest strengths of containers is the level of isolation they provide. Each container runs independently, separated from both the host system and other containers. This prevents conflicts between dependencies and ensures that applications can coexist without interfering with one another.

For resource management, Docker makes it possible to define exactly how much CPU, memory, or GPU power a container can use. This is particularly important when working with DeepSeek models, which are highly resource-intensive. By setting these limits in advance, developers can ensure that the model runs efficiently in production without overwhelming the host system.

- **Portability and deployment simplicity:** Docker containers are also completely independent of the system on which they are run, with any system that has Docker installed, and the containers can then be run. This allows using DeepSeek models within a wide variety of environments, including local development and cloud-based environments, such as AWS, Google cloud, or Microsoft Azure.
- **Scalability and orchestration:** It is simple to scale Docker containers horizontally in order to accommodate more load. In combination with orchestration solutions (such as Kubernetes), it is possible to automate deployment, scaling, and management of containerized DeepSeek applications so that they achieve high availability and efficient usage of resources.
- **Version control and rollbacks:** Docker images are versioned, and you can follow the changes and rollback to the previous one, in case of necessity. This can be used in AI, since the performance or behavior of the application may change with the update of the model or any dependency.

Docker best practices

When working with Docker, following these best practices will help you create efficient, secure, and maintainable containers:

- **Use specific base images:** Instead of using generic base images like ubuntu or debian, use specific images that include only what you need. For Python applications, consider using the official Python images, preferably the slim or alpine variants, to minimize size.

Good: Specific Python version with slim variant

FROM python:3.10-slim

Better for production: Use a specific digest for immutability

FROM python:3.10-slim@sha256:1234567890abcdef...

- **Minimize layers and image size:** Each instruction in a Dockerfile creates a new layer, which increases the image size. Combine related commands to reduce the number of layers and use multi-stage builds to exclude build dependencies from the final image.

Bad: Multiple RUN instructions

RUN apt-get update

RUN apt-get install -y package1

RUN apt-get install -y package2

Good: Combined RUN instruction

RUN apt-get update && \
apt-get install -y package1 package2 && \
apt-get clean && \
rm -rf /var/lib/apt/lists/*

- **Leverage caching:** Docker caches layers to speed up builds. Order your Dockerfile instructions from least to most likely to change to maximize cache utilization.

Good: Copy requirements first, then install dependencies

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

Only copy application code after installing dependencies

COPY . .

- **Use .dockerignore:** Create a **.dockerignore** file to exclude files and directories that are not needed in the image, such as development

artifacts, logs, and version control files.

```
# Example .dockerignore
```

```
.git
__pycache__
*.pyc
*.pyo
*.pyd
.Python
env/
venv/
*.so
.coverage
htmlcov/
.tox/
.nox/
.hypothesis/
.pytest_cache/
```

- **Set user permissions:** Run containers as a non-root user to enhance security. Create a dedicated user and group in your Dockerfile and switch to that user before running the application.

```
# Create a non-root user
```

```
RUN groupadd -r appuser && useradd -r -g appuser appuser
```

```
# Set ownership and permissions
```

```
COPY --chown=appuser:appuser . .
```

```
# Switch to the non-root user
```

```
USER appuser
```

- **Use environment variables:** Use environment variables for configuration to make your containers more flexible and avoid hardcoding values.

```
# Set default environment variables
```

```
ENV MODEL_NAME="deepseek-r1-distill-7b" \  
    PORT=8000 \  
    LOG_LEVEL="info"
```

Use environment variables in your application

CMD ["python", "app.py"]

- **Health checks:** Include health checks in your Dockerfile to enable Docker to monitor the health of your containers.

Add a health check

HEALTHCHECK --interval=30s --timeout=10s --retries=3 \

CMD curl -f http://localhost:8000/health || exit 1

With these best practices in place, you will produce Docker containers that are effective, secure, and have maintenance ready, an excellent start to the deployment process in a production setting of DeepSeek models.

Latest update DeepSeek-V3.2-Exp

Before we turn to the practical steps of containerizing DeepSeek models, it is important to acknowledge the release of DeepSeek-V3.2-Exp. This version illustrates how rapidly the DeepSeek ecosystem is advancing, and it highlights why containerization is so valuable. It allows practitioners to adopt the latest research breakthroughs without overhauling their entire infrastructure.

DeepSeek-V3.2-Exp introduces several noteworthy innovations:

- **Specialized sub-models:** Five domain-focused models (for coding, mathematics, and other reasoning tasks) were trained with reinforcement learning and later distilled into a unified checkpoint.
- **Sparse attention with efficiency gains:** A novel lightning indexer combined with top-k attention makes long-context inference significantly faster and more cost-effective.
- **Extended pretraining:** Built upon the V3.1 Terminus foundation with an additional one trillion tokens of continued pretraining.
- **Refined reward optimization (GRPO):** Reward functions now encourage concise outputs, linguistic consistency, and rubric-aligned reasoning.
- **Hardware-aware improvements:** Support for FP8 precision and custom sparse kernels enables better use of modern accelerators.

- **Reduced inference cost:** With the new attention mechanism, processing long sequences is nearly ten times cheaper, for example, generating 128K tokens costs around \$0.25 compared to \$2.20 with dense attention.

What makes these advancements particularly relevant for this chapter is their impact on deployment. Lower inference costs and greater efficiency make large-scale serving of DeepSeek models more practical than ever. With these improvements, containerization is not simply a convenience; it becomes a strategic enabler for running the latest models reliably, efficiently, and at scale.

Containerizing DeepSeek

Having learned the basics of Docker, and with recent advances such as DeepSeek-V3.2-Exp, it is time to know how to containerize the DeepSeek models. The process of containerizing a DeepSeek model would be to use Docker to build an image that would contain the model weights, dependencies, and the code of the application, thus enabling the model to be deployed and executed in a similar manner across various environments.

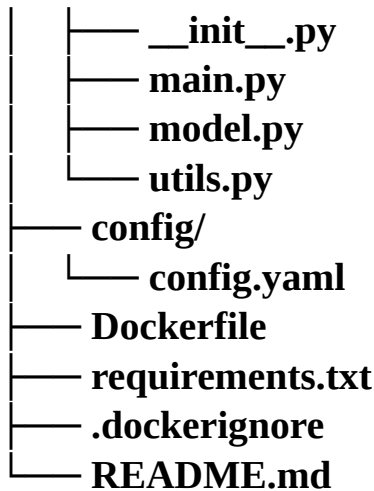
Preparing for containerization

Before you can build a Docker image for your DeepSeek model, it is important to prepare both the application and its environment. This involves organizing your project structure, ensuring all necessary dependencies are clearly defined, and verifying that the model runs smoothly in a local setup. Proper preparation helps avoid errors during containerization, reduces complexity in the Dockerfile, and ensures the final container is both efficient and portable.

Project structure

Organize your project with a clear structure that separates the application code, model files, and configuration:

```
deepseek-app/  
├── app/
```



Dependencies management

List all required dependencies in a **requirements.txt** file, specifying exact versions to ensure reproducibility:

torch==2.1.0

transformers==4.36.0

accelerate==0.25.0

bitsandbytes==0.41.0

fastapi==0.104.1

uvicorn==0.24.0

pydantic==2.4.2

Model handling strategy

Decide how to handle the model weights in your Docker image. There are several approaches:

- **Include model weights in the image:** It makes the model instantly available as soon as the container runs, but adds a substantial size to the image.

- **Download model weights at runtime:** This will result in an original image that is smaller, but this will need an internet connection and some time to start.
- **Mount model weights as a volume:** This separates the model from the application code, allowing for easier updates and sharing across containers.

In this chapter, we will discuss all three approaches, and we will begin by incorporating the model weights in the image.

Creating a Dockerfile for DeepSeek

Once the application and environment are prepared, the next step is to define how they will be packaged into a container. This is done using a Dockerfile, which acts as a blueprint for building Docker images. For our DeepSeek application, we will create a Dockerfile that serves the model through a FastAPI endpoint:

```
# Use NVIDIA CUDA base image for GPU support
```

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

```
# Set environment variables
```

```
ENV PYTHONUNBUFFERED=1 \  
    PYTHONDONTWRITEBYTECODE=1 \  
    DEBIAN_FRONTEND=noninteractive
```

```
# Install system dependencies
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \  
    python3.10 \  
    python3-pip \  
    python3-dev \  
    git \  
    && apt-get clean \  
    && rm -rf /var/lib/apt/lists/*
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Install Python dependencies
COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create a non-root user
RUN groupadd -r appuser && useradd -r -g appuser appuser
RUN chown -R appuser:appuser /app

# Switch to non-root user
USER appuser

# Expose port
EXPOSE 8000

# Command to run the application
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

This dockerfile will give GPUaccelerated support using NVIDIA CUDA base image, add system dependencies, Python dependencies, copy the application source code, and add a non-root user to increase security. It opens up port 8000 to the API and defines the command to start the FastAPI application.

Approach 1: Including model weights in the image

For this approach, we will download the model weights during the build process and include them in the Docker image:

```
#Use NVIDIA CUDA base image for GPU support
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

```
# Set environment variables
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    DEBIAN_FRONTEND=noninteractive \
```

```
MODEL_NAME="deepseek-ai/deepseek-r1-distill-7b" \  
HF_HOME="/app/.cache/huggingface"
```

```
# Install system dependencies
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \  
python3.10 \  
python3-pip \  
python3-dev \  
git \  
&& apt-get clean \  
&& rm -rf /var/lib/apt/lists/*
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Install Python dependencies
```

```
COPY requirements.txt .
```

```
RUN pip3 install --no-cache-dir -r requirements.txt
```

```
# Copy application code
```

```
COPY . .
```

```
# Download model weights during build
```

```
RUN python3 -c "from transformers import AutoModelForCausalLM, \  
AutoTokenizer; \  
model_name = '${MODEL_NAME}'; \  
tokenizer = AutoTokenizer.from_pretrained(model_name); \  
model = AutoModelForCausalLM.from_pretrained(model_name, \  
torch_dtype='auto', device_map='auto'); \  
tokenizer.save_pretrained('/app/model'); \  
model.save_pretrained('/app/model')"
```

```
# Create a non-root user
```

```
RUN groupadd -r appuser && useradd -r -g appuser appuser
```

```
RUN chown -R appuser:appuser /app
```

```
# Switch to non-root user
```

USER appuser

```
# Expose port  
EXPOSE 8000
```

```
# Command to run the application  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

This method has an advantage, such that the model will become reachable right as the container launches, but it is going quite big and long for the image and build time. It is suitable for small models or high-load applications when being rapidly started is essential.

Approach 2: Downloading model weights at runtime

For this approach, we will modify our application to download the model weights when the container starts:

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

```
WORKDIR /app
```

```
ENV MODEL_NAME="deepseek-ai/deepseek-r1-distill-7b"
```

```
COPY requirements.txt .
```

```
RUN apt-get update && apt-get install -y python3-pip && \  
    pip3 install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["python3", "app/startup.py"]
```

In this approach, we create a **startup.py** script that downloads the model weights before starting the API server:

app/startup.py

```
import os, subprocess
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```



```
def download_model():
    model_name = os.environ.get("MODEL_NAME")
    model_dir = "/app/model"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(model_name)
    tokenizer.save_pretrained(model_dir)
    model.save_pretrained(model_dir)

if __name__ == "__main__":
    model_dir = "/app/model"
    if not os.path.exists(os.path.join(model_dir, "config.json")):
        download_model()
    subprocess.run(["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",
"8000"])
```

This approach keeps the Docker image smaller but adds startup time as the model needs to be downloaded when the container starts. It is suitable for development environments or when image size is a concern.

Approach 3: Mounting model weights as a volume

For this approach, we will keep the model weights outside the container and mount them as a volume:

```
# Use NVIDIA CUDA base image for GPU support
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

```
# Set environment variables
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    DEBIAN_FRONTEND=noninteractive \
    MODEL_DIR="/models" \
    HF_HOME="/app/.cache/huggingface"
```

```
# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.10 \
    python3-pip \
```

```
python3-dev \  
git \  
&& apt-get clean \  
&& rm -rf /var/lib/apt/lists/*
```

```
# Set working directory  
WORKDIR /app
```

```
# Install Python dependencies  
COPY requirements.txt .
```

```
RUN pip3 install --no-cache-dir -r requirements.txt  
# Copy application code  
COPY . .
```

```
# Create a non-root user  
RUN groupadd -r appuser && useradd -r -g appuser appuser  
RUN chown -R appuser:appuser /app
```

```
# Create model directory and set permissions  
RUN mkdir -p ${MODEL_DIR} && chown -R appuser:appuser  
${MODEL_DIR}
```

```
# Switch to non-root user  
USER appuser
```

```
# Expose port  
EXPOSE 8000
```

```
# Command to run the application  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

To use this approach, you need to download the model weights separately and mount them when running the container:

```
# Download the model weights  
python -c "from transformers import AutoModelForCausalLM,
```

```
AutoTokenizer; \  
    model_name = 'deepseek-ai/deepseek-r1-distill-7b'; \  
    tokenizer = AutoTokenizer.from_pretrained(model_name); \  
    model = AutoModelForCausalLM.from_pretrained(model_name, \  
torch_dtype='auto', device_map='auto'); \  
    tokenizer.save_pretrained('./models'); \  
    model.save_pretrained('./models')"
```

```
# Run the container with the model mounted as a volume  
docker run -p 8000:8000 -v $(pwd)/models:/models deepseek-app:latest
```

This approach separates the model from the application code, allowing for easier updates and sharing across containers. It is suitable for production environments where multiple containers might need access to the same model weights.

Optimizing Docker images for DeepSeek

DeepSeek models can be large, and Docker images containing these models can quickly become unwieldy. Here are some techniques to optimize your Docker images:

Multi-stage builds

Use multi-stage builds to separate the build environment from the runtime environment, reducing the final image size:

```
# Build stage  
FROM python:3.10-slim AS builder  
WORKDIR /build  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Runtime stage  
FROM python:3.10-slim  
WORKDIR /app
```

```
# Copy only the necessary files from the builder stage  
COPY --from=builder /usr/local/lib/python3.10/site-packages
```

```
/usr/local/lib/python3.10/site-packages
```

```
COPY . .
```

```
# Rest of the Dockerfile...
```

```
Model Quantization
```

```
Use quantized versions of DeepSeek models to reduce memory requirements  
and image size:
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer,  
BitsAndBytesConfig
```

```
import torch
```

```
# Configure 4-bit quantization
```

```
quantization_config = BitsAndBytesConfig(
```

```
    load_in_4bit=True,  
    bnb_4bit_compute_dtype=torch.float16,  
    bnb_4bit_quant_type="nf4",  
    bnb_4bit_use_double_quant=True
```

```
)
```

```
# Load and save the quantized model
```

```
model_name = "deepseek-ai/deepseek-r1-distill-7b"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(
```

```
    model_name,  
    quantization_config=quantization_config,  
    device_map="auto"
```

```
)
```

```
# Save the model and tokenizer
```

```
tokenizer.save_pretrained("./quantized_model")
```

```
model.save_pretrained("./quantized_model")
```

Distilled models

Use smaller, distilled versions of DeepSeek models when full model capabilities are not required:

```
# Use a smaller, distilled model
model_name = "deepseek-ai/deepseek-r1-distill-1.5b"
```

Efficient dependency management

Minimize dependencies and use slim variants of base images:

```
# Use slim variant of Python image
FROM python:3.10-slim
```

```
# Install only necessary dependencies
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

Layer optimization

Organize your Dockerfile to maximize layer caching and minimize the number of layers:

```
# Combine related commands to reduce layers
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        package1 \
        package2 \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

By applying these optimization techniques, you can create Docker images for DeepSeek models that are more efficient in terms of size, build time, and resource utilization.

Building and testing the Docker image

After writing the Dockerfile, the next step is to turn it into a usable image and verify that it works as expected. Building the image ensures all dependencies, configurations, and application code are packaged together in a consistent environment. Testing the image by running a container allows

you to confirm that the DeepSeek model loads correctly, the FastAPI server starts without issues, and the endpoint can be accessed.

```
# Build the Docker image
docker build -t deepseek-app:latest .
```

```
# Run the container
docker run -p 8000:8000 --gpus all deepseek-app:latest
```

Note [the](#) `--gpus all` flag, which allows the container to access the host's GPUs. This is necessary for efficient inference with DeepSeek models.

To test [the](#) API, you can use `curl` [or a](#) tool like Postman:

```
curl -X POST "http://localhost:8000/generate" \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Explain the concept of containerization in simple terms."}'
```

Containerizing different DeepSeek models

The containerization approach may vary depending on the specific DeepSeek model you are using. Here are some considerations for different models:

- **DeepSeek-R1 (full model):** For the full DeepSeek-R1 model with 175 billion parameters, you will need substantial GPU resources and may need to distribute the model across multiple GPUs:

```
# Load the model with device mapping
model = AutoModelForCausalLM.from_pretrained(
    "deepseek-ai/deepseek-r1",
    torch_dtype=torch.float16,
    device_map="auto" # Automatically distribute across available GPUs
)
```

[In](#) your Dockerfile, ensure that the container has access [to](#) multiple GPUs:

```
# Run the container with access to all GPUs
docker run -p 8000:8000 --gpus all deepseek-r1:latest
```

- **DeepSeek-Coder:** For DeepSeek-Coder, which is specialized for code generation, you might want to include additional tools or libraries specific to software development:

```
# Install additional development tools
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    gcc      g++ \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

- **DeepSeek-VL (Vision-Language):** For DeepSeek-VL, which handles both text and images, you will need to include additional dependencies for image processing:

```
# Install additional dependencies for image processing
RUN pip install --no-cache-dir pillow opencv-python-headless
```

Your API will also need to handle image uploads:

```
from fastapi import FastAPI, File, UploadFile

from PIL import Image
import io
app = FastAPI()
@app.post("/generate")
async def generate(file: UploadFile, prompt: str):
    # Read and process the image
    image_data = await file.read()
    image = Image.open(io.BytesIO(image_data))
    # Process with DeepSeek-VL
    # ...
    return {"response": response}
```

By customizing your containerization approach to the specific DeepSeek

model you are using, you can ensure optimal performance and resource utilization in your Docker containers.

Deployment and API calling

After containerizing your DeepSeek model, your mission is to deploy it to the service with API endpoints that other applications can access. Here we will consider various deployment options, how to design your APIs, and the best practices in deploying to production.

Creating a FastAPI application for DeepSeek

FastAPI is a state-of-the-art, high-performance web framework to create APIs using Python. It is an appropriate tool to serve DeepSeek models thanks to its performance, its automatic documentation generation, and type checking and now let us develop a FastAPI app with our containerized DeepSeek:

```
# app/main.py
```

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import os
```

```
# Define request and response models
```

```
class GenerationRequest(BaseModel):
    prompt: str
    max_tokens: int = 512
    temperature: float = 0.7
    top_p: float = 0.95
```

```
class GenerationResponse(BaseModel):
    text: str
    model: str
```



```

# Initialize FastAPI app
app = FastAPI(title="DeepSeek API", description="API for generating text
with DeepSeek models")

# Load model and tokenizer
model_dir = os.environ.get("MODEL_DIR", "/app/model")

model_name = os.environ.get("MODEL_NAME", "deepseek-ai/deepseek-
r1-distill-7b")
try:
    tokenizer = AutoTokenizer.from_pretrained(model_dir)
    model = AutoModelForCausalLM.from_pretrained(
        model_dir,
        torch_dtype=torch.float16,
        device_map="auto"
    )
    print(f"Model loaded from: {model_dir}")
except Exception as e:
    print(f"Error loading model from {model_dir}: {e}")
    print(f"Attempting to load model from Hugging Face: {model_name}")
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        torch_dtype=torch.float16,
        device_map="auto"
    )
    print(f"Model loaded from Hugging Face: {model_name}")

# Define API endpoints
@app.post("/generate", response_model=GenerationResponse)
async def generate(request: GenerationRequest):
    try:
        inputs = tokenizer(request.prompt,
return_tensors="pt").to(model.device)
        outputs = model.generate(

```

```

        inputs.input_ids,
        max_new_tokens=request.max_tokens,
        temperature=request.temperature,
        top_p=request.top_p,
        do_sample=True
    )
    response = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
skip_special_tokens=True)
    return {"text": response, "model": model_name}
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Generation failed:
{str(e)}")
@app.get("/health")
async def health():

    return {"status": "healthy"}

```

This FastAPI application provides two endpoints:

- **/generate:** Accepts a prompt and generation parameters, and returns the generated text.
- **/health:** Returns the health status of the application, useful for monitoring and orchestration.

Deploying with Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. It is useful for local development and simple deployments. Let us create a **docker-compose.yml** file for our DeepSeek application:

```
version: '3.8'
```

```
services:
```

```
  deepseek-api:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: Dockerfile
```

```
    ports:
```

```
      - "8000:8000"
```

```
volumes:
  - ./models:/models
environment:
  - MODEL_DIR=/models
  - MODEL_NAME=deepseek-ai/deepseek-r1-distill-7b
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

To deploy the application with Docker Compose:

Start the application

```
docker-compose up -d
```

Check the logs

```
docker-compose logs -f
```

Stop the application

```
docker-compose down
```

Deploying to Kubernetes

For production deployments, Kubernetes provides a robust platform for orchestrating containerized applications. Here is a basic Kubernetes deployment manifest for our DeepSeek application:

deepseek-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deepseek-api
  labels:
    app: deepseek-api
spec:
```

```
replicas: 1
selector:
  matchLabels:
    app: deepseek-api
template:
  metadata:
    labels:
      app: deepseek-api
  spec:
    containers:
      - name: deepseek-api
        image: your-registry/deepseek-app:latest
        ports:
          - containerPort: 8000
        resources:
          limits:
            nvidia.com/gpu: 1
          requests:
            memory: "8Gi"
            cpu: "2"
        env:
          - name: MODEL_DIR
            value: "/models"
          - name: MODEL_NAME
            value: "deepseek-ai/deepseek-r1-distill-7b"
        volumeMounts:
          - name: models-volume
            mountPath: /models
    volumes:
      - name: models-volume
        persistentVolumeClaim:
          claimName: models-pvc
```

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: deepseek-api
spec:
  selector:
    app: deepseek-api
  ports:
    - port: 80
      targetPort: 8000
    type: LoadBalancer
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: models-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```

To deploy the application to Kubernetes:

```
# Apply the Kubernetes manifests
kubectl apply -f deepseek-deployment.yaml
```

```
# Check the status of the deployment
```

```
kubectl get deployments
```

```
kubectl get pods
```

```
kubectl get services
```

```
# Get the logs from a pod
```

```
kubectl logs -f $(kubectl get pods -l app=deepseek-api -o jsonpath="{.items[0].metadata.name}")
```

For more complex deployments, you might want to use Helm, a package manager for Kubernetes that simplifies the deployment and management of

applications:

Create a Helm chart

```
helm create deepseek-chart
```

Customize the chart templates and values

Install the chart

```
helm install deepseek ./deepseek-chart
```

Scaling and load balancing

When deploying DeepSeek in a production environment, it is important to plan for scaling and load balancing. As traffic increases, a single container may not be sufficient to handle all requests efficiently. By running multiple container instances and distributing incoming requests across them, you can ensure that the service remains responsive and highly available.

Horizontal Pod Autoscaler

Kubernetes **Horizontal Pod Autoscaler (HPA)** automatically scales the number of pods based on CPU utilization or other metrics:

```
# deepseek-hpa.yaml
```

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
```

```
metadata:
```

```
  name: deepseek-api-hpa
```

```
spec:
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: deepseek-api
```

```
  minReplicas: 1
```

```
  maxReplicas: 10
```

```
  metrics:
```

```
  - type: Resource
```

```
    resource:
```

```
      name: cpu
```

```
target:
  type: Utilization
  averageUtilization: 70
```

Load balancing

For load balancing across multiple replicas, you can use Kubernetes Services or an Ingress controller:

```
# deepseek-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: deepseek-api-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: deepseek-api.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: deepseek-api
            port:
              number: 80
```

Monitoring and logging

For production deployments, monitoring and logging are essential to ensure the health and performance of your application:

Prometheus and Grafana

Prometheus is a monitoring system that collects metrics from your applications, and Grafana provides visualization of these metrics:

```
# deepseek-monitoring.yaml
```

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: deepseek-api-monitor
labels:
  release: prometheus
spec:
  selector:
    matchLabels:
      app: deepseek-api
  endpoints:
    - port: http
      path: /metrics
```

To expose metrics from your FastAPI application, you can use the prometheus-fastapi-instrumentator library:

```
from prometheus_fastapi_instrumentator import Instrumentator
# Initialize FastAPI app
app = FastAPI(title="DeepSeek API", description="API for generating text
with DeepSeek models")
# Add Prometheus metrics
Instrumentator().instrument(app).expose(app)
```

Elasticsearch, Logstash, Kibana stack

The **Elasticsearch, Logstash, Kibana (ELK)** stack provides a comprehensive logging solution:

- **Elasticsearch:** Stores and indexes logs
- **Logstash:** Processes and transforms logs
- **Kibana:** Visualizes logs

To integrate with the ELK stack, you can use a logging library like **python-json-logger** to output logs in JSON format:

```
import logging

from pythonjsonlogger import jsonlogger
```



```
# Configure JSON logging
logger = logging.getLogger()
logHandler = logging.StreamHandler()
formatter = jsonlogger.JsonFormatter()
logHandler.setFormatter(formatter)
logger.addHandler(logHandler)
logger.setLevel(logging.INFO)

# Use logging in your application
logger.info("Model loaded", extra={"model_name": model_name})
```

API calling from client applications

Once your DeepSeek API is deployed, client applications can call it to generate text. Here are examples of how to call the API from different types of clients:

- **Python client:**

```
import requests
import json

def generate_text(prompt, max_tokens=512, temperature=0.7,
top_p=0.95):
    url = "http://localhost:8000/generate"
    headers = {"Content-Type": "application/json"}
    data = {
        "prompt": prompt,
        "max_tokens": max_tokens,
        "temperature": temperature,
        "top_p": top_p
    }
    response = requests.post(url, headers=headers,
data=json.dumps(data))
    if response.status_code == 200:
        return response.json()["text"]
    else:
        raise Exception(f"API call failed: {response.status_code} -
```

```
{response.text}"))
```

```
# Example usage
```

```
prompt = "Explain the concept of containerization in simple terms."
```

```
generated_text = generate_text(prompt)
```

```
print(generated_text)
```

- **JavaScript client:**

```
async function generateText(prompt, maxTokens = 512, temperature =
```

```
0.7, topP = 0.95) {
```

```
    const url = "http://localhost:8000/generate";
```

```
    const headers = {
```

```
        "Content-Type": "application/json"
```

```
    };
```

```
    const data = {
```

```
        prompt,
```

```
        max_tokens: maxTokens,
```

```
        temperature,
```

```
        top_p: topP
```

```
    };
```

```
    try {
```

```
        const response = await fetch(url, {
```

```
            method: "POST",
```

```
            headers,
```

```
            body: JSON.stringify(data)
```

```
        });
```

```
        if (!response.ok) {
```

```
            throw new Error(`API call failed: ${response.status} - ${await  
response.text()}`);
```

```
        }
```

```
        const result = await response.json();
```

```
        return result.text;
```

```
    } catch (error) {
```

```
        console.error("Error calling DeepSeek API:", error);
```

```
        throw error;
```

```
    }
```

```
}
```

```
// Example usage
generateText("Explain the concept of containerization in simple terms.")
  .then(text => console.log(text))
  .catch(error => console.error(error));
```

- **Curl command:**

```
curl -X POST "http://localhost:8000/generate" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Explain the concept of containerization in simple
terms.",
    "max_tokens": 512,
    "temperature": 0.7,
    "top_p": 0.95
  }'
```

Real-world applications

Since we have discussed the method of containerizing and deployment of DeepSeek models, it is appropriate to do the same with example applications of DeepSeek models. Deployed as containerized DeepSeek infrastructures, real-world solutions that use intelligent customer service bots, adaptive educational assistants, and more can be found as applications in many industries. In this section, we are going to present some of the best personalized practices where Dockerized DeepSeek models are being employed to find solutions to innovative problems on a massive scale.

Customer support

In customer support, DeepSeek chatbots can handle complex inquiries that traditional chatbots would struggle with:

- **Technical troubleshooting:** Walking users through multi-step troubleshooting processes, adapting based on user feedback at each step.
- **Policy explanations:** Breaking down complex policies and regulations into understandable explanations.

- **Product recommendations:** Reasoning through user requirements to suggest appropriate products or services.

For example, a telecommunications company implemented a DeepSeek-powered chatbot to handle technical support for its internet services. The chatbot could:

- Understand complex network issues described in natural language.
- Ask clarifying questions to narrow down the problem.
- Guide users through diagnostic steps, adapting based on the results.
- Provide tailored solutions based on the specific issue identified.

This implementation reduced the average resolution time by 45% and increased customer satisfaction scores by 30%.

Educational assistants

In education, DeepSeek chatbots serve as personalized tutors that can do the following tasks:

- **Explain complex concepts:** Breaking down difficult topics into understandable components.
- **Solve problems step-by-step:** Demonstrating reasoning processes for mathematical or scientific problems.
- **Answer follow-up questions:** Providing additional explanations or clarifications based on student queries.
- **Adapt to learning styles:** Tailoring explanations based on the student's level of understanding.

A notable implementation is an educational platform that integrated DeepSeek-R1 to provide personalized tutoring in STEM subjects. The chatbot could:

- Understand the student's current knowledge level based on their questions and responses.
- Provide step-by-step explanations of complex problems.
- Generate practice problems tailored to the student's abilities.
- Offer hints and guidance when students struggle with specific concepts.

This implementation led to a 40% improvement in student engagement and a

25% increase in test scores for users who regularly interacted with the chatbot.

Healthcare assistants

In healthcare, DeepSeek chatbots assist both patients and healthcare providers:

- **Symptom assessment:** Guiding patients through detailed symptom evaluations.
- **Treatment explanations:** Explaining complex treatment plans in understandable terms.
- **Medical literature analysis:** Summarizing and contextualizing research findings for healthcare providers.
- **Clinical decision support:** Assisting healthcare providers in diagnostic reasoning.

A healthcare provider implemented a DeepSeek-powered assistant to help patients understand their treatment plans and medication regimens. The chatbot could do the following:

- Access the patient's treatment plan (with appropriate privacy measures).
- Explain medications, potential side effects, and important precautions.
- Answer questions about interactions between medications.
- Provide reminders and follow-up information.

This implementation improved medication adherence by 35% and reduced unnecessary follow-up calls by 28%.

The application of DeepSeek models in the real world highlights both their potential in reasoning, in addition to the importance of containerization in ensuring such implementation is accomplished. With Docker, companies can maintain a uniform performance across the environments, simplify updates, and optimize the delivery of the services according to the demands. It can be a healthcare assistant that runs on-premises with very specific data compliance requirements or a customer support bot deployed in multiple cloud regions, but, in any case, with Dockerized DeepSeek models, production-ready AI systems have the flexibility, portability, and reliability they require. These advantages render the process of containerization one of

the key enablers in the transformation of high-power AI models into handy solutions.

Conclusion

In the chapter, we discussed the lifecycle of DeepSeek model deployment using Docker. We started out by introducing the concept of Docker and its importance in the production of portable, consistent, and scalable environments. There, we containerized DeepSeek models, got to know how to expose them through API endpoints, and analyzed workflows aimed at deploying using such tools as FastAPI, Docker Compose, and Kubernetes. Lastly, we applied the theory to practice and discussed the real-world scenarios where containerized DeepSeek models are already changing the features of customer support, education, health industries, and finance.

With this book at its end, we went through the entire landscape and story of DeepSeek, including the architecture and unique research in this field, the building blocks of different models, and its perception and reasoning ability, then the installation and configuration, optimization strategies, and deployment modes. All the parts of the course started to develop your knowledge step by step, enabling you to have a theoretical level as well as practical implementation in using DeepSeek in real practices of AI. Regardless of whether you are a researcher, a developer, or a hobbyist, you are now armed with the information you need to create, improve, and release intelligent DeepSeek-based systems with a sense of expertise. This is not only a conclusion to a book, but it is the start of your adventure of creating powerful, logical, and reasoning-based technological solutions to AI using DeepSeek.

Points to remember

- Docker provides a consistent, portable, and isolated environment for deploying DeepSeek models, ensuring they run the same way regardless of the underlying infrastructure.
- When containerizing DeepSeek models, you have several options for

handling model weights: including them in the image, downloading them at runtime, or mounting them as volumes.

- Optimizing Docker images for DeepSeek models is crucial due to their size. Techniques like multi-stage builds, model quantization, and using distilled models can significantly reduce image size and improve performance.
- FastAPI provides a modern, fast framework for building APIs around DeepSeek models, with features like automatic documentation generation and type checking.
- For production deployments, consider using Kubernetes for orchestration, implementing scaling and load balancing, and setting up monitoring and logging.
- Streaming responses can provide a better user experience for longer generations, allowing clients to display text as it is generated rather than waiting for the complete response.
- When deploying containerized DeepSeek models, ensure that the container has access to the necessary GPU resources for efficient inference.
- Different DeepSeek models may require different containerization approaches, depending on their size, capabilities, and resource requirements.

Key terms

- **Docker:** A platform that enables developers to build, package, and distribute applications as containers.
- **Container:** A lightweight, standalone, and executable package that includes everything needed to run an application.
- **Dockerfile:** A text file that contains instructions for building a Docker image.
- **Docker image:** A read-only template with instructions for creating Docker containers.
- **Docker volume:** Persistent data storage that exists outside the container

lifecycle.

- **FastAPI:** A modern, fast web framework for building APIs with Python.
- **Kubernetes:** An open-source platform for automating deployment, scaling, and management of containerized applications.
- **HPA:** A Kubernetes resource that automatically scales the number of pods based on CPU utilization or other metrics.
- **Prometheus:** A monitoring system that collects metrics from applications.
- **ELK stack:** A combination of Elasticsearch, Logstash, and Kibana for comprehensive logging.
- **Streaming response:** A technique for sending data to clients incrementally as it's generated, rather than waiting for the complete response.

Join our Discord space

Join our Discord workspace for latest updates, offers, tech happenings around the world, new releases, and sessions with the authors:

<https://discord.bpbonline.com>



[OceanofPDF.com](https://oceanofpdf.com)

Index

Symbols

4-bit quantization [123](#)

A

Accelerate [91](#)

accuracy reward system [26](#)

advanced agent techniques

 CoT method [212](#)

 Reasoning and Acting (ReAct) [209](#)

 self-reflection and correction [213](#)

 tool learning [210](#), [211](#)

advanced multimodal techniques [196](#)

 multimodal applications, with DeepSeek-VL [201](#), [202](#)

 multimodal chain-of-thought reasoning [198](#)

 multimodal few-shot learning [199](#), [200](#)

 multimodal RAG [196](#), [197](#)

 RAG [196](#)

 response quality with retrieval pipelines, improving [197](#)

advanced RLHF techniques [140](#)

agent applications

 with DeepSeek [215](#)

agents, building with DeepSeek [203](#)

 agent, implementing [208](#)

 language model, setting up [203](#)

 memory, implementing [204](#)

 planning and execution, implementing [207](#)

 tools, defining [205](#)

AHA moments [8](#), [28](#)

Amazon Web Services (AWS) [188](#)

American Invitational Mathematics Examination (AIME) [3](#), [18](#)

API-based deployment [64](#)

 working [64](#), [65](#)

API calling

 from client applications [246](#)

API integration best practices [68](#)

 caching [69](#)

 error handling and retry [68](#)

 prompt engineering [69](#)

API security

- considerations [70, 71](#)
- API versus local LLMs
 - pros and cons [80-84](#)
- artificial intelligence (AI) [1](#)
- AutoGPTQ [94](#)

B

- batch processing [105](#)
- bitsandbytes library [93](#)
- broad context window capability [44](#)

C

- chat application
 - building [107, 108](#)
- cloud deployment with AWS [188](#)
 - dependencies, installing [189](#)
 - FastAPI app [189](#)
 - inference endpoint [189](#)
 - server, running [190](#)
- CoT process [19, 212](#)

D

- DeepSeek [2](#)
 - applications [10, 11](#)
 - capabilities, exploring [103](#)
 - comparison, with traditional LLMs [3, 4, 5](#)
 - development [6](#)
 - development milestones [7](#)
 - distillation of reasoning capabilities [9](#)
 - evolution [6](#)
 - expert architecture [8](#)
 - features [2, 3](#)
 - impact on AI landscape [9, 10](#)
 - key research and contributions [7](#)
 - origins [6](#)
 - reasoning abilities [5](#)
 - research team [6](#)
 - RL innovations [8](#)
 - structure [3](#)
- DeepSeek API services [65-67](#)
 - pricing and quotas [67](#)
- DeepSeek containerization [227](#)
 - dependencies management [228](#)
 - model handling strategy [228](#)
 - preparing for [227](#)
 - project structure [227](#)

- DeepSeek language models
 - applications [46, 47](#)
 - evolution [43, 44](#)
- DeepSeek models
 - comparative analysis [58, 59](#)
 - containerizing [236-238](#)
- DeepSeekMoE [8](#)
- DeepSeek-R1-Distill series [53](#)
 - download and setup summary [55](#)
 - models and specifications [53, 54](#)
 - performance benchmarks [55](#)
- DeepSeek's Distillation process
 - innovations [53](#)
- DeepSeek's RL implementation [27](#)
 - DeepSeek-R1, using hybrid approach [27, 28](#)
 - DeepSeek-R1-Zero [27](#)
- DeepSeek-V3 [44](#)
- DeepSeek-V3.2-Exp [226, 227](#)
- DeepSeek vision models
 - applications [50, 51](#)
- DeepSeek-VL [48](#)
 - architecture and design [48](#)
 - capabilities and performance [49](#)
 - specialized vision processing [49, 50](#)
- DeepSeek-VL2 [47](#)
- direct preference optimization (DPO) [140, 141](#)
 - advantages [142](#)
 - constitutional AI [142](#)
 - implementing [155](#)
 - iterative RLHF [142](#)
- distillation operation [52](#)
- Distillation-Oriented Trainer (DOT) [57](#)
- distilled models [51](#)
 - distillation process [51, 52](#)
 - practical applications [56, 57](#)
 - trade-offs and considerations [57](#)
- Docker [220](#)
 - architecture [221](#)
 - benefits, for AI applications [223, 224](#)
 - best practices [224-226](#)
 - components [221](#)
 - containers [221](#)
 - images [221](#)
 - networks [221](#)
 - objects [221](#)
 - volumes [221](#)
- Docker Compose [240](#)
- Docker Engine [221](#)

- Dockerfile [221](#), [222](#)
- Dockerfile for DeepSeek
 - creating [229](#), [230](#)
 - model, downloading at runtime [231](#), [232](#)
 - model weights, including in image [230](#), [231](#)
 - model weights, mounting as volume [232-234](#)
- Docker image
 - building [236](#)
 - testing [236](#)
- Docker image optimization [234](#)
 - distilled models [235](#)
 - efficient dependency management [235](#)
 - layer optimization [236](#)
 - multi-stage builds [234](#)
- Docker workflow [222](#), [223](#)

E

- Elasticsearch, Logstash, Kibana (ELK) stack [245](#)
- environment setup [94](#)
 - Python environment, setting up [95](#)
 - system requirements [94](#), [95](#)

F

- FastAPI [238](#)
- FastAPI application
 - creating [238](#), [240](#)
 - deploying, to Kubernetes [241-243](#)
 - deploying, with Docker Compose [240](#)
 - load balancing [244](#)
 - monitoring and logging [244](#), [245](#)
- fine-tuning [112](#), [113](#)
 - approaches, comparing [127](#), [128](#)
 - using [113](#)
- Flash Attention [78](#), [94](#)
- format-based rewards [26](#)

G

- GPU setup for NVIDIA cards [96](#)
 - common setup issues, troubleshooting [97](#)
 - CUDA out of memory errors [97](#)
 - dependency conflicts [98](#), [99](#)
 - environment configuration, for optimal performance [96](#), [97](#)
 - slow inference performance [98](#)
- Grafana [244](#)
- Group Relative Policy Optimization (GRPO) [16](#), [31](#), [142](#), [143](#)
 - advantages [36](#), [37](#)

- challenges in LLM policy optimization [31](#), [32](#)
- critic model, eliminating [32](#)
- implementing [157](#), [161](#)
- limitations [37](#)
- policy optimization fundamentals [31](#)
- traditional policy optimization [31](#)
- using [32](#)
- GRPO algorithm [33](#), [34](#)
- GRPO implementation [34](#)
 - DeepSeek-R1 [36](#)
 - DeepSeek-R1-Zero training [34](#), [35](#)

H

- Hello DeepSeek [99](#)
 - Hugging Face Transformers, using [100](#)
 - LM Studio, using [101](#)
 - model, loading [100](#)
 - model, selecting [99](#)
 - Ollama, using [100](#), [101](#)
- Horizontal Pod Autoscaler (HPA) [243](#)
- Hugging Face [170](#)
- Hugging Face Transformers [73](#), [91](#)
- Hypothetical Document Embeddings (HyDE) [178](#)

I

- image captioning [193](#)
- Image-to-Text Generation [195](#)
- inference endpoint
 - with Hugging Face [170](#)
- inference optimization [104](#)
 - batch processing [105](#)
 - parameter tuning [104](#), [105](#)
 - prompt engineering [104](#)
 - streaming generation [106](#)
- intelligent agents [202](#)
 - typical agent architecture [202](#)

K

- Key-Value (KV) cache [77](#)
- Kullback-Lebler (KL) divergent penalties [36](#)

L

- language models [42](#), [43](#)
 - architecture [44](#), [45](#)
 - capabilities and performance [45](#), [46](#)
 - DeepSeek language models [43](#), [44](#)

- large language models (LLMs) [4](#)
- LlamaIndex [75](#)
- LM Studio [93](#)
- local deployment architectures [78](#)
 - best practices [79, 80](#)
 - distributed deployment [78, 79](#)
 - hybrid deployment [79](#)
 - security considerations [80](#)
 - single-server deployment [78](#)
- Local LLM deployment [71](#)
 - approach, selecting [85, 86](#)
 - frameworks and tools [73-75](#)
 - hardware requirements [73](#)
 - options [72](#)
 - working [71, 72](#)
- Local LLM tools [90](#)
 - Accelerate [91](#)
 - core frameworks and libraries [91](#)
 - Hugging Face Transformers [91](#)
 - installation [91](#)
 - VLLM [92](#)
- LoRA adapters
 - advanced techniques and future directions [130](#)
 - merging, with base models [129, 130](#)
- Low-Rank Adaptation (LoRA) [119](#)
 - advantages [120](#)
 - implementing, for DeepSeek models [120, 123](#)
 - target modules, for DeepSeek models [123](#)
 - working [119, 120](#)

M

- Mixture of Experts (MOE) architecture [6, 43](#)
- model sharding [76](#)
- multimodal applications [190](#)
- multimodal applications, with DeepSeek VL
 - building [191](#)
 - DeepSeek-VL, setting up [191](#)
 - image-based reasoning [194](#)
 - Image-to-Text Generation [195](#)
 - imaging captioning [193](#)
 - implementing [196](#)
 - Visual Question Answering [193](#)
- multimodal integration [191](#)
 - challenges [191](#)
- Multi-Stage Distillation [53](#)

N

natural language processing (NLP) tasks [42](#)

O

Ollama [75](#), [92](#)

optimization libraries [93](#)

AutoGPTQ [94](#)

bitsandbytes [93](#), [94](#)

Flash Attention [94](#)

optimization techniques [76](#)

Flash Attention [78](#)

Key-Value (KV) cache management [77](#)

model sharding [76](#), [77](#)

quantization [76](#)

P

parameter-efficient fine-tuning (PEFT) library [119](#), [120](#)

Quantized Low-Rank Adaptation [123](#)

parameter-efficient fine-tuning techniques

best practices [128](#), [129](#)

parameter tuning [104](#), [105](#)

policy optimization [31](#)

preference collection interface

building [146](#)

preference data collection [145](#)

preference data guidelines [148](#)

responses, generating for comparison [145](#)

process rewards [26](#)

Prometheus [244](#)

prompt engineering [104](#)

proximal policy optimization (PPO) [31](#), [138](#), [139](#)

environment, setting up [152](#)

KL penalty and reference model [139](#)

policy optimization, with [152](#)

training loop, implementing [154](#)

Python environment

setting up [95](#)

PyTorch

installing [91](#)

Q

quantization [76](#)

Quantized Low-Rank Adaptation (QLoRA) [123](#)

advantages [124](#)

implementing [124](#), [127](#)

working [123](#)

R

- RAG applications, with DeepSeek 181
 - educational content 183
 - legal research 182
 - medical question answering 181, 182
 - technical support 182
- RAG systems evaluation 179
 - answer quality evaluation 180
 - hallucination assessment 180, 181
 - relevance evaluation 179
- RAG system with DeepSeek
 - building 172
 - document processing and indexing 172, 173
 - generation with DeepSeek 175
 - prompt construction 174
 - RAG system 175
 - retrieval component 173, 174
- real-world applications 247
 - customer support 248
 - educational assistants 248, 249
 - healthcare assistants 249
- Reasoning and Acting (ReAct) 209
- reasoning capabilities 16
 - chain-of-thought (CoT) reasoning 19-21
 - comparative advantage 22
 - core reasoning abilities 18
 - emergence 16, 17
 - emergent behaviors 21, 22
 - performance metrics 18, 19
- Reasoning-Focused Distillation 53
- Reciprocal Rank Fusion (RRF) 176
- reinforcement learning from human feedback (RLHF) 133-135
 - challenges 139, 140
 - paradigm 135, 136
 - role in DeepSeek development 143, 144
- reinforcement learning innovations 8
- reinforcement learning (RL) 1, 23
 - challenges and solutions 29, 30
 - fundamental concepts 23
 - implementing 25
 - pretraining 24
 - process 24
 - role in reasoning capabilities 30
 - versus, traditional training methods 24
- relativization optimization (GRPO) 8
- response quality with retrieval pipelines
 - hybrid search 176
 - improving 176

- query decomposition [177](#), [178](#)
- re-ranking [177](#)
- retrieval-augmented generation (RAG) [171](#)
 - working [171](#)
- reward model training [148](#)
 - dataset, preparing [148](#)
 - implementing [150](#), [151](#)
- reward sparsity [32](#)
- RL concepts, in DeepSeek [26](#)
 - exploration, versus exploitation [26](#)
 - policy optimization [26](#)
 - reward functions [26](#)
- RLHF implementation, with DeepSeek [144](#)
 - preference data collection [145](#)
 - prerequisites [144](#)
 - reward model training [148](#)
- RLHF model evaluation [161](#)
 - preference evaluation [161](#)
 - safety and alignment evaluation [165](#)
 - task-specific evaluation [163](#)
- RLHF process in detail [136](#)
 - policy optimization [138](#)
 - preference data collection [137](#)
 - reward modeling [137](#)
 - reward model training [137](#), [138](#)
 - supervised fine-tuning [136](#)
- running inference, with DeepSeek [101](#)
 - Hugging Face Transformers, using [101](#), [102](#)
 - LM Studio, using [102](#)
 - Ollama, using [102](#)

S

- sample efficiency [32](#)
- Selective Knowledge Transfer [53](#)
- self-learning and emergent behaviors [28](#)
 - aha moment [28](#)
 - self-verification [29](#)
 - thinking time allocation [28](#)
- self-reflection [213](#)
- server-sent events (SSE) [106](#)
- specialized tools, for local deployment [92](#)
 - LM Studio [93](#)
 - Ollama [92](#)
 - Text Generation WebUI [93](#)
- streaming generation [106](#)
- supervised fine-tuning (SFT) [8](#), [25](#), [111-114](#)
 - dataset preparation [114](#), [115](#)
 - DeepSeek models, fine-tuning [116-118](#)

evaluation [116](#)
hyperparameter selection [116](#)
model selection [115](#)
training execution [116](#)

T

Text Generation WebUI [93](#)
token optimization [70](#)
tool learning [210](#), [211](#)
traditional fine-tuning
 challenges [118](#), [119](#)
typical agent architecture [202](#)

V

vision models [47](#), [48](#)
Vision Transformer (ViT) architecture [48](#)
Visual Question Answering (VQA) [193](#)
 tasks [49](#)
VLLM [74](#), [92](#)